<리눅스 사용을 위한 기본 구성>
리눅스 배포판 확인 및 오픈소스 역사

리눅스 소개

오픈소스 프로젝트

- GNU (GNU's Not Unix)

    GNU 시스템을 개발하기 위해 1984년에 시작

- GPL (General Public License)

소프트웨어의 실행, 연구, 공유, 수정의 자유를 최종 사용자에게 보장하며 GPL은 가장 널리 알려진 강한 카피레프트 사용 허가이며, 이 허가를 가진 프로그램을 사용하여 새로운 프로그램을 만들게 되면 파생된 프로그램 역시 같은 카피레프트를 가져야 한다

자신이 만들 소프트웨어에 GPL 라이선스를 가지는 코드가 한 줄 이라도 들어가면 소유권을 포기하라는 뜻이다.
완전한 평등과 완전한 공유 정신을 뜻한다.
오픈소스의 대명사격인 리눅스가 GPL을 대표한다고 할 수 있다.

소프트웨어에 관련된 다섯 가지 의무

GNU 일반 공중 사용 허가서는 누구에게나 다음의 다섯 가지의 의무를 저작권의 한 부분으로서 강제한다. 1. 컴퓨터 프로그램을 어떠한 목적으로든지 사용할 수 있다. 다만 법으로 제한하는 행위는 할 수 없다. 2. 컴퓨터 프로그램의 실행 복사본은 언제나 프로그램의 소스 코드와 함께 판매하거나 소스코드를 무료로 배포해야 한다. 3. 컴퓨터 프로그램의 소스 코드를 용도에 따라 변경할 수 있다. 4. 변경된 컴퓨터 프로그램 역시 프로그램의 소스 코드를 반드시 공개 배포해야 한다. 5. 변경된 컴퓨터 프로그램 역시 반드시 똑같은 라이선스를 취해야 한다. 즉 GPL 라이선스를 적용해야 한다. https://ko.wikipedia.org/wiki/GNU_%EC%9D%BC%EB%B0%98_%EA%B3%B5%EC%A4%91_%EC%82%AC%EC%9A%A9_%ED%97%88%EA%B0%80%EC%84%9C

Open Source

오픈소스 소프트웨어는 사용자가 소프트웨어를 실행하고 복제하고 배포하고 학습하고 개작하고 향상시킬수 있는 소프트웨어이다.

GNU란 무엇인가?

GNU는 운영 체제의 하나이자 컴퓨터 소프트웨어의 모음집이다. GNU는 온전히 자유 소프트웨어로 이루어져 있으며그 중 대부분이 GNU 프로젝트의 GPL로 라이선스된다.

GNU는 "GNU's Not Unix!"(GNU는 유닉스가 아니다!)의 재귀 약자이며 이렇게 선정된 이유는 GNU의 디자인이 유닉스 계열이지만 자유 소프트웨어인 점과 유닉스 코드를 포함하지 않는다는 점에서 차별을 두려는 것이다.

GNU 프로젝트는운영 체제 커널, GNU HURD를 포함하고 있으며 이는 자유 소프트웨어 재단(FSF)에서 본래 초점을 두고 있었던 것이다. 그러나 GNU가 아닌 커널(가장 유명한 것으로 리눅스)은 GNU 소프트웨어와 함께 사용할 수도 있다. 즉, 허드(Hurd) 커널은 아직 운영용으로 준비되지는 않았는데, 이는 어떻게 GNU 시스템이 일반적으로 사용되고 있는지를 설명한다.GNU 소프트웨어와 리눅스 커널의 결합은 리눅스(또는 GNU/리눅스)로 알려져 있다.

GNU 프로젝트의 설립자 리처드 스톨먼

프로젝트의 창립자 리처드 스톨먼은 GNU를 "사회에 대한 기술적 수단'으로 보았다. 이와 관련해 Lawrence Lessig는 스톨먼의 책《자유 소프트웨어, 자유 사회》제2판에서 스톨먼은 소프트웨어의 사회적 관념과 어떻게 자유 소프트웨어가 공동체와 사회 정의를 조성할 수 있는지에 대해 썼다고 언급하였다.

GNU란 GNU는 UNIX가 아니라는 뜻이다. GNU 프로젝트에서 만들어진 중요한 것 중 하나가 GNU 공용 라이선스(GPL)다. 이 라이선스는 해당 소프트웨어가 완벽하게 공개이며 그 누구도 이러한 자유를 없앨 수 없음을 분명하게 명시한다. 소프트웨어는 사용할 수 있고 이득을 얻기 위해 되팔 수 있지만 이러한 재판매가 이워질 경우 판매자는 반드시 변경된 부분을 포함한 모든 소스 코드를 공개해야 한다. 왜냐하면 재판매된 패키지도 역시 GPL 정책을 따르게 되어 있기에 자유롭게 배포될 수 있고 제3자가 이익을

얻기 위해 다시 판매할 수 있다. -위키백과

GNU는 유닉스 형태의 운영체제로 자유 소프트웨어이며 여러분의 자유를 존중합니다. GNU의 개발은 여러분의 자유를 억압할 수 있는 소프트웨어를 사용하지 않도록 하였습니다.- gnu.org

프로그래머는 자신이 제작한 소프트웨어로 인해 발생한 어떠한 손해에도 법적 책임이 없다.
법적 책임은 사용한 당사자에게 있다.

오픈 소스 소프트웨어의 장점

1. 프로그램을 원하는 어떠한 목적으로도 실행할 수 있는 자유.
2. 프로그램이 어떻게 동작하는지 학습하고, 자신의 필요에 맞게 개작할 수 있는 자유. 이것을 위해서는 소스 코드에 대한 접근이 전제되어야 합니다.
3. 이웃을 도울 수 있도록 복제물을 재배포할 수 있는 자유.
4. 프로그램을 개선시킬 수 있는 자유와 개선된 이점을 공동체 전체가 누릴 수 있게 그것을 발표할 자유. 이를 위해서는 역시 소스 코드에 대한 접근이 전제되어야 합니다.
5. 코드상의 오류들이 많은 사람들의 눈에 의해 보다 쉽게 감지되고 빠르게 수정된다.
6. GPL시스템하에서는 프로그래머들이 소송에 대한 두려움 없이 마음껏 코드를 배포할 수 있다.

리눅스란?

- 리눅스는 유닉스 계열의 운영체제
POSIX 호환

- 리눅스는 합성어
리누스 토발즈와 UNIX의 합성어

- 리눅스 커널코드
자유 소프트웨어 재단 라이선스 정책인 GNU GPL을 따른다.

리눅스란 무엇인가?

리눅스(Linux)는 컴퓨터 운영 체제(서버 운영체제)이며, 그 커널을 뜻하기도 한다. 리눅스는 자유 소프트웨어와 오픈 소스 개발의 가장 유명한 표본으로 들 수 있다. 리눅스는 다중 사용자, 다중 작업(멀티태스킹), 다중 스레드를 지원하는 네트워크 운영 체제(NOS)이다.

엄밀하게 따지면 이'리눅스'라는 용어는 리눅스 커널만을 뜻하지만, 리눅스 커널과 GNU 프로젝트의 라이브러리와 도구들이 포함된, 전체 운영 체제(GNU/리눅스라고도 알려진)를 나타내는 말로 흔히 쓰인다. 리눅스 배포판은 핵심 시스템 외에 대다수 소프트웨어를 포함한다. 현재 200여 종류가 넘는 배포판이 존재한다.

초기에 리눅스는 개개인의 애호자들이 광범위하게 개발하였다. 이후 리눅스는 IBM, HP와 같은 거대 IT 기업의 후원을 받으며, 서버 분야에서 Unix와 Microsoft Windows 운영 체제의 대안으로 자리 잡았다.

리눅스는 데스크톱 컴퓨터를 위한 운영 체제로서도 인기가 늘어가고 있다. 지지자와 분석자들은 이와 같은 성공을 벤더 독립성과 적은 개발비, 보안성과 안전성에서 기인한다고 분석한다.

리눅스는 처음에 인텔 386 마이크로프로세서를 위해 개발되었으나 현재는 다양한 컴퓨터 아키텍처를 지원한다. 리눅스는 개인용 컴퓨터에서부터 슈퍼 컴퓨터는 물론 휴대 전화, 개인용 비디오 레코더와 같은 임베디드 시스템까지 광범위하게 이용되고 있다.

리누스 토발즈가 커널을 개발하고 GNU에서 개발된 나머지 명령어 프로그램들을 도입하여 현재의 리눅스로 발전해왔다.

현재는 각 국가들의 국가 운영체제로 리눅스를 채택하고 있다.

한국의 경우에는 일본, 중국, 미국의 영향을 받아 아직은 50:50정도로 가져가고 있으며 원천기술 개발이 부족한 시점이다.

리눅스의 특징

- 리눅스는 누구가 자유롭게 사용할 수 있는 운영체제
- 리눅스는 다중 유저, 다중 작업을 지원하는 시스템
- 리눅스는 강력하고 안정적인 네트워크를 지원하는 운영체제
- 리눅스는 가격대 성능비가 강력한 운영체제
- 리눅스는 이식성이 강력한 운영체제

리눅스의 장점

- 누구나 자유롭게 사용할 수 있는 운영체제이다.

오픈 소스로 인해 소스를 가공 & 변형 & 재배포 할 수 있는 GPL라이센스가 적용된다. 리눅스는 오픈소스나 배포에 치명적인 오류가 발생하더라도 여러 사람이 공동으로 소스를 수정 할 수 있도록 오픈되어 있기 때문에 MS와 같은 비오픈소스에 비해 정확하고 빠른 패치의 사용이 가능하다.

- 여러 사용자(Multi-user)가 동시에 사용할 수 있는 환경을 제공한다.

많은 사용자가 동시에 리눅스 시스템에 접속하더라도 다른 운영체제에 비해 안정적이고 탁월한 성능을 유지한다.

- 다중 작업(Multi Tasking) 및 가상 터미널(Virtual Terminal)환경을 지원한다.

윈도우는 한 화면에서 여러개의 창을 열어 다중작업을 하는 반면, 리눅스는 가상터미널 환경에서 작업할 수 있는 멀티캐스팅을 지원한다. 리눅스는 윈도우에서 도스창과 창은 콘솔화면 창을 6개까지 띄워 작업하는 것을 말하는데, 이는 도스와 같은 환경의 컴퓨터를 6대 사용하는 것과 같다.

- GUI방식의 X Window 를 지원한다.

리눅스에서는 2가지 형태의 작업 환경을 제공해준다. 첫 번째는 콘솔형태와 같은 TUI(Text User Interface)방식이고, 두 번째는 윈도우와 같은 GUI(Graphic User Interface)방식이다.

- CPU 구애가 없는 운영체계이다.

리눅스는 x86을 기반으로 하는 인텔계열의 모든 CPU를 완벽히 지원하며, 인텔 486 CPU 이상의 기종에서도 무리 없이 작동 할 수 있도록 설계되어 다른 운영체제에 비해 적은 시스템 자원을 가진 시스템에서도 리눅스를 사용 할 수 있다.

- 강력하면서 안정적인 네트워크를 지원하는 운영체계이다.

하나의 어플리케이션에 오류가 발생하여도 전체 시스템이 중지되지 않다. 각 어플리케이션은 메모리에서 고유한 개별 위치를 차지하며 사용자 프로세스는 운영 체제 프로세스와 완전히 분리되어 있어, 어플리케이션에서 오류가 발생하더라도 운영 체제는 전혀 영향을 받지 않다.

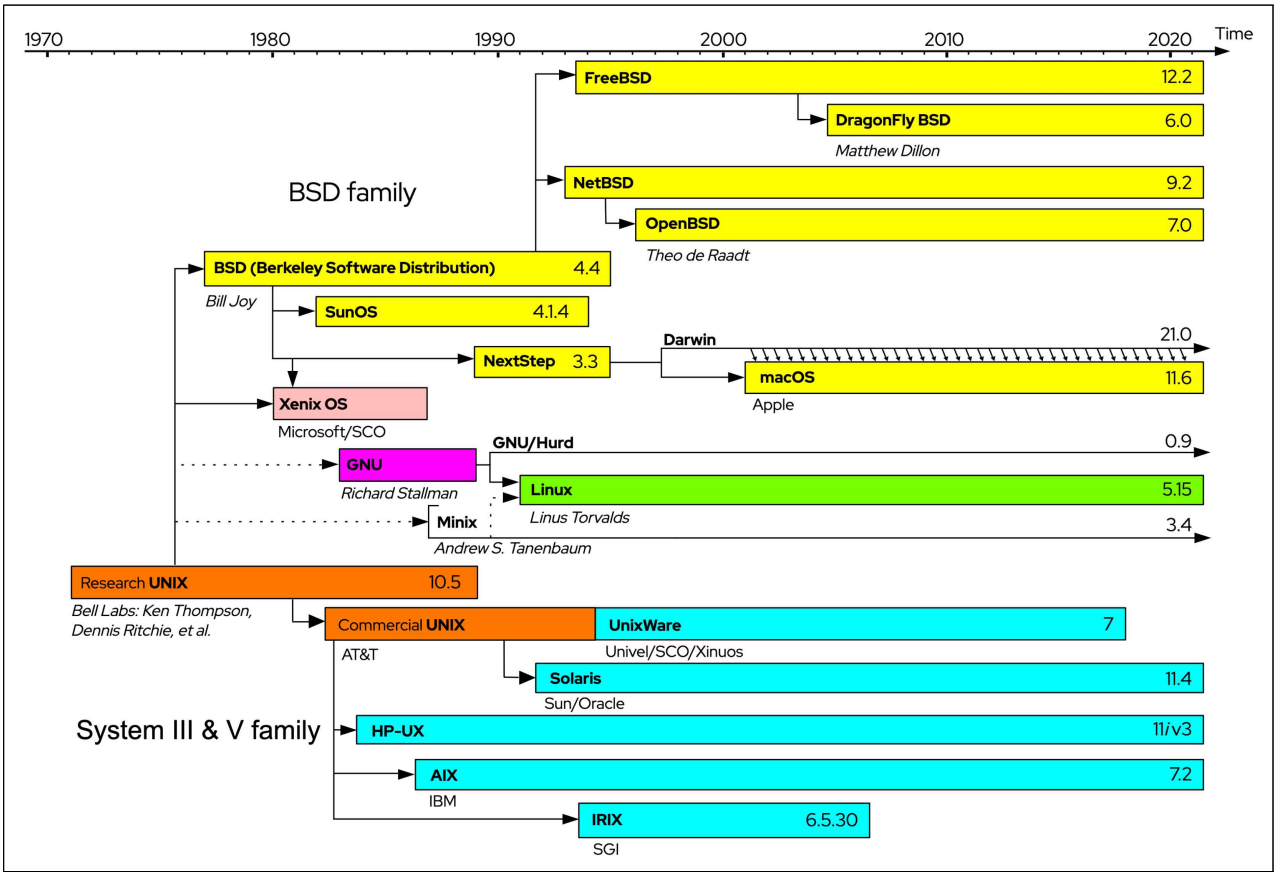
- 리눅스에서의 하드웨어 드라이버 설정 및 하드웨어 사용이 매우 쉽다.

리눅스는 드라이버를 커널에서 모듈의 형태로 지원하고 있어, 제조업체의 드라이버 없이도 동일한 칩셋을 사용한 하드웨어에 대해서는 범용적으로 지원한다.

- 이식성이 강한 운영체계이다.

리눅스는 Unix의 프로그램과 소스를 쉽게 이식하고 컴파일 할 수 있다.

리눅스의 역사



리눅스의 탄생 및 발전

핀란드 헬싱키 대학의 학생이었던 리누스 토발즈에 의해 만들어진 리눅스는 Unix(UNIX)에서부터 시작되었다.

실질적인 모태는 1987년에 Andrew S. Tanenbaum 교수가 개발한 Minix이다. Minix는 말 그대로 작은 Unix란 뜻으로 이것을 접한 리누스 토발즈가 지속적인 개발을 통해 1994년 3월에 커널버전 1.0을 탄생시켰다.

FSF의 설립자인 리차드 스톨만이 GNU 프로젝트를 추진한 후 1990년 무렵쯤 되었을 때 GNU 시스템은 거의 완성단계이 이르렀을 때 핵심 부분인 커널이 완전하지 않은 상태였다. 때마침 리누스가 만들어낸 리눅스가 GNU에 큰 힘을 실어주었고 GNU 시스템의 운영체제로 리눅스를 채택하게 된 것이다. GNU(GNU is not unix)란 쉽게 말해 자유 소프트웨어의 생산과 보급을 장려하기 위해 세워진 재단이라고 할 수 있다.

우리가 리눅스를 'GNU/Linux'라고 부르는 것은 바로 이와같은 배경때문이다.

현재 많이 알려진 리눅스의 배포판으로는 슬랙웨어, 레드햇, 와우리눅스, 데비안, 리눅스-맨드레이크, 오픈 리눅스 등이 있다.

리눅스는 1991년도에 출시되었다. 늦게 출시 되었지만 현재 계속 발전중인 OS의 하나이다.

리눅스(Linux)의 배포판

슬랙웨어, 레드햇, 맨드리바, 수세, 칼데라, 한컴리눅스 등 (엑스 윈도우, 인터넷 서버 및 클라이언트 프로그램과 그 외의 여러 가지 유틸리티들을 안정적으로 작동 될 수 있는 리눅스 배포판을 개발)

리눅스 배포판은 워드프로세서, 스프래드시트, 미디어 플레이어, 데이터베이스 등 여러 가지 소프트웨어 애플리케이션의 모음이다. 운영체제는 리눅스 커널과 GNU 프로젝트에서 가져온 라이브러리와 유틸리티, X 윈도 시스템의 그래픽으로 구성된다.

용량을 맞춰서 X 윈도를 빼거나 용량이 작은 GNU 유틸리티를 선택하기도 한다. 전 세계에 300개 이상의 배포판이 있다고 한다. 커널과 패키지 대부분이 무료 소프트웨어고 오픈 소스를 따르므로 각 배포판의 형태는 매우 다양하다.

페도라(Fedora)는 레드햇(Red Hat), openSUSE는 노벨(Novell), 우분투(Ubuntu)는 캐노니컬 등의 기업이 관리하는 배포판이다. 데비안(Debian)이나 젠투(Gentoo)는 리눅스 커뮤니티 기반의 배포판이다.

이에 해당하지 않는 배포판이 있는데 대표적인 것이 슬랙웨어(slackware)다. 리눅스 배포판이 처음부터 있던 것은 아니다. 리눅스 배포판은 전문가가 아닌 개인 이용자를 위해 만들어졌다.

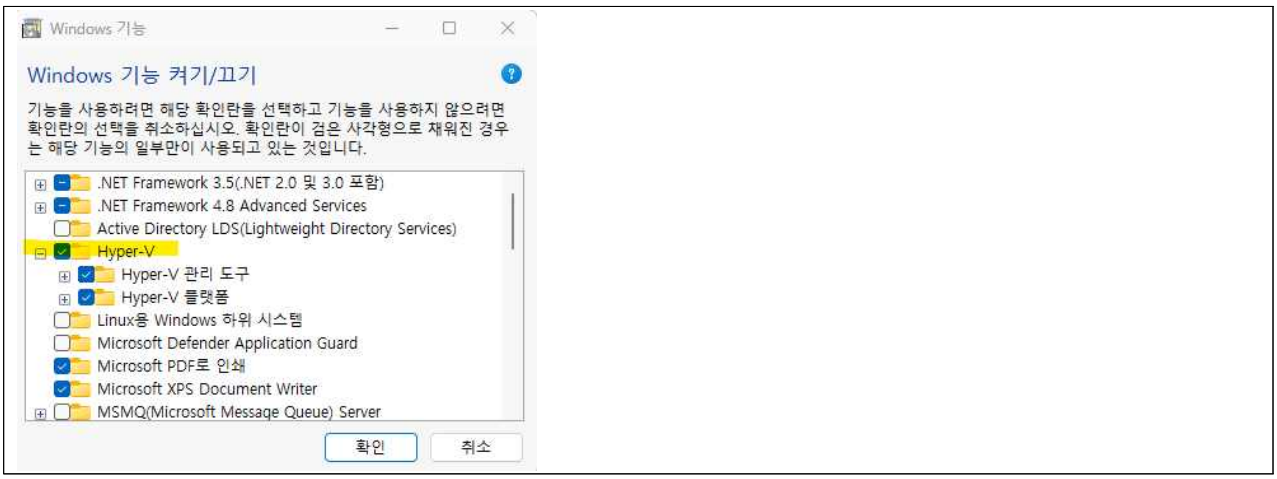
[참고]

기본 리눅스 커널에 자사의 프로그램을 같이 포함하여 특정 사용자를 위한 (서버, 클라이언트, 모바일, 임베디드)OS를 구성하여 만들어 내는 OS를 배포판이라 한다.

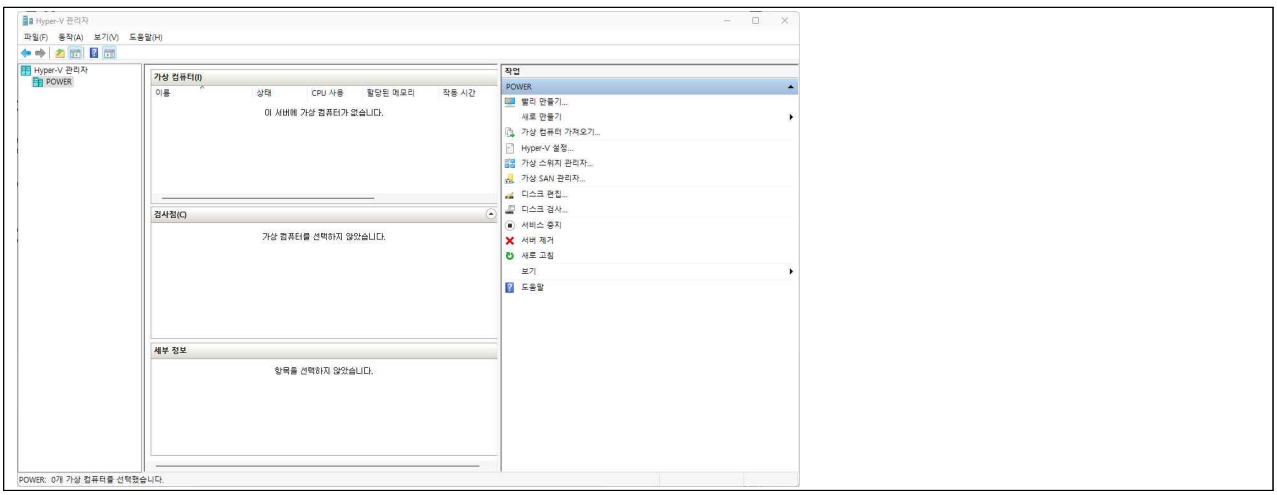
리눅스 설치 및 구성

Hyper-v 구성
MS windows server 가상화
windows 7 8 10 11

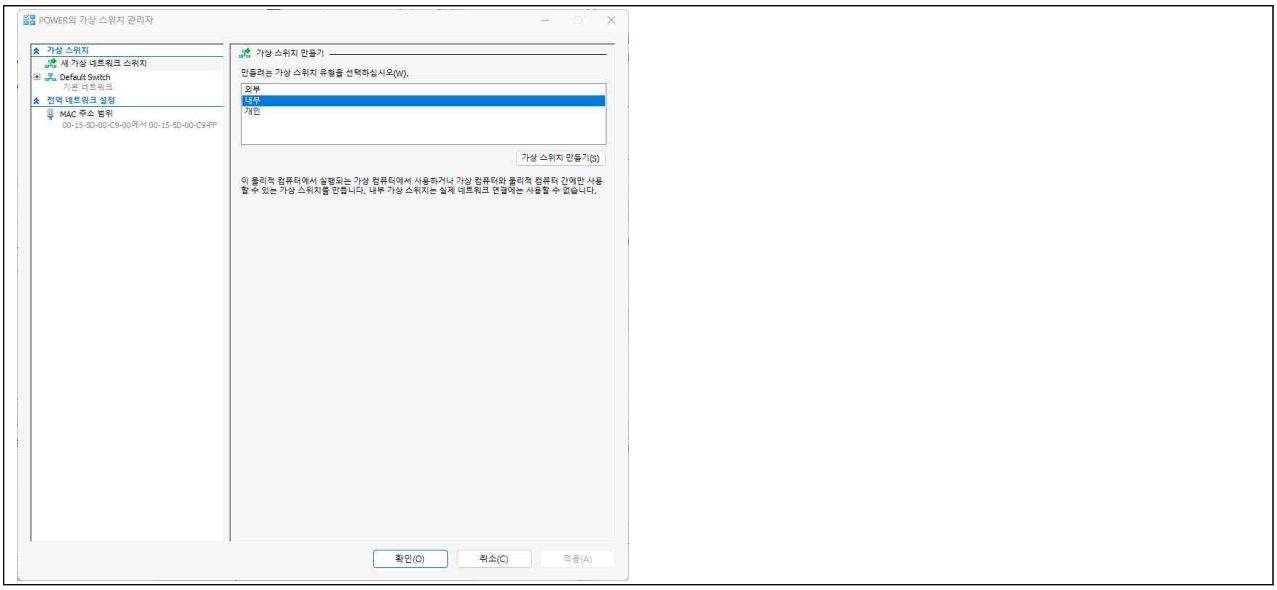
hyper-v 기능 활성화
[win]+ R > control > 범주 > 작은 아이콘 > 프로그램 및 기능



재부팅이 한번 필요하다.
나머지 작업
hyper-v

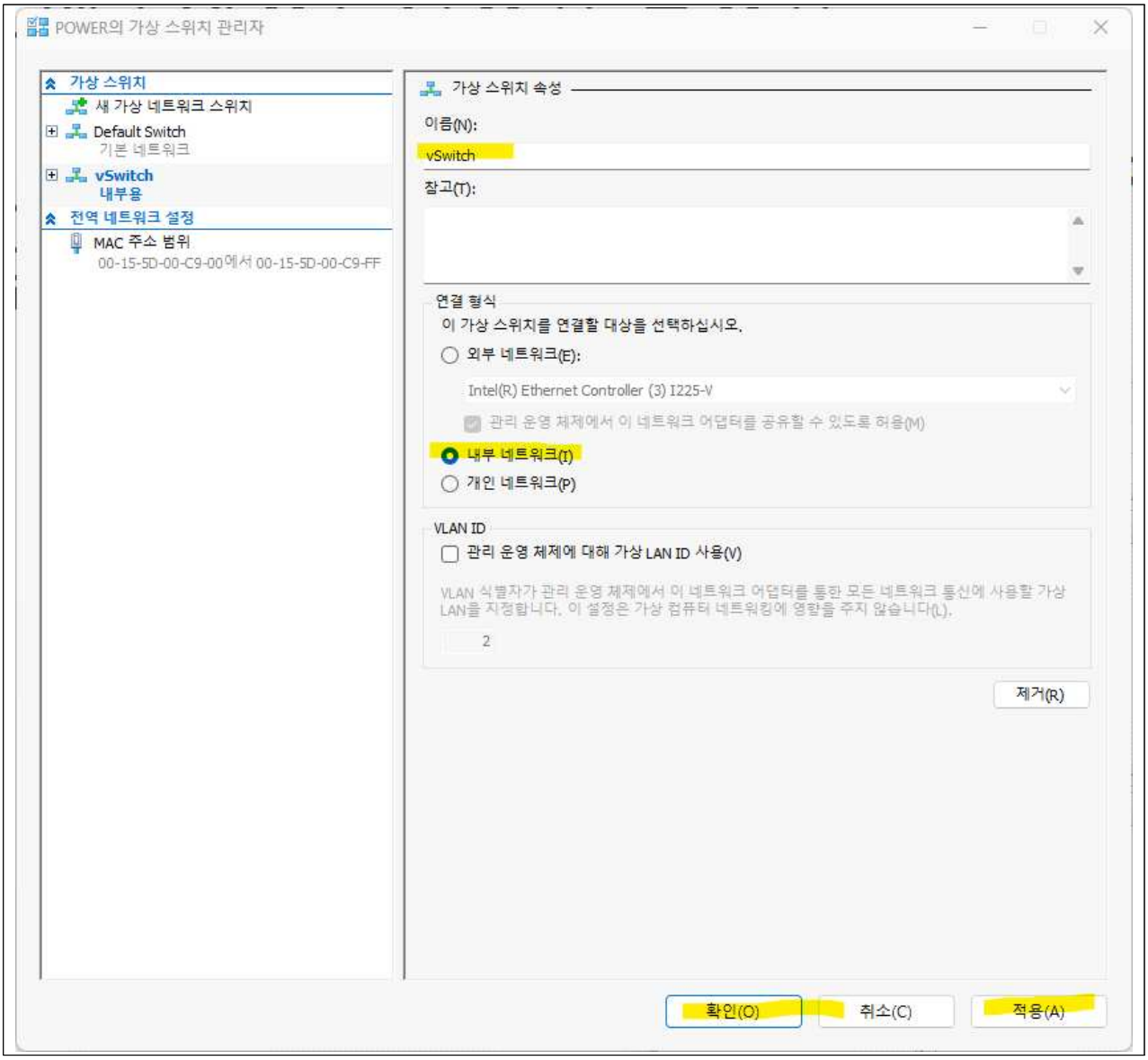


가상 스위치 관리를 통해 추가 가상스위치 등록 사용



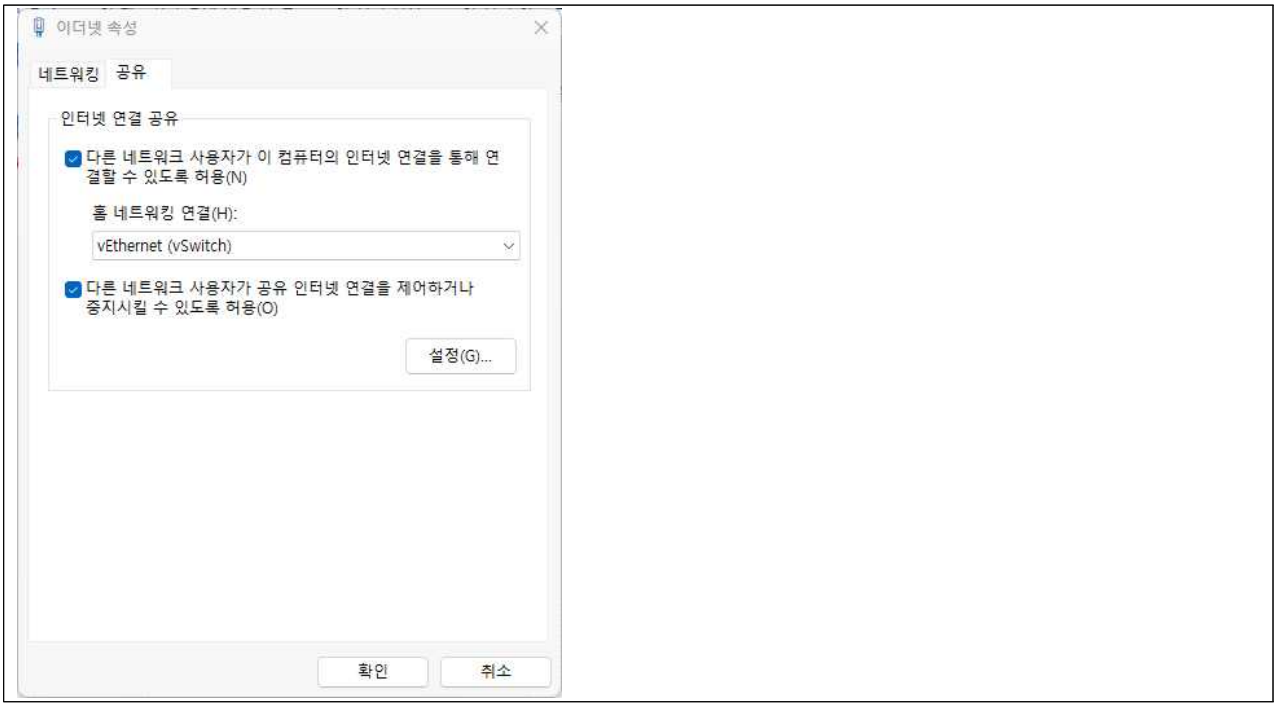
내부로 생성

가상스위치 만들기 선택



[win]+ R > ncpa.cpl
생성된 vSwitch 장치 확인 속성탭으로 진입
IPv4 속성 추가적인 정보를 입력

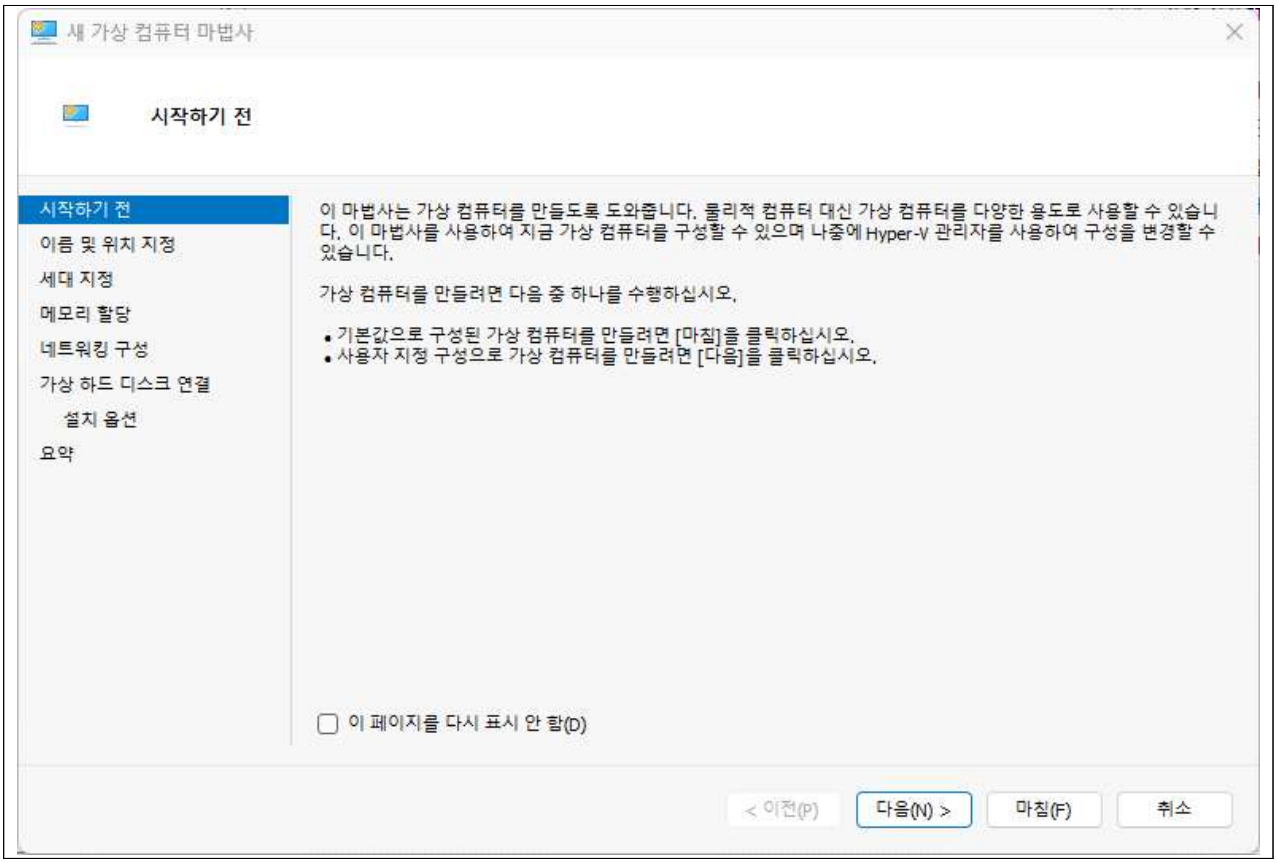
기존에 사용하던 실제 물리적인 NIC의 속성의 공유탭을 확인

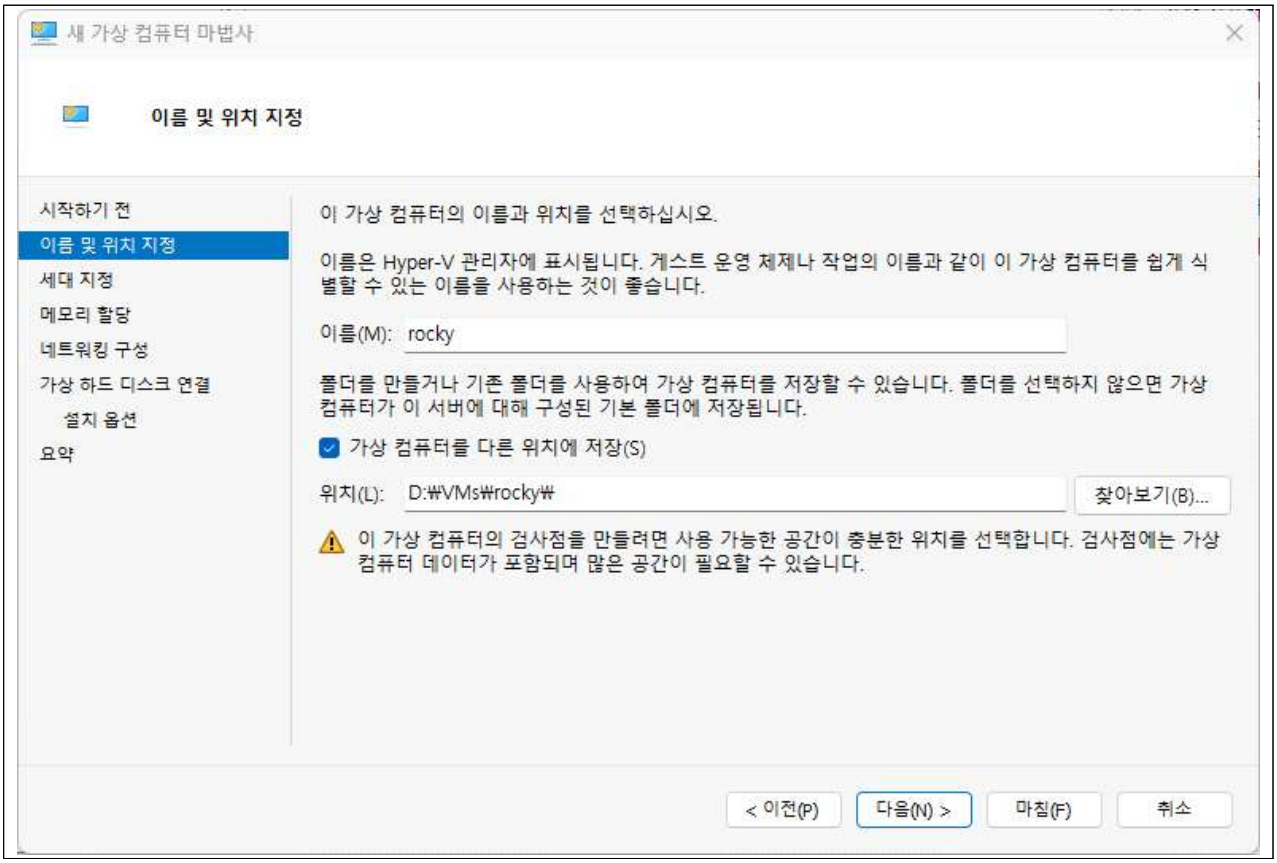


내부 스위치 장치와의 공유를 활성화

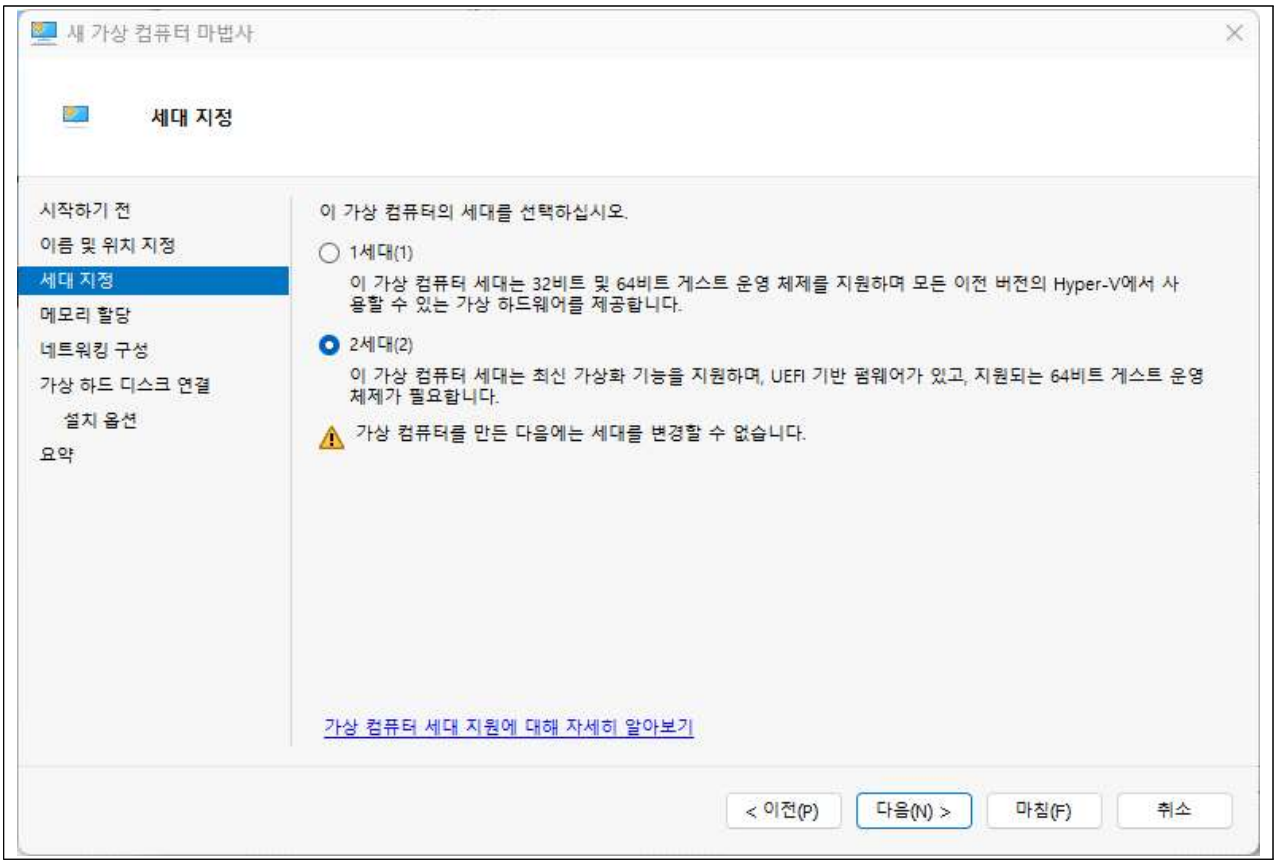
OS 설치

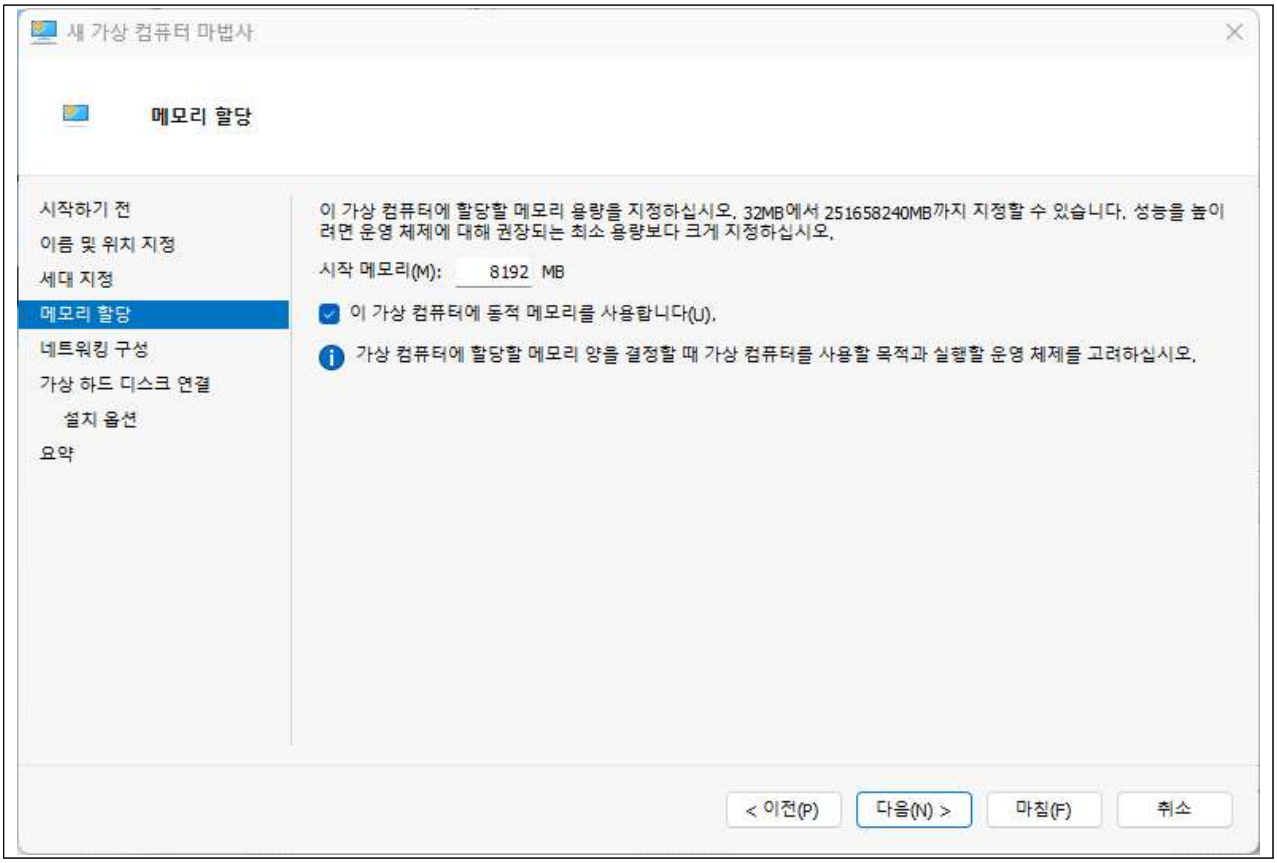
새로만들기 > 가상컴퓨터

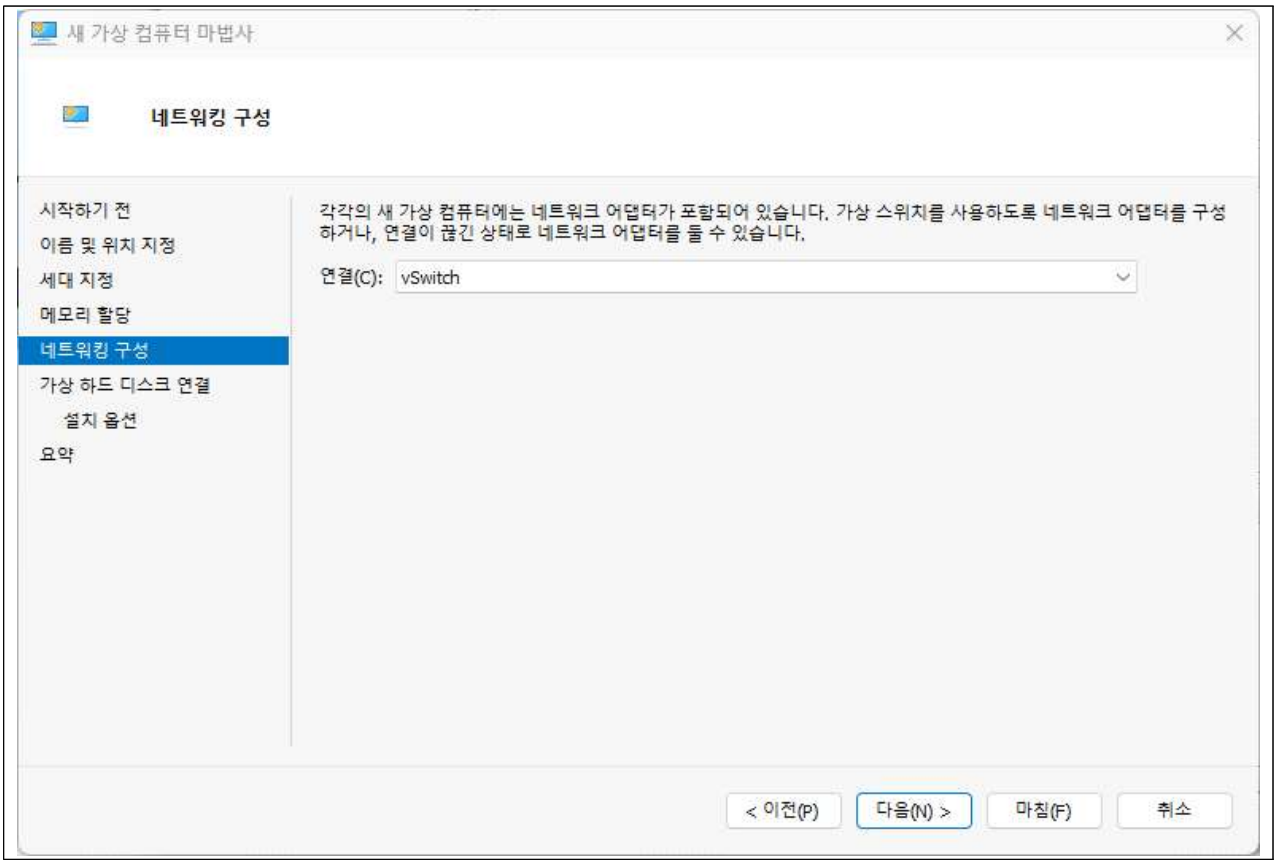


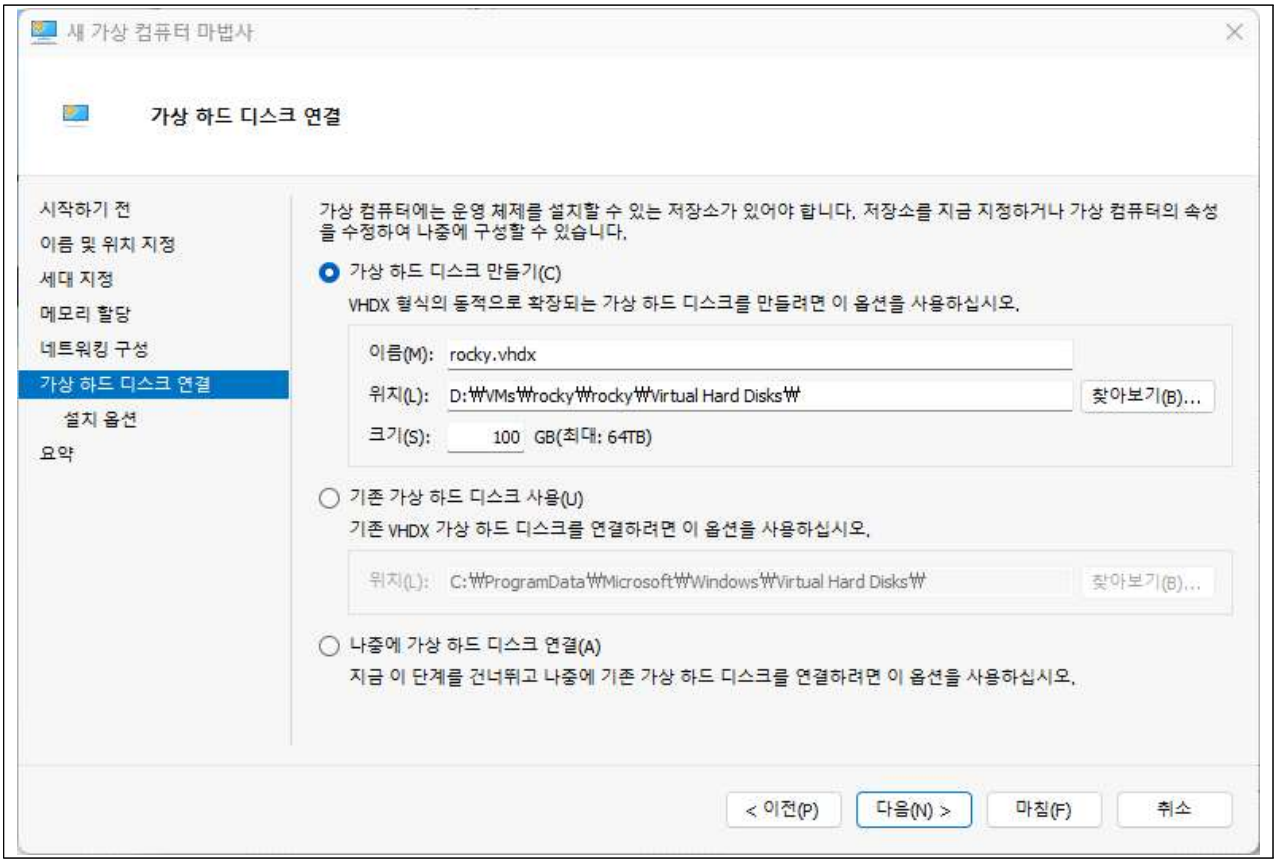


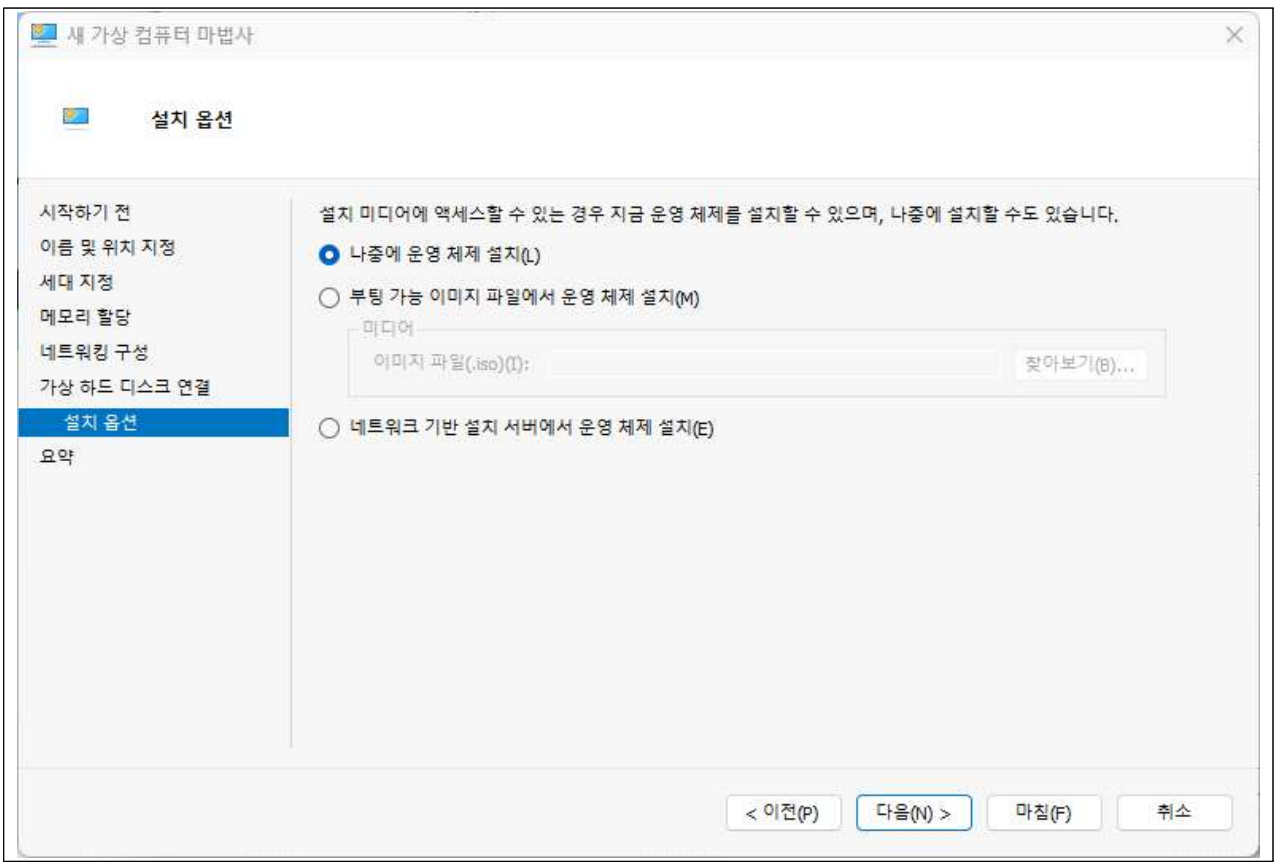
D:₩VMs₩rocky₩
만약에 D가 없다면 C의 동일한 위치에 생성

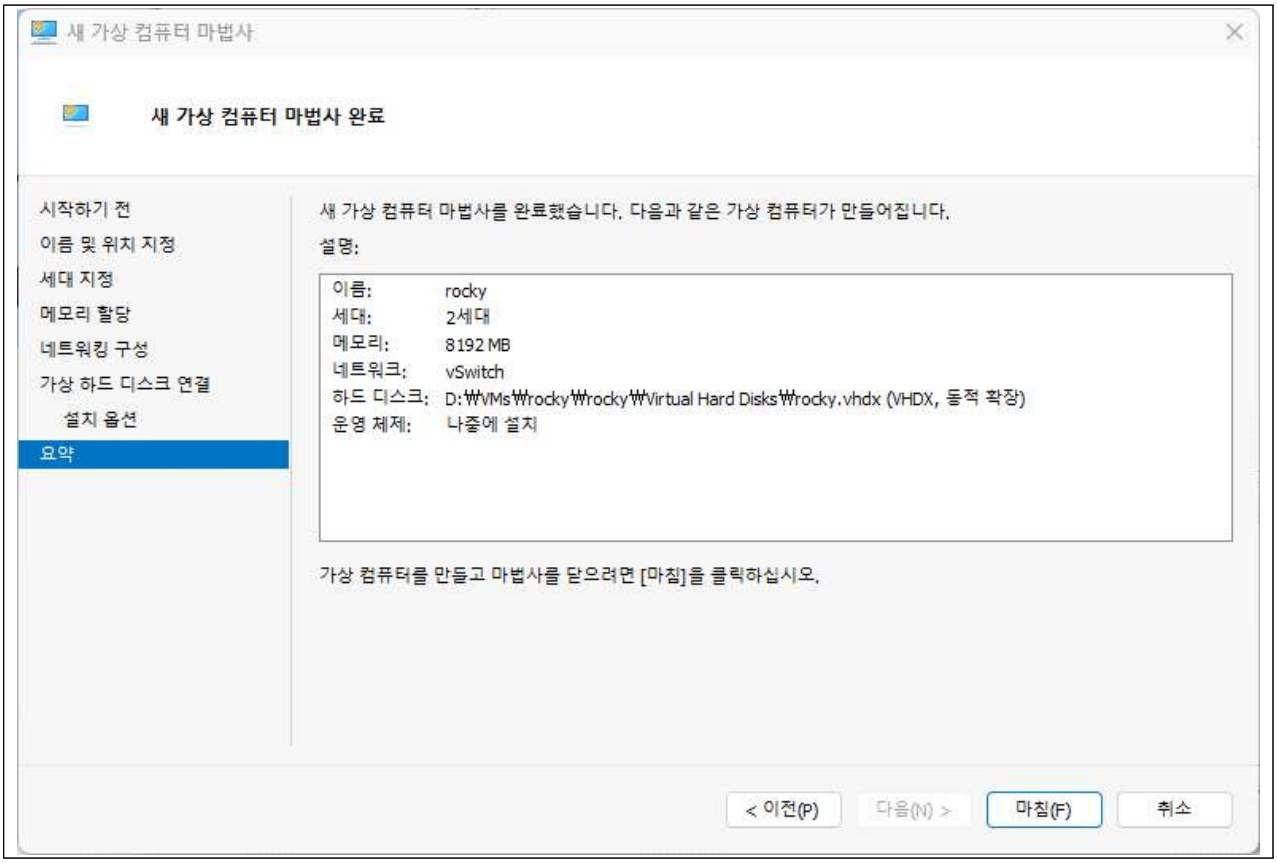


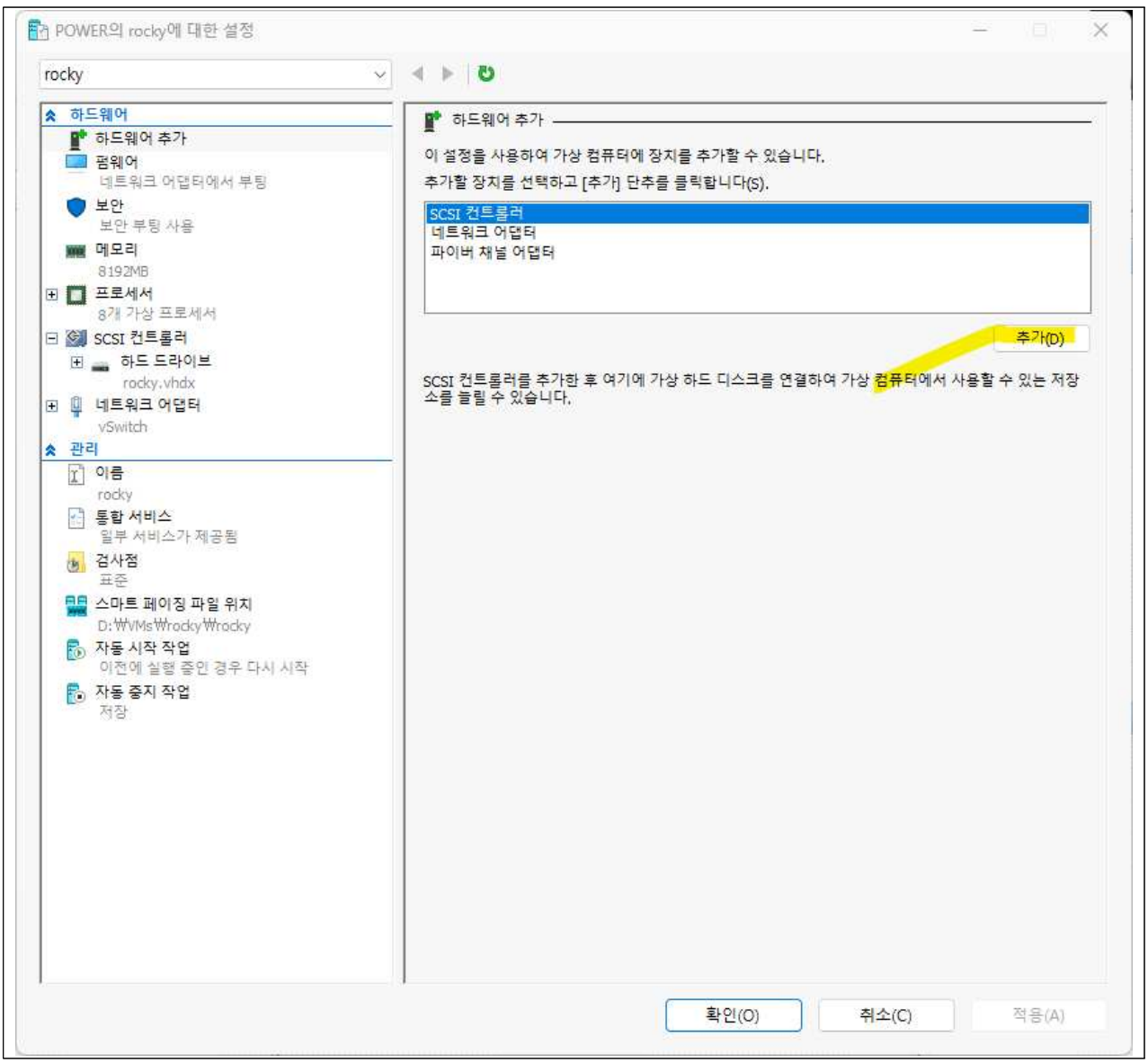

새 가상 컴퓨터 마법사
메모리 할당
시작하기 전
이름 및 위치 지정
세대 지정
메모리 할당
네트워킹 구성
가상 하드 디스크 연결
설치 옵션
요약
이 가상 컴퓨터에 할당할 메모리 용량을 지정하십시오. 32MB에서 251658240MB까지 지정할 수 있습니다. 성능을 높이려면 운영 체제에 대해 권장되는 최소 용량보다 크게 지정하십시오.
시작 메모리(M): 8192 MB
이 가상 컴퓨터에 동적 메모리를 사용합니다(U).
가상 컴퓨터에 할당할 메모리 양을 결정할 때 가상 컴퓨터를 사용할 목적과 실행할 운영 체제를 고려하십시오.
< 이전(P)
다음(N) >
마침(F)
취소

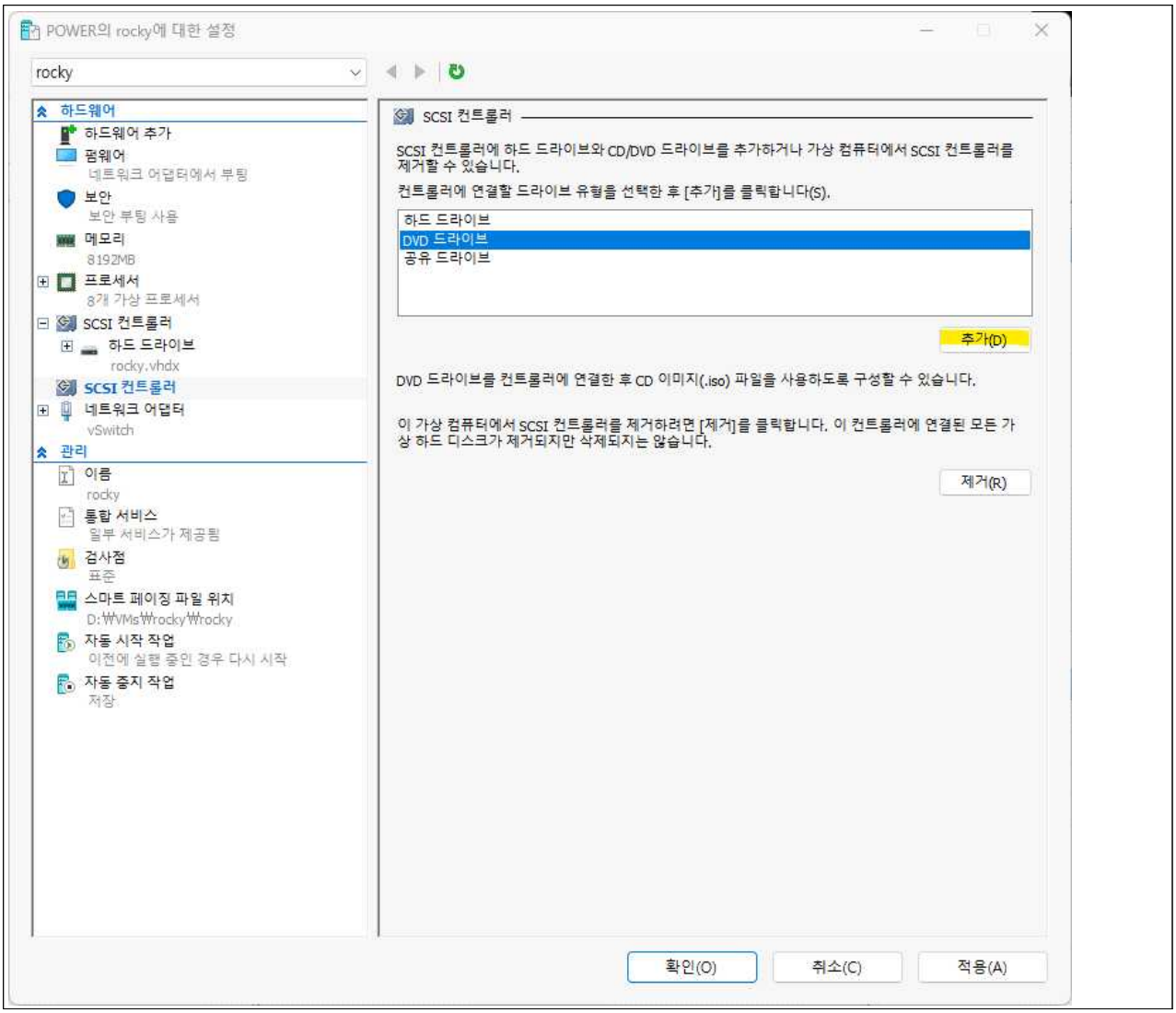

새 가상 컴퓨터 마법사
네트워킹 구성
시작하기 전
이름 및 위치 지정
세대 지정
메모리 할당
네트워킹 구성
가상 하드 디스크 연결
설치 옵션
요약
각각의 새 가상 컴퓨터에는 네트워크 어댑터가 포함되어 있습니다. 가상 스위치를 사용하도록 네트워크 어댑터를 구성하거나, 연결이 끊긴 상태로 네트워크 어댑터를 둘 수 있습니다.
연결(C): vSwitch
< 이전(P)
다음(N) >
마침(F)
취소

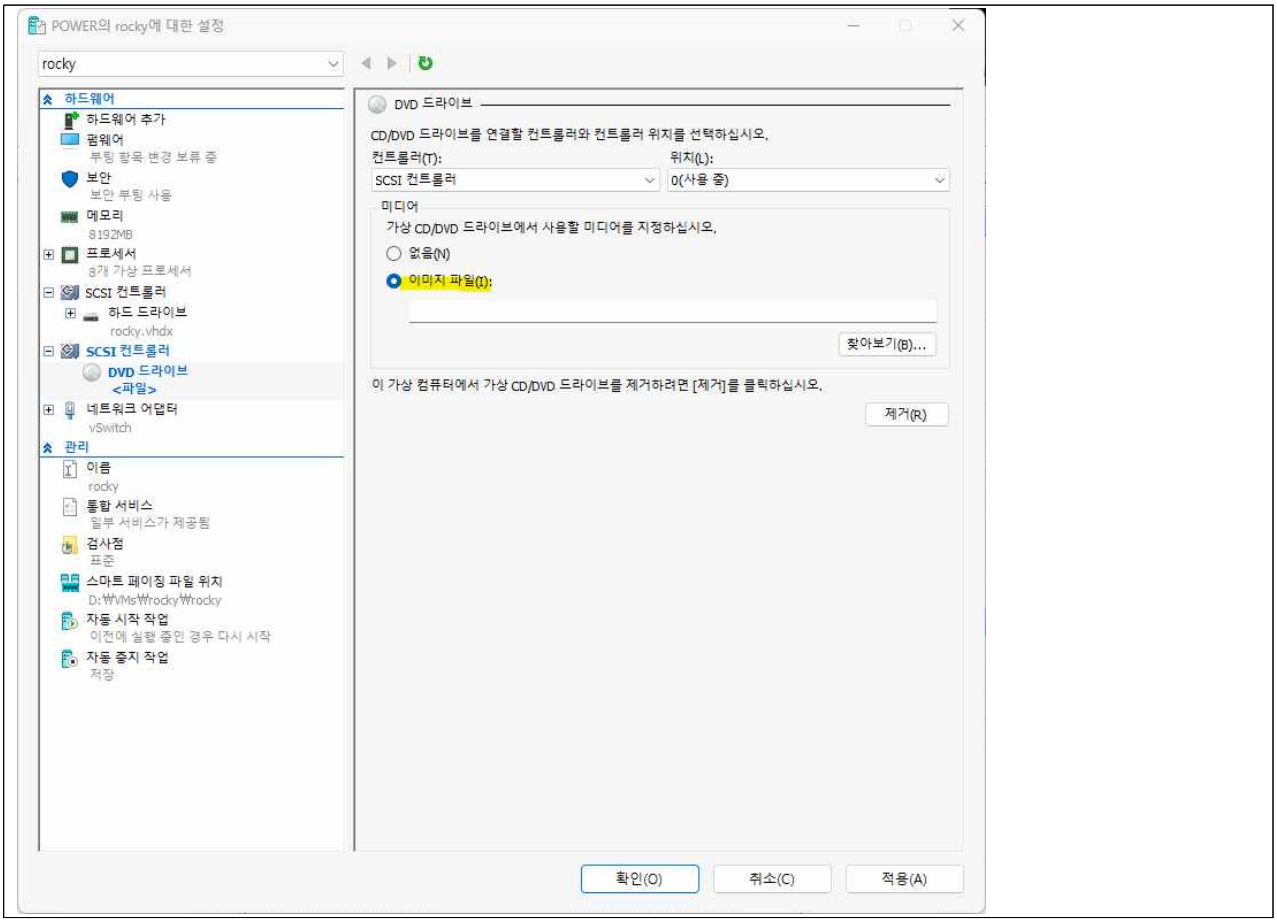

새 가상 컴퓨터 마법사
가상 하드 디스크 연결
시작하기 전
이름 및 위치 지정
세대 지정
메모리 할당
네트워킹 구성
가상 하드 디스크 연결
설치 옵션
요약
가상 컴퓨터에는 운영 체제를 설치할 수 있는 저장소가 있어야 합니다. 저장소를 지금 지정하거나 가상 컴퓨터의 속성을 수정하여 나중에 구성할 수 있습니다.
가상 하드 디스크 만들기(C)
VHDX 형식의 동적으로 확장되는 가상 하드 디스크를 만들려면 이 옵션을 사용하십시오.
이름(M): rocky.vhdx
위치(L): D:₩VMs₩rocky₩rocky₩Virtual Hard Disks₩
찾아보기(B)...
크기(S): 100 GB(최대: 64TB)
기존 가상 하드 디스크 사용(U)
기존 VHDX 가상 하드 디스크를 연결하려면 이 옵션을 사용하십시오.
위치(L): C:₩ProgramData₩Microsoft₩Windows₩Virtual Hard Disks₩
찾아보기(B)...
나중에 가상 하드 디스크 연결(A)
지금 이 단계를 건너뛰고 나중에 기존 가상 하드 디스크를 연결하려면 이 옵션을 사용하십시오.
< 이전(P)
다음(N) >
마침(F)
취소

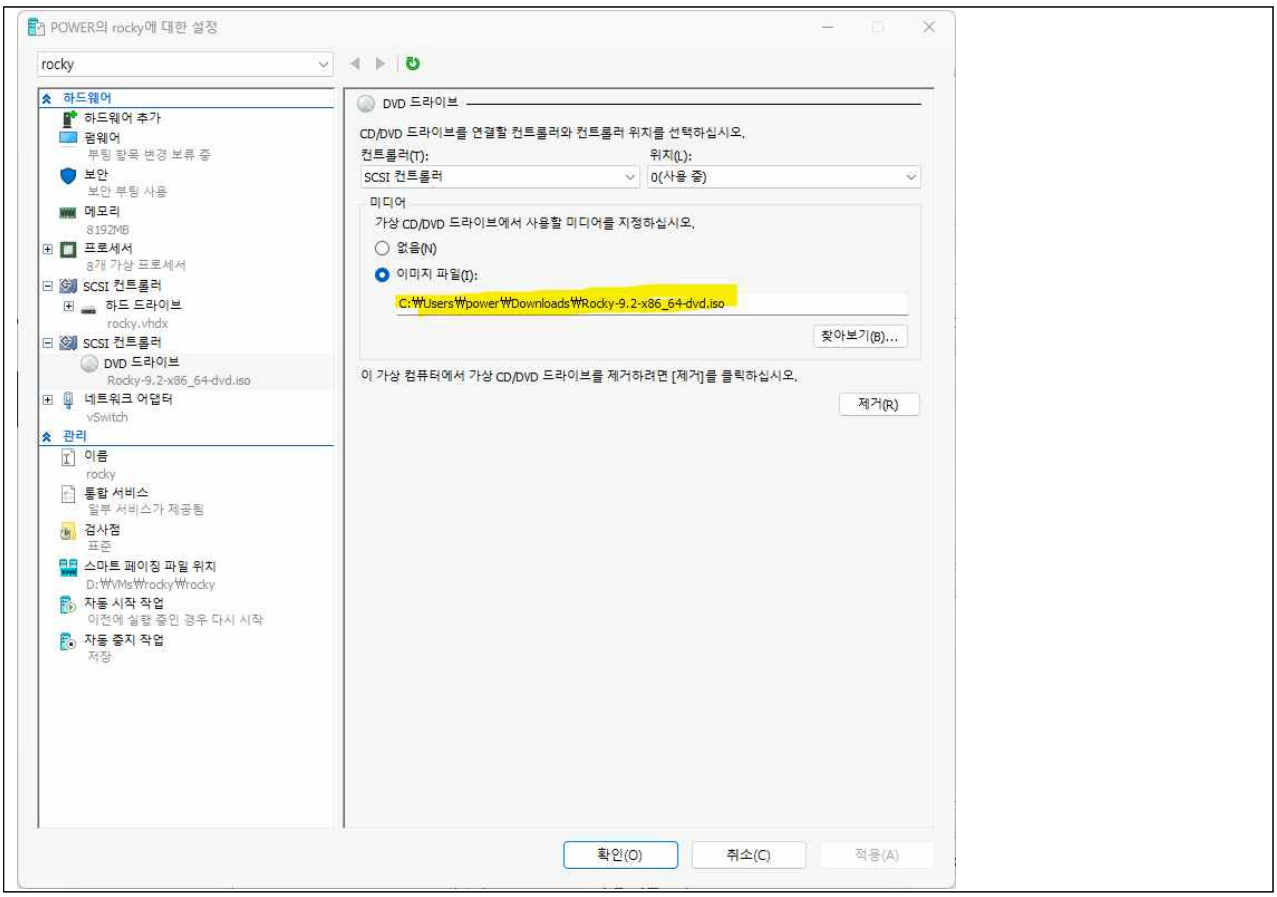

새 가상 컴퓨터 마법사
설치 옵션
시작하기 전
이름 및 위치 지정
세대 지정
메모리 할당
네트워킹 구성
가상 하드 디스크 연결
설치 옵션
요약
설치 미디어에 액세스할 수 있는 경우 지금 운영 체제를 설치할 수 있으며, 나중에 설치할 수도 있습니다.
나중에 운영 체제 설치(L)
부팅 가능 이미지 파일에서 운영 체제 설치(M)
미디어
이미지 파일(.iso)(I):
찾아보기(B)...
네트워크 기반 설치 서버에서 운영 체제 설치(E)
< 이전(P)
다음(N) >
마침(F)
취소

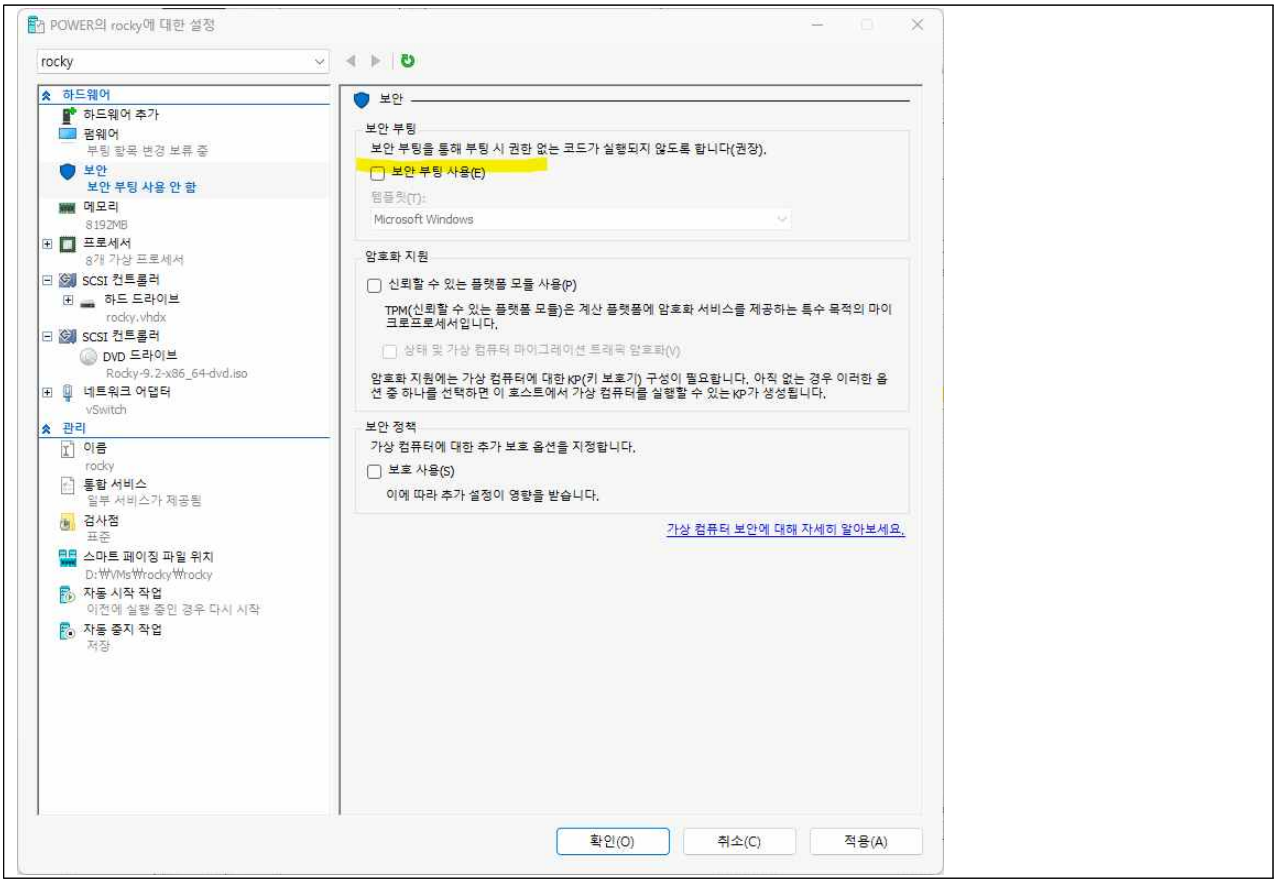

새 가상 컴퓨터 마법사
새 가상 컴퓨터 마법사 완료
시작하기 전
이름 및 위치 지정
세대 지정
메모리 할당
네트워킹 구성
가상 하드 디스크 연결
설치 옵션
요약
새 가상 컴퓨터 마법사를 완료했습니다. 다음과 같은 가상 컴퓨터가 만들어집니다.
설명:
이름: rocky
세대: 2세대
메모리: 8192 MB
네트워크: vSwitch
하드 디스크: D:₩VMs₩rocky₩rocky₩Virtual Hard Disks₩rocky.vhdx (VHDX, 동적 확장)
운영 체제: 나중에 설치
가상 컴퓨터를 만들고 마법사를 닫으려면 [마침]을 클릭하십시오.
< 이전(P)
다음(N) >
마침(F)
취소

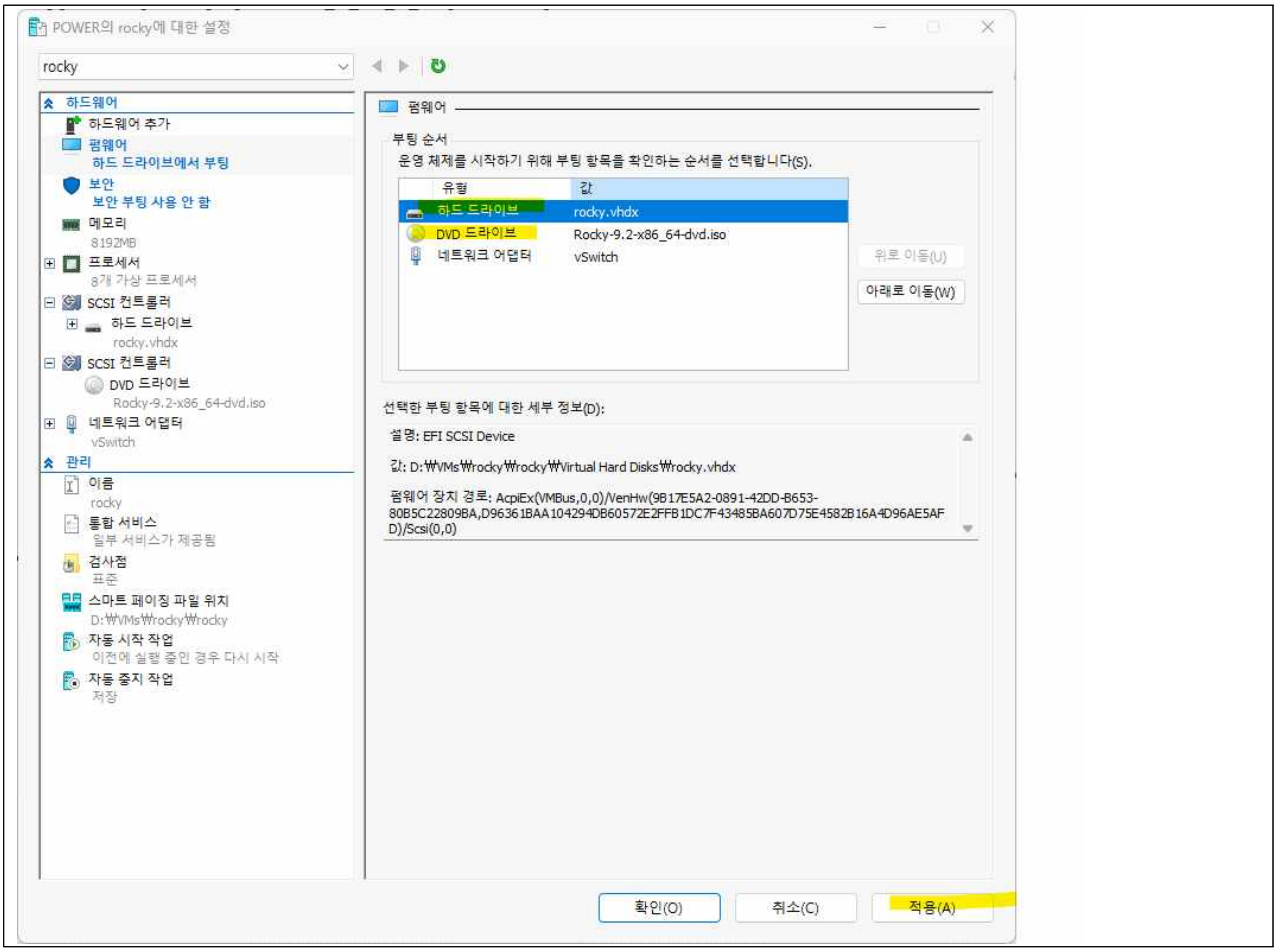

POWER의 rocky에 대한 설정
rocky
하드웨어
하드웨어 추가
펌웨어
네트워크 어댑터에서 부팅
보안
보안 부팅 사용
메모리
8192MB
프로세서
8개 가상 프로세서
SCSI 컨트롤러
하드 드라이브
rocky.vhdx
네트워크 어댑터
vSwitch
관리
이름
rocky
통합 서비스
일부 서비스가 제공됨
검사점
표준
스마트 페이징 파일 위치
D:₩VMs₩rocky₩rocky
자동 시작 작업
이전에 실행 중인 경우 다시 시작
자동 중지 작업
저장
하드웨어 추가
이 설정을 사용하여 가상 컴퓨터에 장치를 추가할 수 있습니다.
추가할 장치를 선택하고 [추가] 단추를 클릭합니다(S).
SCSI 컨트롤러
네트워크 어댑터
파이버 채널 어댑터
추가(D)
SCSI 컨트롤러를 추가한 후 여기에 가상 하드 디스크를 연결하여 가상 컴퓨터에서 사용할 수 있는 저장소를 늘릴 수 있습니다.
확인(O)
취소(C)
적용(A)


POWER의 rocky에 대한 설정
rocky
하드웨어
하드웨어 추가
펌웨어
네트워크 어댑터에서 부팅
보안
보안 부팅 사용
메모리
8192MB
프로세서
8개 가상 프로세서
SCSI 컨트롤러
하드 드라이브
rocky.vhdx
SCSI 컨트롤러
네트워크 어댑터
vSwitch
관리
이름
rocky
통합 서비스
일부 서비스가 제공됨
검사점
표준
스마트 페이징 파일 위치
D:₩VMs₩rocky₩rocky
자동 시작 작업
이전에 실행 중인 경우 다시 시작
자동 중지 작업
저장
SCSI 컨트롤러
SCSI 컨트롤러에 하드 드라이브와 CD/DVD 드라이브를 추가하거나 가상 컴퓨터에서 SCSI 컨트롤러를 제거할 수 있습니다.
컨트롤러에 연결할 드라이브 유형을 선택한 후 [추가]를 클릭합니다(S).
하드 드라이브
DVD 드라이브
공유 드라이브
추가(D)
DVD 드라이브를 컨트롤러에 연결한 후 CD 이미지(.iso) 파일을 사용하도록 구성할 수 있습니다.
이 가상 컴퓨터에서 SCSI 컨트롤러를 제거하려면 [제거]를 클릭합니다. 이 컨트롤러에 연결된 모든 가상 하드 디스크가 제거되지만 삭제되지는 않습니다.
제거(R)
확인(O)
취소(C)
적용(A)

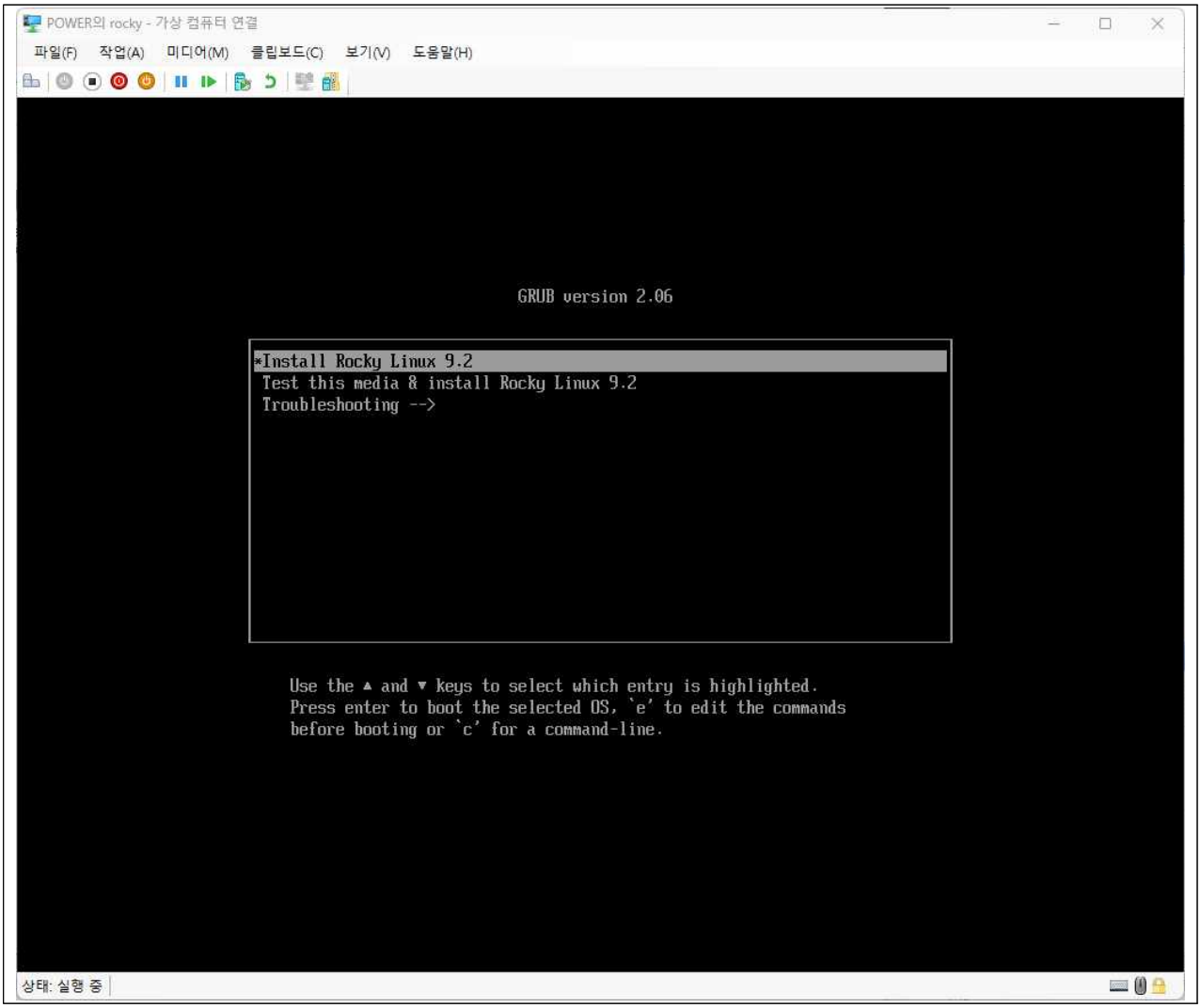

POWER의 rocky에 대한 설정
rocky
하드웨어
하드웨어 추가
펌웨어
부팅 항목 변경 보류 중
보안
보안 부팅 사용
메모리
8192MB
프로세서
8개 가상 프로세서
SCSI 컨트롤러
하드 드라이브
rocky.vhdx
SCSI 컨트롤러
DVD 드라이브
<파일>
네트워크 어댑터
vSwitch
관리
이름
rocky
통합 서비스
일부 서비스가 제공됨
검사점
표준
스마트 페이징 파일 위치
D:₩VMs₩rocky₩rocky
자동 시작 작업
이전에 실행 중인 경우 다시 시작
자동 중지 작업
저장
DVD 드라이브
CD/DVD 드라이브를 연결할 컨트롤러와 컨트롤러 위치를 선택하십시오.
컨트롤러(T):
SCSI 컨트롤러
위치(L):
0(사용 중)
미디어
가상 CD/DVD 드라이브에서 사용할 미디어를 지정하십시오.
없음(N)
이미지 파일(I):
찾아보기(B)...
이 가상 컴퓨터에서 가상 CD/DVD 드라이브를 제거하려면 [제거]를 클릭하십시오.
제거(R)
확인(O)
취소(C)
적용(A)

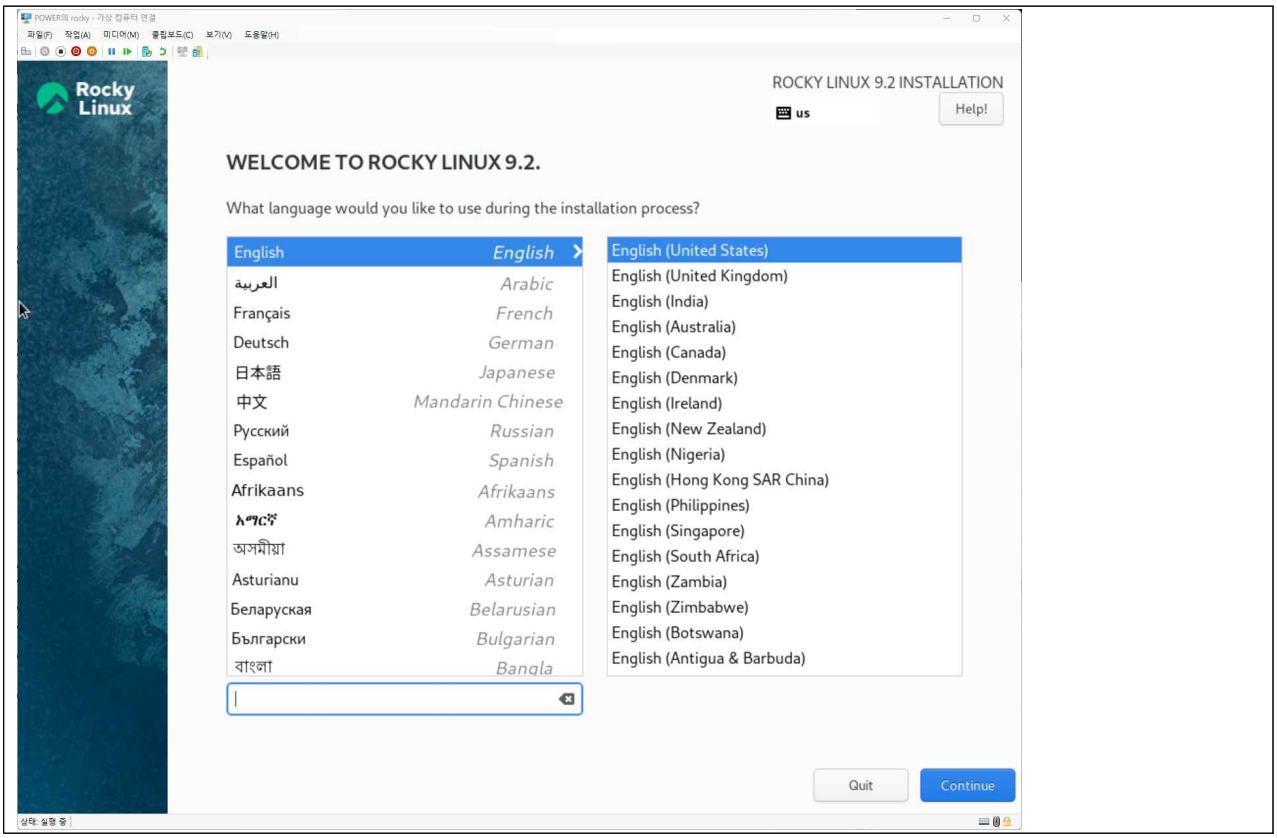

POWER의 rocky에 대한 설정
rocky
하드웨어
하드웨어 추가
펌웨어
부팅 항목 변경 보류 중
보안
보안 부팅 사용
메모리
8192MB
프로세서
8개 가상 프로세서
SCSI 컨트롤러
하드 드라이브
rocky.vhdx
SCSI 컨트롤러
DVD 드라이브
Rocky-9.2-x86_64-dvd.iso
네트워크 어댑터
vSwitch
관리
이름
rocky
통합 서비스
일부 서비스가 제공됨
검사점
표준
스마트 페이징 파일 위치
D:₩VMs₩rocky₩rocky
자동 시작 작업
이전에 실행 중인 경우 다시 시작
자동 중지 작업
저장
DVD 드라이브
CD/DVD 드라이브를 연결할 컨트롤러와 컨트롤러 위치를 선택하십시오.
컨트롤러(T):
SCSI 컨트롤러
위치(L):
0(사용 중)
미디어
가상 CD/DVD 드라이브에서 사용할 미디어를 지정하십시오.
없음(N)
이미지 파일(I):
C:₩Users₩power₩Downloads₩Rocky-9.2-x86_64-dvd.iso
찾아보기(B)...
이 가상 컴퓨터에서 가상 CD/DVD 드라이브를 제거하려면 [제거]를 클릭하십시오.
제거(R)
확인(O)
취소(C)
적용(A)



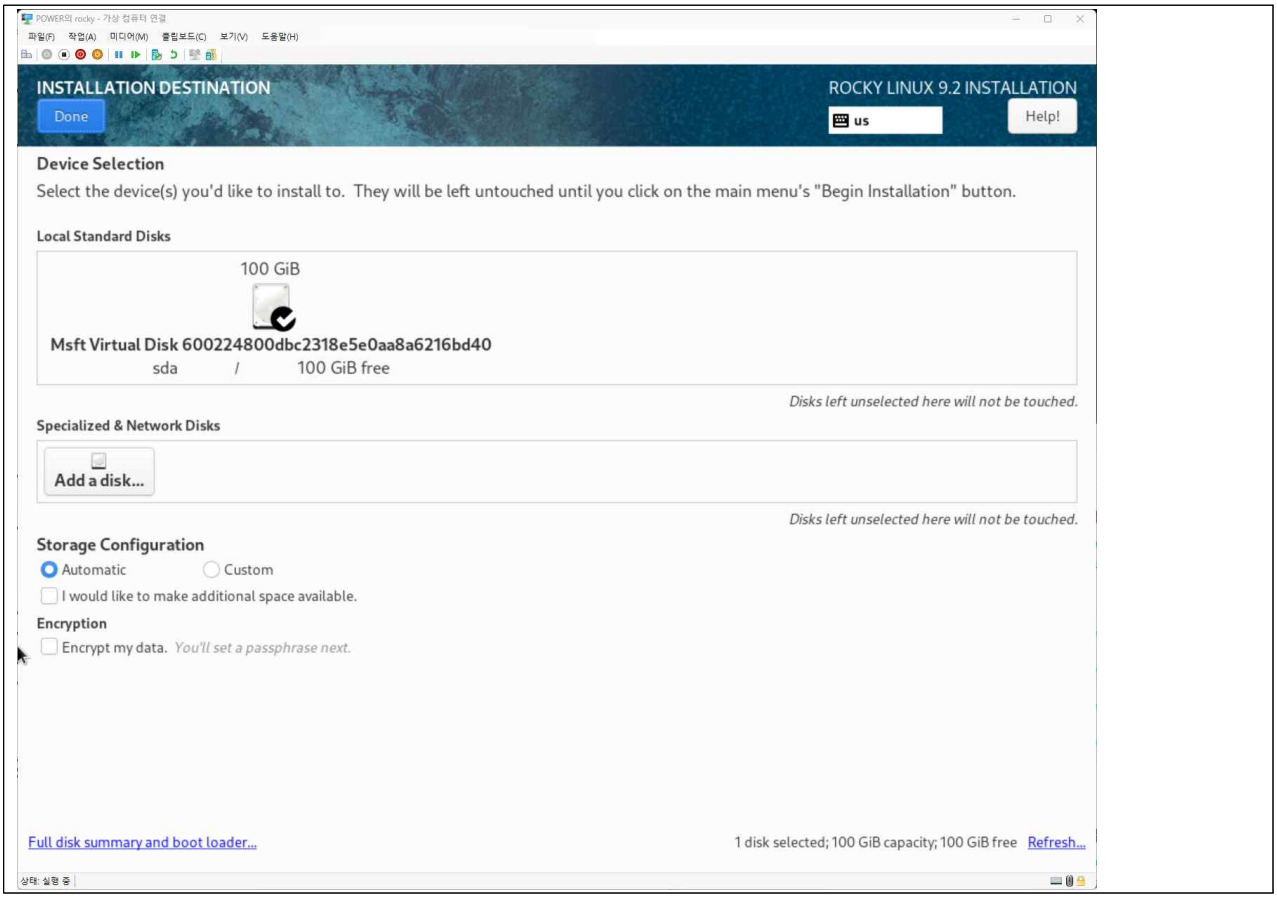


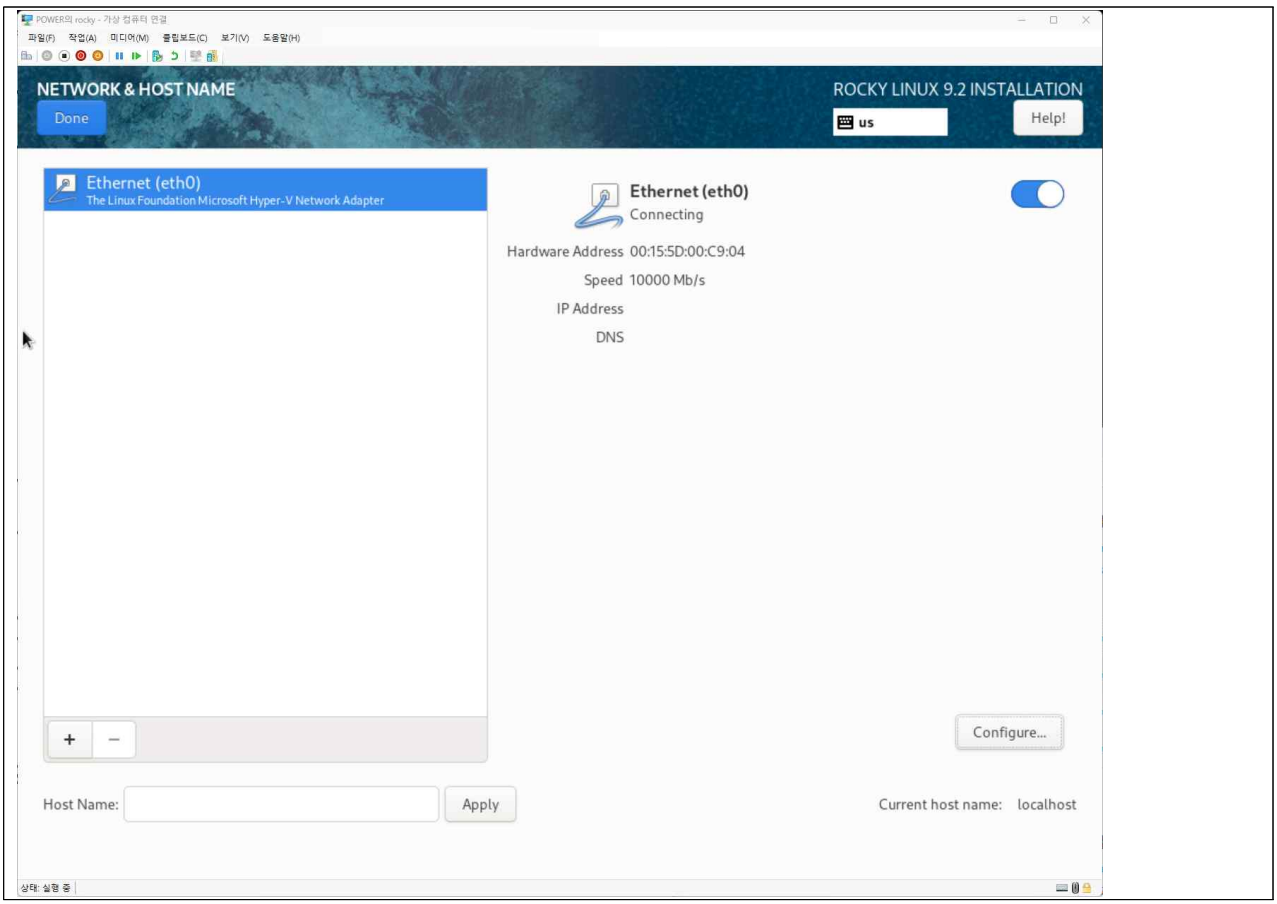
요기까지 구성이 되면 설치에 필요한 기본은 완료

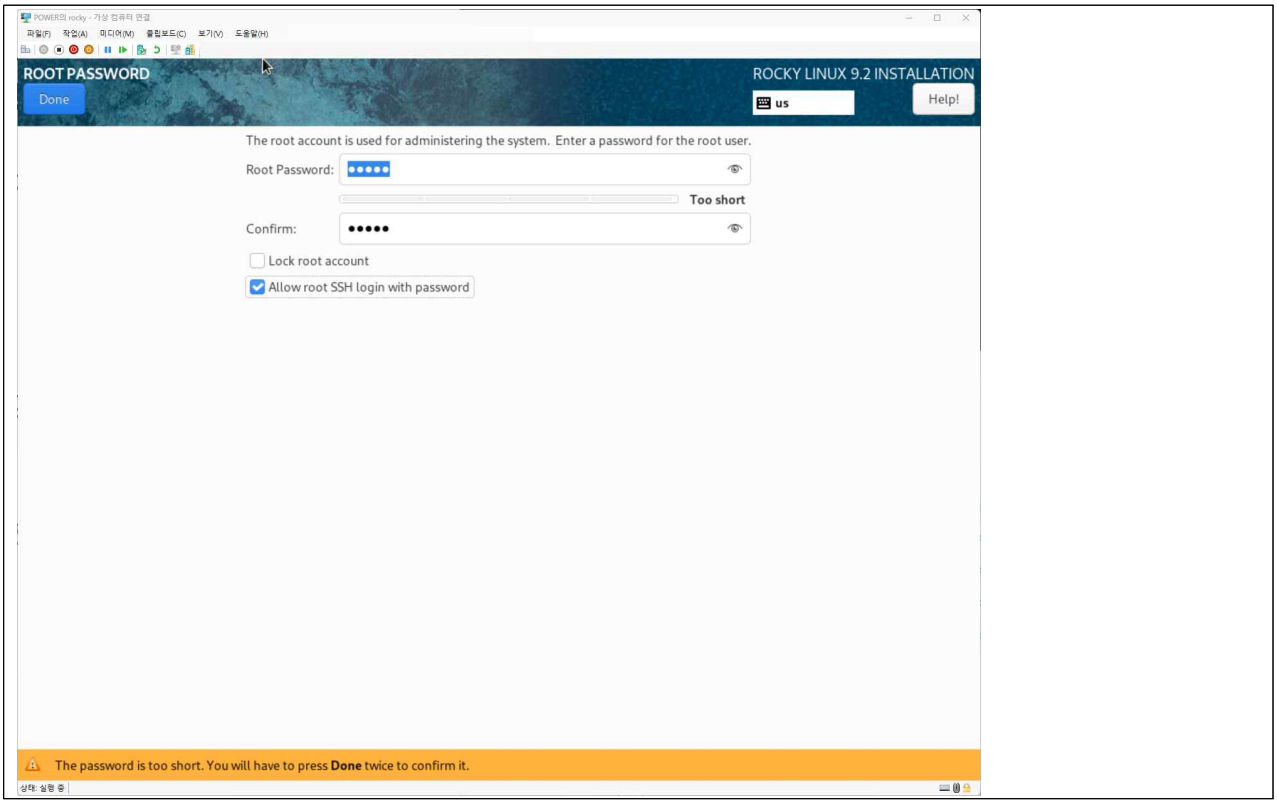

POWER의 rocky - 가상 컴퓨터 연결
파일(F) 작업(A) 미디어(M) 클립보드(C) 보기(V) 도움말(H)
GRUB version 2.06
*Install Rocky Linux 9.2
Test this media & install Rocky Linux 9.2
Troubleshooting -->
Use the ▲ and ▼ keys to select which entry is highlighted.
Press enter to boot the selected OS, `e' to edit the commands
before booting or `c' for a command-line.
상태: 실행 중

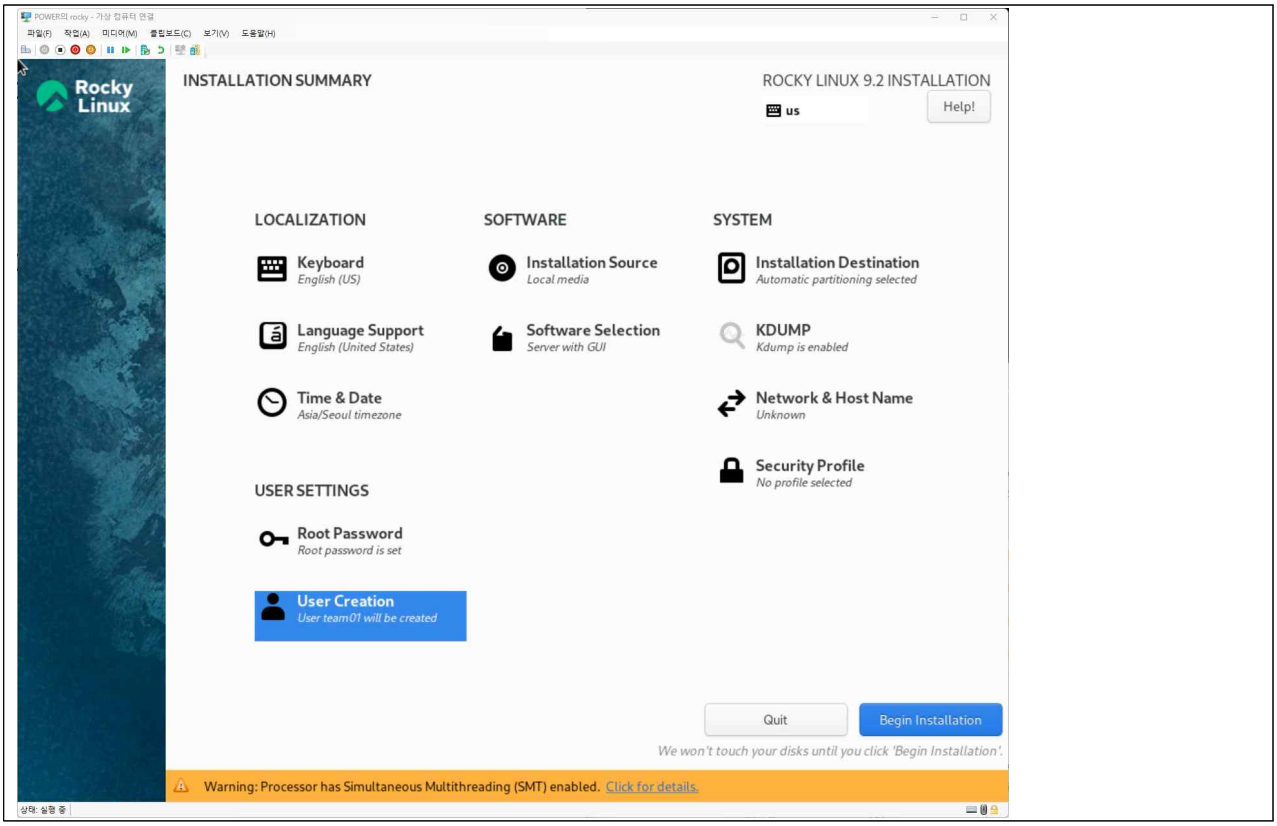

Rocky Linux
ROCKY LINUX 9.2 INSTALLATION
us
Help!
WELCOME TO ROCKY LINUX 9.2.
What language would you like to use during the installation process?
English English
العربية Arabic
Français French
Deutsch German
日本語 Japanese
中文 Mandarin Chinese
Русский Russian
Español Spanish
Afrikaans Afrikaans
አማርኛ Amharic
অসমীয়া Assamese
Asturianu Asturian
Беларуская Belarusian
Български Bulgarian
বাংলা Bangla
English (United States)
English (United Kingdom)
English (India)
English (Australia)
English (Canada)
English (Denmark)
English (Ireland)
English (New Zealand)
English (Nigeria)
English (Hong Kong SAR China)
English (Philippines)
English (Singapore)
English (South Africa)
English (Zambia)
English (Zimbabwe)
English (Botswana)
English (Antigua & Barbuda)
Quit
Continue

영어를 선택


INSTALLATION DESTINATION
Done
ROCKY LINUX 9.2 INSTALLATION
us
Help!
Device Selection
Select the device(s) you'd like to install to. They will be left untouched until you click on the main menu's "Begin Installation" button.
Local Standard Disks
100 GiB
Msft Virtual Disk 600224800dbc2318e5e0aa8a6216bd40
sda / 100 GiB free
Disks left unselected here will not be touched.
Specialized & Network Disks
Add a disk...
Disks left unselected here will not be touched.
Storage Configuration
Automatic
Custom
I would like to make additional space available.
Encryption
Encrypt my data. You'll set a passphrase next.
Full disk summary and boot loader...
1 disk selected; 100 GiB capacity; 100 GiB free
Refresh...


NETWORK & HOST NAME
Done
ROCKY LINUX 9.2 INSTALLATION
us
Help!
Ethernet (eth0)
The Linux Foundation Microsoft Hyper-V Network Adapter
Ethernet (eth0)
Connecting
Hardware Address 00:15:5D:00:C9:04
Speed 10000 Mb/s
IP Address
DNS
Configure...
Host Name:
Apply
Current host name: localhost





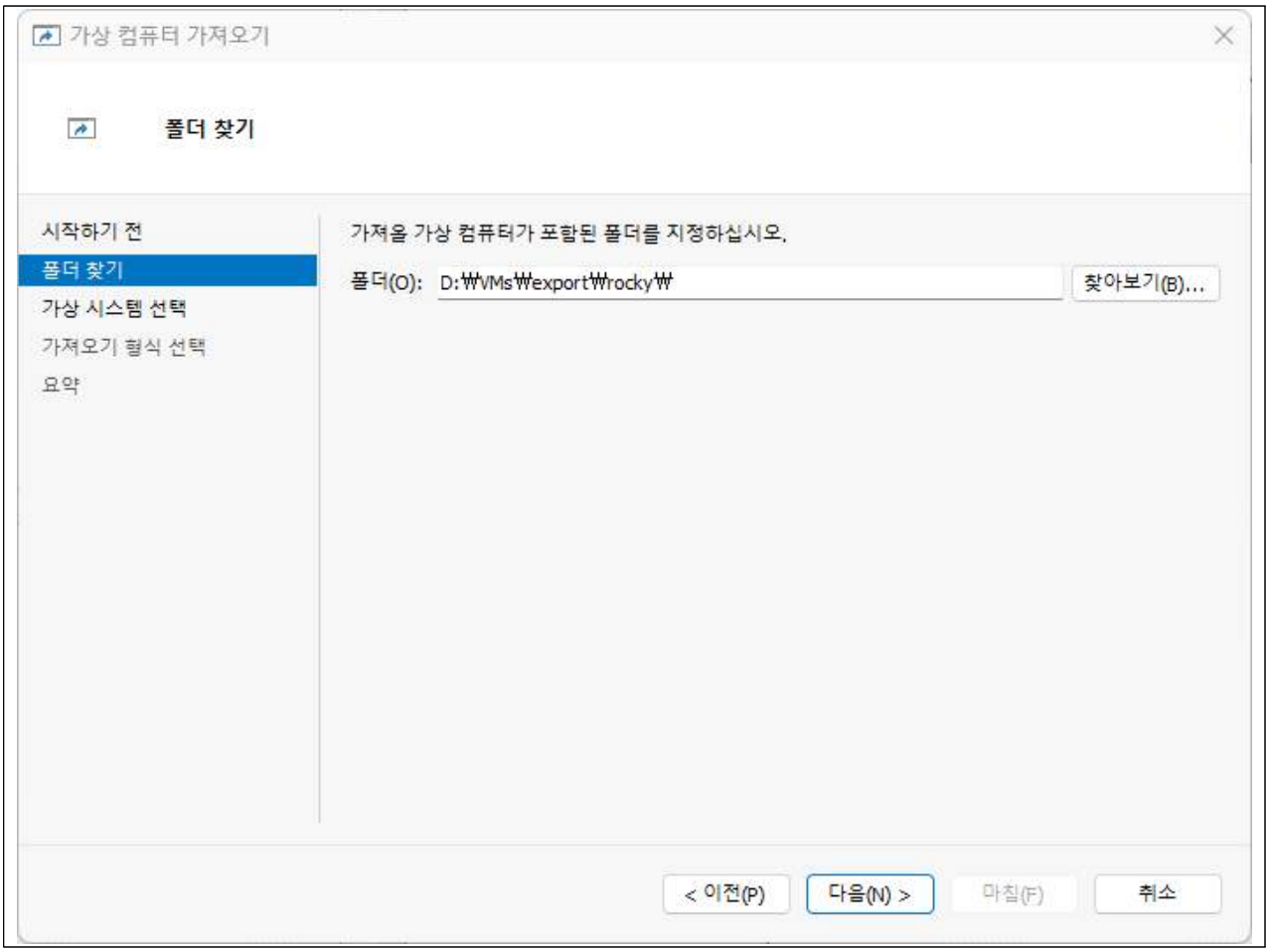
설치가 완료 되면 재부팅을 먼저 해주시면 됩니다.

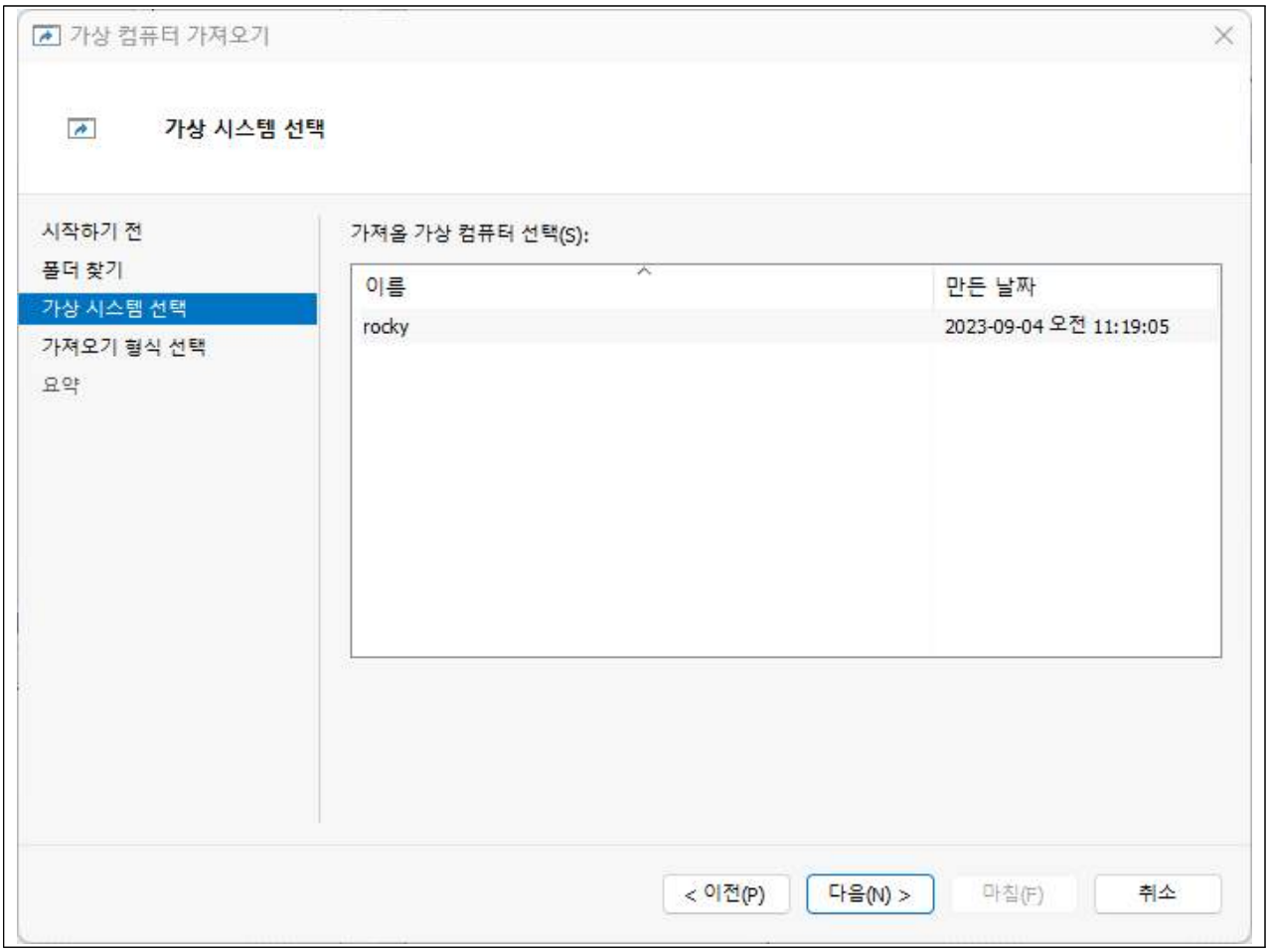
사용해야 하는 머신 두 대
rocky를 설치가 완료 > 내보내기 > 가져오기 rocky2

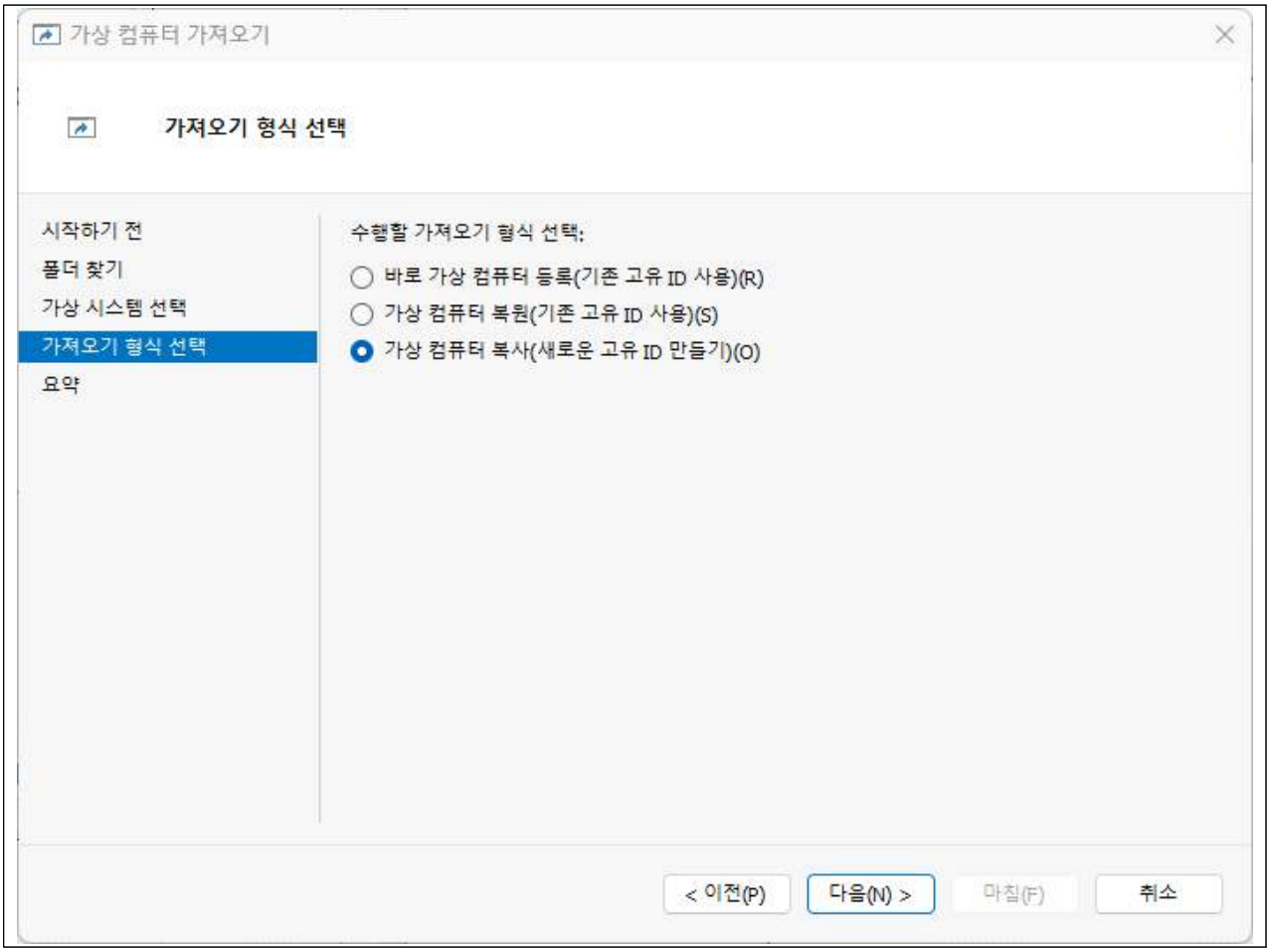
rocky1
rocky2

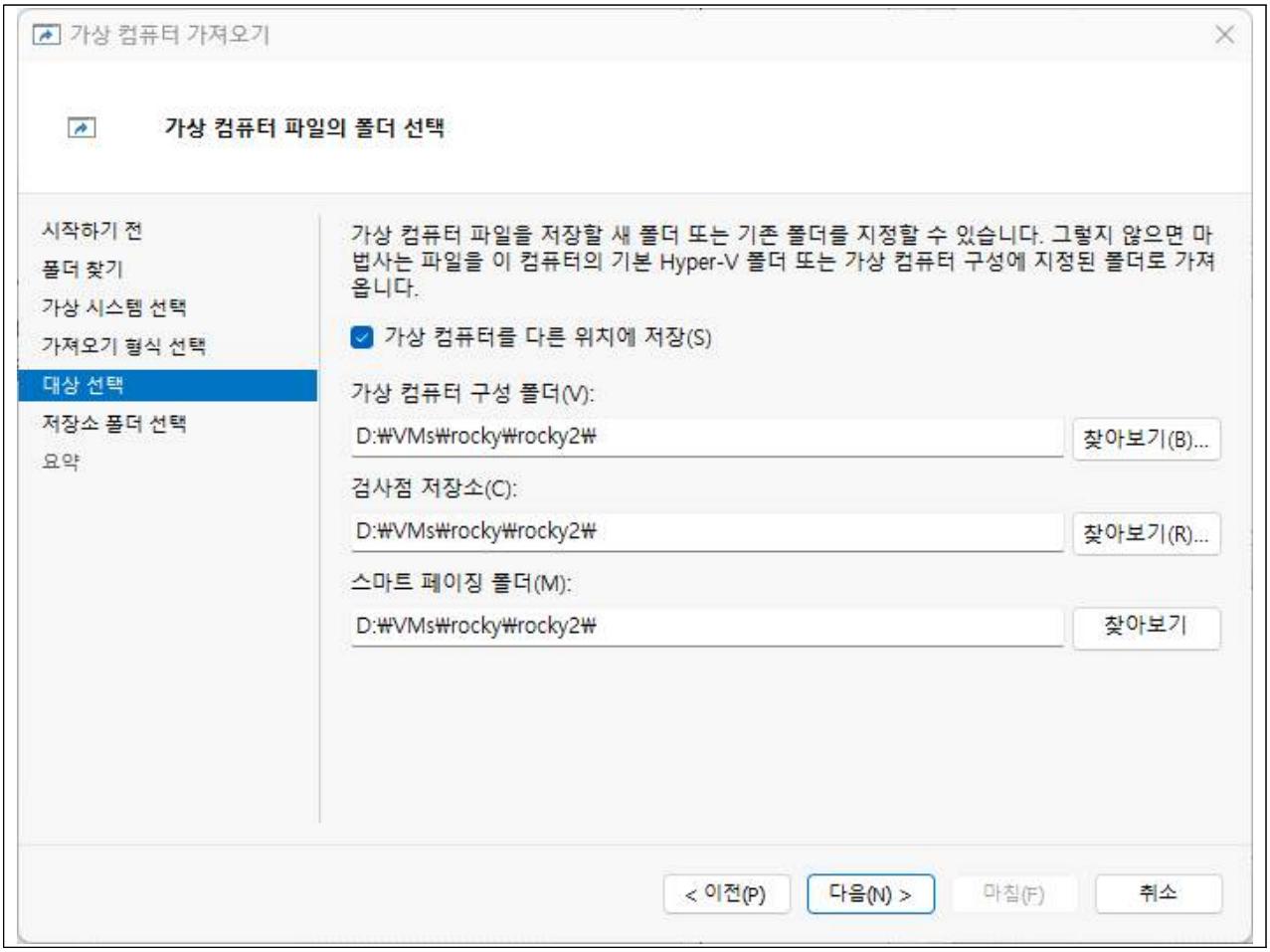
가상머신 > 내보내기 > 원하는 위치에 디렉토리를 만들어 저장

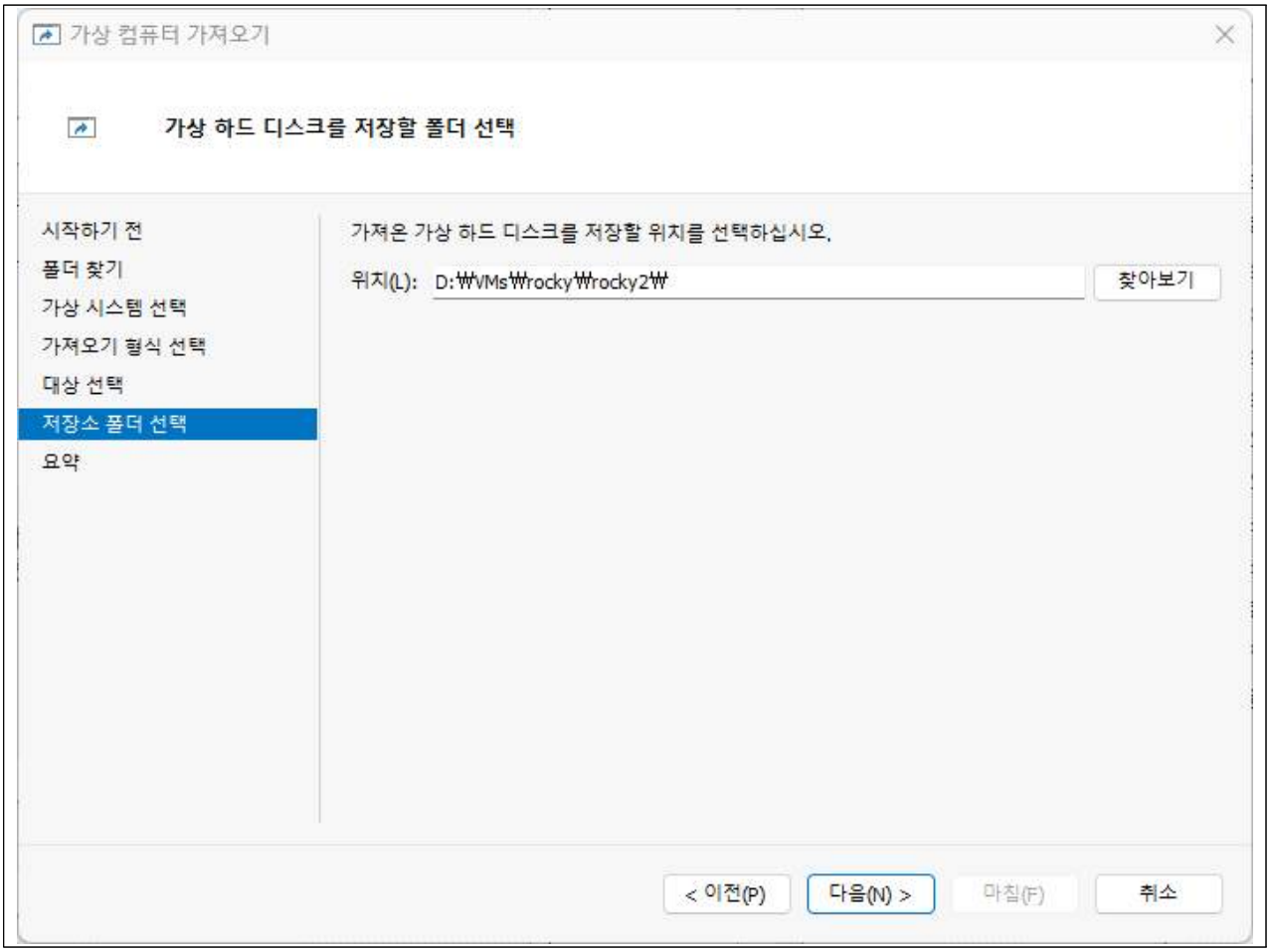
가상컴퓨터 가져오기

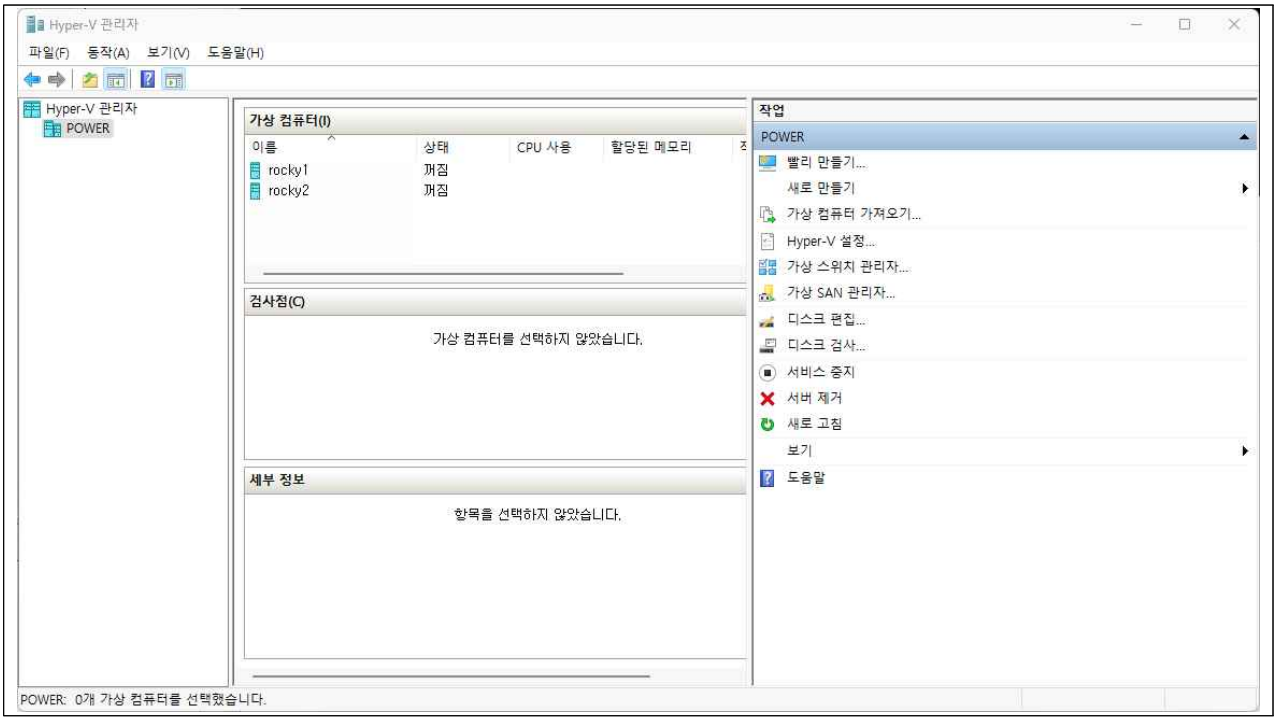

가상 컴퓨터 가져오기
폴더 찾기
시작하기 전
폴더 찾기
가상 시스템 선택
가져오기 형식 선택
요약
가져올 가상 컴퓨터가 포함된 폴더를 지정하십시오.
폴더(O): D:₩VMs₩export₩rocky₩
찾아보기(B)...
< 이전(P)
다음(N) >
마침(F)
취소


가상 컴퓨터 가져오기
가상 시스템 선택
시작하기 전
폴더 찾기
가상 시스템 선택
가져오기 형식 선택
요약
가져올 가상 컴퓨터 선택(S):
이름
만든 날짜
rocky
2023-09-04 오전 11:19:05
< 이전(P)
다음(N) >
마침(F)
취소


가상 컴퓨터 가져오기
가져오기 형식 선택
시작하기 전
폴더 찾기
가상 시스템 선택
가져오기 형식 선택
요약
수행할 가져오기 형식 선택:
바로 가상 컴퓨터 등록(기존 고유 ID 사용)(R)
가상 컴퓨터 복원(기존 고유 ID 사용)(S)
가상 컴퓨터 복사(새로운 고유 ID 만들기)(O)
< 이전(P)
다음(N) >
마침(F)
취소

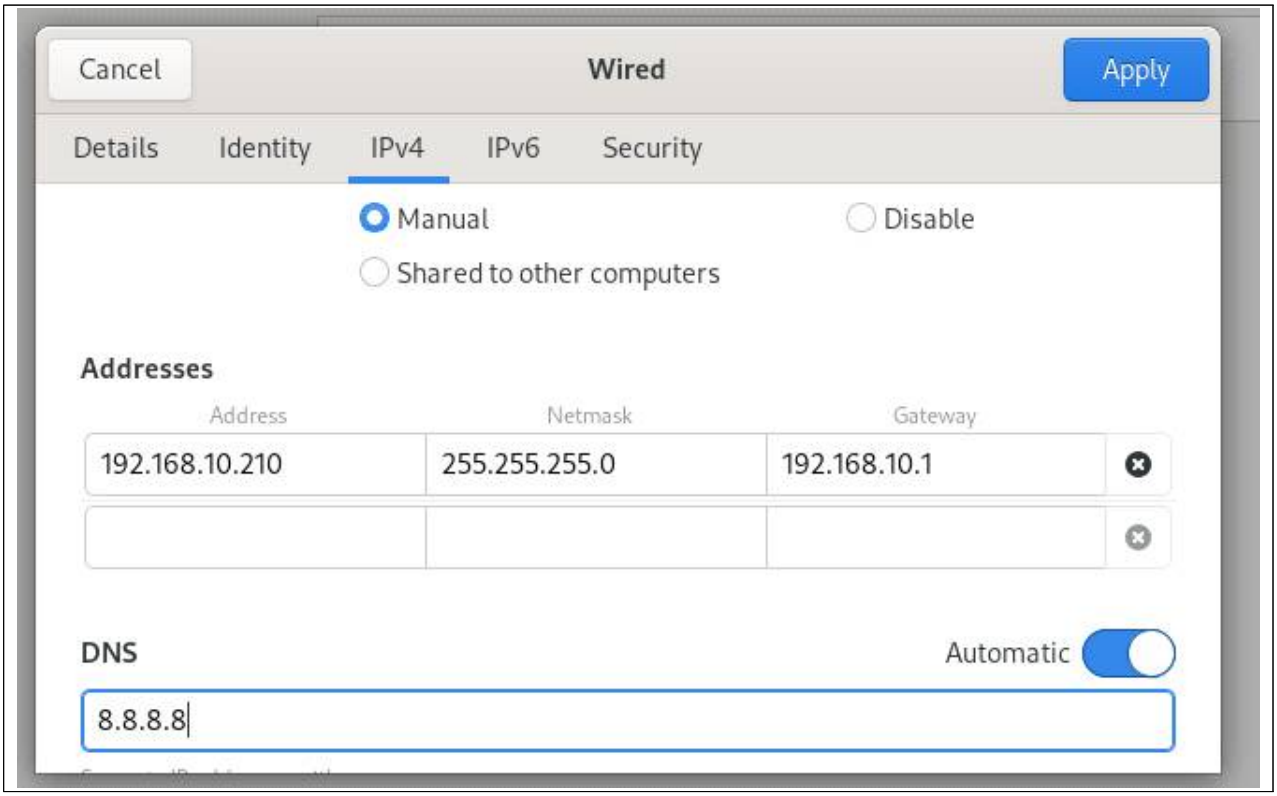

가상 컴퓨터 가져오기
가상 컴퓨터 파일의 폴더 선택
시작하기 전
폴더 찾기
가상 시스템 선택
가져오기 형식 선택
대상 선택
저장소 폴더 선택
요약
가상 컴퓨터 파일을 저장할 새 폴더 또는 기존 폴더를 지정할 수 있습니다. 그렇지 않으면 마법사는 파일을 이 컴퓨터의 기본 Hyper-V 폴더 또는 가상 컴퓨터 구성에 지정된 폴더로 가져옵니다.
가상 컴퓨터를 다른 위치에 저장(S)
가상 컴퓨터 구성 폴더(V):
D:₩VMs₩rocky₩rocky2₩
찾아보기(B)...
검사점 저장소(C):
D:₩VMs₩rocky₩rocky2₩
찾아보기(R)...
스마트 페이징 폴더(M):
D:₩VMs₩rocky₩rocky2₩
찾아보기
< 이전(P)
다음(N) >
마침(F)
취소





네트워크 구성을 완료하고 확인

activity > 전체 어플리케이션 : setting
network



네트워크 장치 비활성화 > 활성화

터미널 오픈:
ip a
ping www.google.com

rocky1
IP 192.168.10.210
SUBNET 255.255.255.0
GW 192.168.10.1
DNS 8.8.8.8

rocky2
IP 192.168.10.220
SUBNET 255.255.255.0
GW 192.168.10.1
DNS 8.8.8.8

네트워크에 대한 테스트가 완료 되었다면 시스템 종료
poweroff

putty / cmd
원격 접속을 시도하시고 정상 접근 까지 완료

검사점 생성

원격 접속과 접속 환경을 위한 설정

1. putty 무료
2. cmd windows 내장
3. termius << 무료 / 유료 / 회원가입

윈도우와 비교하는 리눅스

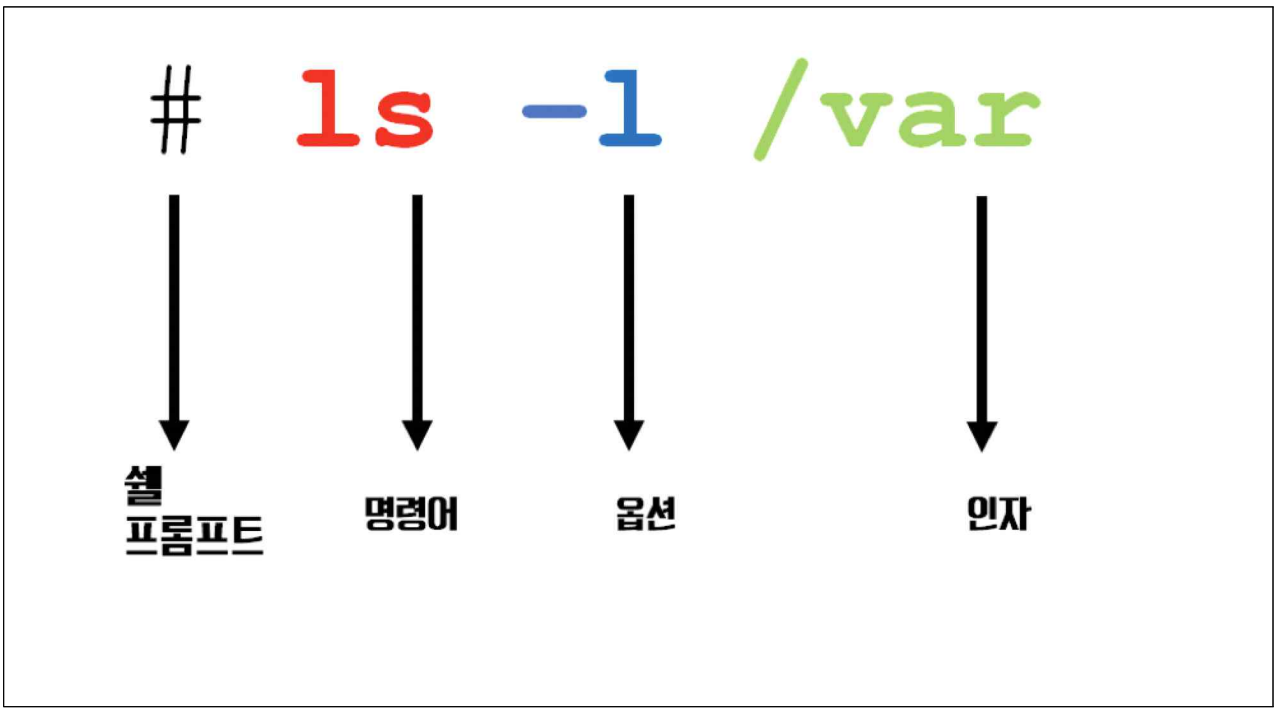
기초 명령어 가이드

리눅스 기본정보

단원목표

- 리눅스 명령어 형식
- 도움말
- 시스템의 기본적인 정보를 확인하는 명령어

리눅스 명령어 형식



Linux CMD Format

리눅스에서는 명령어 입력으로 작업을 수행하는 TEXT 모드(CLI)를 주로 사용한다. 명령어 사용법을 익혀야 하며, 명령어를 입력하면 쉘에서 명령어를 해석하여 작업을 수행한다. 명령어에는 옵션(Options)과 인자(Arguments)가 있는데, 옵션은 명령어의 동작을 다르게 하는데 사용하고, 인자는 명령어의 인자나 옵션의 인자로 사용될 수 있다.

[명령어 형식]

```
# Command [options] [Arguments]
```

각 문자의 구분은 white space(공백문자)로 구분한다.

```
[root@serverX ~]#  ls    -l   /var
        |          |      |    |
       쉘프롬프트  명령어 옵션  인자
```

명령어를 사용하는 다양한 형식은 다음과 같다.

```
- "# Command" 사용하는 경우
[EX] # ls
- "# Command Options" 사용하는 경우
[EX] # ls -l
- "# Command Arguments" 사용하는 경우
[EX] # ls /var
- "# Command options Arguments" 사용하는 경우
[EX] # ls -l /var
- 한 개의 명령어 라인에 여러 명령어 사용 하는 경우
[EX] # cd /test ; ls -l
```

(예) "명령어" 형식을 사용하는 경우

```
# ls
# cal
# uname
```

(예) "명령어 + 옵션" 형식을 사용하는 경우

```
# uname -sr
# uname -a
# ls -l
```

(예) "명령어 + 인자" 형식을 사용하는 경우

```
# cd /etc/sysconfig/network-scripts
# ls /var
# cal 2002
```

(예) "명령어 + 옵션 + 인자" 형식을 사용하는 경우

```
# ls -l /var
# find / -name core
```

[참고] 옵션에 대한 설명

```
명령어의 옵션은 자리를 바꾸거나 혹은 합쳐서 사용해도 같은 의미로 동작한다.
# ls -a -l -F
# ls -al -F
# ls -alF
# ls -Fla

< 명령어의 옵션을 약자로 사용하지 않을 경우 >
명령어의 옵션을 약자 형태로 쓰지 않기 위해서는 "--(double dash)"을 붙인다.
# ls --all    (# ls -a)
```

[참고 & 주의] 옵션의 인자가 존재하는 경우의 예

```
예) # find / -name core -type f
EX : # find . -name file1 -type f

# cmd -i -f arg1(O)
# cmd -f arg1 -i(O)
# cmd -if arg1(O)
# cmd -fi arg1(X)

특정 값이 인자에 포함되어야 하는 경우는 인자 뒤에 값이 들어가야 한다.
옵션의 순서는 변경이 가능하지만 옵션과 같이 지정되는 인자의 경우는 옵션뒤에 와야하는 것을 주의한다.
```

도움말
- man 명령어

```
# man ls
# man -k calendar
# whatis passwd
# man -s 5 passwd
# man ls | col -b > ls.txt
# info ls
# which ls
# whatis ls
```

[명령어 형식]

```
# man ls /* 명령어나 파일의 이름으로 검색하는 경우 */
# man -k calendar  /* keyword로 검색하는 경우 (색인파일 생성 필요)*/
# whatis passwd
# man -s 1 passwd/* section번호로 검색하는 경우 */
```

[EX1] 명령어에 대한 man 페이지 확인

```
(간략하게 정보 확인) # ls --help   (# CMD --help)
(자세하게 정보 확인) # man ls      (# man CMD)
```

```
# man ls
빠져나오는경우 q
```

[참고] man 페이지 내에서 페이지 제어

```
<Enter> 하나의 라인씩 밀리면서 출력한다.
<Space> <--> <CTRL + B> 한 페이지씩 밀리면서 출력한다.
      CTRL + B로 이전 페이지로 돌아간다.
/-a     <--> n, N    (n: next) 특정 단어를 검색하여 찾는다. 두 개 이상 존재할경우는 n으로 다음 단
    어를 찾는다
: 자주 사용되는 기능
/OPTIONS
/EXAMPLES
/Examplesls
q
h
```

[EX2] 명령어를 알지 못하는 경우

```
# man -k calendar
calendar: nothing appropriate  -> 에러메시지가 나오면
```

처음 -k 옵션을 실행하게 된다면 색인에 대해 생성이 되어있지 않기 때문에 에러 메시지를 출력한다

[참고] CENTOS7은 에러메시지가 나오지 않는다.(100%는 아니다.)

```
# makewhatis
-> 약간 시간이 걸린다.(약 1분 정도) [ 명령어에 대한 색인을 생성한다 ] << centos 5,6 버전에 해당
```

[참고] 7버전 이상에서 색인 데이터베이스 생성

```
처음 서버를 설치한 경우에 스케쥴러 시간이 되지 않으면 아직 색인이 생성되어 있지 않다.
그렇기 때문에 수동으로 색인을 생성하는 작업을 해준다.
# cd /etc/cron.daily/
# ls
grotate  man-db.cron  mlocate
# ./man-db.cron
#
```

[참고] 8버전 이상에서 색인 데이터베이스 생성

```
man-db.cron 파일이 없기 때문에 레포지토리에서 설치 이후 진행한다.
# dnf -y install man-db-cron
# cd /etc/cron.daily
# ls
grotate  man-db.cron
# ./man-db.cron
>> 색인 생성중
#
```

[참고] 9버전 이상에서 색인 데이터베이스 생성

```
# mandb
```

```
# man -k calendar
cal (1)              - display a calendar
cal (1p)             - print a calendar
difftime (3p)        - compute the difference ...
```

[EX3] 섹션 별로 검색 할 경우

```
# whatis passwd
passwd (5)           - 패스워드 파일
sslpasswd (1ssl)     - compute password hashes
passwd (1)           - update user's authentic...
```

```
# man -s 1 passwd /* passwd 명령어 매뉴얼 페이지 */
# man -s 5 passwd /* /etc/passwd 파일 매뉴얼 페이지 */
```

섹션번호로 보는 방법이 필요한 이유는 # passwd CMD형식으로 검색하게 된다면 1번 섹션에 대해서만 출력된다. 2~ 이후의 섹션에 대하여 검색하기 위해서는 -s 옵션을 사용하는 섹션으로 검색하는 방법에 대하여 알고있어야한다.

```
# man passwd
# man -s 1 passwd
# man -s 5 passwd
```

매뉴얼 페이지 각 섹션별 정보

```
매뉴얼 페이지 각 섹션별 정보
  1     누구나 실행할 수 있는 사용자 명령 (User Commands)
  2     시스템 호출 (System Calls), 즉 커널에서 제공하는 함수
  3     C 라이브러리 함수 (C Library functions)
  4      디바이스와 네트워크 인터페이스 (Devices and Network Interfaces), 즉 /dev 디렉토리에
있는 특별한 파일들
  5     파일 포맷 (File Formats), 예를 들어 /etc/passwd
  6     게임과 데모 (Games and Demos)
  7     환경, 테이블, 매크로 (Environments, Tables, and Macros) 등의 잡다한 여러 가지들
  8     시스템 관리자만 실행할 수 있는 시스템 관리용 명령들 (Maintenance Commands)
  9     커널 루틴에 관련된 문서들의 리눅스만의 독자적인 저장소
  n     새로운 문서들의 저장소. 보다 적당한 장소로 이동된다.
  o     오래된 문서들의 저장소. 유예 기간으로 보관되고 있는 것들이다.
  l     특정 시스템에 대한 로컬 문서들
```

[EX4] 추가적인 정보 확인 명령어

```
# info ls
man 페이지에서 간략하게 나오고 info에서 확인하라는 메시지가 하단에 나타나는 경우에 info 명령
어를 확인한다.
# pinfo ls
map 형식으로 나타내주며 색깔표시가 보기좋게 되어있다.
info로 확인해야 할 일이 있다면 pinfo를 사용하기 바란다.

# apropos ls     (# man -k calendar)
# man -k ls

# whatis ls    명령어의 메뉴얼 파일 섹션 확인
ls (1)              - list directory contents
ls (1p)             - list directory contents
# whereis ls   명령어의 바이너리 파일 경로, 메뉴얼 파일 실제 경로
ls: /usr/bin/ls /usr/share/man/man1/ls.1.gz /usr/share/man/man1p/ls.1p.gz
# which ls     명령어 실제 경로
alias ls='ls --color=auto'
        /usr/bin/ls
# ls --help
```

```
# whatis ls 명령어의 모든 섹션 확인
ls (1)              - 경로의 내용을 나...
ls (1p)             - list directory contents
# whereis ls 메뉴얼 페이지의 경로 확인
ls: /usr/bin/ls /usr/share/man/man1/ls.1.gz /usr/share/man/man1p/ls.1p.gz
# which ls 명령어의 실제 경로 확인
alias ls='ls --color=auto'
        /usr/bin/ls
# ls --help 명령어의 간단한 사용 형식 확인
```

passwd 명령어
암호 변경 권한 차이

- root 사용자 : 모든 사용자의 암호를 변경 가능
- 일반 사용자 : 자신의 암호만 변경 가능

암호 변경 시 이전 암호 입력 여부

- root 사용자 : 암호 변경시 이전 암호 물어보지 않고 변경 가능
- 일반 사용자 : 이전 암호를 반드시 맞추어야만 새로운 암호 입력 가능

[명령어 형식]

```
# passwd
# passwd fedora
```

[EX1] 관리자가 자신의 암호 변경하는 경우

[TERM1] root 사용자 터미널

```
# passwd
Changing password for user root. <변경하는 계정의 ID가 출력된다>
New UNIX password: (fedora)
BAD PASSWORD: it is based on a dictionary word
Retype new UNIX password: (fedora)
passwd: all authentication tokens updated successfully. <연속으로 동일한 비밀번호를 입력시 변경>
```

```
# passwd root
Changing password for user root.
New UNIX password: (soldesk)
BAD PASSWORD: it is based on a dictionary word
Retype new UNIX password: (soldesk)
passwd: all authentication tokens updated successfully.
```

```
# passwd fedora
Changing password for user fedora.
New UNIX password: (123user)
BAD PASSWORD: it is based on a dictionary word
Retype new UNIX password: (123user)
passwd: all authentication tokens updated successfully.
```

```
# passwd fedora
Changing password for user fedora.
New UNIX password: (123user)
BAD PASSWORD: it is based on a dictionary word
Retype new UNIX password: (123user)
passwd: all authentication tokens updated successfully.
```

[TERM2] fedora 사용자 터미널

```
# ssh fedora@localhost
The authenticity of host 'localhost (127.0.0.1)' can't be established.
RSA key fingerprint is 3b:25:5b:9a:ae:0f:ba:fc:85:66:73:d4:fc:e2:78:c6.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added 'localhost' (RSA) to the list of known hosts.
fedora@localhost's password: (123user)
$ id
uid=1001(fedora) gid=1001(fedora) groups=1001(fedora)
$ passwd
Changing password for user fedora.
Changing password for fedora
(current) UNIX password: (123user)    /* 이전 비밀번호 확인 */
New UNIX password: (P@ssw0rd##1.)         /* 새로운 비빌번호 입력 */
Retype new UNIX password: (P#ssw0rd##1.)  /* 새로운 비빌번호 동일하게 재 입력 */
passwd: all authentication tokens updated successfully.
$ exit
#
```

비밀번호를 변경하는 일 외에도 passwd 삭제, 사용자 계정에 대한 잠금, 해제 등 여러 기능이 있다.

(복원) 관리자가 사용자의 암호 변경

```
fedora 사용자의 암호를 fedora로 변경
# passwd fedora
-> 일반사용자는 아이디와 암호를 동일하게 설정할수 없다.
```

stdin옵션

```
# echo 'password' | passwd --stdin [USERNAME]
```

시스템의 기본적인 정보를 확인하는 명령어

- uname 명령어

uname -a
cat /etc/redhat-release

- date 명령어
date
date +%m%d

uname 명령어

운영체제에 맞는 애플리케이션(Application)을 설치하려면 운영체제의 종류와 버전에 대한 정보등 확인

할 수 있어야 한다. 또한 운영체제의 시간을 확인하고 서버와 동기화를 시키는 작업등을 할 수 있어야 한다.

uname 명령어를 사용하는 경우

- 운영체제의 종류와 버전 확인(OS + Version)
- 커널 버전 확인(Kernel Version)

[명령어 형식]

```
# uname [OPTIONS]
```

[명령어 옵션]

```
옵션 : 설  명
-a : 모든 정보
-s : 운영체제 종류 (예: Linux)
-r : 운영체제 릴리즈 (예: 2.6.18-164.11.1.el5)
-n : 호스트 이름 (예: linux200)
-i : 머신아키텍쳐 (예: i386)
-p : 프로세스 종류 (예: athlon)
```

[EX1] uname 명령어

```
# uname -a
Linux server1.example.com 4.18.0-147.el8.x86_64 #1 SMP Wed Dec 4 21:51:45 UTC 2019 x86_64
x86_64 x86_64 GNU/Linux
```

uname 명령어의 출력 결과 해석

```
Linux                              (-s) 커널 이름(kernel name)
server1.example.com                (-n) 호스트 이름(Hostname)
4.18.0-147.el8.x86_64             (-r) 커널 릴리즈
커널 버전 : 주버전.부버전.fix버전-몇개의업데이트가있었는가
#1 SMP Wed Aug 7 18:08:02 UTC 2019 (-v) 커널 버전
x86_64                             (-m) 머신 하드웨어 이름
x86_64                             (-p) CPU 종류
x86_64                             (-i) 하드웨어 구현 플랫폼
GNU/Linux                          (-o) 운영체제 종류
```

```
# uname -s  /* 커널 이름 */
# uname -n  /* 호스트 이름 */
# uname -r  /* 커널 릴리즈 */
# uname -sr /* 커널 이름과 릴리즈 */
# uname -v  /* 커널 버전 */
# uname -m  /* 머신 하드웨어 이름 */
# uname -p  /* CPU 종류 */
# uname -i  /* 하드웨어 구현 플랫폼 */
# uname -o  /* 운영체제 종류 */
# uname -nsr
```

[EX2] 설치된 리눅스 운영체제 종류 확인

```
# uname -a
Linux server1.example.com  4.18.0-147.el8.x86_64 #1 SMP Wed Aug 7 18:08:02 UTC 2019 x86_64
x86_64 x86_64 GNU/Linux
```

uname 대신에 시스템 정보를 확인하는 명령어

```
# hostnamectl
Static hostname: server1.example.com
        Icon name: computer-vm
          Chassis: vm □
       Machine ID: e6c3061aab7d4db1904499fdfb0c8b0d
          Boot ID: 35a2166869f547679dcd448834ae9096
  Virtualization: vmware
Operating System: Rocky Linux 9.0 (Blue Onyx)
     CPE OS Name: cpe:/o:rocky:rocky:9::baseos
          Kernel: Linux 5.14.0-70.17.1.el9_0.x86_64
    Architecture: x86-64
 Hardware Vendor: VMware, Inc.
  Hardware Model: VMware Virtual Platform
```

릴리즈 정보 확인하기

```
# ls /etc/*release
/etc/os-release  /etc/redhat-release  /etc/rocky-release  /etc/system-release

/etc/centos-release
/etc/os-release
/etc/redhat-release > /etc/centos-release의 링크파일
/etc/system-release
# cat /etc/redhat-release
Rocky Linux release 9.0 (Blue Onyx)
```

[EX3] 문서(Documentation) 사이트(운영체제 버전별 정보 확인)

```
https://docs.redhat.com/
http://www.centos.org/docs/
http://www.centos.org/docs/3/CentOS 3.X
http://www.centos.org/docs/4/CentOS 4.X
http://www.centos.org/docs/5/CentOS 5.X
centos6부터는 RHEL 레드햇 공식 웹사이트를 이용해야한다.
[참고]
centos는 RHEL을 전신으로 개발된 무료용버전이기 때문에 기술들이 레드햇에서 문서가 있는 경우가 많다.
두가지 도메인 주소를 모두 기억해두자.
```

리눅스 기본환경

- CLI / GUI 환경전환과 로그인 로그아웃
- 언어변경
- 제어문자
- 운영체제 종료와 재부팅
- 관리자 암호 변경

CLI / GUI 환경전환과 target name

- system runlevel - 시스템의 현재 상태를 나타내는 숫자.
- init telinit
- system runlevel type

```
0 - halt                        시스템종료
1 - single user mode            싱글유저모드(시스템복구, 관리자 비밀번호 변경)
2 - multiuser                   멀티유저가능(단 nfs 사용불가)
3 - full multiuser              멀티유저가능(모든 기능 사용 가능)
4 - unused                      이후를 위해 예약된 번호
5 - full multiuser, graphical   그래픽모드를 지원하는 멀티유저
6 - reboot                      재부팅
```

runlevel 5는 runlevel3의 모든 기능과 GUI기능의 결합이다.

시스템 런레벨의 확인

```
# who -r
            run-level 5  2020-03-11 09:40
            운영레벨=5    운영레벨 전환 시간
# runlevel
N 5
# init 3
login: root
password: redhat
# runrevel
5 3
# init 5
# who -r
run-level 5  2020-03-11 10:23                   last=3
현재 구성 정보   시스템 전환시간                  이전 구성 정보 레벨 확인

# systemctl get-default
graphical.target     => init 5과 동일
```

system run level은 무엇인가?

런레벨은 하나의 시스템에서 여러 가지의 운영 환경을 제공하는 것을 의미한다.
리눅스 서버의 경우 운영 환경의 전환을 통해 GUI CLI 의 변경이 가능하다.

EX: GUI > TUI,CLI

언어변경
- 현재 사용중인 언어 확인
echo $LANG
locale

- 현재 지원되는 언어 종류 확인
locale -a

- 임시적인 언어 변경
export LANG=C ; echo $LANG

- 영구적인 언어 변경
localectl set-locale LANG='변경할 언어'

현재 사용중인 언어 확인하기

```
# set | grep ^LANG    (# locale -a | grep LANG)
LANG=ko_KR.UTF-8
```

```
# locale

LANG=ko_KR.UTF-8
LC_CTYPE="ko_KR.UTF-8"
LC_NUMERIC="ko_KR.UTF-8"
LC_TIME="ko_KR.UTF-8"
LC_COLLATE="ko_KR.UTF-8"
LC_MONETARY="ko_KR.UTF-8"
LC_MESSAGES="ko_KR.UTF-8"
LC_PAPER="ko_KR.UTF-8"
LC_NAME="ko_KR.UTF-8"
LC_ADDRESS="ko_KR.UTF-8"
LC_TELEPHONE="ko_KR.UTF-8"
LC_MEASUREMENT="ko_KR.UTF-8"
LC_IDENTIFICATION="ko_KR.UTF-8"
LC_ALL=
```

임시적으로 언어를 변경하기

```
# export LANG=ko_KR.euckr

# export LANG=C
```

임시적으로 언어를 변경해야 하는 경우 :
현재 지정된(설정된) 언어셋을 실행해야 하는 프로그램에서 지원하지 않을 경우에 사용한다.
TOOL을 실행시에 $LANG의 변수를 읽어들인다. 만약 지원하지 않는 언어로 지정되어있을 경우에는 글자가 정상적으로 출력되지 않을 가능성이 있다.

제어문자

Ctrl + C Terminates the command currently running.

Ctrl + D Indicates end-of-file or exit.

---

① <CTRL + C> = ^C

- 현재 실행중인 프로세스를 인터럽트(Interrupt) 걸어서 종료한다.

인터럽트 - 인터럽트는 컴퓨터에 장착된 장치나 컴퓨터 내의 프로그램으로부터 오는 신호로서 운영체계가 하던 일을 멈추고 다음에 무엇을 할 것인지를 결정하게 한다. 인터럽트 신호가 입력되게 된다면 이전의 상태에 상관없이 인터럽트된 신호를 먼저 처리한다.

```
# sleep 500
        # ls
        # locale
  반응이 없다.
<CTRL + C>
#    < 명령어 실행이 가능해 진다.
```

sleep 명령어를 수행중에 <CTRL + C>를 이용하여 실행을 종료한다.

② <CTRL + D>

- "파일의 끝(EOF)" 의미 또는 "현재 쉘 종료(exit)" 의미를 갖는다.

(파일의 끝(EOF) 의미의 <CTRL + D>)

```
# mkdir -p /test        /* 최상위 디렉토리 아래 test 디렉토리 생성 */
# cd /test            /* 생성한 test 디렉토리로 이동 */
# pwd             /* 현재 작업 디텍토리 확인*/
 /test
# cat > file1       /* 파일에 내용 입력 */
Hello, Linux <Enter>
<CTRL + D>        /* "파일의 끝" 의미 */
```

```
# cat file1  /* 입력된 파일 내용 확인 */
Hello, Linux
```

(쉘 종료(exit) 의미의 <CTRL + D>)

```
# ps /* 프로세스 확인*/
# bash /* 쉘 실행 */
# ps/* 프로세스 확인 */

PID  TTY          TIME CMD
3290 pts/0    00:00:00 bash
3608 pts/0    00:00:00 bash
```

[참고] BASH : 명령어 해석기

ls 라는 명령어 하나만의 결과를 보기 위해서도 하드디스크, 키보드, 모니터등 여러 가지 장치를 제어해야 하는데 bash를 통한 명령어가 명령어 해석이 된 다음 커널에서 각종 장치를 제어하여 결과를 출력해 준다.

현재 ps로 나오는 결과의 bash는 사용중인 명령어 해석기가 bash라는 것을 나타내준다.

```
bash                      bash
----------+--wait------>+----------> 부모쉘은 자식 쉘이 실행되는동안 wait상태로 대기하다가
          |             |            반환즉시 wait상태를 종료하고 명령어 입력을 받는다
          | bash        |
          +------------>+ 자식 쉘이 실행된다

# <CTRL + D>/* 쉘 종료 (logout이 되는 것을 확인) */

bash
----------+            +--------->
          |            |
          | bash       |
          +------------+
```

```
# ps
PID TTY          TIME CMD
3290 pts/0    00:00:00 bash /* 이전에 실행한 bash shell이 종료 */

# <CTRL + D> /* 쉘 종료 (logout이 되는 것을 확인) */
GUI - 터미널이 종료된다
CLI - login 창으로 돌아간다

# init 3
login: root
password: redhat
# ps
# bash
# ps
^d입력
# ps
^d
login:
```

운영체제 종료와 재부팅

- 시스템 종료

```
# halt
# poweroff

# init 0

# shutdown -h now
```

- 시스템 재부팅

```
# reboot

# init 6

# shutdown -r now
```

- 다양한 시스템 종료 명령어
- 시스템을 종료하는 경우(halt)

```
이 명령어들은 시스템을 바로 종료한다.
# shutdown -h now      /* -h : halt */ /* 만약 시간을 지정하지 않는다면 기본 60초 이후 종료된다 */
# halt
# poweroff
# init 0
```

시스템을 재부팅(reboot)

```
# shutdown -r now      /* -r : reboot */
# reboot
# init 6
```

예약 종료

```
# shutdown -h +5        /* -h : halt, +5 : 5분의 유예기간 */

Shutdown scheduled for 수 2018-02-28 20:01:27 KST, use 'shutdown -c' to cancel.

Broadcast message from root@server1 (Wed 2018-02-28 19:56:27 KST):

The system is going down for power-off at Wed 2018-02-28 20:01:27 KST!
 /* 현재 상태에서 5분간 대기후 종료 */

# shutdown -c      /* 종료 취소 */

Broadcast message from root@server1 (Wed 2018-02-28 19:57:20 KST):

The system shutdown has been cancelled at Wed 2018-02-28 19:58:20 KST!
/* 시스템 종료를 취소 */
이전버전에서는 제어문자 <CTRL + C>를사용하여 취소를 했지만 centos7에서는 shutdown -c 명령어를
사용하여 취소해야한다.
```

예약 종료 취소

```
# shutdown -c       /* -c: cancel, 관리자 및 다른 관리자가 예약종료를 취소시 사용 */

Broadcast message from root@server1 (Wed 2018-02-28 20:00:10 KST):

The system shutdown has been cancelled at Wed 2018-02-28 20:01:10 KST!
```

시스템을 종료하거나 재부팅 하는 경우

```
1. 정기 PM
2. 하드웨어 업그레이드
3. 대규모 업데이트
4. * 해킹의 경우 *
```

관리자 암호 변경

- 관리자 암호 변경 방법
passwd

- 관리자 암호(password) 변경

초기 설치 시에는 비밀번호를 쉽게 주고 설치를 완료할 수 있다. 하지만 실제로 외부에서 접속을 하거나 사용을 하기 전에는 조직의 정책에 맞게 비밀번호를 변경해야하기 때문에 비밀번호를 변경하는 방법을 알고 있어야한다.

root 사용자는 모든 사용자의 암호 변경이 가능하고, 일반 사용자는 자신의 암호 변경만 가능하다.

passwd 명령어 다음에 지정된 사용자가 없으면 자신의 암호를 변경하는 경우이다.

```
# passwd    /* 변경된 root 계정의 비밀번호를 복구한다. */
Changing password for user root.
New UNIX password: (centos)                        /* 새 패스워드 입력 */
BAD PASSWORD: it is based on a dictionary word
Retype new UNIX password: (centos)                 /* 새 패스워드 확인 메시지 */
passwd: all authentication tokens updated successfully.

root 사용자의 비밀 번호 변경 중
새  암호:
잘못된 암호: 암호는 8 개의 문자 보다 짧습니다
새  암호 재입력:
passwd: 모든 인증 토큰이 성공적으로 업데이트 되었습니다.
```

date

date를 이용해서 시간을 확인한다. 만약 시간이 잘못 설정되어 있으면 date나 rdate,ntp명령어를 통해 시간을 재설정 해준다. 이때 rdate,ntp명령어를 사용하는 경우 서버로부터 시간을 동기화하여 사용 할 수 있으므로 시스템의 시간을 정확하게 설정 할 수 있다.

일반 사용자(EX: fedora)는 date 명령어를 통해 시간 확인만 가능하고, 관리자(EX: root)는 date 명령어를 통해 시간 확인이나 변경이 가능하다.

- root 사용자 : 현재 시간을 설정 및 확인 할 수 있다.
- 일반 사용자 : 현재 시간만 확인 할 수 있다. (즉, 관리자가 아니면 시간을 변경 할 수 없다.)

[명령어 형식]

```
# date /* 시스템 시간 출력 */
# date 07241300/* 시스템 시간 변경 */
# date +%m%d /* 시간 출력 형태 변경 */
```

[EX1] 운영체제 시간 확인 및 변경

```
# date /* 현재 운영체제에 설정되어 있는 시간 출력 */
요일 월 일 시:분:초 UTC 년
Tue Jan 26 05:16:44 KST 2010
```

시간정보 변경후 확인하기

```
테스트를 위하여 터미널을 하나 더 실행
[TERM1]# watch -n 1 date

아래 작업들은 [TERM2]에서 작업한다.
# date 08161300     /* 8월 16일 13:00 로 설정 년도는 생략가능하다 */
       월.일.시.분 단위로 작성
# date 081613002009 /* 2009년 8월 16일 13:00 */
       월.일.시.분.연도
```

[EX3] date 명령어의 시간 출력 형식 변경하여 출력하기

대부분의 프로그램은 하루에 한번 씩 로그 파일을 만들도록 하는 경우가 많이 존재한다.
이런 경우 date 명령어를 아래와 같이 응용하여 사용하는 경우가 많다.

실무에서 많이 사용되는 로그 파일 이름 형식 : file_0807.log

[시간 포맷 형식]

```
# date +%m%d        /* %m : month, %d : day */
# date +%m%d_%H%M   /* %H : hour, %M : Minute */
```

[실습] 시간정보 포맷 변경하여 파일 저장

```
# cd /test
# rm -rf /test/*
# touch file_`date +%m%d`.log
# ls -l
# ls -l
total 0
-rw-r--r--. 1 root root 0 Mar 11 15:18 file_0311.log

date 명령어에서 시간형식 포맷 변경시 자주 사용되는 옵션
%m 월
%d 일
%H 시
%M 분
%S 정수 초
%s 1970.01.01 기준으로 흐른 초시간
[참고]
역 따옴표를 표현하기 위한 또 다른 방법
# touch file_$(date +%m%d).log
```

[참고] 시간의 종류(Clock Type)

```
시간(Clock)
- Software Clock : 운영체제의 시간
- Hardware Clock : 하드웨어(펌웨어) 시간
```

```
■ Software clock
# date

■ Hardware Clock
# hwclock --show

HW-OS 시간 동기화

# hwclock --systohc    /* System Time -> Hardware Clock */
# hwclock --hctosys    /* Hardware Clock -> System Time */
```

관리자 비밀번호 변경

관리자 비밀번호를 변경하는 경우

- 주기적인 비밀번호 변경
- 비밀번호를 잊어버린경우
- 해킹들의 외부 침입으로 인한 경우

[EX] 기본적인 비밀번호 변경

```
# passwd

일상적으로 변경하는 경우에는 passwd 명령어로 변경한다.
```

디렉토리와 파일관리

디렉토리와 파일관리

단원목표

- 디렉토리 이동 관련 명령어
- 디렉토리 관리 명령어
- 파일 관리 명령어
- 파일 내용 확인 명령어
- 기타관리용 명령어

디렉토리 이동 관련 명령어

- pwd 명령어

pwd

- cd 명령어

경로(PATH)

- 상대경로
- 절대경로

PWD 명령어

파일 작업 중이나 자료의 위치로 이동 할 경우 현재 내가 작업하고 있는 디렉토리의 위치를 알고 이동 할 디렉토리를 지정해야 한다. 디렉토리 이동 관련 명령어에는 pwd 명령어, cd 명령어가 있다.

현재 내가 작업하고 있는 디렉토리의 위치를 절대경로를 통해 / (root)부터의 전체 경로를 출력해 준다.

[명령어 형식]

```
# pwd
```

[EX1] pwd 명령어 실습

```
# cd
# pwd
/root

# cd /etc/sysconfig/network-scripts
# pwd           /* 위치가 변경되었다. */
/etc/sysconfig/network-scripts/
# echo $PWD

# cd
# pwd
/root
```

CD 명령어

작업 디렉토리를 변경하고자 할 때 사용한다. 인자(Argument)인 디렉토리명은 이동하고자 하는 경로명을 나타낸다. 디렉토리 명을 지정하지 않고 cd 명령어를 단독으로 사용하면 현재 작업디렉토리가 어디에 있든지 사용자의 홈 디렉토리로 이동한다. cd 명령어는 디렉토리를 변경하는 명령어이다. cd 명령어를 사용하여 디렉토리 경로를 변경하는 경우 상대경로(Relative Path)나 절대경로(Absolute Path)를 사용할 수 있다.

- 상대경로(Relatvie Path)는 이동하는 기준이 현재 디렉토리이며, 현재 디렉토리를 기준으로 위, 아래로 이동하는 할 때 사용한다.

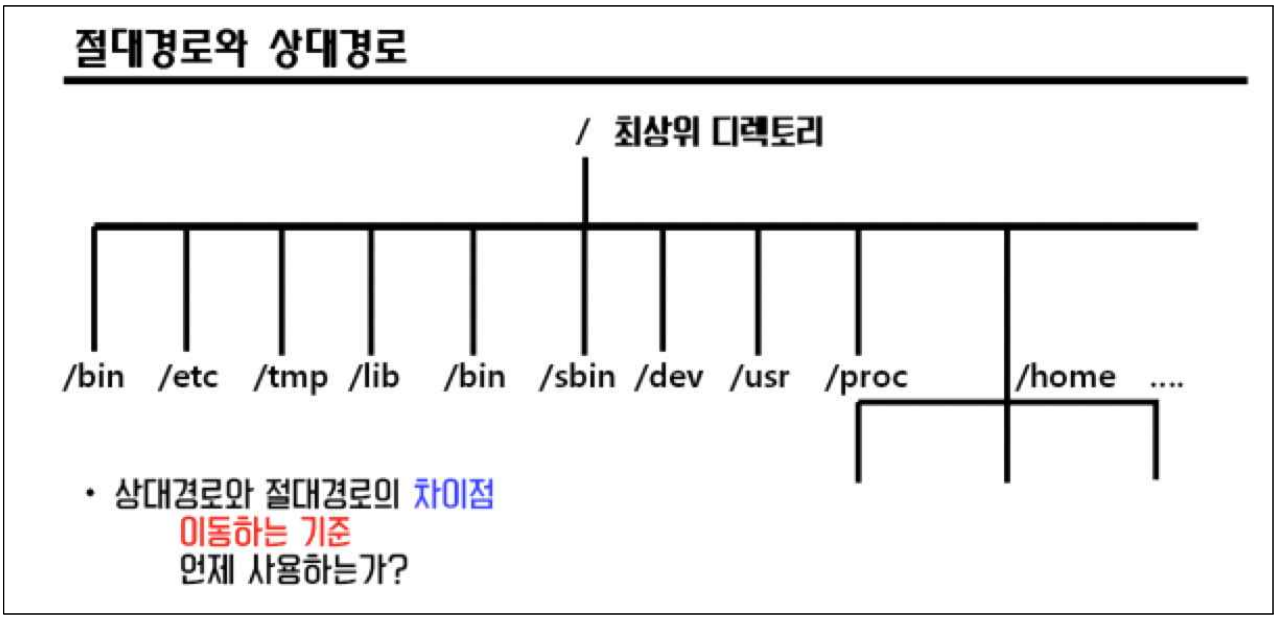
- 절대 경로(Absolute Path)는 이동하는 기준이 최상위 디렉토리(/)이며, root(/) 디렉토리를 기준으로 이동할 때 사용한다.

경로(PATH)

- 상대경로(Relative PATH) # cd dir1

- 절대경로(Absolute PATH) # cd /dir1

작업시에는 상대경로를 사용하여 이동하는 경우, 즉 작업 디렉토리안에 들어가서 직접 파일을 다루는 방법을 권장한다. 이것은 명령어 실수로 인해 불의의 사고를 예방할 수 있거나, 최소화 할 수 있기 때문이다.



상대경로(Relative PATH)

[EX1] 상대경로 사용하는 경우

```
# cd /etc
# pwd
/etc
# cd sysconfig
# cd network-scripts
# ls -a /* .디렉토리 , ..디렉토리 */
.  ..  ifcfg-ens33

# cd .     /* 현재 디렉토리 (cd /etc/sysconfig/network-scripts/와 같은 뜻) */
# pwd
/etc/sysconfig/network-scripts/
# cd .. /* 상위 디렉토리 (내 기준에서 상위 디렉토리로 이동) */
# pwd
/etc/sysconfig/
```

[참고] 디렉토리 표시

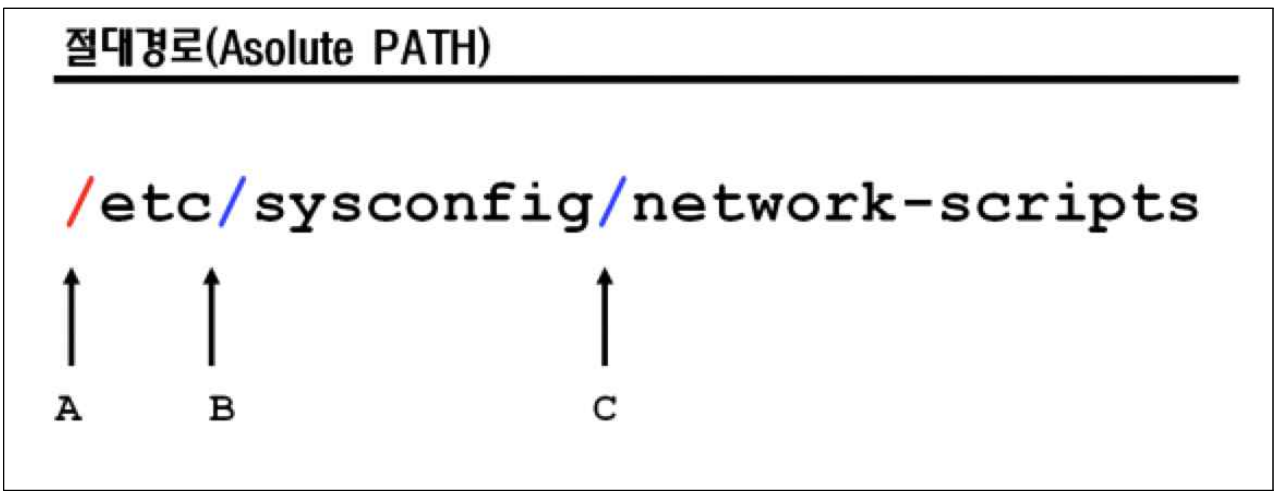
```
.  디렉토리는 현재 디렉토리
.. 디렉토리는 상위 디렉토리
```

```
# pwd
/etc/sysconfig/
# cd ../../ /* 상위 디렉토리, 상위 디렉토리 */
# pwd
/
```

[EX2] 상위 디렉토리에 파일 만들기

```
# cd /etc/sysconfig/network-scripts
# touch ../../test.txt
# cd ../../
# pwd
/etc/
# ls -l test.txt
```

절대경로



- A는 최상위 디렉토리인 / 디렉토리를 뜻 한다.
- B,C는 디렉토리의 구분자

처음 위치한 /는 최상위 디렉토리를 나타내지만 뒤에 나와 있는 /는 구분자의 역할이다.
따라서 /etc/sysconfig/network-scripts 와 /etc/sysconfig/network-scripts/는 같은 뜻을 가지게 된다.

```
# cd /tmp , cd /tmp/
# cd /etc/sysconfig
# cd /usr
```

로그인 된 사용자의 홈 디렉토리 이동

root 사용자로 로그인 하였기 때문에 root의 홈디렉토리로 이동한다.
만약 fedora 사용자로 로그인 하였다면 fedora의 홈디렉토리로 이동하게 된다.

사용자의 홈디렉토리

root 사용자 -> /root

일반 사용자 -> /home/$USER

root 사용자인 경우

```
# id
uid=0(root) gid=0(root) groups=0(root)
-> root 사용자인 경우
```

fedora 사용자인 경우

```
# ssh fedora@localhost
fedora 사용자로 로그인

$ id
uid=1001(fedora) gid=1001(fedora) groups=1001(fedora)
$ cd
$ pwd
/home/fedora
$ exit
# cd ~fedora
```

[차이점]

```
# cd ~fedora
# cd ~/fedora

~fedora의 경우는 fedora의 홈 디렉토리 /home/fedora로 이동하는 경우의 명령어이며
~/fedora의 경우는 fedora의 홈 디렉토리로 이동하지 않고 현재 계정의 홈 디렉터리 하위의 fedora
디렉터리로 이동하는 /$HOME/fedora의 의미를 가진다.

[참고]

$ cd
$ cd ~
$ cd $HOME

동일 하게 자신의 홈 디렉토리로 이동한다.
```

사용한 이전 디렉토리로 이동

[시나리오] 사용자가 /etc 디렉토리에서 작업을 하다가 /home/user01 디렉토리로 이동하는 경우 상대경로나 절대경로를 사용하는 경우는 불편하게 된다. 이런 경우 이전 디렉토리로 바로 이동하기 위해서 "cd - " 명령어를 사용한다.

/etc <------> /etc/sysconfig/network-scripts

```
# cd /etc
# ls


# cd /etc/sysconfig/network-scripts
# ls


# cd /etc
# ls


# cd -
# pwd <cd - 로 이동이후 현재 디렉토리를 확인한다.>
/etc/sysconfig/network-scripts
-> 반복적으로 사용해본다.
```

옆에 있는 디렉토리로 이동하는 경우

```
                 /
                 |
-------+---------+-------+--------+------
       |         |       |
.... test ..... export   etc
        |
 --+----+-----+---    • 현재 디렉토리 : dir2
 dir1       dir2      • 목적 디렉토리 : dir1
```

[EX] 디렉토리 이동 실습

```
# cd /test
# rm -rf /test/*

# mkdir dir1 dir2
# cd dir2
# pwd
/test/dir2

# cd ..
# pwd
/test

# cd dir1
# pwd
/test/dir1

# cd ../dir2
# pwd
/test/dir2

# cd ../dir1
# pwd
/test/dir1
```

```
최상위 디렉터리에서부터 다시 이동하는 것보다는 근처에 있는 디렉터리라면 상대 경로를 통해 이동
하는 것이 더 빠르게 이동할 수 있는 경우도 있다.

(실무 예) 프로그램 설치
        /usr/local/apache2
                 |
                 +---- bin   (# cd ../bin)
                 |
                 +---- conf  (# cd ../conf)
                 |
                 +---- docs  (# cd ../docs)
                 |
                 +---- ....
```

디렉토리 관리 명령어

- ls 명령어

ls -l dir1
ls -ld dir1

- mkdir 명령어
- rmdir 명령어

ls 명령어

디렉토리에 있는 내용을 확인하고자 할 때 (ls 명령에 대해서 확인 : # man ls)

[명령어 형식]

```
# ls -l
# ls -l dir1

# ls -ld
# ls -ld dir1

# ls -altr
```

옵션 : 설명

-a : 모든 파일 표시, 여기에는 숨김 파일(점(.)으로 시작하는 파일)도 표시한다.

-l : 디렉토리가 지정되는 경우 디렉토리의 내용을 자세히 보여준다. 또한 파일의 내용 이 지정되는 경우 파일의 속성 정보를 자세히 보여준다. 파일 종류, 링크 수, 소유자명, 그룹명, 파일 크기, 최종 수정일 및 용량, 파일명 표시한다.

-R : 해당 디렉토리와 서브디렉토리의 모든 내용을 표시

-F : 디렉토리인 경우에는 디렉토리 "/"표시를 하고 실행 파일인 경우는 뒤에 "*"표시를 한다.

-i : 해당 파일의 inode 번호를 표시한다.

-n : 파일의 소유자와 그룹을 숫자로(UID : GID)표시한다.

-d : 찾고자 하는 디렉토리에 관한 정보만을 표시한다.

[EX1]] ls 명령어의 기본 사용법 테스트

```
(실습용 구조)

/test ---+-- file1
         |
         +-- dir1 --- file2
```

실습 준비

```
# cd /test
# rm -rf /test/*
# touch file1
# mkdir dir1
# touch dir1/file2

[root@server1 test]# tree
.
├── dir1
│   └── file2
└── file1


# ls -lR(# find . )

.:
total 0
drwxr-xr-x. 2 root root 18 Mar 25 13:23 dir1
-rw-r--r--. 1 root root  0 Mar 25 13:23 file1

./dir1:
total 0
-rw-r--r--. 1 root root 0 Mar 25 13:23 file2
```

```
(ls -l 출력 결과 해석)
-----------------------------------------------------------------------------
-            파일의 종류(File Type), -(일반파일), d(디렉토리파일)
rw-r--r--.   퍼미션 모드(Permission Mode) <마지막의.은 ACL 관련 코드>
1            링크 수(Hard Link Count)
root         소유자(Owner)
root         그룹(Group)
0            파일의 크기(File Size), 기본 단위 : bytes
Mar 25 13:23 수정 또는 생성 시간(Mtime: Modify Time) [자세한 정보를 보려면 stat명령어를 사용]
file1        파일의 이름(File Name)
-----------------------------------------------------------------------------
```

```
# ls -ld . /* 현 디렉토리 정보를 자세히 출력( .디렉토리 출력) */
drwxr-xr-x. 3 root root 29 Mar 25 13:23 .
# ls -l dir1 /* dir1디렉토리의 내용을 출력 */
total 0
-rw-r--r--. 1 root root 0 Mar 25 13:23 file2
# ls -ld dir1 /* dir1디렉토리의 정보를 출력 */
drwxr-xr-x. 2 root root 18 Mar 25 13:23 dir1
# ls -lR /test /* -R: Recursive, 하위 디렉토리까지 */
/test: <해당하는 디렉토리명을 출력한뒤 디렉토리 내부의 파일과 디렉토리를 출력한다.>
total 0
drwxr-xr-x. 2 root root 18 Mar 25 13:23 dir1
-rw-r--r--. 1 root root  0 Mar 25 13:23 file1

/test/dir1:
total 0
-rw-r--r--. 1 root root 0 Mar 25 13:23 file2
```

[EX2] "ls -a" 옵션 실습

```
# cd
# pwd
/root
```

일반파일 출력

```
# ls -l
drwxr-xr-x 2 root root  4096  4월  6 12:17 Desktop
-rw------- 1 root root  4210  1월 25 22:54 anaconda-ks.cfg
-rw-r--r-- 1 root root     0  4월  7 12:38 file_0407.log
-rw-r--r-- 1 root root 54631  1월 25 22:54 install.log
-rw-r--r-- 1 root root  9641  1월 25 22:50 install.log.syslog
```

숨김파일과 일반파일 출력

```
# ls -al
drwxr-x--- 18 root root  4096  8월 15  2010 .
drwxr-xr-x 26 root root  4096  8월 15  2010 ..
-rw-------  1 root root  2395  8월 15  2010 .bash_history
-rw-r--r--  1 root root    24  1월  6  2007 .bash_logout
-rw-r--r--  1 root root   231  4월  7 10:44 .bash_profile
-rw-r--r--  1 root root   223  4월  7 10:33 .bashrc
..... (중략) .....
drwxr-xr-x  2 root root  4096  4월  6 12:17 Desktop
-rw-------  1 root root  4210  1월 25 22:54 anaconda-ks.cfg
-rw-r--r--  1 root root     0  4월  7 12:38 file_0407.log
-rw-r--r--  1 root root 54631  1월 25 22:54 install.log
-rw-r--r--  1 root root  9641  1월 25 22:50 install.log.syslog

# cd /test
# ls -l
합계 0
drwxr-xr-x. 2 root root 19  3월 12 11:49 dir1
-rw-r--r--. 1 root root  0  3월 12 11:49 file1
# touch .file2
# ls -l
합계 0
drwxr-xr-x. 2 root root 19  3월 12 11:49 dir1
-rw-r--r--. 1 root root  0  3월 12 11:49 file1
# ls -la   /* 숨김파일은 a 옵션을 사용해야만 확인가능하다. */
합계 0
drwxr-xr-x.  3 root root  45  3월 12 12:01 .
dr-xr-xr-x. 19 root root 249  3월 11 18:08 ..
-rw-r--r--.  1 root root   0  3월 12 12:01 .file2
drwxr-xr-x.  2 root root  19  3월 12 11:49 dir1
-rw-r--r--.  1 root root   0  3월 12 11:49 file1
```

[EX3] "ls -F" 옵션 실습

```
# cd /test
# rm -rf /test/*

# cp /etc/passwd file1    /* 일반 파일 */
# ln -s file1 file2       /* 링크 파일 */
# cp /bin/ls file3        /* 실행 파일 */
# mkdir dir1              /* 디렉토리 파일 */

# ls -F   (dir1/  file1  file2@  file3*)
# ls (dir1   file1  file2   file3 )
```

[참고]

리눅스의 경우는 파일, 디렉터리등 종류가 다르다면 다른 색깔로 표시를 해주기 때문에 –F의 경우 그렇게 사용량이 많지는 않다. 하지만 다른 UNIX계열의 OS는 동일하게 표시되기 때문에 알아두자.

[EX4] ls -i' 옵션 실습

-i는 Index NODE인 inode를 표시해주기 위한 옵션이다. 파일의 속성정보를 담고있는 inode의 구분번호를 표시해 주고 [있다. ls](http://xn--2j1bk50b.ls/) –l로 명령어를 입력한 경우의 출력되는 결과들이 저장되어 있다.

데이터는 Data Block이라는 공간에 별도로 저장되어있다

```
# ls -li /test/file1 /* inode 번호까지 확인 */
70133672 drwxr-xr-x 2 root root 18  3월  5 12:42 /test/dir1

# ls -ldi /test/dir1 /* inode는 각각 고유한 값을 지님 */
701325642 drwxr-xr-x 2 root root 18  3월  5 12:42 /test/dir1
inode - 파일과 디렉토리에 지정되어 있는 번호
```

[EX5] 파일 또는 디렉토리만 출력

alias(별칭) 명령어를 간편하게 사용하기 위해 사용한다

(선언) # alias ls='ls -h --color=tty'
(확인) # alias
(해제) # unalias ls

```
# alias lsf='ls -l | grep "^-"' /* 파일인 경우 속성 정보에 -로 표시 */
# alias lsd='ls -l | grep "^d"'/* 디렉토리인 경우 속성 정보에 d로 표시 */
```

```
# cd
# lsf
-rw------- 1 root root  4210 Jan 25 22:54 anaconda-ks.cfg
-rw-r--r-- 1 root root 54631 Jan 25 22:54 install.log
-rw-r--r-- 1 root root  9641 Jan 25 22:50 install.log.syslog
-rw-r--r-- 1 root root     0 Jan 26 07:34 local-host-names
-rw------- 1 root root 11665 Jan 26 07:32 mbox
# lsd
drwxr-xr-x 2 root root  4096 Jan 26 02:13 Desktop
```

(참고) 선언된 alias 확인

```
# alias        alias의 전체목록을 확인
# alias lsf    인자값으로 생성되어 있는 alias가 있다면 출력
```

```
# alias lsf /* lsf의 alias의 정보 확인 */
alias lsf='ls -l | grep "^-"'

# alias lsd /* lsd의 alias의 정보 확인 */
alias lsd='ls -l | grep "^d"'
```

[EX6] ls -h 옵션 설정(-h : human)

- alias를 지정하지 않은 경우

```
# cd /test
# cat /etc/services >> file1
# cat /etc/services >> file1
# cat /etc/services >> file1
# cat /etc/services >> file1
# cat /etc/services >> file1
# ls -l /etc/services
-rw-r--r--. 1 root root 670293  6월  7  2013 /etc/services
# ls -l file1
-rw-r--r--. 1 root root 5364652  3월 12 12:13 file1
# ls -lh file1
-rw-r--r--. 1 root root 5.2M  3월 12 12:13 file1
```

이전과 달리 현재는 byte 단위로 용량을 보게 된다면 단위가 너무 많기 때문에 힘들다. 용량이 정확해야 하지 않는다면 -h 옵션을 사용하여 사람이 보기 쉬운형태로 변환하여 사용한다.

```
# ls -lh /etc/services
-rw-r--r--. 1 root root 670293  6월  7  2013 /etc/services
```

```
# vi ~/.bashrc     (# gedit ~/.bashrc)

..... (중략) .....
#
# Sfecific Configuration
#
alias ls='ls --color=tty -h'

# . ~/.bashrc     (# source ~/.bashrc)
```

alias를 지정한 경우

```
# ls -l /etc/services
-rw-r--r--. 1 root root 655K  6월  7  2013 /etc/services
```

보기 편한 형태로 용량이 변경되었다.

[EX7] "ls -altr" 명령어 실습

```
# ls -altr /tmp    /* -t : time sort, -r : reverse sort */
```

이전파일이 상단에 출력된다.

```
[참고]
# cd /Log_dir
# ls -altr
```

```
ls 명령어에서 이정도는 알아두자.
l 자세하게 출력
a 모든파일 출력(숨김파일 포함)
i inode 번호 출력
h 용량을 사람이 보기 편하게 변경하여 출력
t 시간순 정렬
u 접근시간 으로 변경
r 역방향 정렬때 사용(t옵션과 같이 쓴다.)
F 파일형식 확인
```

mkdir 명령어

새로운 디렉토리를 생성하며, 빈 디렉토리를 생성한다. 옵션을 통하여 여러개의 디렉토리를 한꺼번에 생성 할 수도 있다.

[명령어 형식]

```
# mkdir dir1              /* 현 디렉토리에 dir1 디렉토리 1개 생성 */
# mkdir dir1 dir2         /* 현 디렉토리에 dir1, dir2 디렉토리 2개 생성 */
# mkdir -p dir3/dir2/dir1 /* dir3 디렉토리 안에 dir2를 생성하고 dir2 안에 dir1을 생성 */
```

[명령어 옵션]

옵션 : 설명

-m : 디렉토리의 퍼미션 권한을 지정 (기본값 : 755)

-p : 디렉토리 경로로 생성 (디렉토리를 만들 때 상위 디렉토리가 없을시 상위 디렉토리까지 생성)

[EX1] mkdir -p 옵션 사용법

```
# cd /test
# rm -rf /test/*
# pwd
/test
# mkdir dir4
# ls -l
drwxr-xr-x 2 root root 6  3월  5 12:58 dir4
# mkdir dir4/dir2/dir1
mkdir: cannot create directory 'dir4/dir2/dir1': No such file or directory
# mkdir -p dir4/dir2/dir1
# ls -R   (# find .)
-> 출력 내용 생략
```

rmdir 명령어

[명령어 형식]

```
# rmdir dir1              /* dir1 디렉토리 1개 삭제 */
# rmdir dir1 dir2         /* dir1, dir2 디렉토리 2개 삭제 */
# rmdir -p dir4/dir2/dir1 /* 경로에 포함되어 있는 디렉토리 삭제 (비워있어야 함) */
```

[명령어 옵션]

옵션 : 설명

-p : 하위항목을 같이 지움 (조건 : 하위항목도 비워 있어야 함)

[참고] 비어 있지 않는 디렉토리 삭제

```
# rm -rf dir1       /* -r : recursive, -f : force */
```

파일 관리 명령어

- touch 명령어
touch -t 08301300 file1

- cp 명령어
cp file1 file2
cp file1 dir1
cp -r dir1 dir2

- mv 명령어
mv file1 file2
mv file1 dir2
mv dir1 dir2

- rm 명령어
rm -rf dir1

touch 명령어

파일의 이름을 지정하였다면 기존에 존재하지 않는 빈 파일을 만들어주고 기존에 존재하는 파일이라면 지정된 파일이나 디렉토리의 수정시간(mtime, Modify Time)이나 접근시간(atime, Access Time)등을 현재 시간으로 업데이트 시켜준다.

만약 touch 명령어에 -t 옵션을 사용 하여 파일이나 디렉토리의 수정시간을 특정한 시간으로 변경 가능하다.

(해커가 침입하여 파일을 수정하고 아래의 옵션을 통해 수정과 접근 시간을 바꿔놓을 수도 있다.)

[명령어 형식]

```
# touch file2             /* file2 파일 1개 생성 */
# touch file1 file2       /* file1, file2 파일 2개 생성 */
# touch -t 08081230 file1 /* file1 수정 시간 변경(월,일,시,분) */
```

[명령어 옵션]

옵션 : 설명

-a : 최근 파일 사용기간만 변경

-c : 파일을 생성하지 않는 명령어

-d [시간] : 현재 시간 대신 지정한 시간(시분)으로 변경

-m : 최근 파일 변경 시간만 변경 (파일 수정시간)

-r [파일] : 현재 시간 대신 지정한 파일의 시간으로 변경

-t MMDDhhmm [[CC]YY][.SS] : 현재 시간 대신 지정한 시간(월일시분)으로 변경

[EX1] 빈 파일 생성

```
# cd /test
# rm -rf /test/*

# touch file1
# ls -l file1
-rw-r--r-- 1 root root 0  8월 11 12:30 file1
-> 파일의 크기 0
```

[EX2] 파일의 생성 시간 현재시간으로 변경

```
# date
# cp -p /etc/passwd file2
# ls -l file2
-rw-r--r-- 1 root root 2.0K Jan  7 15:52 file2
-> 시간 정보를 확인한다.

# touch file2
# ls -l file2
-rw-r--r-- 1 root root 2.0K Jan  8 11:08 file2
-> 시간 정보를 확인한다.

# date
Wed Jan  8 11:08:40 KST 2014
-> 현재 시간을 확인한다.
```

[EX3] touch -t 옵션 사용

```
# touch -t 08301300 file2
# ls -l file2
-rw-r--r-- 1 root root 2.0K Aug 30  13:00 file2
```

cp 명령어

파일이나 디렉토리의 내용을 다른 파일 또는 다른 디렉토리에 복사 할 때 사용. 파일을 복사하는 것은 물리적으로 새로운 파일을 하나 생성하며 새로운 파일의 이름과 새로운 inode, 복사된 데이터 블록을 가지게 된다.

[명령어 형식]

```
# cp file1 file2   /* file1 파일내용을 file2로 생성 */
# cp file1 dir1   /* file1 파일내용을 dir1디렉토리에 file1 생성 */
# cp -r dir1 dir2 /* dir1 디렉토리를 dir2디렉토리로 생성 */
# cp 원본 타겟
```

[명령어 옵션]

옵션 : 내용

-a : 원본 파일의 속성, 링크 정보를 유지 하면서 복사
-b : 복사할 대상을 덮어쓰거나 지울 때를 대비해서 백업 파일 만듦 백업파일의 파일명 뒤에는 ~가 표시된다.
-d : 심볼릭 파일 자체를 심볼릭 정보와 함께 복사할 때 사용
-f : 복사할 파일이 존재할 때 삭제하고 복사
-i : 복사할 파일이 존재하는 경우 복사할 것인지 물어봄
-l :디렉토리가 아닌 경우 복사 대신 하드 링크로 만듦
-p : 원본 파일의 소유, 그룹, 권한, 허용 시간을 보존한 채로 복사
-r : 서브 디렉토리 내에 있는 모든 파일까지 통째로 복사
-s : 디렉토리가 아닌 경우 복사 대신 심볼릭 링크로 만듦
-u : 대상 파일보다 원본 파일이 새로운 것일 때 복사
-v : 복사 상태를 보여줌

[EX1] 파일에 대한 inode를 확인

```
■ 파일의 일반적인 구조(EX: 일반 파일)

                +-------------+
      file1 ---->| Inode       |   정보 블럭
                +-------------+
                |             |
                | Data Block  |   데이터 블럭
                |             |
                +-------------+

■ Inode Number ?
```

```
# cd /test
# rm -rf /test/*

# touch file1
# ls -li file1
4039445 -rw-r--r-- 1 root root 0 Jan 26 11:34 file1

# cp file1 file2
# ls -li file2      /* 복사하면서 inode가 바뀐 걸 볼 수 있음. */
4039452 -rw-r--r-- 1 root root 0 Jan 26 11:35 file2
```

[EX2] cp 명령어의 -r 옵션

```
# cd /test
# rm -r /test/*

# mkdir dir1
# touch dir1/file1 dir1/file2

# cp -r dir1 dir2
# ls -lR
drwxr-xr-x 2 root root 4096 Jan 26 11:36 dir1
drwxr-xr-x 2 root root 4096 Jan 26 11:37 dir2

./dir1:
total 0
-rw-r--r-- 1 root root 0 Jan 26 11:36 file1
-rw-r--r-- 1 root root 0 Jan 26 11:36 file2

./dir2:
total 0
-rw-r--r-- 1 root root 0 Jan 26 11:37 file1
-rw-r--r-- 1 root root 0 Jan 26 11:37 file2

# cp -r dir1 dir2
# ls -lR
        # cp -r /home/fedora/test1 /tmp/fedorahomefile
        # cp -r /test /tmp
```

[EX3] cp를 이용하여 같은 파일에 덮어 쓰기(Overwrite)하는 경우

```
# cd /test
# rm -rf /test/*

# mkdir dir1
# echo "linux200" > file1
# cat file1
linux200

# touch dir1/file1 /* 동일한 파일명으로 파일 생성 */
# cat dir1/file1   /* 이름만 동일 할 뿐 내용 다름 */
#

# cp file1 dir1    /* 동일한 파일명에 덮어쓰기 */
cp: overwrite `dir1/file1'? y

# ls -lR
.:
total 8
drwxr-xr-x 2 root root 4096 Jan 26 23:39 dir1
-rw-r--r-- 1 root root    9 Jan 26 23:39 file1

./dir1:
total 4
-rw-r--r-- 1 root root 9 Jan 26 23:40 file1

# cat dir1/file1    /* copy 내용 확인 가능 */
linux200

-f 옵션 실습하기
# cp file1 file2
# echo "Linux252" >> file1
# unalias cp
# cp -f file1 file2 /* 따로 응답없이 덮어쓰기 가능 */

unalias 하지 않고 복사
# which cp
        /usr/bin/cp
# /usr/bin/cp -f cpfile file1
# command cp -f file1 file2
```

[EX4] 원본 파일의 소유, 그룹, 권한, 허용 시간을 보존한 채로 복사(cp -p)

```
# touch file1
# ls -l
합계 0
-rw-r--r--. 1 root root 0  3월 12 13:02 file1
# chmod 777 file1 /* 파일에 대한 퍼미션 변경 */
# ls -l
합계 0
-rwxrwxrwx. 1 root root 0  3월 12 13:02 file1  파일권한에 대한 변경
# chown .team01 file1
# ls -l
합계 0
-rwxrwxrwx. 1 root team01 0  3월 12 13:02 file1  파일그룹소유자에 대한 변경 확인

# cp file1 file2
# ls -l
합계 0
-rwxrwxrwx. 1 root team01 0  3월 12 13:02 file1
-rwxr-xr-x. 1 root root   0  3월 12 13:05 file2

# cp -p file1 file3
# ls -l            /* 퍼미션 값과 생성 시간등을 그대로 이어 받아오고 있다 */
합계 0
-rwxrwxrwx. 1 root team01 0  3월 12 13:02 file1
-rwxr-xr-x. 1 root root   0  3월 12 13:05 file2
-rwxrwxrwx. 1 root team01 0  3월 12 13:02 file3


# chmod 777 /test        /* 다른 사용자들이 접근하여 파일을 생성 할 수 있도록 퍼미션 변경 */
# su - fedora
$ id
uid=1001(fedora)                    gid=1001(fedora)                    groups=1001(fedora)
context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023

$ pwd
/home/fedora

$ cd /test
$ cp file1 file4
$ ls -l
-rwxrwxrwx 1 root   root   0  5월 11 20:30 file1
-rwxr-xr-x 1 root   root   0  5월 11 20:31 file2
-rwxrwxrwx 1 root   root   0  5월 11 20:30 file3
-rwxrwxr-x 1 fedora fedora 0  5월 11 20:32 file4

$ cp -p file1 file5
$ ls -l
합계 0
-rwxrwxrwx 1 root   root   0  5월 11 20:30 file1
-rwxr-xr-x 1 root   root   0  5월 11 20:31 file2
-rwxrwxrwx 1 root   root   0  5월 11 20:30 file3
-rwxrwxr-x 1 fedora fedora 0  5월 11 20:32 file4
-rwxrwxrwx 1 fedora fedora 0  5월 11 20:30 file5

$ exit
#
```

```
(실무 예) 로그 파일(EX: file.log) 비우기
# cp /dev/null file.log
# cat /dev/null > file.log
# > file.log

# cd /test
# cp /var/log/messages file.log
# cp /dev/null file.log
-> y
# ls -l file.log
-> 파일의 크기 '0' 확인
```

mv 명령어

파일과 디렉토리의 내용을 다른 파일 또는 다른 디렉토리로 옮길 때 사용하며 파일의 이름이나 디렉토리의 이름을 바꿀 수 있음. 같은 파티션 안에서 파일을 옮긴다는 것은 물리적으로 파일 이름만 변경하며, Inode 정보나 데이터 블록은 그대로 유지가 되고 다른 파티션으로 파일을 옮기는 경우는 새로운 파일 이름과 Inode, 데이터 블록을 할당 받게 됨.

[명령어 형식]

```
# mv file1 file2       /* file1 파일이 이름이 file2로 변함 */
# mv file1 dir1        /* file1 파일이 dir1 디렉토리에 하위경로로 이동 */
# mv dir1 dir2         /* dir1 디렉토리가 dir2 디렉토리에 하위경로로 이동 */
```

[명령어 옵션]

옵션 : 내용

-b : 복사할 대상을 덮어쓰거나 지울 때를 대비해서 백업 파일 만듦
-f : 복사할 파일이 존재할 때 삭제하고 복사
-i : 복사할 파일이 존재하는 경우 복사할 것인지 물어봄
-u : 대상 파일보다 원본 파일이 새로운 것일 때 복사
-v : 파일 옮기기 전의 과정을 보여 줌

[EX1] mv 명령어 사용법

```
# cd /test ; rm -rf /test/*

# touch file1
# mkdir dir1
# touch dir1/file2
/test ---+--- file1
         |
         +--- dir1-----file2

# mv file1 file3         /* mv를 동일한 디렉토리에서 사용하는 경우에는 이름변경으로 동작한다.
*/
/test ---+--- file3
         |
         +--- dir1-----file2

# mv file3 dir1
/test --- dir1---+-----file2
                 |
                 +-----file3

# mv dir1 dir2
/test --- dir2---+-----file2
                 |
                 +-----file3

# cp -r dir2 dir1
/test -+- dir2---+-----file2
       |         |
       |         +-----file3
       +- dir1---+-----file2
                 |
                 +-----file3

# mv dir1 dir2
/test --- dir2 ---+---- file2
                  |
                  +---- file3
                  |
                  +--- dir1 --+-- file2
                  |
                  +-- file3
```

[EX2] Inode 확인

```
# cd /test ; rm -rf /test/*

# touch file1
# ls -li file1                /* Inode 확인 */
97774 -rw-r--r-- 1 root root 0 Jan 26 23:57 file1

# mv file1 file3
# ls -li file3                /* Inode 가 변하지 않은 걸 알 수 있음 */
97774 -rw-r--r-- 1 root root 0 Jan 26 23:57 file3
단지 파일의 이름정보만 변경되었다.
```

[EX3] 여러 개의 파일을 동시에 이동

```
# cd /test
# touch file1 file2 file3 file4
# ls
file1 file2 file3 file4

# mkdir dir1
# mv file* dir1                /* file이라는 파일들을 dir1 디렉토리로 이동 */
# ls dir1
file1 file2 file3 file4
```

rm 명령어

파일과 디렉토리를 지우고자 할 때 사용하며 한꺼번에 여러 개를 지울 수도 있으며 지운 파일들은 되살릴 수 없으므로 주위 해서 사용해야 하는데 -i 옵션을 사용하면 한번 더 묻게 되므로 부주의로 인한 파일 삭제를 막을 수 있으며, 옵션 -r를 사용 시 시스템의 모든 파일이 삭제되는 경우도 있으니 신중하게 사용해야 한다.

[명령어 형식]

```
# rm file1        /* file1 파일 1개 삭제 */
# rm file1 file2  /* file1, file2 파일 2개 삭제 */
# rm -r dir1      /* dir1 디렉토리 하위경로까지 삭제 */
```

[명령어 옵션]

옵션 : 설명

-f : 강제로 파일을 지우고 삭제할 파일이 없을 경우에도 아무런 메시지를 보여주지 않는다.
-i : 파일을 삭제할 것인지 사용자에게 물어봄
-r, -R : 일반파일이면 그냥 지우고 디렉토리일 경우 그 하위경로와 파일을 모두 지움
-v : 삭제되는 파일의 정보를 보여줌

[참고] 비어 있지 않은 디렉토리 삭제

```
# rm -rf dir1
```

[EX1] rm 명령어 -r 옵션 사용에 대해서

```
# cd /test
# rm -r /test/*

# mkdir dir1
# touch dir1/file1 dir1/file2
```

```
# cat ~/.bashrc
--------------------------------------------
# .bashrc

# User specific aliases and functions

alias rm='rm -i'     /* -i : interactive */
alias cp='cp -i'
alias mv='mv -i'

# Source global definitions
if [ -f /etc/bashrc ]; then
        . /etc/bashrc
fi

#
# Sfecific Configuration
#
export PS1='[₩u@₩h ₩w]₩$ '
alias ls='ls --color=tty -h'
---------------------------------------------
```

[참고] alias 사용법

(선언) # alias cp='cp -i'
(확인) # alias (# alias cp)
(해제) # unalias cp

```
# rm -r dir1
rm: descend into directory `dir1'? <CTRL + C>

# ls
-> dir1 존재 확인

# rm -rf dir1    /* -f : force */
# ls
```

파일 내용 확인 명령어
- cat 명령어
cat /etc/passwd
cat -n /etc/passwd
cat file1 file2 > file3

- more 명령어
cat /etc/services
more /etc/services
CMD | more
ps -ef | more
rpm -qa | more
cat /etc/services | more

- head 명령어
head /etc/passwd
head -5 /etc/passwd

- tail 명령어
tail /etc/passwd
tail -5 /etc/passwd
tail -f var/log/messages

cat 명령어

cat 명령어는 텍스트 파일을 읽고., 결합하고 표준 출력으로 내보내는 명령어이다.

cat의 경우 파일이 화면이상으로 길게 작성된 경우 순식간에 지나가버린다. 파일의 내용을 화면으로 출력. 파일의 내용을 화면에 연속적으로 출력하기 때문에 파이프(Pipe Line)을 사용하여 more 명령어에 연결하여 사용 가능.

[명령어 형식]

```
# cat file1                  /* file1 파일 내용을 출력 */
# cat file1 file2            /* file1, file2 파일 내용을 출력 */
# cat -n file1               /* file1 파일내용을 줄번호와 함께 출력 */
# cat file1 file2 > file3    /* file1, file2 출력 결과를 file3에 저장 */
```

[명령어 옵션]

옵션 : 설명

-e : 제어 문자를 ^ 형태로 출력하며 끝에 $를 추가
-n : 줄번호를 공백을 포함하여 화면 왼쪽에 나타냄
-s : 중복되고 겹치는 빈 행은 하나의 빈 행으로 처리
-v : 행바꿈 문자, tab를 제외한 제어문자를 ^ 형태로 출력
-E : 각 행 끝에 $ 문자 출력
-T : tab 문자를 출력
-A : -vET 옵션과 동일

```
-v, --show-nonprinting: 인쇄되지 않는 문자(탭 및 줄 끝 문자 제외)를 이스케이프 시퀀스로 표시합니다.
-E, --show-ends: 각 라인의 끝에 "$" 문자를 표시합니다.
-T, --show-tabs: 탭을 "^I"로 표시합니다.
```

[EX1] /etc/passwd 파일 출력

```
# cat /etc/passwd
-> 출력내용 생략
```

```
# cat -n /etc/passwd | more (# nl /etc/passwd | more)       /* 내용 앞에 번호가 붙어 출력 */
     1  root:x:0:0:root:/root:/bin/bash
     2  bin:x:1:1:bin:/bin:/sbin/nologin
     3  daemon:x:2:2:daemon:/sbin:/sbin/nologin
     4  adm:x:3:4:adm:/var/adm:/sbin/nologin
     5  lp:x:4:7:lp:/var/spool/lpd:/sbin/nologin
     6  sync:x:5:0:sync:/sbin:/bin/sync
     7  shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown
     8  halt:x:7:0:halt:/sbin:/sbin/halt

# cat /etc/passwd | grep fedora
fedora:x:1001:1001::/home/fedora:/bin/bash
# cat -n /etc/passwd | grep fedora
48      fedora:x:1001:1001::/home/fedora:/bin/bash
                 라인번호와 UID, GID는 차이가 있을 수 있습니다.
```

[EX2] file1, file2 두 개의 파일을 하나의 file3으로 합치기

```
# cd /test
# rm -rf /test/*
# echo 1111 > file1
# echo 2222 > file2
# cat file1 file2
1111
2222
# cat file1 file2 > file3
# cat file3
1111
2222
```

[참고] cat 명령어를 사용하여 바이너리 파일 확인 시

```
바이너리 파일을 cat 명령어로 보는 경우 비프(bepp)음이 계속 나오게 되며 이러한 경우 터미널 창을 종료 해야 함.

ex) # cat /bin/ls                          /* 바이너리 파일은 cat으로 열지 않는다 */
    # strings -f /bin/ls


미니멀 버전 : # dnf -y install binutils
```

```
# man strings
NAME
       strings - print the strings of printable characters in files.

SYNOPSIS
       strings [-afov] [-min-len]
               [-n min-len] [--bytes=min-len]
               [-t radix] [--radix=radix]
               [-e encoding] [--encoding=encoding]
               [-] [--all] [--print-file-name]
               [-T bfdname] [--target=bfdname]
               [--help] [--version] file...

DESCRIPTION
       For  each  file  given,  GNU  strings  prints  the  printable character
       sequences that are at least 4 characters long (or the number given with
       the  options  below)  and are followed by an unprintable character.  By
       default, it only prints the strings from  the  initialized  and  loaded
       sections  of  object  files;  for  other  types of files, it prints the
       strings from the whole file.

       strings is mainly useful  for  determining  the  contents  of  non-text
       files.

OPTIONS
       -f
       --print-file-name
           Print the name of the file before each string.
```

확실하지 않은 파일의 경우는 파일의 형식을 확인해본다

```
# cd /
# ls -F
bin@  boot/  data1/  data2/  data3/  data4/  dev/  etc/  home/  lib@  lib64@  media/  mnt/
opt/  proc/  restore/  root/  run/  sbin@  srv/  sys/  test/  tmp/  usr/  var/

# file *
bin:     symbolic link to `usr/bin'
boot:    directory
data1:   directory
data2:   directory
data3:   directory
data4:   directory
dev:     directory
etc:     directory
home:    directory
lib:     symbolic link to `usr/lib'
lib64:   symbolic link to `usr/lib64'
media:   directory
mnt:     directory
opt:     directory
proc:    directory
restore: directory
root:    directory
run:     directory
sbin:    symbolic link to `usr/sbin'
srv:     directory
sys:     directory
test:    directory
tmp:     sticky directory
usr:     directory
var:     directory
```

more 명령어

큰 파일을 출력할 때 화면 크기 페이지 단위로 출력하며 하단에 "--More--(20%)"는 현재 내용을 20% 보았고 80% 남았다고 표현하며 화면에서 엔터(Enter)키를 누르면 한 개의 라인(line) 단위로 넘어가고 스페이스(space) 키를 누르면 한 페이지 단위로 넘어가는데 less 명령과 함께 사용 하면 더 효율적이다.

[명령어 형식]

```
# more file1      /* file1 파일을 출력 */
# more -c file1   /* file1 파일을 한행씩 지우면서 출력 (more와 같은 형태로 출력~) */
```

[명령어 옵션]

옵션 : 설명

-n(숫자) : 출력 행수를 지정
-c : 위에서부터 한 행씩 지운 후 한 행씩 출력
-d : 스페이스나 q를 누르라는 프롬프트를 출력
-f : 보통은 긴 칼럼의 행은 화면에서 행 바꿈을 하여 새로운 행으로 계산되는데
      -f 옵션은 새로운 행으로 계산 하지 않으며 화면이 행이 아닌 논리적인 행 수를 계산
-s : 여러 개의 빈 공백행은 하나로 취급
-p : 스크롤하지 않으며 화면을 지우고 출력
-u : 밑줄 치기를 금지

[참고] cat & more CMD 차이점

```
# cat /etc/services
# more /etc/services
```

cat명령어의 경우에는 모든 출력결과가 한번에 표시되어 지나가기 때문에 확인이 어렵고 more 명령어어 경우에는 한 페이지씩 걸려 확인 가능하기 때문에 모든 내용을 정상적으로 확인할수 있다.

[EX1] "CMD | more" 형식 실습

```
# CMD       (EX: # help)              /* 쉘 내부(내장) 명령어의 목록 확인 */
# CMD | more (EX: # help | more)

# ps -ef | more
# cat /etc/services | more
# rpm -qa | more
```

```
# cat /etc/services | more / * 상단부터 출력한다 */
----------------------------------------------------
# /etc/services:
# $Id: services,v 1.42 2006/02/23 13:09:23 pknirsch Exp $
#
# Network services, Internet style
#
# Note that it is presently the policy of IANA to assign a single well-known
# port number for both TCP and UDP; hence, most entries here have two entries
# even if the protocol doesn't support UDP operations.
# Updated from RFC 1700, ``Assigned Numbers'' (October 1994).  Not all ports
# are included, only the more common ones.
--More--
----------------------------------------------------

# more /etc/services
----------------------------------------------------
# /etc/services:
# $Id: services,v 1.42 2006/02/23 13:09:23 pknirsch Exp $
#
# Network services, Internet style
#
# Note that it is presently the policy of IANA to assign a single well-known
# port number for both TCP and UDP; hence, most entries here have two entries
# even if the protocol doesn't support UDP operations.
# Updated from RFC 1700, ``Assigned Numbers'' (October 1994).  Not all ports
# are included, only the more common ones.
--More--(0%)                          /* 0% 확인 */
----------------------------------------------------
```

[참고] more 안에서 사용할 수 있는 여러 가지 키보드 키 값

more화면 창에서 h 또는 ?를 치면 아래와 같은 화면이 나오며 여러 가지 기능을 확인 할 수 있다.

```
Star (*) indicates argument becomes new default.
--------------------------------------------------------------------------------
<space>                 Display next k lines of text [current screen size]
z                       Display next k lines of text [current screen size]*
<return>                Display next k lines of text [1]*
d or ctrl-D             Scroll k lines [current scroll size, initially 11]*
q or Q or <interrupt>   Exit from more
s                       Skip forward k lines of text [1]
f                       Skip forward k screenfuls of text [1]
b or ctrl-B             Skip backwards k screenfuls of text [1]
'                       Go to place where previous search started
=                       Display current line number
/<regular expression>   Search for kth occurrence of regular expression [1]
n                       Search for kth occurrence of last r.e [1]
!<cmd> or :!<cmd>       Execute <cmd> in a subshell
v                       Start up /usr/bin/vi at current line
ctrl-L                  Redraw screen
:n                      Go to kth next file [1]
:p                      Go to kth previous file [1]
:f                      Display current file name and line number
.                       Repeat previous command
--------------------------------------------------------------------------------

spacebar : 매뉴얼 페이지에서 한 화면 단위로 넘어 갈 때 사용
Enter : 매뉴얼 페이지에서 한 라인씩 넘어 갈 때 사용
b : Back Screen, 한 화면 전 화면으로 넘어갈 때 사용
/pattern : 특정한 패턴을 빨리 찾을 때 사용
n : Next, 특정한 패턴을 찾은 후 다음 번째 똑같은 문자열을 찾을 때 사용
h : help 매뉴얼 페이지 안에서 사용 할 수 있는 명령어 소개
q : quit, 빠져나옴
```

head 명령어

파일의 처음 시작 부분의 몇 줄을 출력 하고 자 할 때 사용. 따라서 긴 파일의 내용의 앞 부분만을 출력 하고자 할 때 유용하게 사용 되며 head 명령어에 아무런 옵션 없이 사용된 경우 문서의 처음 10줄을 보여 준다.

[명령어 형식]

```
# head /etc/passwd            (# head -10 /etc/passwd, # head -n 10 /etc/passwd)
# head -n 5 /etc/passwd/*     숫자에 해당하는 라인 번호 수 만큼만 출력 (기본은 10줄) */
# head -c 10 /etc/passwd/*    -c 옵션 다음에 오는 숫자 byte 수에 해당하는 만큼 출력 */
```

[명령어 옵션]

옵션 : 설명

-n (숫자) : 위쪽 행에서부터 출력할 행수를 지정
-c (숫자) : byte 수 만큼만 출력

[EX1] "head -n #" 실습

```
# head -n 5 /etc/passwd    (# head -5 /etc/passwd)
root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/sbin/nologin
daemon:x:2:2:daemon:/sbin:/sbin/nologin
adm:x:3:4:adm:/var/adm:/sbin/nologin
lp:x:4:7:lp:/var/spool/lpd:/sbin/nologin
```

```
# head -c 10 /etc/passwd
root:x:0:0
```

[EX2] head 명령어를 이용한 프로세스의 헤더 부분 출력

```
# ps -ef
# ps -ef | more
# ps -ef | grep ssh
root       1488      1  0 16:51 ?        00:00:00 /usr/sbin/sshd -D
root       4092   1488  0 18:58 ?        00:00:00 sshd: root@pts/0
root         4354    4217   0 19:00 ?          00:00:00 /usr/bin/ssh-agent /bin/sh -c exec -l
/bin/bash -c "env GNOME_SHELL_SESSION_MODE=classic gnome-session --session gnome-classic"
root       6513   1488  0 19:48 ?        00:00:00 sshd: root@pts/1
root       7119   6518  0 20:30 pts/1    00:00:00 grep --color=auto ssh
```

각각의 출력 내용이 무엇인지 정확히 알 수 없다.

```
# ps -ef | head -1
UID        PID  PPID  C STIME TTY          TIME CMD
```

```
# ps -ef | head -1 ; ps -ef | grep sshd  ( ; =  하나의 명령줄에 다수의 명령을 순차적으로 실행
할 때)
UID         PID   PPID  C STIME TTY          TIME CMD
root       1266      1  0 12:38 ?        00:00:00 /usr/sbin/sshd -D
root       7878   1266  0 14:24 ?        00:00:00 sshd: root@pts/1
root       8047   7880  0 14:32 pts/1    00:00:00 grep --color=auto sshd
-----------------------------------------------------------------------

출력 결과 분할
ps -ef | head -1 명령어의 결과
UID         PID   PPID  C STIME TTY          TIME CMD

ps -ef | grep sshd 명령어의 결과
root       1266      1  0 12:38 ?        00:00:00 /usr/sbin/sshd -D
root       7878   1266  0 14:24 ?        00:00:00 sshd: root@pts/1
root       8047   7880  0 14:32 pts/1    00:00:00 grep --color=auto sshd

ps -ef | head -1 ; ps -ef | grep sshd   두 명령어를 한번에 출력
```

```
# alias pps='ps -ef | head -1 ; ps -ef | grep $1'   /* $1 : 첫번째 인자(Argument) */
# pps sshd    (# pps syslogd)
UID         PID   PPID  C STIME TTY          TIME CMD
root       1266      1  0 12:38 ?        00:00:00 /usr/sbin/sshd -D
root       7878   1266  0 14:24 ?        00:00:00 sshd: root@pts/1
root       8047   7880  0 14:32 pts/1    00:00:00 grep --color=auto sshd
```

```
# vi ~/.bashrc (# gedit ~/.bashrc)
..... (중략) .....
#
# Sfecific Configuration
#
alias pps='ps -ef | head -1 ; ps -ef | grep $1'
```

```
# vi ~/.bashrc (# gedit ~/.bashrc)
..... (중략) .....
#
# Sfecific Configuration
#
alias pps='ps -ef | head -1 ; ps -ef | grep $1'
```

```
# . ~/.bashrc   (# source ~/.bashrc)
# pps syslogd
-> 정상 출력 내용 확인
```

[참고] Unix 계열의 초창기 head와 유사한 명령어

```
# sed 5q /etc/passwd        /* 파일의 5줄을 출력하고, 끝내라(q)는 뜻 */
root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/sbin/nologin
daemon:x:2:2:daemon:/sbin:/sbin/nologin
adm:x:3:4:adm:/var/adm:/sbin/nologin
lp:x:4:7:lp:/var/spool/lpd:/sbin/nologin
# sed -n '1,5p' /etc/passwd
```

tail 명령어

tail은 텍스트파일이나 지정된 데이터의 마지막 몇 줄을 보여주는 데 사용하는 Unix 및 Unix계열 시스템

에서의 프로그램이다. 파일의 끝 부분만 출력 하고자 할 때 사용하며, 아무런 옵션 없이 사용된 경우 문서의 마지막 10줄을 보여는데, 예를 들어서 사용자가 추가되면 /etc/passwd 파일에 마지막에 추가 된다. 이때 tail 명령어에 -1 옵션을 사용하여 사용자 추가를 확인 할 수 있다.

```
file1
+----------+
|          |
|          |
|          |
|          |
|          |
+----------+
```

[명령어 형식]

```
# tail /etc/passwd     (# tail -10 /etc/passwd, # taill -n 10 /etc/passwd)
# tail -5 /etc/passwd
# tail -n +5 /etc/passwd < 상단의 라인에서 4개의 라인을 제거하고 나머지를 출력
# tail -f /var/log/messages
```

[명령어 옵션]

옵션 : 설명

-c (숫자) : 끝에서부터 지정된 수만큼의 바이트에 해당하는 정보를 보여준다.

-f : 파일의 크기가 변할때마다 추가된 정보를 출력한다.

-F : 위 -f옵션의 경우 파일크기가 변하여 5Mbytes 정도 되면 확장자에 숫자를 붙여 백업파일을 생성하고, 다시 본 파일은 0byte 부터 저장된다. 그래서 tail -f 파일명으로 실행중인 명령이 멈춰버린다는 문제가 생겨 재실행시켜줘야 하는 번거로움이 있다. -F옵션을 사용하면 이런 재실행문제 및 용량변화로 인한 문제를 걱정하지 않아도 된다. -f옵션때와 마찬가지로 Ctrl+C로 빠져나올 수 있다.

-(숫자) : 끝에서부터 지정된 수만큼의 줄을 보여준다.

-q : 출력결과에서 맨 윗줄에 입력파일명을 표시하지 않게 설정한다.

-v : -q와 반대로 출력결과에서 맨 윗줄에 입력파일명을 항상 표시해준다.

--help : 도움말을 보여준다.

--version : 버젼 정보를 보여준다.

[EX1] tail 명령어의 기본 사용법

```
# tail -n 5 /etc/passwd   (# tail -5 /etc/passwd)
postfix:x:89:89::/var/spool/postfix:/sbin/nologin
sshd:x:74:74:Privilege-separated SSH:/var/empty/sshd:/sbin/nologin
tcpdump:x:72:72::/:/sbin/nologin
user01:x:1000:1000:user01:/home/user01:/bin/bash
fedora:x:1001:1001::/home/fedora:/bin/bash
```

```
# tail -c 15 /etc/passwd
dora:/bin/bash
```

[EX2] 로그 파일 모니터링(EX: tail -f CMD)

tail은 실시간으로 파일의 변화를 감지할 수 있게 해주는 -f 옵션이라는 특별한 명령행 옵션을 가지고 있다.

마지막 몇줄을 출력하고 끝내는 것에 그치지 않고, tail은 그 줄들을 표시하고 파일을 감독한다. 새 줄들이 다른 프로세스에 의해 그 파일에 추가될 때, tail의 –f 옵션은 그 표시또한 실시간으로 업데이트한다. 이 옵션은 특히 로그파일(입출력정보파일)들을 감독할때 유용하다. 다음의 명령구문은 messages라는 파일의 마지막 열줄을 보여주고 새 줄들이 추가되면 그 줄들을 추가하여 보여준다.

[질문] 파일이 30 라인으로 되어져 있는데 파일 중간의 10라인만 출력해 볼 수 있나요?

```
# cp /etc/passwd /test/filename
# cd /test
# head -20 filename | tail -10
  head로 상단에서 20개 라인을 출력하고 PIPE로 받아 tail로 아래에서 10개 라인 출력
# nl filename | head -20 | tail -10
                11  uucp:x:10:14:uucp:/var/spool/uucp:/sbin/nologin
     12  operator:x:11:0:operator:/root:/sbin/nologin
     13  games:x:12:100:games:/usr/games:/sbin/nologin
     14  gopher:x:13:30:gopher:/var/gopher:/sbin/nologin
     15  ftp:x:14:50:FTP User:/var/ftp:/sbin/nologin
     16  nobody:x:99:99:Nobody:/:/sbin/nologin
     17  vcsa:x:69:69:virtual console memory owner:/dev:/sbin/nologin
     18  apache:x:48:48:Apache:/var/www:/sbin/nologin
     19  exim:x:93:93::/var/spool/exim:/sbin/nologin
     20  rpc:x:32:32:Portmapper RPC user:/:/sbin/nologin
```

```
[EX3] 서버를 실시간적으로 모니터링
[TERM1] # top                        (# gnome-system-monitor)
[TERM2] # tail -f /var/log/messages  (# gnome-system-log)

# tail -f /var/log/messages | grep -i DHCP
# tail -f /var/log/messages | grep -i DNS
# tail -f /var/log/messages | grep oracle
# tail -f /var/log/messages | grep wasuser  (특정 서비스에 대한 모니터링)

또 다른 활용방안

# cat /var/log/messages | egrep -i '(warn|err|alert|emerg)'
```

기타 관리용 명령어

- wc 명령어
wc -l /etc/passwd
ps -ef | wc -l
cat /etc/passwd | wc -l
rpm -qa | wc -l

[참고] 데이터수집
ps -ef | grep httpd | wc -l
df -h | tail-1 | awk '{print $5}'
cat /var/log/messages | grep 'START: telnet' | wc -l

---

wc 명령어

파일 내의 문자수, 단어 수 그리고 라인수를 확인하고자 할 때 사용한다. wc 명령어를 사용하여 프로세스의 수, 시스템에 설치된 패치의 수, 시스템에 설치된 패치의 수 등을 확인 할 때 사용 할 수 있다. wc 명령어에 -l 옵션은 쉘스크립트나 파일에 대한 무결성 체크 등 많은 곳에서 활용이 가능하다.

[명령어 형식]

```
# wc /etc/passwd
# wc -l /etc/passwd
# wc -w /etc/passwd
# wc -c /etc/passwd
```

[명령어 옵션]

옵션 : 설명

-c : 문자수만 출력
-l : 라인수만 출력
-w : 단어수만 출력
-L : 가장 긴줄 한줄만 출력

[EX1] wc 명령어 사용법

```
# wc /etc/passwd
59    104   2902    /etc/passwd
# wc -l /etc/passwd
59       /etc/passwd
# wc -w /etc/passwd /* 단어수 (word) */
104       /etc/passwd
# wc -c /etc/passwd /* 문자수 (byte) */
2902      /etc/passwd
```

[EX2] 시스템 사용자 수 확인

```
# cat /etc/passwd | wc -l
59
# wc -l /etc/passwd
59 /etc/passwd
```

[EX3] 실행중인 프로세스의 수 확인

```
# ps -ef | wc -l
98
해당 숫자에서 제일 상단위 헤더를 제외하고 읽는다.8
```

[EX4] 설치된 패키지 수 확인

```
# rpm -qa | wc -l
1451
```

[EX5] 시스템성능/사용량 카운트 수집 : Data Gathering

```
Apache Webserver
- apache 1.3.X(Process 방식)
- apache 2.X  (Thread 방식)

Web Client ------------> Web Server(www.daum.net)
http://www.daum.net      httpd
```

```
# yum -y install httpd
# systemctl restart httpd
# systemctl status httpd
httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled)
   Active: active (running) since 월 2018-03-05 14:54:49 KST; 12s ago
     Docs: man:httpd(8)
           man:apachectl(8)
 Main PID: 9019 (httpd)
   Status: "Total requests: 0; Current requests/sec: 0; Current traffic:   0 B/sec"
   CGroup: /system.slice/httpd.service
....(중략)

-> 에러메세지 무시
```

```
# ps -ef | grep httpd
root       9019      1  0 14:54 ?        00:00:00 /usr/sbin/httpd -DFOREGROUND
apache     9020   9019  0 14:54 ?        00:00:00 /usr/sbin/httpd -DFOREGROUND
apache     9021   9019  0 14:54 ?        00:00:00 /usr/sbin/httpd -DFOREGROUND
apache     9022   9019  0 14:54 ?        00:00:00 /usr/sbin/httpd -DFOREGROUND
apache     9023   9019  0 14:54 ?        00:00:00 /usr/sbin/httpd -DFOREGROUND
apache     9024   9019  0 14:54 ?        00:00:00 /usr/sbin/httpd -DFOREGROUND
```

```
# ps -ef | grep httpd | wc -l > apache.count
# cat apache.count
9
```

[EX6] 디스크 사용량을 모니터링

```
# df -k
Filesystem     1K-blocks    Used Available Use% Mounted on
/dev/sda8       58223848 4624432  53599416   8% /
devtmpfs          926916       0    926916   0% /dev
tmpfs             935380      84    935296   1% /dev/shm
tmpfs             935380    9132    926248   1% /run
tmpfs             935380       0    935380   0% /sys/fs/cgroup
/dev/sda7         508588   25852    482736   6% /home
/dev/sda6         508588   25760    482828   6% /data4
/dev/sda5         508588   25760    482828   6% /data3
/dev/sda3         508588   25760    482828   6% /data2
/dev/sda2         508588   25760    482828   6% /data1
```

```
# df -k /
Filesystem           1K-blocks      Used Available Use% Mounted on
/dev/sda8       58223848 4624432  53599416   8% /
```

```
# df -k / | tail -1
/dev/sda8       58223848 4624432  53599416   8% /
```

```
# df -k / | tail -1 | awk '{print $5}' > df.count
# cat df.count
8%
```

```
# df -k / | tail -1 | awk '{print $5}' | awk -F% '{print $1}' > df.count
# cat df.count
8
```

[참고] awk 명령어

awk는 라인을 받아와서 구분자를 통해 구분하고 print 명령으로 출력하게 된다.

awk 명령어를 사용하기 전에 꼭 알아두어야 할 것은 기본적으로 탭 또는 공백으로 구분되는 각각의 단어들을 하나의 변수로 처리한다는 것이다. 탭과 공백을 무시하고 특정문자 콜론(:)이나, 세미콜론(;) 콤마(,) 등을 구분문자로 사용하고자 한다면 -F 옵션을 사용하면 된다.

```
명령어 형식 = awk [option] [-F] ['{script}']

# cd /test
# touch file1 file2 file3
# ls -l | grep "^-" | awk '{print "vi "$9}'
vi file1
vi file2
vi file3
```

ls -l 명령어로 현재 디렉토리의 목록을 출력하여 나온 결과를 grep 명령을 통해 "^-" 에 해당되는 값을 출력하여 awk 명령으로 "vi "$9 포맷형식으로 출력한 내용을 test.sh 파일에 복사하여 넣어라 라는 것입니다.

기타 관리용 명령어

- su 명령어
su user01
su - user01

- id 명령어
id
id user01

- groups 명령어
groups
groups user01
groups user01 root

su 명령어

다른 사용자의 권한으로 쉘을 실행한다. 서버에 접속한 상태에서 로그아웃 없이 다른 사용자로 전환할 수 있음(windows의 사용자 전환 - terminal service (application)를 자동으로 만들어 놓아야 사용자 전환이 실행가능) 원격 접속시에 다른 사용자로 로그인하기 위해 로그아웃하면 접속이 종료가 되는데 접속이 종료 되지 않은 상태에서 다른 사용자로 로그인하고자 할 때 사용한다.

한명의 관리자가 여러 계정을 사용하는 경우 유용하다.

일반사용자가 다른 사용자가 되는 것을 권한이 높아지는 것이기 때문에 전환(Switching) 되는 사용자의 암호를 맞추어야만 전환이 가능하고, root 사용자가 다른사용자로 전환하는 경우에는 권한이 낮아지는 것이기 때문에 암호입력 없이 전환이 가능하다.

- [ ] (사용자 전환의 예)

일반사용자(user01) ----> 다른 일반사용자(user02)
일반사용자(user01) ----> 관리자(root)
관리자(root) ----> 일반사용자(user01)

su 명령어 다음에 전환하고 싶은 사용자 이름이 없는 경우 root 사용자로 전환된다.
그리고 su 명령어에 '-' 기호 없이 다른 사용자로 전환하는 경우 지정된 사용자로 전환이 되지만 이전 사용자가 쓰고 있던 export 변수들의 설정이 그대로 따라온다. su 명령어에 '-' 기호를 붙이고 다른 사용자로 전환하는 경우 지정된 사용자로 새로 로그인한 것 처럼 동작을 시켜서 사용자 환경변수로 모두 변경한다.(DB를 운영하는 oracle 사용자)

[명령어 형식]

su [fedora]
su - [fedora]

[EX1] su 명령어의 전환 형식 실습

```
# cd /etc
# pwd
/etc
```

```
# su fedroa
$ id
uid=1001(fedora) gid=1001(fedora) groups=1001(fedora)
```

```
$ pwd
/etc
```

```
$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/root/bin
```

```
$ exit

# cd /etc
# pwd
/etc
```

```
# su - fedora
$ id
uid=1001(fedora) gid=1001(fedora) groups=1001(fedora)
```

```
$ pwd
/home/fedora
```

```
$ echo $PATH
/usr/local/bin:/bin:/usr/bin:/usr/local/sbin:/usr/sbin:/home/fedora/.local/bin:/home/fedora/b
in
```

```
$ exit
#
```

[EX2] fedora 사용자가 다른 사용자로 전환하는 경우

```
# ssh fedora@localhost
fedora@localhost's password: (fedora)
Last login: Wed Jan  8 12:34:36 2014 from localhost.localdomain
-> 암호입력
```

```
$ cat /etc/shadow
cat: /etc/shadow: Permission denied
권한이 없기 때문에 파일의 내용을 확인할수 없다.
```

```
$ ls -l /etc/shadow
---------- 1 root root 1276  3월  5 14:54 /etc/shadow
자신의 계정이상으로 권한이 필요한 파일,디렉토리의 접근,수정에 사용한다.
```

```
$ su -        ($ su - root)
Password: (redhat)
-> root 사용자의 암호 입력
```

```
# id
uid=0(root) gid=0(root) groups=0(root)
```

```
# cat /etc/shadow
-> 파일 내용 확인
```

```
# exit
$ exit
#
```

id 명령어

사용자의 uid, gid, groups을 보여줌 (사용자 이름의 길이가 긴 경우 메모리 기억공간의 낭비가 심함. but 정수로 사용자의 이름을 구분하는 경우 메모리 기억공간을 효율적으로 사용가능. 정수로 사용하여 4byte씩 사용될 경우 0 ~ 4294967295명까지 구분이 가능)

[명령어 형식]

```
# id
# id -u root
# id -g root
```

[명령어 옵션]

옵션 : 기능

-g : 기본 그룹의 gid를 출력 (사용자가 소속되어 있는 기본 그룹)
-G : 사용자가 속한 모든 그룹의 gid를 출력
-u : 사용자의 uid를 출력
-n : -u과 함께 사용하여 숫자 대신 이름 출력

[EX] id 명령어 사용

```
# id
uid=0(root) gid=0(root) groups=0(root)
```

```
# id -g root
0
```

```
# id -G root
0
```

```
# id -Gn root
root
```

동일한 방법으로 fedora 계정에 대하여 확인해봅시다.

groups 명령어

[명령어 형식]

```
# groups                          /* 현재 사용중인 사용자의 그룹을 보여줌 */
# groups user1          /* user1의 그룹을 보여줌 */
# groups user1 user2    /* user1, user2의 그룹을 보여줌 */
```

[EX1] groups 명령어 사용법

```
# groups
root
```

```
# groups fedora
fedora : fedora
```

```
# groups fedora root
fedora : fedora
root : root
```

```
# cat /etc/group
root:x:0:root
bin:x:1:root,bin,daemon
daemon:x:2:root,bin,daemon
sys:x:3:root,bin,adm
adm:x:4:root,adm,daemon
..... (중략) ....
sabayon:x:86:
fedora:x:500:
user01:x:501:

-> 시스템에 선언된 모든 그룹들에 대한 기본적인 정보를 표시한다.
```

group에 사용자를 추가할 경우

```
# adduser -G [GROUP] [USER]
```

[참고사항]

```
UNIX 계열 계정이 그룹에 가입할때 그룹의 갯수가 16개 이상이 되면 최초 가입된 그룹부터 삭제
LINUX 계열 리눅스에서는 UNIX에서 일어나는 문제가 없다.
```

기타 관리용 명령어
- last 명령어
last
last user01
last reboot
last -20
last -f /var/log/wtmp.0

- lastlog 명령어
lastlog
lastlog -u user01
lastlog -t 10

- lastb 명령어
lastb
lastb -20

last 명령어

서버를 이용하는 각 계정사용자들의 로그인 정보를 보여주는 명령어이다. 흔히 관리자는 각 계정별로 서버에 접속한 시간과 IP주소 등을 확인해야 할 경우가 있다. 또한 특정 계정의 서버 접속정보를 확인해야 할 때 에도 마찬가지이다. 다양한 방법으로 사용자들의 로그인정보를 조사한다.

[명령어 형식]

```
# last
# last root
# last -5 /* 5행의 결과만을 확인 */
# last -R /* last의 결과에서 호스트(IP주소)접속기록을 제외한 결과만을 출력 */
# last -a /* last의 결과중 호스트(IP주소)정보를 맨 마지막에 출력 */
```

[명령어 옵션]

옵션 : 설명

-n : (num) 지정한 num 만큼의 줄만 보여준다.
-f : (file) 지정한 파일에서 정보를 불러온다.
-R : 보여주는 목록에서 hostname(IP주소)필드는 보여주지 않는다.
-a : 보여주는 목록에서 hostname(IP주소)필드를 마지막 필드에 보여준다.
-d : 다른 호스트에서 접속한 것만 보여준다.
-x : shutdown이 일어난 상태나, run level이 바뀐 상태를 보여준다.

[EX1] last 명령어 사용

[TERM1] fedora 사용자의 터미널

```
# ssh fedora@localhost
-> fedora 사용자의 암호 입력
$ tty
/dev/pts/#
```

[TERM2] 관리자 터미널

```
# last                /* 최근기록이 상단에 출력된다. head 명령어와 같이 사용된다 */
fedora   pts/2        192.168.0.1      Wed Feb 10 10:27   still logged in
root     pts/1        192.168.0.1      Wed Feb 10 10:25   still logged in
reboot   system boot  2.6.18-164.el5   Wed Feb 10 10:23          (00:08)
root     pts/2        linux200         Wed Jan 27 14:11 - 14:28  (00:16)
root     pts/1        192.168.0.1      Wed Jan 27 14:11 - 14:28  (00:17)
fedora   pts/2        192.168.0.1      Wed Jan 27 13:52 - 14:10  (00:18)
root     pts/1        192.168.0.1      Wed Jan 27 13:40 - 14:10  (00:29)
root     pts/4        :0.0             Wed Jan 27 13:39 - 13:39  (00:00)
root     pts/3        192.168.0.1      Wed Jan 27 13:30 - 13:39  (00:09)
root     pts/3        192.168.0.1      Wed Jan 27 13:29 - 13:30  (00:00)
root     :0                            Wed Jan 27 13:27 - crash (13+20:55)
root     :0                            Wed Jan 27 13:27 - 13:27  (00:00)
```

출력 내용 설명

```
# last -5
fedora   pts/2        192.168.10.1     Fri Mar 13 10:08   still logged in
root     pts/1        192.168.10.1     Fri Mar 13 10:04   still logged in
fedora   pts/1        localhost        Thu Mar 12 16:55 - 17:00  (00:04)


root            LOGIN 이름(어떤 계정으로 접속하였는지 확인)
pts/1           터미널 번호
192.168.10.1    접속지 IP

Fri Mar 13 10:04 , Wed Jan 27 14:11 - 14:28  LOGIN 시간을 나타낸다. 만약 로그아웃까지 완료하
였으면 (14:11 - 14:28)의 형식으로 출력된다. 시간은 분단위로 측정되지만 절대 시간으로 초단위로
계산 되는 것이 아니기 때문에 실제 접속 시간이 59초에 접속하는 경우 1분 정도 더 길게 시간이 측
정 될 수 있다.
still logged in  현재 로그인중. 만약 로그아웃까지 완료하였으면 (00:04 작업 진행시간)으로 출력
된다.
```

```
# last -5                        (# last | head -5)/* 상위 5개 라인 출력 */
fedora   pts/2        192.168.0.1      Wed Feb 10 10:27   still logged in
root     pts/1        192.168.0.1      Wed Feb 10 10:25   still logged in
reboot   system boot  2.6.18-164.el5   Wed Feb 10 10:23          (00:09)
root     pts/2        linux200         Wed Jan 27 14:11 - 14:28  (00:16)
root     pts/1        192.168.0.1      Wed Jan 27 14:11 - 14:28  (00:17)

wtmp begins Tue Jan 26 01:25:16 2010

상위라인으로 출력하는 이유는 최신의 정보가 위에서 부터 쌓이고 있다.
```

[EX2] /var/log/wtmp.# 파일 읽기

```
# file /var/log/wtmp                                  /* data 파일 타입을 가지고 있다 */
/var/log/wtmp: data
```

```
# file /etc/passwd                                    /* text 파일의 경우에 확인 가능 */
/etc/passwd: ASCII text
```

```
# cat /var/log/wtmp                                   /* 파일을 열수 있는 형식 아님 */
췕:0root?rKNJ췕:0:0root?r?
pts/1R.rKP(mpts/3S.smpts/4S.rK?쫘pts/1ts/1root192.168.0.1^.rKNp웸npts/1홂rKblpts/2JNrK懲
...... (중략) ......
```

```
# last -f /var/log/wtmp    (# last)
root     pts/1        192.168.0.1      Wed Feb 10 15:47   still logged in
root     pts/1        192.168.0.1      Wed Feb 10 12:56 - 14:40  (01:44)
root     :0                            Wed Feb 10 12:44   still logged in
root     :0                            Wed Feb 10 12:44 - 12:44  (00:00)
root     pts/4        linux200         Wed Feb 10 12:43 - 12:56  (00:12)
root     pts/3        linux200         Wed Feb 10 12:32 - 12:56  (00:23)
root     pts/2        192.168.0.1      Wed Feb 10 11:53 - 15:12  (03:18)
fedora   pts/2        192.168.0.1      Wed Feb 10 10:27 - 11:33  (01:05)
root     pts/1        192.168.0.1      Wed Feb 10 10:25 - 12:56  (02:30)
```

[참고] last -f 옵션 사용하는 경우

```
# cd /var/log
# ls wtmp*
wtmp  wtmp.0  wtmp.1  wtmp.2 ....

# last   -----> /var/log/wtmp
# last -f /var/log/wtmp.0  < 정책에 의해 로그가 일정 기간이 지나면 다른 파일로 저장된다 >
/etc/logrotate.conf 파일에 의해 결정
기간이 지난 파일의 로그인기록을 확인할 경우 사용한다.
```

[EX3] last 명령어의 사용 예

```
(개발자 요청 내용) 어제 파일(예: file.log)을 삭제한 사용자를 검색해 달라.

(정보1) 어제 파일이 지워졌다.
# last | grep 'Nov 30'
(정보2) 지워진 파일의 이름 : file.log
# cat ~/.bash_history
# cat ~/.bash_history | grep 'file.log' | grep rm
```

[작업 준비]

```
# cd /test
# chmod 777 /test
# touch file.log

# ssh fedora@localhost
fedora 사용자로 로그인

$ rm -rf /test/file.log
$ ls /test
-> file.log 파일이 지워졌는지 확인

$ exit
```

```
# last | grep 'Sep  9'
fedora   pts/4        linux249         Fri Sep  9 10:29 - 10:30  (00:00)
root     pts/3        :0.0             Fri Sep  9 10:11   still logged in
fedora   pts/2        linux249         Fri Sep  9 10:11   still logged in
root     pts/2        linux249         Fri Sep  9 10:10 - 10:10  (00:00)
root     pts/1        :0.0             Fri Sep  9 10:00   still logged in
root     pts/1        :0.0             Fri Sep  9 09:59 - 10:00  (00:00)
root     :0                            Fri Sep  9 09:59   still logged in
root     :0                            Fri Sep  9 09:59 - 09:59  (00:00)
reboot   system boot  2.6.18-164.el5   Fri Sep  9 09:56          (00:35)
```

```
# egrep -l "file.log" /home/*/.bash_history
/home/fedora/.bash_history
```

```
# egrep "file.log" /home/fedora/.bash_history
rm -rf /test/file.log
```

lastlog 명령어

사용자의 마지막 로그인 정보만을 출력 해준다. lastlog는 /var/log/lastlog라는 파일의 내용을 출력해 준다. 로그인할 때 마지막 로그인 정보가 출력되는데 이때 출력되는 정보가 /var/log/lastlog의 정보이다. 사용자 계정을 지우게 되면 /var/log/lastlog 파일에도 사용자계정에 해당 정보도 삭제된다.

[명령어 형식]

```
# lastlog
# lastlog -u feodra /* 지정한 로그인명의 lastlog 정보만을 보여준다. */
# lastlog -t 3 /* 지정한 날짜기간 안에 로그인한 정보만을 보여준다 */
```

[명령어 옵션]

옵션 : 설명

-u : 지정된 사용자의 lastlog 기록을 보여줌
-t : 지정된 날짜기간 안의 로그인 정보만 출력해 준다.

[EX1] lastlog 명령어 사용법

```
# telnet localhost (# ssh localhost)
login as: root
root@192.168.0.200's password:
Last login: Wed Feb 10 10:25:22 2010 from 192.168.0.1
# exit
```

```
# lastlog
Username         Port     From             Latest
root             pts/2    linux249         Fri Sep  9 10:41:24 +0900 2011
bin                                        Never logged in
daemon                                     Never logged in
adm                                        Never logged in
lp                                         Never logged in
sync                                       Never logged in
shutdown                                   Never logged in
halt                                       Never logged in
mail                                       Never logged in

..... (중략) .....

rpcuser                                    Never logged in
named                                      Never logged in
hsqldb                                     Never logged in
sshd                                       Never logged in
haldaemon                                  Never logged in
avahi-autoipd                              Never logged in
xfs                                        Never logged in
gdm                                        Never logged in
sabayon                                    Never logged in
fedora           pts/4    linux249         Fri Sep  9 10:29:54 +0900 2011
user01           pts/3    linux249         Thu Sep  8 10:26:39 +0900 2011
```

출력내용설명

```
Username         Port     From             Latest
root             pts/2    linux249         Fri Sep  9 10:41:24 +0900 2011
sabayon                                    Never logged in

Username     계정이름
Port         터미널 정보
From         접속을 시도한 호스트
Latest       최종 접속 일자,시간  ( Never logged in)> 단 한번도 접속한 적이 없다는 의미
```

```
# lastlog -u fedora
Username         Port     From             Latest
fedora           pts/2    192.168.0.1      Wed Feb 10 10:27:47 +0900 2010
```

lastb 명령어

접속 실패 로그를 출력해준다. /var/log/btmp파일에 로그가 저장된다. cat이라는 명령어를 통해서 안의 내용을 열어보면 파일의 내용이 제대로 나오지 않는다. 이 파일의 내용을 보기 위해서 우리가 사용하는 명령어가 바로 lastb라는 명령어다. 보통 블루투스 공격(임의의 사용자 계정과 임의의 패스워드로 내 시스템에 접속하려는 것)을 당하는지, 당했는지와 어떤 사용자가 잘못 로그인을 했는지와 같은 정보를 확인 할 수 있다.

[명령어 형식]

```
# lastb
```

[명령어 옵션]

옵션 : 설명

-n : (num) 지정한 num 만큼의 줄만 보여준다.
-f : (file) 지정한 파일에서 정보를 불러온다.
-R : 보여주는 목록에서 hostname(IP주소)필드는 보여주지 않는다.
-x : shutdown이 일어난 상태나, run level이 바뀐 상태를 보여준다.

[EX1] lastb 명령어 실습

(전제 조건) user01 사용자가 존재해야 한다.

```
[192.168.10.252]
# cat /etc/passwd | grep user01
# useradd user01
# passwd user01
```

```
# ssh user01@192.168.10.252
user01@192.168.10.252 is password:  <----- 잘못된 암호 입력
Permission denied, please try again.
user01@192.168.10.252 is password:  <----- 잘못된 암호 입력
Permission denied, please try again.
user01@192.168.10.252's password:   <----- 잘못된 암호 입력
Permission denied (publickey,gssapi-keyex,gssapi-with-mic,password).
```

```
192.168.10.252로 root 접속
# lastb
user01   ssh:notty    localhost.locald Sat Mar 23 15:49 - 15:49  (00:00)
user01   ssh:notty    localhost.locald Sat Mar 23 15:49 - 15:49  (00:00)
user01   ssh:notty    localhost.locald Sat Mar 23 15:49 - 15:49  (00:00)
rppt     :0                            Wed Feb 13 17:48 - 17:48  (00:00)
root     ssh:notty    172.16.9.1       Mon Feb  4 15:35 - 15:35  (00:00)
root     ssh:notty    172.16.9.1       Mon Feb  4 15:35 - 15:35  (00:00)
ftp      ssh:notty    172.16.9.1       Mon Feb  4 15:35 - 15:35  (00:00)
..... (중략) .....
```

```
# lastb -5             /* 지정된 만큼만 기록을 보여줌 */
user01   ssh:notty    localhost.locald Sat Mar 23 15:49 - 15:49  (00:00)
user01   ssh:notty    localhost.locald Sat Mar 23 15:49 - 15:49  (00:00)
user01   ssh:notty    localhost.locald Sat Mar 23 15:49 - 15:49  (00:00)
rppt     :0                            Wed Feb 13 17:48 - 17:48  (00:00)
root     ssh:notty    172.16.9.1       Mon Feb  4 15:35 - 15:35  (00:00)

btmp begins Wed Apr 11 13:33:02 2012
```

```
# lastb | grep user01
user01   ssh:notty    localhost.locald Sat Mar 23 15:49 - 15:49  (00:00)
user01   ssh:notty    localhost.locald Sat Mar 23 15:49 - 15:49  (00:00)
user01   ssh:notty    localhost.locald Sat Mar 23 15:49 - 15:49  (00:00)
```

```
# lastb | grep user01 | wc -l
3
```

기타 관리용 명령어

- who 명령어
who

- w 명령어
w
w user01
while true
> do
> w user01
> sleep 2
> done

who 명령어

who 명령을 통해 누가 로그인해 있는지, 어떤 장치를 이용하고 있는지, 언제 로그인했는지, 어디에서 로그인했는지 등의 정보를 알 수 있다. /var/run/utmp의 내용을 보여줌!

[명령어 형식]

```
# who      /* 현재 시스템에 접속 중인 모든 사용자 */
# who -r   /* 현재 사용자의 Runlevel 확인 */
# who am i /* 로그인한 사용자 정보 확인 */
# who -H   /* 헤드라인과 같이 출력 */
# whoami   /* 현재 사용자명 확인 , 유효사용자를 확인한다*/
```

[명령어 옵션]

옵션 : 기능

-i : idle time 과 함께 사용자 출력
-m : who 명령을 실행한 사용자 표시

-q : 사용자 이름과 사용자수 출력
-w, -T : 각 사용자의 메시지 설정 상태 출력
-H : 헤드라인 정보 표시
-r : run-level 확인

[EX1] who 명령어 실습

192.168.10.252 서버에 설치되어 있는 telnet 서비스로 접속을 하여 현재 사용자에 대하여 확인해보자.

[TERM1] fedora 사용자의 윈도우

```
# telnet 192.168.10.252
-> fedora 사용자로 로그인

$ id
uid=1001(fedora)                    gid=1001(fedora)                    groups=1001(fedora)
context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
```

```
$ pwd
/home/fedora
```

```
$ tty
/dev/pts/[NUMBER]
[NUMBER]는 부여받는 터미널 번호에 따라 다르다.
```

[TERM2] root 사용자의 윈도우

```
# who
root     pts/1        Feb 10 15:47 (192.168.0.1)          /* Putty로 로그인하여 작업중 */
root     pts/2        Feb 10 16:34 (192.168.0.1)
fedora   pts/[NUMBER] Feb 10 22:50 (192.168.0.1)
root     :0           Feb 10 12:44
root     pts/3        Feb 10 21:33 (192.168.0.1)
```

```
(명령어 출력 결과 해석)
-------------------+-------------------------
필드 설명            |
-------------------+-------------------------
root               | 사용자 정보
pts/1              | 제어 터미널
Feb 10 15:47       | 로그인 시간
192.168.0.1        | 원격호스트
-------------------+-------------------------
```

[TERM1] fedora 사용자의 윈도우

```
$ exit
#
```

[EX2] who 명령어 실습

```
# ssh root@192.168.10.252
The authenticity of host '192.168.10.252 (192.168.10.252)' can't be established.
RSA key fingerprint is 3b:25:5b:9a:ae:0f:ba:fc:85:66:73:d4:fc:e2:78:c6.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '192.168.10.252' (RSA) to the list of known hosts.
root@172.16.9.249's password: (centos)
Last login: Thu Jan  9 11:38:41 2014 from 192.168.10.200
```

```
# hostname
server2.example.com
```

```
# id
uid=0(root)                         gid=0(root)                         groups=0(root)
context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
```

```
# tty
/dev/pts/4
```

```
# who
root     :0           2014-02-12 22:29
root     pts/1        2014-02-12 22:29 (:0.0)
root     pts/2        2014-02-13 10:29 (linux205.example.com)
root     pts/3        2014-02-13 10:31 (linux206.example.com)
root     pts/4        2014-02-13 10:29 (linux249.example.com)
root     pts/5        2014-02-13 10:29 (linux208.example.com)
root     pts/6        2014-02-13 10:29 (linux218.example.com)
root     pts/7        2014-02-13 10:29 (linux201.example.com)
root     pts/8        2014-02-13 10:30 (linux207.example.com)
root     pts/9        2014-02-13 10:30 (linux219.example.com)
root     pts/10       2014-02-13 10:30 (linux214.example.com)
root     pts/11       2014-02-13 10:31 (linux200.example.com)
..... (중략) .....

# exit
#
```

w 명령어

시스템에 login한 사용자가 어떤 명령어를 실행하고 있는지 알아보는 명령어이며, /proc 디렉토리로 부터 사용자에 대한 정보와 실행중인 명령어에 대한 정보를 추출해 낸다.

[명령어 형식]

```
# w
# w user01
```

```
# w
10:34:14 up 28 min,  3 users,  load average: 0.00, 0.01, 0.01
USER     TTY      FROM              LOGIN@   IDLE   JCPU   PCPU WHAT
root     :0                        10:07   ?xdm?  12.60s  0.19s /usr/bin/gnome-ses
root     pts/1    :0.0             10:10    0.00s  0.03s  0.00s w
root     pts/3    :0.0             10:27    7:00   0.02s  0.02s bash
```

[EX1] w 명령어 실습

[TERM1] root 사용자 윈도우

```
-> putty 프로그램을 통해 리눅스 서버(192.168.10.252)에 접속
login : root
passwd : (redhat)
```

[TERM2] fedora 사용자 윈도우

```
# ssh fedora@192.168.10.252
fedora@192.168.10.252's password: (fedora)
Activate the web console with: systemctl enable --now cockpit.socket

Last login: Wed Aug 26 10:01:21 2020 from 192.168.10.252
```

[TERM3] user01 사용자 윈도우

```
# telnet localhost
user01 사용자로 로그인
```

```
$ vi /etc/passwd
```

[TERM4] root 사용자 윈도우(분석하는 터미널)

```
# w
10:43:47 up 38 min,  7 users,  load average: 0.00, 0.03, 0.01
USER     TTY      FROM              LOGIN@   IDLE   JCPU   PCPU WHAT
root     :0                        10:07   ?xdm?  14.78s  0.19s /usr/bin/gnome-ses
root     pts/1    :0.0             10:10    2:56   0.03s  0.00s ssh fedora@localho
root     pts/3    :0.0             10:27   48.00s  0.02s  0.00s telnet localhost
root     pts/2    172.16.9.1       10:39    3:43   0.01s  0.01s -bash
fedora   pts/4    localhost.locald 10:40    2:51   0.01s  0.01s -bash
user01   pts/5    localhost.locald 10:42   48.00s  0.04s  0.03s vim /etc/passwd
root     pts/6    :0               10:43    0.00s  0.01s  0.00s w
```

[EX2] 악의적인 사용자 로그아웃 시키기

[TERM4] 관리자 윈도우

```
# w
10:43:47 up 38 min,  7 users,  load average: 0.00, 0.03, 0.01
USER     TTY      FROM              LOGIN@   IDLE   JCPU   PCPU WHAT
root     :0                        10:07   ?xdm?  14.78s  0.19s /usr/bin/gnome-ses
root     pts/1    :0.0             10:10    2:56   0.03s  0.00s ssh fedora@localho
root     pts/3    :0.0             10:27   48.00s  0.02s  0.00s telnet localhost
root     pts/2    172.16.9.1       10:39    3:43   0.01s  0.01s -bash
fedora   pts/4    localhost.locald 10:40    2:51   0.01s  0.01s -bash
user01   pts/5    localhost.locald 10:42   48.00s  0.04s  0.03s vim /etc/passwd
root     pts/6    :0.0             10:43    0.00s  0.01s  0.00s w
```

```
# w user01
10:48:23 up 42 min,  7 users,  load average: 0.00, 0.02, 0.00
USER     TTY      FROM              LOGIN@   IDLE   JCPU   PCPU WHAT
user01   pts/5    localhost.locald 10:42    5:24   0.04s  0.03s vim /etc/passwd
```

```
# while true
> do
> echo
> echo "============`date`============"
> w user01
> sleep 2
> done
```

```
# ps -u user01    (# ps -fu user01)
PID TTY          TIME CMD
 8064 pts/5    00:00:00 bash
 8100 pts/5    00:00:00 vim
```

```
# kill -9 8064
#
```

[참고] 원격서버에 접속시

```
원격서버에 접속하여 접속한 서버의 이름, 사용자, 작업 디렉토리 확인
# ssh 172.16.9.252
root 사용자로 로그인

# hostname
# id
# pwd
```

[참고] 관리 서버에 접속시

```
자신이 관리하는 서버에 접속하여 오늘 로그인/로그아웃 사용자 정보 확인
현재 접속되어져 있는 사용자 확인
현재 접속된 사용자가 수행하는 명령어 확인
# last
# who
# w
```

exit 명령어

현재의 프로세스(현재쉘)를 종료한다.

[명령어 형식]

```
# exit [Number] /* 값을 지정해준다면 0은 정상종료, 1~255는 비정상 종료 */
```

[EX1] 서브 쉘 종료 해보자

```
# ps
PID TTY          TIME CMD
21224 pts/11   00:00:00 bash
```

```
# bash
```

```
# ps
PID TTY          TIME CMD
21224 pts/11   00:00:00 bash
21316 pts/11   00:00:00 bash
```

```
# exit
exit
```

```
# ps
PID TTY          TIME CMD
21224 pts/11   00:00:00 bash
```

[EX2] 서버에 접속 후 로그 아웃

```
# ssh 172.16.9.252
root 사용자로 접속

# hostname
# id
# pwd

# ps
# exit
#
```

[EX3] 사용자 전환후에 원래 사용자로 전환

```
# su - user01
$ ps
$ exit
#
```

[참고]

```
쉘 스크립트 작성시에 사용되는 exit
상태값 지정
```

<윈도우 환경과 비교하며 리눅스 환경 이해하기>
서버에서의 다수 사용자에 대한 이해
파일에 대한 정의와 이해

리눅스 파일 관리
파일종류

단원목표

- 파일의 구조
- 일반 파일
- 디렉토리 파일
- 링크 파일
    - 하드 링크
    - 소프트 링크
- 디바이스 파일

파일의 구조

```
모든 파일은 다음과 같은 기본 구조를 가지고 있다.

                        +-----------------+
      filename          |    INODE        |
                        +-----------------+
                        |                 |
                        |    data block   |
                        |                 |
                        +-----------------+
```

■ 파일의 종류
■ 일반 파일(Egular File)
■ 디렉토리 파일(Directory File)
■ 링크 파일(Link File)
■ 장치 파일(Device File)
■ 소켓 파일(Socket File)
■ 도어 파일(Door File)
■ 파이프 파일(Pipe File)

파일과 디렉토리의 내용들을 다루기 위해서는 위와 같은 기본적인 명령어에 익숙해져야 한다. 리눅스 시스템에서 파일의 구조(File Structure) 파일이름, Inode (Index Node)와 데이터 블럭(Data block)로 구분할 수 있다. 파일의 이름은 유일해야 하고 대소문자는 다른 문자로 인식하게 된다. Inode는 파일의 속성 정보와 데이터 블럭을 포인트하는 정보가 들어 있으며 ls 명령어에 -l 옵션을 사용하여 대부분의 정보를 확인 할 수 있다. 데이터 블록안에는 실제 파일에 저장되는 내용이 들어가게 된다. 일반파일, 디렉토리, 심볼릭링크 파일들은 모두 하나이상의 데이터 종류를 저장한다. 하지만 디바이스 파일은 데이터를 저장하지 않는다. 대신에 디바이스 파일은 디바이스 제어접근(Access)을 제공한다.

일반 파일
리눅스에서는 모든 것을 파일로 다룬다. 일반 파일, 디렉토리 파일, 링크 파일등이 있다. 파일의 종류에 관한 정보는 ls 명령어의 -l 옵션을 사용하여 출력되는 화면에 첫 번째 문자에서 확인이 가능하다. 또한 file 명령어에 파일의 이름을 인자로 받아서 파일의 종류를 확인 할 수도 있다.

문자 : 파일 종류

- : 일반 파일(Egular File)
d : 디렉토리 파일(Directory File)
b : 블록 디바이스 파일 (Block Device File, (예) /dev/sha, /dev/hda, /dev/fd0)
c : 문자 디바이스 파일 (Character Device File, 입출력 장치)
l : 심볼릭 링크(Symbolic Link File)

[참고] 자주 사용되는 기본 명령어

- 디렉토리에 관련된 기본 명령어: ls, pwd, mkdir, cd, mv 등
- 파일과 관련된 기본 명령어: cat, more, cp, rm, head, tail 등
- 파일과 디렉토리 검색에 관련한 기본 명령어: grep, find 등

[1] 파일의 이름

리눅스 시스템에서 파일의 이름은 파일을 접근하기 위해 사용한다. 파일은 Inode와 함께 파일의 구성요소로서 같은 디렉토리 안에서 파일의 이름은 유일한 것이어야 한다. 예를 들어 현재 디렉토리 밑에 존재하는 디렉토리와 같은 이름은 일반파일을 생성 할 수는 없다.

- [ ] 파일의 이름 생성 시 규칙
    - 파일의 이름은 최대 255자까지 생성이 가능하다.
    - 파일의 이름은 문자와 숫자를 사용 할 수 있으며 대소문자를 구별한다.
    - 파일의 이름으로 .(Dot), _(Underbar), -(Dash) 사용가능하다.
    - 파일의 이름으로 특수문자(Metacharater)를 사용하면 안된다.

대표적인 특수문자는 *(Aterisk), /(Slash), ₩(Back Slash), ,(Comma), '(Single Quote), "(Double Quote) (Semicolon), &(Ampersand), |(Pipeline), <(Angle Blocket)등이다.

- ".(Dot)"로 시작하는 파일의 이름은 시스템의 환경파일들이므로 일반파일의 이름으로 권장하지 않는다.
- 파일의 이름에 공백이나 탭을 사용하는 것을 권장하지 않는다.

[2] Inode

Inode는 파일에 대한 정보를 담고 있는 부분이다. 일반적으로 Inode에는 크게 두가지 부분을 포함하고 있다. 첫 번째는 파일에 대한 속성정보(Ownership, Groupship, File Permission Mode등)와 데이터 블록을 가리키고 있는 포인터(Direct/Indirect Pointer)이다. Inode는 숫자로 되어져 있으며 각 파일 시스템은 자신의 Inode 테이블(Inode Tables)을 가지고 있다. 파일이 새로운 파일시스템에 만들어지는 경우 파일시스템 안에서는 새로운 Inode 번호를 할당 받게 된다.

- [ ] Inode의 정보
  - 파일의 종류(File Type)
  - 파일의 퍼미션모드(File Permission Mode)
  - 파일의 소유자, 그룹(File Ownership, Groupship)
  - 파일의 링크수(Hard Link Count)
  - 파일의 마지막 접근 시간(Access Time), 수정시간(Modification Time)
  - 파일의 크기(Bytes, 할당된 또는 사용중인 데이터블럭의 수)
  - 두가지 형태의 포인터(Direct Pointers and Indirect Pointers)

[3] Data Block

데이터블럭은 디스크 공간에 대한 단위(Units of Disk Space)로서 데이터를 저장하는 역할을 가진다. 일반파일, 디렉토리, 심볼릭 파일들은 데이터 블록을 사용하지만 일반파일과 다른 구조를 가지고 있는 디바이스 파일은 데이터블럭에 데이터를 저장하지 않고 주 디바이스 숫자(Major Device Number)와 부 디바이스 숫자(Minor Device Number)를 담고 있다.

- [ ] Data Block의 내용
  - 일반파일의 경우 파일의 내용이 들어 있다.
  - 디렉토리인 경우 안에 포함된 파일과 디렉토리이름이 들어 있다.

일반파일은 리눅스에서 찾을 수 있는 거의 대부분의 파일종류이다. 데이터 블록에 들어가는 데이터는 많은 형태는 ASCII (American Standard Code for Information Interchange) 텍스트, 바이너리 데이터, 이미지 데이터, 데이터베이스 데이터, 애플리케이션 데이터 등이 있다. 일반파일을 만드는 방법 또한 많다.

예를 들어 vi 편집기를 사용 할 수도 있고, 컴파일을 통해 바이너리를 생성 할 수도 있고, touch 명령어를 통해 빈 파일을 생성 할 수도 있다.

디렉토리 파일

디렉토리의 데이터 블록 안에는 실제 디렉토리가 포함하고 있는 파일과 디렉토리에 대한 정보를 가지고 있다. 파일의 이름, 디렉토리의 이름 Inode 번호를 저장하고 있다. 디렉토리의 데이터 블록 안에는 어떤 파일 종류이든 파일의 종류에 맞는 정보가 들어 갈수 있다.

링크파일
- 링크 파일
  - 하드링크 파일(Hard Link File)
  - 심볼릭링크 파일(Symbolic Link File), 소프트 링크 파일(Soft Link File)

[1] 하드 링크 (Hard link)

원본파일의 경로를 저장하고 있는 파일을 말한다. 파일을 실제 경로가 아니라 사용하기 편리한 다른 경로로 접근할 수 있도록 지정하는 명령어이다. 링크파일의 type은 "ls -l" 했을 때 맨 앞의 글자가 "l"로 표시되어 있다.

하드 링크는 똑같은 파일크기로 원본 파일이 수정될 경우, 하드 링크된 파일도 원본과 동일하게 변경되며 항상 같은 내용을 유지할 수 있다. f으므로 `자원을 공유하되 데이터를 안전하게 관리하고자 할 때 유용하게 사용할 수 있다`. 하드링크는 디렉토리에는 만들 수 없다.

하드 링크는 파일명이 하나 더 만들어져 동일한 I-node 번호를 가르치게 된다. 하드 링크는 같은 I-node 번호를 저장하고 있으므로 원본 파일을 지워도 원본파일의 파일명만 지워질 뿐 inode가 지워지는 것이 아니기 때문에 링크로 inode값을 가르키는 링크 파일에 영향을 미치지 않는다.

원본 파일과 링크 파일의 권한은 항상 같다. 그 이유는 I-node에 권한이 저장이 되어 있기 때문이다. 즉 파일명에 대해서 권한을 부여하는 것이 아니고 inode번호에 대해 권한을 설정하기 때문이다. 이는 각기 다른 이름을 가진 하드링크의 파일명에 대한 권한을 변경해줘도 파일명에 권한 설정이 되는 것이 아니

기 때문에 서로 다른 이름을 가진 하드링크 파일의 권한은 동일하게 유지가 되는 것이다.

[2] 심볼릭 링크, 소프트 링크 (Symbolic link, soft link)

일반적으로 링크라고 하면 심볼릭 링크를 말한다. 심볼릭 링크는 소프트 링크라고도 하며 불필요한 파일의 복사를 하지 않아도 된다. 보통 여러 디렉토리에서 동일한 라이브러리를 요구할 경우나, 하나의 파일을 여러 사람이 공통으로 사용할 경우도 많이 쓴다. 소프트 링크의 퍼미션은 모든 유저에게 모든 권한(rwxrwxrwx=777)을 준다.

포인트 하는 정보가 들어 있을 뿐 원본파일은 아니다. 심볼릭 size는 포인트 정보만 들어 있어 cat으로 보여지는 정보가 많던 적던간에 동일한 size를 갖는다. 따라서 윈도우에서 바탕화면에 바로가기가 되어 있어도 많은 양의 리소스를 차지하지 않고 포인트 정보를 담은 size만이 disk의 공간을 차지하게 된다.

원본파일이 지워지게 되면 링크 파일에 영향을 미치게 된다. 소프트링크 파일의 권한은 항상 모든 권한이다.

[참고] 소프트 링크파일의 퍼미션은 항상 777인 이유는?

소프트링크를 만드는 이유는 모든 사용자가 자원을 공유하려 할 때 많이 사용하기 때문이다. 그런데 소프트링크파일은 그 안의 데이터 블록에 원본파일의 inode의 경로를 포함하는 것이므로 링크파일의 퍼미션이 모든 권한이라 하더라도 실제로는 원본 파일의 권한을 따르게 된다. 링크파일의 퍼미션을 변경하면 원본 파일의 퍼미션이 변경이 된다.

```
# cd /etc/
# ls -l grub.conf
lrwxrwxrwx 1 root root 22  1월 25 22:54 grub.conf -> ../boot/grub/grub.conf
                                        링크파일      원본파일
# ls -l /boot/grub/grub.conf
-rw------- 1 root root 600  1월 25 22:54 /boot/grub/grub.conf
# su - fedora
$ cat /etc/grub.conf
cat: /etc/grub.conf: 허가 거부됨
```

ln 명령어

[명령어 형식]

```
# ln file1 file2
# ln -s file1 file2

(하드링크)  # ln file1 file2
(심볼릭링크)# ln -s file1 file2
```

[명령어 옵션]

옵션 : 설명

-b, --backup : 대상 파일이 있다면 백업파일을 생성한다.
-f, --force : 링크를 생성할 대상 파일이 있더라도 강제적으로 새로운 링크를 생성한다.
-i, --interactive : 링크를 생성할 대상 파일이 있을 경우, 삭제 유무를 사용자에게 물어 본다.
-n, --no-dereference : 링크할 원본이 심볼릭 파일이면, 그 심볼릭 파일의 대상 파일을 추적하여 링크한다.
-s, --symbolic : 링크할 원본이 심볼릭 파일이면, 심볼릭 파일을 링크한다.
-S, --suffix backup-suffix : 링크를 생성할 대상 파일이 이미 있을 경우, 이전의 대상파일을 백업할 파일의 확장자를 지정한다.
-V, --version-control : {numbered, existing,simple} 백업하는 방법을 지정한다.

t,numbered : 항상 번호로 된 백업파일을 만든다.
nil, existing : 대상파일이 있을 경우에만 백업파일을 만든다.
never, simple : 간단한 백업을 만든다.

하드링크
[실습 준비]

```
# cd /test ; rm -rf /test/*

# echo 1111 > file1
# ls -l file1
-rw-r--r-- 1 root root 5 Feb 13 13:06 file1
# cat file1
1111
```

[EX1] 하드링크 실습

```
# ln file1 file2
# ls -li
5489017 -rw-r--r-- 2 root root 5 Feb 13 13:06 file1
5489017 -rw-r--r-- 2 root root 5 Feb 13 13:06 file2
# echo 2222 >> file2
# cat file2
1111
2222
# cat file1
1111
2222
# rm file1
# cat file2
1111
2222
# ls -l
-rw-r--r-- 1 root root 10 Feb 13 13:14 file2
```

[EX2] 하드링크 활용

```
하드링크를 이용한 백업파일 만들기
1. 기존방식의 백업파일 만들기
# ls -l
합계 4
-rw-r--r--. 1 root root 10  3월 13 12:42 file2
[root@server1 test]# cp -p file2 .file2
[root@server1 test]# ls -la
합계 8
drwxrwxrwx.  2 root root  33  3월 13 12:44 .
dr-xr-xr-x. 19 root root 249  3월 11 18:08 ..
-rw-r--r--.  1 root root  10  3월 13 12:42 .file2
-rw-r--r--.  1 root root  10  3월 13 12:42 file2
# cp -p file2 .file2.bak /* 백업파일이라는 것을 명시하기위해서 .bak를 붙인다. */
1-1. cp를 이용한 백업의 단점
현시간 이후의 자료에 대해서는 백업이 불가능.
                # cat file2
                1111
                2222
                # cat .file2
                1111
                2222
                # echo 3333 >> file2
                # cat file2
                1111
                2222
                3333
                # cat .file2
                1111
                2222
                # rm -rf file2
                # cp -p .file2 file2
                # cat file2  /* 최신본이 아닌 백업한 당시의 파일로 저장되어 있다. */
                1111
                2222
                # rm -rf .file2

2. 하드링크를 이용한 백업
/etc 디렉토리같은 설정파일이 주로 있는 곳에서 사용하면 좋다
# ls
file2
# ln file2 .file2
# ls -l
합계 4
-rw-r--r--. 2 root root 10  3월 13 12:42 file2
# echo 4444 >> file2
# cat .file2
1111
2222
4444
# rm -rf file2
# ln .file2 file2
# cat file2  /* 항상 백업파일이 최신본으로 유지되며 동일한 데이터를 이용하기 때문에 용량증가
의 걱정이 없다. */ 6565655
1111
2222
4444
```

소프트링크 / 심볼릭링크

[EX1] 소프트링크 / 심볼링링크 실습

```
# ln -s file2 file3
#

# ls -li
5489017 -rw-r--r-- 1 root root 10 Feb 13 13:14 file2
5488913 lrwxrwxrwx 1 root root  5 Feb 13 13:18 file3 -> file2
file3은 file2번을 링크하며 file2번의 INODE가 저장되어있다.

# echo 3333 >> file3
# cat file3
1111
2222
3333
# cat file2
1111
2222
3333
# rm file2
#

# cat file3
cat: file3: No such file or directory
```

[EX2] 디렉토리 넘어 소프트링크

```
# mkdir /test1
# mkdir /test2
# touch file1
# ls
file1
# echo 11111 >> file1
# ls
file1
# ln -s file1 file2
# ls -l
합계 4
-rw-r--r--. 1 root root 6  3월 13 12:59 file1
lrwxrwxrwx. 1 root root 5  3월 13 13:00 file2 -> file1
# mv file2 /test2     file2번을 /test2 디렉토리로 이동
# cd /test2
# ls
file2
# cat file2
cat: file2: 그런 파일이나 디렉터리가 없습니다
[이유] -> 링크파일을 만들 당시에 상대경로로 생성을 하였기 때문에 /test2 디렉토리로 이동하고
나서 /test1/file1을 찾을수가 없다.
# ls -l
합계 0
lrwxrwxrwx. 1 root root 5  3월 13 13:00 file2 -> file1

[해결책] -> 절대경로로 파일 링크 생성
# cd /test1
# ls
file1
# ln -s /test1/file1 file2  /* 원본 파일 경로를 절대 경로로 지정 */
# ls -l
합계 4
-rw-r--r--. 1 root root  6  3월 13 12:59 file1
lrwxrwxrwx. 1 root root 12  3월 13 13:05 file2 -> /test1/file1
# mv file2 /test2
# cd /test2
# ls
file2
# ls -l
합계 0
lrwxrwxrwx. 1 root root 12  3월 13 13:05 file2 -> /test1/file1 /* 이동된 파일이지만 원본 경로
가 절대경로로 지정되어 있기때문에 파일을 제어하는데 아무런 문제가 없다. */
# cat file2
11111
# echo "test2 file create" >> file2
# cd /test1
# ls
file1
# cat file1
11111
test2 file create
```

하드링크(# ln file1 file2)

- 두개의 파일의 inode 번호가 동일한가?
- ls -li 특이한 변화는?
- 두개의 파일 사이즈는 동일한가?
- file2의 편집하면?
- 원본파일을 삭제한 경우 파일을 열수 있는가?

심볼릭 링크(# ln -s file1 file2)

- 두개의 파일의 inode 번호가 동일한가?
- ls -li 특이한 변화는?
- 두개의 파일 사이즈는 동일한가?
- file2의 편집하면?
- 원본파일을 삭제한 경우 파일을 열수 있는가?

하드 링크 & 심볼릭 링크의 차이점

- 파일시스템을 넘어서 링크를 걸수 있는가?

- 디렉토리에 링크를 걸수 있는가?

[참고]

```
심볼릭 링크 -> 윈도우의 바탕화면 아이콘
"아이콘".lnk



(주의) 심볼릭 링크는 일반적으로 상대경로 사용하지 않는다.
(X) # ln -s dir1 dir2
(O) # ln -s /test/dir1 /test/dir2




(실무 예) 웹서버(WAS 서버)에서 웹소스 디렉토리 마이그레이션 하는 작업

----- DAUM Web Server ---------- Web Client -----
httpd(80)        <----------------   http://www.daum.net:80
/was/index.html

작업: /was ---> /zeus
# ln -s /was /zeus

작업2: 스토리지
# ln -s /newstorage /stage


(실무 예) 관리 디렉토리 통합
FTP : /etc/vsftpd
DNS : /var/named
WEB : /var/www/html

         # cd /test ; rm -rf /test/*
# ln -s /etc/vsftpd FTP       /* FTP 서비스 프로그램 디렉토리 */
# ln -s /var/named DNS        /* DNS 서비스 프로그램 디렉토리 */
# ln -s /var/www/html WEB     /* WEB 서비스 프로그램 디렉토리 */
# cd /test ; ls
# cd FTP
# cd ../DNS
# cd ../WEB


(실무 예) 버전 관리
/usr/local/tomcat-4.X ----> /usr/local/tomcat
/usr/local/tomcat-5.X ----> /usr/local/tomcat
/usr/local/tomcat-6.X ----> /usr/local/tomcat

(초기 프로그램을 설치하는 경우)
# cd /usr/local
# ls
tomcat-4.X

# ln -s tomcat-4.x tomcat
# cd tomcat
설정 작업(EX: /usr/local/tomcat)

(버전 업그레이드 하는 경우)
# cd /usr/local
# ls
tomcat-4.X  tomcat-5.X  tomcat

# rm tomcat
# ln -s tomcat-5.x tomcat

# cd tomcat
이전설정(/usr/local/tomcat) 복사
```

디바이스 파일

- [ ] Block Device File
- [ ] Character Device File(= Raw Device File)

- [ ] 디바이스 파일(장치 파일, Device File)
  - 블럭 디바이스 파일(Block Device File)
  - 캐릭터 디바이스 파일(Character Device File) = Raw Device File
- [ ] 블럭 디바이스 파일
  - 블럭 단위로 I/O 발생
  - 디스크 디바이스(Disk Device)이면 I/O 단위는 4K(4096 Bytes)
- [ ] 캐릭터 디바이스 파일
  - 바이트 단위로 I/O 발생
  - 디스크 디바이스(Disk Device)이면 I/0 단위는 512 Bytes(1 Sector = 512 bytes)
- [ ] Major Device 번호
  - 장치의 종류
  - 장치의 종류가 틀리면 Major Device 번호가 틀리다.
- [ ] Minor Device 번호
  - 개별적인 장치의 종류 또는 동작 방법의 차이
  - 개별적인 장치의 종류가 틀리면 Minor Device 번호가 틀리다.
  - 또는 같은 장치라도 동작 방법이 틀리면 Minor Device 번호가 틀리다.

[EX1] 블럭 디바이스/캐릭터 디바이스 파일 확인

```
# ls -l /dev | grep '^b'
brw-rw---- 1 root floppy   2,    0  6월 11 13:02 fd0
brw-rw---- 1 root floppy   2,   84  6월 11 13:02 fd0u1040
brw-rw---- 1 root floppy   2,   88  6월 11 13:02 fd0u1120
brw-rw---- 1 root floppy   2,   28  6월 11 13:02 fd0u1440
brw-rw---- 1 root floppy   2,   44  6월 11 13:02 fd0u1680
..... (중략) .....
brw-rw---- 1 root floppy   2,   68  6월 11 13:02 fd0u830
brw-rw---- 1 root disk    22,    0  6월 11 13:02 hdc
brw-r----- 1 root disk     7,    0  6월 11 13:02 loop0
brw-r----- 1 root disk     7,    1  6월 11 13:02 loop1
..... (중략) .....
brw-r----- 1 root disk     9,    0  6월 11 13:02 md0
brw-r----- 1 root disk     1,    0  6월 11  2010 ram0
brw-r----- 1 root disk     1,    1  6월 11  2010 ram1
brw-r----- 1 root disk     1,   10  6월 11  2010 ram10
brw-r----- 1 root disk     1,   11  6월 11  2010 ram11
..... (중략) .....
brw------- 1 root root     8,    1  6월 11  2010 root
brw-r----- 1 root disk     8,    0  6월 11  2010 sda
brw-r----- 1 root disk     8,    1  6월 11 13:02 sda1
brw-r----- 1 root disk     8,    2  6월 11  2010 sda2
..... (중략) .....
```

```
# ls -l /dev | grep '^c'
crw-rw---- 1 root audio   14,   12  6월 11 13:02 adsp
crw------- 1 root root    10,  175  6월 11 13:02 agpgart
crw-rw---- 1 root audio   14,    4  6월 11 13:02 audio
crw------- 1 root root    10,   61  6월 11 13:02 autofs
crw------- 1 root root     5,    1  6월 11 13:02 console
crw-rw---- 1 root root    14,    9  6월 11 13:02 dmmidi
crw-rw---- 1 root audio   14,    3  6월 11 13:02 dsp
crw-rw-rw- 1 root root     1,    7  6월 11 13:02 full
crw------- 1 root root    10,  228  6월 11 13:02 hpet
crw------- 1 root root     1,   11  6월 11 13:02 kmsg
crw-rw---- 1 root lp       6,    0  6월 11 13:02 lp0
crw-r----- 1 root kmem     1,    1  6월 11 13:02 mem
crw-rw---- 1 root audio   14,    2  6월 11 13:02 midi
crw-rw---- 1 root audio   14,    0  6월 11 13:02 mixer
crw-rw-rw- 1 root root     1,    3  6월 11 13:02 null
crw-rw---- 1 root root    10,  144  6월 11 13:02 nvram
crw------- 1 root root     1,   12  6월 11 13:02 oldmem
crw-rw---- 1 root lp      99,    0  6월 11 13:02 parport0
crw-rw---- 1 root lp      99,    1  6월 11 13:02 parport1
crw-rw---- 1 root lp      99,    2  6월 11 13:02 parport2
crw-rw---- 1 root lp      99,    3  6월 11 13:02 parport3
crw-r----- 1 root kmem     1,    4  6월 11 13:02 port
crw------- 1 root root   108,    0  6월 11 13:02 ppp
crw-rw-rw- 1 root tty      5,    2  6월 11 13:59 ptmx
crw-rw-rw- 1 root root     1,    8  6월 11 13:02 random
crw------- 1 root root   162,    0  6월 11 13:02 rawctl
crw-r--r-- 1 root root    10,  135  6월 11  2010 rtc
crw-rw---- 1 root audio   14,    1  6월 11 13:02 sequencer
crw-rw---- 1 root audio   14,    8  6월 11 13:02 sequencer2
crw------- 1 root root    21,    0  6월 11 13:02 sg0
crw------- 1 root root    10,  231  6월 11 13:02 snapshot
crw------- 1 root root     4,    0  6월 11  2010 systty
crw-rw-rw- 1 root tty      5,    0  6월 11 13:02 tty
crw-rw---- 1 root root     4,    0  6월 11  2010 tty0
crw------- 1 root root     4,    1  6월 11 13:02 tty1
crw-rw---- 1 root tty      4,   10  6월 11  2010 tty10
crw-rw---- 1 root tty      4,   11  6월 11  2010 tty11
..... (중략) .....
cr--r--r-- 1 root root     1,    9  6월 11 13:02 urandom
crw------- 1 root root   442,    0  6월 11 13:02 usbdev1.1_ep00
crw------- 1 root root   442,    0  6월 11 13:02 usbdev1.1_ep81
crw------- 1 root root   442, 2048  6월 11 13:02 usbdev2.1_ep00
crw------- 1 root root   442, 2048  6월 11 13:02 usbdev2.1_ep81
```

권한 및 소유자 이해

파일의 속성 관리

단원목표

- 파일의 속성 정보 확인
- 파일의 속성 정보 변경
- 파일의 소유권한 / 그룹권한 변경
- 파일의 퍼미션 변경
- 파일의 특수퍼미션 변경

파일의 속성 정보 변경 명령어에는 chown, chgrp, chmod, umask와 같은 명령어 등이 있다.

파일에 대한 정보로 소유권에 대한 기록을 담고 있다. 단, 소유권에 대한 권한 설정은 속성 정보가 아닌 사용자와 그룹에게 권한을 부여해 주는 것이므로 파일의 속성정보는 권한을 뺀 나머지의 정보가 모두 파일 속성에 해당 하는 것이다. (권한은 사용자가 가지고 있다!)

(파일의 속성 정보 변경 명령어)

```
# ls -l file1
-rw-r--r-- 1 root root 1945  6월 11 14:13 file1

File Type:
Permission Mode: chmod
Link Count: ln
Owner: chown
Group: chgrp
File Size:
Mtime: touch -t
File Name: mv
```

파일의 소유권한 / 그룹권한 변경

chown 명령어

chown 명령어는 Unix 계통 시스템에서 파일의 소유권을 바꾸기 위해서(change the owner of a file)사용된다. 대부분의 경우, 이것은 오직 슈퍼 사용자에 의해서만 실행될 수 있다. 그들이 소유하고 있는 파일의 그룹을 바꾸고 싶어하는 비특권화된 (일반적인) 사용자들은 chgrp을 사용해야 한다.

```
# ls -l file1
-rw-r--r-- 1 root root 0 Jan 10 12:43 file1
```

[명령어 형식]

```
# chown user01 file1
# chown user01.other file1  (# chown user01:other file1)
# chown .other file1
# chown -R user01 dir1
# chown -R user01:other dir1
```

[명령어 옵션]

옵션 : 설명

-c : 바뀌어지는 파일들에 대해서만 자세하게 보여준다.
-f : 바뀌어 지지 않는 파일들에 대해서 오류 메시지를 보여주지 않는다.
-v : 작업 상태를 자세히 보여준다.
-R : 경로와 그 하위 파일들 모두를 바꾼다.

[EX1] chown 명령어 실습

```
# cd /test ; rm -rf /test/*
# touch file1
# ls -l file1
-rw-r--r-- 1 root root 0 Feb 14 10:41 file1
# chown fedora file1
# ls -l file1
-rw-r--r-- 1 fedora root 0 Feb 14 10:41 file1
# chown .user01 file1
# ls -l file1
-rw-r--r-- 1 fedora user01 0 Feb 14 10:41 file1
# chown root:root file1  (# chown root.root file1)
# ls -l file1
-rw-r--r-- 1 root root 0 Feb 14 10:41 file1
```

[참고] 그룹 생성 여부 확인

```
# cat /etc/group | grep root         /* user01라는 그룹이 있는지 파일 안에서 검색 */
root:x:0:

# cat /etc/group | grep root          /* root 라는 그룹 확인 */
root:x:0:      /* root 그룹 */         /* root 사용자가 소속된 그룹 */

 > RHEL7 버전에서는 root:x:0: 하나의 그룹만 존재한다.

        [참고] useradd 명령어의 수행
                # useradd user02
                # passwd user02

                # grep user02 /etc/passwd
                # grep user02 /etc/group
# cat /etc/passwd UID
# cat /etc/group GID
```

[EX2] chown -R 옵션 실습

```
(실습용 구조)

/test -- dir1 --+-- dir2 -- dir3
          |
          +-- file1
          |
          +-- file2
```

```
# cd /test ; rm -rf /test/*

# mkdir -p dir1/dir2/dir3
# touch dir1/file1
# touch dir1/file2

# ls -lR dir1      (# find . -ls)
dir1:
total 4.0K
drwxr-xr-x 3 root root 4.0K Feb 14 10:47 dir2/
-rw-r--r-- 1 root root    0 Feb 14 10:47 file1
-rw-r--r-- 1 root root    0 Feb 14 10:47 file2

dir1/dir2:
total 4.0K
drwxr-xr-x 2 root root 4.0K Feb 14 10:47 dir3/

dir1/dir2/dir3:
total 0
```

```
# chown -R fedora:fedora dir1
# ls -lR dir1    (# find . -ls)
dir1:
total 4.0K
drwxr-xr-x 3 fedora fedora 4.0K Feb 14 10:47 dir2/
-rw-r--r-- 1 fedora fedora    0 Feb 14 10:47 file1
-rw-r--r-- 1 fedora fedora    0 Feb 14 10:47 file2

dir1/dir2:
total 4.0K
drwxr-xr-x 2 fedora fedora 4.0K Feb 14 10:47 dir3/

dir1/dir2/dir3:
total 0
# chown -R .user01 dir1
# ls -lR dir1    (# find . -ls)
dir1:
total 4.0K
drwxr-xr-x 3 fedora user01 4.0K Feb 14 10:47 dir2/
-rw-r--r-- 1 fedora user01    0 Feb 14 10:47 file1
-rw-r--r-- 1 fedora user01    0 Feb 14 10:47 file2

dir1/dir2:
total 4.0K
drwxr-xr-x 2 fedora user01 4.0K Feb 14 10:47 dir3/

dir1/dir2/dir3:
total 0
# chown -R root:root dir1
# ls -lR dir1
dir1:
total 4.0K
drwxr-xr-x 3 root root 4.0K Feb 14 10:47 dir2/
-rw-r--r-- 1 root root    0 Feb 14 10:47 file1
-rw-r--r-- 1 root root    0 Feb 14 10:47 file2

dir1/dir2:
total 4.0K
drwxr-xr-x 2 root root 4.0K Feb 14 10:47 dir3/

dir1/dir2/dir3:
total 0
```

[참고] 대량의 파일의 권한 변경시 문제점

```
만약에 변경할 파일 및 디렉토리가 200.000개 이상인경우에는 95% 확률로 에러가 난다.
chown을 비롯한 많은 파일작업에는 sort작업이 들어간다.
sort 명령어가 입력되며 생기는 문제점 :
해결책 :
1. find 명령어로 모든 파일 및 디렉토리를 절대경로로 검색 > 출력
2. 출력된 파일을 반복문(for문)으로 작업
3. 이렇게 시간이 오래 걸리는 작업의 경우에는 tmux 명령어를 이용하여 해당 터미널을 백그라운드
에서 작업하는 것을 권장
```

chgrp 명령어

파일의 속성정보 중 그룹명을 변경하는 명령어이다.

```
# ls -l file1
-rw-r--r-- 1 root root 0 Jan 10 12:43 file1
```

[명령어 형식]

```
# chgrp user01 file1
```

[명령어 옵션]

옵션 : 설명

-c : 작업 상태를 자세히 보여주나, 바뀌어 지는 것만 보여준다.
-f : 그룹이 바뀌어 지지 않는 파일들에 대한 오류 메시지를 보여주지 않는다.
-v : 작업 상태를 자세히 보여준다.
-R : 주로 file 이름으로 경로를 사용해서, 그 안에 있는 모든파일도 함께 gourp으로 바꾼다.

[EX1] chgrp 명령어 실습

```
# cd /test
# touch file1
# ls -l file1
-rw-r--r-- 1 root root 0 Feb 14 10:52 file1
# chgrp fedora file1
# ls -l file1
-rw-r--r-- 1 root fedora 0 Feb 14 10:52 file1
```

[EX2] chgrp -R 옵션 실습

```
# mkdir -p dir1/dir2/dir3
# ls -lR
.:
total 4.0K
drwxr-xr-x 3 root root   4.0K Feb 14 10:47 dir1/
-rw-r--r-- 1 root fedora    0 Feb 14 10:52 file1

./dir1:
total 4.0K
drwxr-xr-x 3 root root 4.0K Feb 14 10:47 dir2/
-rw-r--r-- 1 root root    0 Feb 14 10:47 file1
-rw-r--r-- 1 root root    0 Feb 14 10:47 file2

./dir1/dir2:
total 4.0K
drwxr-xr-x 2 root root 4.0K Feb 14 10:47 dir3/

./dir1/dir2/dir3:
total 0
# chgrp fedora dir1
# ls -lR
.:
total 4.0K
drwxr-xr-x 3 root fedora 4.0K Feb 14 10:47 dir1/
-rw-r--r-- 1 root fedora    0 Feb 14 10:52 file1

./dir1:
total 4.0K
drwxr-xr-x 3 root root 4.0K Feb 14 10:47 dir2/
-rw-r--r-- 1 root root    0 Feb 14 10:47 file1
-rw-r--r-- 1 root root    0 Feb 14 10:47 file2

./dir1/dir2:
total 4.0K
drwxr-xr-x 2 root root 4.0K Feb 14 10:47 dir3/

./dir1/dir2/dir3:
total 0
# chgrp -R user01 dir1    (# chown -R root:user01 dir1)
# ls -lR
.:
total 4.0K
drwxr-xr-x 3 root user01 4.0K Feb 14 10:47 dir1/
-rw-r--r-- 1 root fedora    0 Feb 14 10:52 file1

./dir1:
total 4.0K
drwxr-xr-x 3 root user01 4.0K Feb 14 10:47 dir2/
-rw-r--r-- 1 root user01    0 Feb 14 10:47 file1
-rw-r--r-- 1 root user01    0 Feb 14 10:47 file2

./dir1/dir2:
total 4.0K
drwxr-xr-x 2 root user01 4.0K Feb 14 10:47 dir3/

./dir1/dir2/dir3:
total 0
```

[참고]

일반사용자의 경우에 그룹소유권 변경을 해주는 경우 해당 그룹을 소유(개인 권한 소유)하고 있어야 한다.

파일의 퍼미션 변경
chmod 명령어

파일이나 디렉토리를 새로운 권한으로 변경하는 명령이다. 파일의 소유자나 관리자만이 chmod를 사용할 수 있으며 파일의 소유자, 파일의 그룹, 다른 사용자로 나누어 각각의 권한을 설정 할 수 있다.

```
# ls -l file1
-rw-r--r-- 1 root root 0 Jan 10 12:43 file1
```

[명령어 형식]

```
# chmod u+x file1
# chmod 755 file1
```

- 퍼미션(Permission)을 변경하는 방법
- 심볼릭 모드(Symbolic Mode) : # chmod u+x file1
- 옥탈 모드(Octal Mode) : # chmod 744 file1

[1] 심볼모드(symbolic mode)를 이용한 권한 변경

[사용자 기호]

```
기호                        설명
u      user            파일/디렉토리의 소유자
g      group           파일/디렉토리의 그룹
o      other           다른 사용자
a      all             소유자, 그룹, 다른 사용자 모두(아무 표시 안할 경우 기본적으로 설정됨)
```

[설정 기호]

```
기호                        설명
+      퍼미션 허가       지정한 퍼미션을 허가한다.
-      퍼미션 금지       지정된 퍼미션을 금지시킨다.
=      퍼미션 지정       지정한 퍼미션만 허가하고 나머지는 금지 시킨다.
                       설명 : 이전에 권한을 어떻게 줬던 지금 설정해주는 권한으로 대체시켜서
사용 가능
                       (+,-는 내가 이전에 어떠한 권한을 설정해 줬는지 확인해야 하지만
                       =는 이전에 어떻게 설정하였던 상관없이 설정이 가능한 것이다.)
```

[권한 기호]

r w x
read write excute

설명 : 읽기 권한이 없으면 파일 안에 있는 내용을 볼 수 없다. 따라서 파일을 수정하기 위해서는 반드시 read권한을 부여해야지 파일을 열어서 보고 수정 할 수가 있는 것이다. 만약에 write권한만 있다면 파일을 열어 볼 수 없으므로 수정이 불가능 한 것이다. 따라서 파일을 수정하려면 read&write권한이 모두 있어야 한다.

```
# ls -l file1
-rw-r--r-- 1 root   fedora    0 Aug 17 16:06 file1

# chmod  u + r  file1
         g - w
         o = x
         a
```

```
심볼의 정의
u(User)
g(Group)
o(Other)
a(all)

+(Add)
-(Deny)
=(equal)

r(Read)
w(Write)
x(Excute)
```

[EX1] 심볼 모드를 사용한 퍼미션 변경

```
# cd /test
# rm -rf /test/*

# touch file1
# ls -l file1
-rw-r--r-- 1 root root 0 Feb 14 11:12 file1

# chmod u+x file1
# ls -l file1
-rwxr--r-- 1 root root 0 Feb 14 11:12 file1*

# chmod g-r file1
# ls -l file1
-rwx---r-- 1 root root 0 Feb 14 11:12 file1*

# chmod u-x,g+x file1
# ls -l file1
-rw---xr-- 1 root root 0 Feb 14 11:12 file1*

# chmod a=rwx file1
# ls -l file1

-rwxrwxrwx 1 root root 0 Feb 14 11:12 file1*
```

[EX1] 심볼 모드를 사용한 퍼미션 변경

```
소유자권한비트  그룹권한비트  기타권한비트
r  w  x     r  w  x    r  w  x
1  1  1     1  1  1    1  1  1
4  2  1     4  2  1    4  2  1
```

파일 소유자 권한 : 400 = 읽기 권한, 200 = 쓰기 권한, 100 = 실행 권한
그룹 사용자 권한 : 40 = 읽기 권한, 20 = 쓰기 권한, 10 = 실행 권한
기타 사용자 권한 : 4 = 읽기 권한, 2 = 쓰기 권한, 1 = 실행 권한

```
--- : 권한 없음 0          000
--x : 실행 권한 1          001
-w- : 쓰기 권한 2          010
-wx : 쓰기 실행 3          011
r-- : 읽기 권한 4          100
r-x : 읽기 실행 5          101
rw- : 읽기 쓰기 6          110
rwx : 읽기 쓰기 실행 7      111
```

```
# chmod 744 file1   (rwxr--r--)
```

[EX1] Octal Mode 실습

```
# cd /test ; rm -rf /test/*
# touch file1
# ls -l file1
-rw-r--r-- 1 root root 0 Feb 14 11:20 file1
# chmod 744 file1
# ls -l file1
-rwxr--r-- 1 root root 0 Feb 14 11:20 file1*
# chmod 754 file1
# ls -l file1
-rwxr-xr-- 1 root root 0 Feb 14 11:20 file1*
```

파일과 디렉토리 퍼미션의 정확한 의미

```
(파일과 디렉토리의 퍼미션의 정확한 의미)

-----------------------------------
파일 디렉토리
-----------------------------------
r (ls CMD)
w (생성 & 삭제)
x (cd CMD)
-----------------------------------
```

[TERM1] fedora 사용자 터미널

```
# ssh fedora@localhost
fedora 사용자로 로그인

$ ls -ld /home/fedora
drwx------. 3 fedora fedora 115  6월 25 18:19 /home/fedora
$ mkdir dirtest
$ touch dirtest/test2.txt
$ ls -lR
.:
합계 0
drwxrwxr-x. 2 fedora fedora 23  6월 26 10:09 dirtest

./dirtest:
합계 0
-rw-rw-r--. 1 fedora fedora 0  6월 26 10:09 test2.txt
```

[TERM2] user01 사용자 터미널

```
# ssh user01@localhost
user01 사용자로 로그인

$ cd ~fedora          ($ cd /home/fedora)
-bash: cd: /home/fedora: 허가 거부

drwx------ 7 fedora fedora ..... /home/fedora
                     A       A
                     |       |
                     V       V
                  user01 user01

$ id fedora
$ id user01
$ cd ~fedora/dirtest   ($ cd /home/fedora/dirtest)
-bash: cd: /home/fedora/dirtest: 허가 거부
-> 하위디렉토리에 접근 권한이 있더라도 상위 디렉토리에 접근 불가하므로 하위디렉토리인 dirtest
디렉토리도 접근 불가
$ rm -f ~fedora/dirtest/test2.txt   ($ rm -f /home/feodra/dirtest/test2.txt)
rm: cannot remove '/home/fedora/dirtest/test2.txt': 허가 거부
```

[TERM1] fedora 사용자 터미널

```
$ id
uid=1001(fedora)                    gid=1001(fedora)                    groups=1001(fedora)
context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
$ chmod 757 /home/fedora
$ ls -ld /home/fedora
drwxr-xrwx. 4 fedora fedora 130  6월 26 10:08 /home/fedora
$ chmod 755 dirtest
$ touch dirtest/test3
$ chmod 646 dirtest/test3
$ ls -lR
.:
합계 0
drwxr-xr-x. 2 fedora fedora 36  6월 26 10:15 dirtest

./dirtest:
합계 0
-rw-rw-r--. 1 fedora fedora 0  6월 26 10:09 test2.txt
-rw-r--rw-. 1 fedora fedora 0  6월 26 10:16 test3
/home/fedora (rwxr-xrwx fedora fedora)
      |
      +----- dirtest (rwxr-xr-x fedora fedora)
      |
      +----- test3 (rw-r--rw- fedora fedora)
```

[TERM2] user01 사용자 터미널

```
$ id
uid=1002(user01)                    gid=1002(user01)                    groups=1002(user01)
context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
$ cd ~fedora       ($ cd /home/fedora)
$ cd dirtest
$ rm -f test3
rm: cannot remove 'test3': 허가 거
$ mkdir dirtest1
mkdir: `dirtest1' 디렉토리를 만들 수 없습니다: 허가 거부
```

[TERM1] fedora 사용자 터미널

```
$ id
uid=1001(fedora)                    gid=1001(fedora)                    groups=1001(fedora)
context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
$ chmod 757 dirtest
$ ls -lR
.:
합계 0
drwxr-xrwx. 2 fedora fedora 36  6월 26 10:15 dirtest

./dirtest:
합계 0
-rw-rw-r--. 1 fedora fedora 0  6월 26 10:09 test2.txt
-rw-r--rw-. 1 fedora fedora 0  6월 26 10:16 test3

/home/fedora (rwxr-xrwx fedora fedora)
      |
      +----- dirtest (rwxr-xrwx fedora fedora)
      |
      +----- test3 (rw-r--rw- fedora fedora)
```

[TERM2] user01 사용자 터미널

```
$ id
uid=1002(user01)                    gid=1002(user01)                    groups=1002(user01)
context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
$ rm -f test3
$ touch test4.txt
$ ls -l
합계 0
-rw-rw-r--. 1 fedora fedora 0  6월 26 10:09 test2.txt
-rw-rw-r--. 1 user01 user01 0  6월 26 10:17 test4.txt
```

umask 명령어

파일이나 디렉토리 생성시에 파일과 디렉토리에는 기본적으로 적용되는 퍼미션이 있다. 기본적으로 설정되는 퍼미션의 경우 umask에 의해 결정이 된다. umask는 디렉토리와 파일의 기본 퍼미션을 결정해주는 명령어이다.

[기본 퍼미션(Default Permission) 변경]

```
                    파일 디렉토리
Default Permission  666 777
umask               022 022
생성 기본퍼미션         644 755
```

[명령어 형식]

```
# umask
# umask 027
# umask 022
```

[EX1] umask 간단한 실습

```
# cd /test
# rm -rf /test/*

# umask
0022
# touch file1
# mkdir dir1
# ls -l
-rw-r--r-- 1 root root    0 Feb 11 10:45 file1
drwxr-xr-x 2 root root 4096 Feb 11 10:47 dir1

(666 - 022 = 644)
(777 - 022 = 755)

■ Default Permission
File  Directory
-------------------------------------------
666 777 666 777
022 022 027 027
-------------------------------------------
644 755 640 750

# umask 002
# umask
0002
# touch file2
# mkdir dir2
# ls -ld *2
drwxrwxr-x 2 root root 4.0K Feb 14 12:04 dir2/
-rw-rw-r-- 1 root root    0 Feb 14 12:04 file2

■ Default Permission
File  Directory
------------------
666 777
002 002
------------------
664 775

# umask 027
# umask
0027

# touch file3 (666 - 027 = 640)
# mkdir dir3  (777 - 027 = 750)
# ls -ld *3
drwxr-x--- 2 root root 4.0K Feb 14 12:05 dir3/
-rw-r----- 1 root root    0 Feb 14 12:05 file3

■ Default Permission
File Directory
------------------
666 777
027 027
------------------
640 750
```

[EX2] /etc/bashrc 파일에 등록된 umask 확인

```
(관리자) /etc/bashrc
(사용자) $HOME/.bashrc

# cat /etc/bashrc
..... (중략) .....
    # By default, we want umask to get set. This sets it for non-login shell.
    # You could check uidgid reservation validity in
    # /usr/share/doc/setup-*/uidgid file
    if [ $UID -gt 99 ] && [ "`id -gn`" = "`id -un`" ]; then
       umask 002
    else
       umask 022
    fi
..... (중략) .....
-> umask 확인
```

SetUID / SetGID

파일에 대한 소유권을 잠시 다른 사용자에게 빌려 줌으로 인해 소유권이 없는 사용자가 잠시 동안 파일에 대한 소유권으로 권한을 행사 할 수 있는 것을 말한다.

공유한 디렉토리나 파일에 대한 특별한 퍼미션을 부여하는 것으로서 파일 소유자(owner)나 superuser만이 파일에 대해서는 setuid와 setgid를 설정하고,디렉토리에 대해서는 sticky-bit 퍼미션을 설정 할 수 있다.

absolute mode (octal mode)나 symbolic mode를 사용하여 지정하거나 해지할 수 있다.

[참고] SetUID와 SetGID를 실행할 수 있는 프로그램

실행 권한(x)이 주어진 프로그램에 setuid 퍼미션이 주어지면 누구에게나 그 프로그램의 소유자처럼 그 프로그램을 실행할 권한이 주어지고, 또한 어느 누구에게나 그 프로그램의 그룹에 속한 것처럼 할 수 있다. 즉, 실행 권한(x)을 가진 프로그램이 setuid와 setgid 퍼미션이 주어지면 그 프로그램의 owner나 group으로부터 UID와 GID를 얻는다. 이는 해당 프로그램이 시작될 때 프로세스로부터 UID와 GID를 상속받는 것과는 다르다.

[참고] SetUID를 가진 파일의 예시

패스워드를 변경할 수 있는 권한을 root의 권한이다. 그렇기에 일반 사용자는 패스워드 권한을 변경할 수 없다. 일반 사용자도 패스워드를 변경할 때 만큼은 root가 되도록 만들어주어야 한다. 그렇기 위해선 /usr/bin/passwd실행파일의 권한이 root의 SetUID 권한이 부여되어 있으면 된다.

[특수권한 SetUID, SetGID, sticky bit 퍼미션]

```
특수권한                    소유자권한비트  그룹권한비트  기타권한비트
SetUID SetGID sticky-bit  r w x       r w x      r w x
1      1      1           1 1 1       1 1 1      1 1 1
4      2      1

       천                  백          십          일
```

```
# chmod 755 file1
# chmod 0755 file1

# chmod 22 file1
# chmod 0022 file1

# chmod 4755 file1
# chmod 6755 file1
(0755 : rwxr-xr-x)(0755 : rwxr-xr-x)
(4755 : rwsr-xr-x)(6755 : rwsr-sr-x)

# chmod 2755 file1
(0755 : rwxr-xr-x)
(2755 : rwxr-sr-x)

# chmod 1777 dir1

(0777 : rwxrwxrwx)
(1777 : rwxrwxrwt)
```

[EX1] SetUID,SetGID 권한 부여

```
# cd /test
# rm -rf /test/*

# touch file1
# ls -l file1
-rw-r--r-- 1 root root 0 Feb 11 09:37 file1
# chmod 755 file1
# ls -l file1
-rwxr-xr-x 1 root root 0 Feb 11 09:37 file1
# chmod 4755 file1
# ls -l file1
-rwsr-xr-x 1 root root 0 Feb 11 09:37 file1
# chmod 2755 file1
# ls -l file1
-rwxr-sr-x 1 root root 0 Feb 11 09:37 file1
# chmod 6755 file1    /* 참고사항이며 실제로 이렇게 사용하지는 않는다 */
# ls -l file1
-rwsr-sr-x 1 root root 0 Feb 11 09:37 file1
```

[참고] 권한 설정시 유의사항

```
# chmod 4755 file1 (rwxr-xr-x  ->  rwsr-xr-x)
# chmod 4100 file1 (--x------  ->  --s------)
# chmod 4655 file1 (rw-r-xr-x  ->  rwSr-xr-x)

# chmod 2755 file1 (rwxr-xr-x  ->  rwxr-sr-x)
# chmod 2010 file1 (-----x---  ->  -----s---)
# chmod 2765 file1 (rwxrw-r-x  ->  rwxrwSr-x)
원래는 권한이 없어 - 로 표시되어야 하는 자리지만 SETID가 들어가면서 이전 권한이 보이지 않게되
어 대문자로 표시된다.
```

[EX2] SetUID 확인

```
(일반사용자) $ passwd
-> /etc/passwd(user01:x:501:501::/home/user01:/bin/bash)
-> /etc/shadow(user01:$1$D0a001Ns$Src3NIrAeQH8YIQwJ44bp1:15911:0:99999:7:::)
```

```
[passwd 명령어의 동작]
# /etc/passwd /etc/shadow

1. passwd 명령어 입력
2. passwd 명령어 속성에 (권한) 있는 setUID bit 동작을 하며 사용자의 권한을 잠시 root권한을 획
득
3. 비밀번호를 변경하며 /etc/shadow (암호파일)에 변경된 비밀번호를 root권한으로 수정한다.
4. 모든 작업이 종료되면 권한을 반환하고 일반계정으로 돌아간다.
```

```
# ls -l /usr/bin/passwd /etc/passwd /etc/shadow
-rw-r--r-- 1 root root 2936  Feb 11 06:29 /etc/passwd
-r-------- 1 root root 1275  6월 11 15:17 /etc/shadow
-rwsr-xr-x 1 root root 22984 Jan  7 2007  /usr/bin/passwd
# chmod 755 /usr/bin/passwd
# ls -l /usr/bin/passwd
-rwxr-xr-x 1 root root 22984  1월  7  2007 /usr/bin/passwd
# su - fedora
$ passwd
Changing password for user fedora.
Changing password for fedora
(current) UNIX password: (fedora)
passwd: Authentication token manipulation error
$ exit
# passwd fedora
Changing password for user fedora.
New UNIX password: (fedora)
BAD PASSWORD: it is based on a dictionary word
Retype new UNIX password: (fedora)
passwd: all authentication tokens updated successfully.

(원복) /usr/bin/passwd (rwxr-xr-x  ->  rwsr-xr-x)
      # chmod 4755 /usr/bin/passwd
```

[EX3] fedora가 user01 사용자의 권한을 빌리는 경우

```
# ssh user01@localhost
user01 사용자로 로그인

$ chmod 775 /home/user01
$ cp /bin/touch /home/user01
$ ls -l
-rwxr-xr-x 1 user01 user01 42284  3월 17 01:20 touch

                /bin/touch         (rwxr-xr-x 1 root root)
                /home/user01/touch (rwxr-xr-x 1 user01 user01)

$ ./touch file1
$ ls -l
-rw-rw-r-- 1 user01 user01     0  3월 17 01:20 file1
-rwxr-xr-x 1 user01 user01 42284  3월 17 01:20 touch
                /home/user01 (rwxrwxr-x  user01  user01)
                |
                +--- file1((0)생성가능)

$ chmod 4755 touch
$ ls -l
-rw-rw-r-- 1 user01 user01     0  3월 17 01:20 file1
-rwsr-xr-x 1 user01 user01 42284  3월 17 01:33 touch
$ su - fedora
-> fedora 암호 입력

$ cd /home/user01
$ touch file2
touch: cannot touch `file2': 허가 거부됨
                /home/user01 (rwxrwxr-x  user01  user01)
                |
                +--- file2((X)생성 불가능)

/bin/touch         (rwxr-xr-x 1 root root)
/home/user01/touch (rwsr-xr-x 1 user01 user01)

$ ./touch file2
$ ls -l
-rw-rw-r-- 1 user01 user01     0  3월 17 01:20 file1
-rw-rw-r-- 1 user01 fedora     0  3월 17 01:20 file2
-rwsr-xr-x 1 user01 user01 42284  3월 17 01:33 touch
$ exit
$ id
-> user01 사용자 확인

$ chmod 2755 touch  (rwxr-xr-x   ->   )
$ ls -l touch
-rwxr-sr-x  user01 user01  size mtime  touch
$ su - fedora
-> fedora 사용자 암호 입력

$ cd /home/user01
$ ./touch file3
$ ls -l
-rw-rw-r-- 1 user01 user01     0 Apr 21 15:59 file1
-rw-rw-r-- 1 user01 fedora     0 Apr 21 16:02 file2
-rw-rw-r-- 1 fedora user01     0 Apr 21 16:04 file3
-rwxr-sr-x 1 user01 user01 42284 Apr 21 15:59 touch
                /home/user01   (rwxrwxr-x  user01 user01)
                |
                +--- file3((0)생성가능)
$ exit
$ exit
#
```

[EX4] bash쉘의 SetUID 권한 부여

[TERM1] root 사용자 터미널

```
# ls -l /bin/bash
-rwxr-xr-x 1 root root 735004 Jan 22  2009 /bin/bash
# cp /bin/bash /test
# cd /test
# ls -l bash
-rwxr-xr-x 1 root root 735004 Feb 11 09:06 bash
                /bin/bash   (-rwxr-xr-x 1 root root 719K Jul 22  2011 /bin/bash*)
                /test/bash  (-rwxr-xr-x 1 root root 719K Jan 13 11:33 bash*)
# bash
# ps
# exit
exit
# ./bash
# ps
# exit

# chmod 4755 bash
# ls -l bash
-rwsr-xr-x 1 root root 735004 Feb 11 09:06 bash
```

[TERM2] root 사용자의 두번째 터미널

```
# su - fedora
$ cd /test
$ ls -l bash
-rwsr-xr-x 1 root root 735004  Feb 11 09:06 bash
$ ./bash
$ id         /* 커널 2.6버전부터는 쉘은 SetUID, SetGID권한 부여 막음 */
uid=1001(fedora)                    gid=1001(fedora)                    groups=1001(fedora)
context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
$ exit
$ exit
#
```

[참고] gcc 프로그램 설치가 되어 있지 않으면

```
# which gcc
# rpm -qa | grep gcc
# dnf -y install gcc
```

[TERM1] root 사용자의 첫번째 터미널

```
# cd /test
# vi backdoor.c /* 함수를 이용하여 프로그램 내에서 쉘을 실행 시키는 것 */
------------------------------
#include <stdio.h>

main()
{
setuid(0);
setgid(0);
system("/bin/bash");
}
------------------------------

------------------------------     < 변경된 버전
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#define _GNU_SOURCE

int main ()
{
        setuid(0);
        setgid(0);
        system("/bin/bash");
}
------------------------------
# gcc -o bashshell backdoor.c
# ls -l bashshell
-rwxr-xr-x 1 root root 4934 Feb 11 09:22 bashshell
                # file /bin/ls
                # file /test/bashshell
```

[TERM2] fedora 사용자 터미널

```
# ssh fedora@localhost
-> fedora 사용자로 로그인

$ cd /test
$ ls -l bashshell
-rwxr-xr-x 1 root root 4934 Feb 11 09:22 bashshell
$ ./bashshell
$ ps
 PID TTY          TIME CMD
 7195 pts/2    00:00:00 bash
 7226 pts/2    00:00:00 bashshell
 7227 pts/2    00:00:00 bash
 7249 pts/2    00:00:00 ps
$ id
uid=500(fedora) gid=500(fedora) groups=500(fedora)
$ exit
$
```

[TERM1] root 사용자 터미널

```
# cd /test
# chmod 4755 bashshell
# ls -l bashshell
-rwsr-xr-x 1 root root 4934 Feb 11 09:22 bashshell
```

[TERM1] root 사용자 터미널

```
$ ./bashshell
# id
uid=0(root) gid=0(root) groups=500(fedora)
# pwd
/test
# chmod 755 bash
# ls -l
-rw-r--r--  1 root root     77  2??11 09:22 backdoor.c
-rwxr-xr-x  1 root root 735004  2??11 09:06 bash
-rwsr-xr-x  1 root root   4934  2??11 09:22 bashshell
# ./bash
# id
uid=0(root) gid=0(root) groups=500(fedora)
# exit
# exit
$ exit
#
```

[참고] 특수 퍼미션을 가진 파일 찾기

```
[TERM1] # find / -perm -4000 -type f
[TERM2] # find / -perm -2000 -type f
[TERM3] # find / ₩( -perm -4000 -o -perm -2000 ₩) -type f   (-o : OR, -a : AND)
```

Sticky Bits

파일에 쓰기 권한 없어도 디렉토리에 쓰기 권한이 있는 경우 디렉토리 권한에 의해 파일은 삭제 된다. 특정 디렉토리의 경우 공유의 목적으로 사용 할 때 사용자들에 의해 파일이 생성 될 수 있으나 디렉토리에 파일을 마음대로 삭제 할 수 없도록 sticky 권한을 부여해 줄 수 있다. 디렉토리에 쓰기 (파일생성: touch, vi등) 권한이 있어도 파일에 삭제권한(rm)을 제거하려 할 때 사용한다. 파일의 소유자나 그룹의 경우 또는 관리자의 경우 파일에 대한 소유권을 행사 할 수 경우는 제외 된다. 일반적으로 sticky bit는 디렉토리가 777 권한일 때 사용된다.

ex) 게시판의 경우uid=1002(user01) gid=1002(user01) groups=1002(user01)
context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023

```
특수권한                    소유자권한비트  그룹권한비트  기타권한비트
SetUID  SetGID  sticky-bit r w x      r w x      r w x
0       0       1          1 1 1      1 1 1      1 1 1

        천                  백         십         일
```

[참고] sticky bit로 설정된 디렉토리

```
# ls -ld /tmp
drwxrwxrwt 13 root root 4096 Feb 11 09:22 /tmp
# rm -rf /tmp/*

t권한 표기
# chmod 1777 dir1 (drwxrwxrwx  ->  drwwrwxrwt)
# chmod 1001 dir1 (d--------x  ->  d--------t)
# chmod 1776 dir1 (drwxrwxrw-  ->  drwxrwxrwT)
```

[EX1] sticky bit로 파일 삭제 권한 제거

[TERM1] user01 사용자 터미널

```
# ssh user01@localhost
user01 사용자로 로그인

$ id
uid=1002(user01)                    gid=1002(user01)                    groups=1002(user01)
context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
$ cd /tmp
$ mkdir stickybit
$ echo 1111 > file10
$ echo 2222 > stickybit/file2
$ ls -l | grep user01
-rw-rw-r--. 1 user01 user01  5  6월 26 12:02 file10
drwxrwxr-x. 2 user01 user01 19  6월 26 12:02 sticybit
```

[TERM2] fedora 사용자 터미널

```
# ssh fedora@localhost
fedora 사용자로 로그인

$ id
uid=1001(fedora)                    gid=1001(fedora)                    groups=1001(fedora)
context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
$ cd /tmp
$ mkdir linux
$ echo 3333 > file3
$ echo 4444 > linux/file4
$ ls -l | grep fedora
-rw-rw-r--. 1 fedora fedora  5  6월 26 12:03 file3
drwxrwxr-x. 2 fedora fedora 19  6월 26 12:03 linux
$ vi file3  ($ echo 4444 >> file3 )
3333
4444       <----- '새로운 라인 추가'
$ rm -rf linux
$ rm file3
$

$ ls -l file10
-rw-rw-r-- 1 user01 user01 5  3월 18 12:44 file10
$ rm file10
rm: remove write-protected 일반 파일 'file10'? y
rm: cannot remove 'file1': 명령이 허용되지 않음
$ mv file10 file2
mv: cannot move 'file1' to 'file2': 명령이 허용되지 않음
$ cp file10 file2
$ ls -l file10 file2
-rw-rw-r-- 1 user01 user01 5  4월 14 10:46 file1
-rw-rw-r-- 1 fedora fedora 5  4월 14 10:51 file2
[결론] 자신이 소유권을 가지지 않은 파일에 대해서는 원본 파일을 수정할 수 있는 명령어는 허용되지 않는다.

웹 사이트 게시판에서 많이 사용하는것이 sticyBIT
```

VIM [텍스트 편집기] 사용

VI 편집기

단원목표

- VI 실행
- VI의 세가지 모드 변경 방법
- 입력모드
- 명령모드
- 최하위행 모드

- [ ] 현재 많이 사용되는 편집기 종류
  - vi 편집기(Visual Editor)

  vi 편집기는 유닉스 계열에서 가장 많이 사용되는 텍스트 편집기이다. 1976년 빌 조이(Bill Joy)가 개발하였다. vi 편집기는 한 화면을 편집하는 비쥬얼 에디터(Visual Editor)이다.
  - vim(vi improved)

  브람 무레나르(Bram Moolenaar)가 vi 편집기와 호환되면서 독자적으로 다양한 기능 추가하여 만든 편집기이다. 편집시에 다양한 색상을 이용하여 가시성을 높였으며, 패턴 검색시에 하이라이트 기능을 제공하여 빠른 검색이 가능하게 해준다. (http://www.vim.org)
  - emacs 편집기(Editor Macros)

  리처드 스톨만이 개발한 고성능 문서 편집기이다. 매크로 기능이 있는 텍스트 교정 및 편집기이다. 초기 리처드 스톨만에 의해 개발 되었고, 이후에 제임스 고슬링(James Gosling)이 LISP(LISt Processor) 언어에 의한 환경 설정 및 에디터 기능을 확장시킬 수 있는 기능을 포함하여 '고슬링 이맥스'라는 이름으로 배포하였다.
  - pico(Pine Composer)

  워싱턴 대학의 Aboil Kasar가 개발한 유닉스 기반의 텍스트 에디터로 pine 이메일 클라이언트 프로그램에 통합되어서 배포되었다. pico의 기본 인터페이스는 윈도우의 메모장(Notepad)와 유사하여 매우 단순한다.
  - nano

  pico의 복제 버전이다. nano는 pico의 기능 이외에도 마우스 지원, 자동 들여쓰기, 정규 표현식 검색, 구문 강조 등의 기능을 추가로 제공한다.

VI 실행

```
# vi
# vi filename
# vi -r filename
# vi -L
# vi +38 filename
# vi +/"word" filename
```

vi 편집기 특징

```
▪ VI(VIsual editor)           -> /bin/vi
▪ VIM(Visual editor IMproved) -> /usr/bin/vim

# ls -l /bin/vi
-rwxr-xr-x 1 root root 581K Jan  9  2013 /bin/vi*
# ls -l /usr/bin/vim
-rwxr-xr-x 1 root root 2.7M Jan  9  2013 /usr/bin/vim*


# vi ~/.bashrc     (# gedit ~/.bashrc)
alias vi='/usr/bin/vim'
# . ~/.bashrc      (# source ~/.bashrc)
```

vi('브이아이'로 부른다)는 Emacs와 함께 Unix 환경에서 가장 많이 쓰이는 문서 편집기이다. 1976년 빌 조이가 초기 BSD 릴리즈에 포함될 편집기로 만들었다. vi라는 이름은 한 줄씩 편집하는 줄단위 편집기가 아니라 한 화면을 편집하는 비주얼 에디터(visual editor)라는 뜻에서 유래했다. 간결하면서도, 강력한 기능으로 열광적인 사용자가 많다.

현재는 오리지널 vi를 사용하는 경우는 거의 없고, 일반적으로 기능을 모방하여 만들어진 클론을 사용하고 있다. 이런 클론 중 많이 쓰이는 것은 기능이 다양한 것을 장점으로 내세우며, 리눅스 배포판에 포함되는 Vim, 그리고, BSD 라이선스로 제공되며 원본 vi의 동작과 호환성으로 정평이 나 있는 nvi, 독자적인 팬층을 확보한 elvis등이 있다.

vi의 특징

VI 편집기가 동작하는 원리는 버퍼에서 작업을 하게 된다.
그러므로 저장을 시키는 명령어를 입력하지 않는 이상 디스크상에 파일의 내용이 저장되는 것은 아니다. (.swp 파일 제외)

```
명령어 형식          설명
$ vi              새 파일을 편집할 수 있는 화면이 나온다. 여기서 작업을 한후에는 반드시 파일
이름을 지정하여 저장하여 주어야 한다. 예) : w filename (최하위행 모드)

$ vi filename     filename 이름을 가진 파일이 존재하는 경우 해당 파일을 편집하고 filename 이
름을 가진 파일이 존재하지 않는 경우 새 파일을 편집할 수 있는 화면 상태가 된다.

$ vi -R filename  Readonly, 파일을 Readonly 상태로 열어준다. 중요한 파일, 여러 사람이 동시에
수정가능한 파일을 다룰때 편리하게 사용 될수 있다.view 명령어와 동일한 기능을 수행할 수 있다.
$ view filename

$ vi -r filename  Recovery, 이전 vi 편집 작업 중 비정상적으로 작업이 끝난 경우 편집하던 파일
복구시에 사용된다. 이 경우 사용자의 메일로 복구할 파일에 대한 정보가 오게 된다.

$ vi -L           이전 vi 편집 작업 중 비정상적으로 작업이 끝난 경우 복구할 파일들에 대한 전
체적인 목록을 볼수 있다.
$ vi +38 filename
$ vi -c 38 filename
$ vi +/"school" filename 편집작업에 들어갈 때 특별한 명령어를 수행하면서 시작하는 경우 사용한
다.

                        예) vi +38 filename(38번째 라인부터 시작)
                            vi -c 38 filename(위와 같은 의미)
                            vi +/"school" filename (school 단어가 있는 라인부터 시작)
```

```
# vi -L/* fedora 사용자가 작업중 파일이 비정상 종료 (작업중 shutdown) */
Swap files found:
   In current directory:
      -- none --
   In directory ~/tmp:
      -- none --
   In directory /var/tmp:
1.    file1.swp
          owned by: fedora   dated: Thu Feb 11 09:47:26 2010
         file name: /test/file1
          modified: YES
         user name: fedora   host name: linux200
        process ID: 24710
   In directory /tmp:
      -- none --
# ls -l /var/tmp/* 비 정상 종료된 파일의 목록이 보임 */
total 12
-rw------- 1 fedora fedora 12288 Feb 11 09:47 file1.swp
# vi -r /var/tmp/file1.swp
linux
# vi +/"fedora" /etc/passwd
# vi +30 /etc/passwd
# cat > /test/file1
Linux
Vi Editor
<Ctrl + D>
# su - fedora/* 사용자 전환 */
$ vi -R /test/file1/*
Linux
Vi Editor
[본인 이름 입력]
:wq!                    /* 저장 안됨 q! 로 빠져나옴 */
```

기본 작업 테스트

```
# cd /test
```

filename이라는 파일로 기본 틀 생성

VI의 세가지 모드 변경 방법
vi 편집기에서는 3가지 모드가 지원된다. 명령행 모드(Comand Mode). 입력행 모드(Input Mode), 최하위행(라인) 모드(Last Line Mode)가 지원이 된다. 명령행 모드는 편집작업 및 기타 명령어를 입력할 수 있

는 모드이고 입력행 모드는 입력만 할 수 있는 모드이다. 최하위행(라인) 모드는 저장 및 기타 검색 작업등을 할 수 있는 모드이다.

- [ ] Command Mode(Edit Mode)키 입력을 명령어로 해석 (키보드의 키가 기능키로서 동작을 한다.)
- [ ] Input Mode(Insert Mode) 키 입력을 파일에 입력
- [ ] Last Line Mode(Ex Mode) ex 명령어를 수행

입력행 모드 ((Input Mode)

vi 편집기 실행시 기본 모드는 명령행 모드이다. 명령행 모드에서 입력을 하기 위해서는 입력행 모드로 전환해야 한다. 이런 경우 다음과 같은 문자를 통해 입력 모드로 전환할 수 있다.
• a, A, i, I, o, O

▶ Insert
- i : 현재 커서 위치로부터 입력한다.
- I : 현재 커서가 있는 행의 처음부터 입력한다.

▶ Append
- a : 현재 커서 위치 이후부터 입력한다.
- A : 현재 커서가 있는 행의 마지막부터 입력한다.

▶ Open Line
- o : 현재 커서가 있는 아래행부터 입력한다.
- O : 현재 커서가 있는 위행부터 입력한다.

```
# vi filename
Input : k, j

Input : i

File Contents
============================
hello
hello linux
Welcome To My Server!!!
hello linux
linux CentOS
Have a Good Time !!

============================

2 line : linux -> liNux

move -> delete -> insert -> save & quit

(1). move
h, j, k, l
←  ↓  ↑  →

(2). delete
x, dd

(3). input
i

(4). save & quit
shift + :

: w!
: wq      /* w: write, q: quit */
: q!
: wq!

(5). verification
[TERM1]
# vi filename

[TERM2]
# cd /test
# cp /etc/services /test
# vi services

: set number   (: set nu) 파일의 LINE번호를 붙여 표시해준다
: set nonumber (: set nonu)

: 30     (30G)    nG특정 LINE으로 이동한다
/ftp -> n -> N
```

명령행 모드(Command Mode)

이동(Move)

■ 좌우 이동 : (h, l), (w, b), (0(^), $)
■ 상하 이동 : (j, k), (<CTRL + F>, <CTRL + B>), (H, L), (G, nG, 1G)

[참고] :5 (=5G), :10 (=10G), 5, 10

▪ h: 한 문자 왼쪽으로 이동(←)
▪ l: 한 문자 오른쪽으로 이동(→)
▪ w: 한 단어 오른쪽으로 이동
▪ b: 한 단어 왼쪽으로 이동

▪ 0(^): 라인의 처음 문자(라인의 처음)으로 이동
▪ $: 라인의 마지막으로 이동

▪ j: 한 문자 아래로 이동(↓), 다음 라인으로 이동
▪ k: 한 문자 위로 이동(↑), 이전 라인으로 이동

▪ <CTRL + F>: 다음 페이지로 이동
▪ <CTRL + B>: 이전 페이지로 이동

▪ H : 현재 페이지의 가장 첫번째 줄로 이동
▪ L : 현재 페이지의 가장 마지막 줄로 이동

▪ 1G: 문서의 첫 번째 라인으로 이동 (gg)
▪ G: 문서의 마지막 라인으로 이동

삭제(Delete)

■ 좌우 삭제: x, dw, (d0, d$), dd
■ 상하 삭제: dd, 3dd, (:1,3d), (dG, d1G)

[참고] dd(=D), 3dd(=3D)

▪ x: 현재 커서 한 글자 삭제
▪ dw: 현재 커서 앞의 한 단어 삭제
▪ db : 현재 커서 뒤로 한 단어 삭제
▪ d0/d^: 현재 커서부터 라인의 처음까지 삭제
▪ d$: 현재 커서부터 라인의 마지막까지 삭제
▪ dd: 현재 라인 삭제

▪ 3dd: 현재 커서 라인을 포함해서 아래로 3개 라인 삭제
▪ :1,3d: 1번째 라인부터 3번째 라인까지 삭제

▪ dG: 현재 커서 라인부터 문서 마지막까지 삭제
▪ d1G: 현재 커서 라인부터 문서 처음까지 삭제

Undo

■ u, U
▪ u: 바로 이전에 상태로 되돌림
▪ U: 라인전체에 대해 이전 상태로 되돌림

Join Line

■ J
▪ J: 현재라인에 아래 라인 붙이기

Replace

■ r, R
▪ r: 현재 글자를 대치
▪ R: <ESC>키를 누르기 전까지 연속으로 현재 글자 대치

최하위행(라인) 모드(Last Line Mode)

복사/붙이기(Copy/Yank & Paste)

```
■ yy(=Y), 3yy(3Y), p or P
■ :1,3 co 5
■ :1,3 m 5

▪ yy: 현재 라인 복사(Yank)
▪ 3yy: 현재 커서 라인 포함해서 하위의 3개의 라인 복사
▪ :1,3 co 5: 첫 번째 라인부터 3번째 라인까지 복사하여 5번째 라인 아래에 붙이기
▪ :1,3 m 5: 첫 번째 라인부터 3번째 라인까지 5번째 라인 아래에 이동하기

▪ p : 현재 커서 아래에 붙이기
▪ P: 현재 커서 위에 붙이기
```

검색(Search)

```
■ /New,n or N
■ ?Newn or N

▪ /New: 현재 커서 라인부터 찾으려는 문자열(예: New) 검색
▪ ?New: 문서의 마지막 라인부터 찾을려는 문자열(예: New) 검색

▪ n: n(Next), 정방향다음번째 검색
▪ N: N(Next), 역방향으로 다음번째 검색

/ 정방향 검색
? 역방향 검색
```

최하위행 모드

- 저장 & 종료

:w
:w filename
:w!
:q
:q!
:wq
:wq!
:!cmd

Save & Quit

```
■ :w/* w(write), 현재 파일에 저장 하기 */
■ :w filename/* 다른 이름으로 저장 하기 */
■ :w!(root use)  /* 현재 파일에 강제적으로 저장 하기 */
■ :w! file       /* 현재까지의 변경사항을 file로 저장 */
■ :3,10w file    /* 3번째 라인부터 10번째 라인까지 file로 저장 */
■ :q             /* q(quit), 편집기 종료 */
■ :q!            /* 저장 안하고 편집기 종료 */
■ :wq/* 저장하고 편집기 종료 */
■ :wq!(root Use)/* 현재 파일에 강제적으로 저장하고 편집기 종료 */
■ :r file        /* file의 내용을 현재 커서 위치에서 읽어 들임 */
■ :!CMD          /* vi 편집기를 빠져나가지 않은 상태에서 쉘 명령어를 수행 */
```

[참고] 저장하고 빠져나가기(Save & Quit)의 여러가지 방법

```
■ :x
■ :wq
■ ZZ
```

[참고] vi 편집기 실행에 대해서

```
[TERM1] # vi filename
: wq!
        # cat filename
[TERM2] # vi services
: q!
        # cat services
```

VI 편집기 환경파일 설정

vi 편집기의 동작하는 기능을 변경하기 위해서는 set 명령어를 사용한다. set 명령어 다음에 all을 사용하면 현재 편집기에 사용가능한 모든 기능변수들에 대한 현재 설정값을 표시한다.

vi 편집기의 기능을 현재 실행되는 편집 화면에서만 변경하기 위해서는 최하위행 모드에서 다음과 같은

방법을 사용한다.

```
# cd /test
# vi filename
: set all       (Last Line Mode)
: set number    (: set nu)
: set nonumber  (: set nonu)
--- Options ---
  ambiwidth=single  nohidden            nopreserveindent      termencoding=
noautoindent          history=50          prompt            noterse
noautoread          nohlsearch          noreadonly            textauto
noautowrite         noignorecase          remap             notextmode
noautowriteall        iminsert=0          report=2            textwidth=0
  background=light    imsearch=0          scroll=23         notildeop
nobackup            noincsearch           scrolljump=1        timeout
  backupcopy=auto   noinfercase           scrolloff=0         timeoutlen=1000
  backupext=~       noinsertmode        nosecure            nottimeout
  backupskip=/tmp/*   isprint=@,161-255   shell=/bin/bash     ttimeoutlen=-1
nobinary              joinspaces          shellcmdflag=-c     ttybuiltin
nobomb                keywordprg=man      shellquote=         ttyfast
  buflisted         nolazyredraw          shelltemp           ttyscroll=999
  cmdheight=1         lines=48            shellxquote=        ttytype=xterm
  columns=80        nolist              noshiftround          undolevels=1000
nocompatible          listchars=eol:$     shiftwidth=8        updatecount=200
nocopyindent          loadplugins       noshortname           updatetime=4000
  cpoptions=aABceFs   magic             noshowfulltag         verbose=0
  debug=              matchtime=5       noshowmatch           verbosefile=
nodelcombine          maxcombine=2        showmode          novisualbell
  display=            maxmapdepth=1000    sidescroll=0        warn
noedcompatible        maxmem=257666       sidescrolloff=0   noweirdinvert
  encoding=utf-8      maxmemtot=257666  nosmartcase           whichwrap=b,s
  endofline         nomodeline          nosmarttab            wildchar=<Tab>
  equalalways         modelines=5         softtabstop=0       wildcharm=0
  equalprg=           modifiable          startofline         wildignore=
noerrorbells          modified            swapfile            wildmode=full
  esckeys             more                swapsync=fsync      window=47
noexpandtab           mouse=              switchbuf=          wrap
noexrc                mousemodel=extend   tabstop=8           wrapmargin=0
  fileencoding=       mousetime=500       tagbsearch          wrapscan
  fileformat=unix   nonumber              taglength=0         write
  formatoptions=tcq   operatorfunc=       tagrelative       nowriteany
..... (중략) .....

: set tabstop=10/* 탭간격 조정 */
: set nu/* 라인 번호 달기 */
: set noshowmode/* 상태표시행에 모드를 표시하거나 표시하지 않거나를 설정 */
: set directory=/tmp /* 지정된 기능 변경 */
```

[참고] VIM 편집기의 환경 설정 파일

```
■ 사용자 정의 $HOME/.vimrc 파일 생성
# vi ~/.vimrc   (# vi ~/.exrc)
set number
# vi /etc/passwd
                        1 root:x:0:0:root:/root:/bin/bash
      2 bin:x:1:1:bin:/bin:/sbin/nologin
      3 daemon:x:2:2:daemon:/sbin:/sbin/nologin
      4 adm:x:3:4:adm:/var/adm:/sbin/nologin
■ $HOME/.vimrc 파일의 샘플 파일 생성
# cp /usr/share/vim/vim70/vimrc_example.vim ~/.vimrc

■ VIM 편집기 참고 파일(VIM 편집기 사용법 문서)
# vi /usr/share/vim/vim70/tutor/tutor.ko.ufs-8

■ VIM 편집기의 Plugin 모음들
http://saelly.tistory.com/296
http://ethanschoonover.com/solarized
```

bash shell

단원목표
- 리다이렉션(Redirection)
- 파이프(Pipe)
- 배쉬쉘 기능(Bash Shell Funtion)
- 변수(Variable)
- 메타캐릭터(Metacharcter)
- 히스토리(History)
- 환경 파일(Environmnet File)

쉘은 명령행에서 입력되거나 스크립트 파일에서 읽어 들이는 명령어들을 해석함으로써 사용자와 커널의 중계(Interface) 역할을 담당한다. 사용자가 로그인하면 쉘이 시작되고 사용자가 명령어를 입력하면 쉘은 명령행의 구문을 분석하고, 리다이렉션, 파이프, 와일드 카드, 작업 제어등을 처리하고, 명령어를 PATH변수에서 검색하고 존재하면 실행한다. 쉘의 주요 기능들 중 하나는 명령행 프롬프트에서 입력된 명령어들을 해석하는 대화형 기능이다. 쉘은 명령행에서 입력된 명령어들을 분석해서 '토큰'이라는 단어 단위로 해석한다. 토큰은 탭이나 빈칸, 개행문자 등 [공백]으로 구분한다. 입력된 명령어들에 메타문자들이 포함되어 있으면 적절하게 처리한다. 쉘은 파일 입출력과 백그라운드 작업을 처리하며, 명령행에서 입력이 정상적으로 분석되었을 경우 해당 명령어를 찾아 실행한다.

쉘의 기능을 요약해 보면 다음과 같다.

- 쉘은 PATH 변수에 정의된 모든 디렉토리를 참고 하여 명령어를 찾아 실행한다.
- 쉘은 파이프, 입/출력 리다이렉션, 백그라운드 프로세싱(Background Processing)을 설정한다.
- 쉘은 명령어의 모임인 별칭(alias), 쉘 함수(Function) 등을 찾아 실행시킨다.
- 쉘은 TERM 변수를 사용하여 서로 다른 터미널 환경을 초기화 한다.
- 쉘은 명령어 자동 완성 기능을 사용하여 입력한 명령어를 기록한다.
- 쉘은 사용자 환경 정의 파일을 가지고 사용자의 환경을 초기화 한다.

다음과 같은 쉘의 기능에 대해서 배운다.

■ 쉘의 기능

- 명령어 해석기(Command interpreter)

- 프로그램 언어(Programable Language) ---> 쉘 스크립트(쉘 프로그램)

- 리다이렉션(Redirection)
- 파이프(Pipe)
- 쉘 기능(Shell Function)
- 변수(Variable)
- 히스토리(History)
- 환경파일(Environment Files)

리다이렉션
- 파일기술자
- 입력 리다이렉션(Redirection stdin)
mail –s "OK:linuxXXX" root < report.txt

- 출력 리다이렉션(Redirection stdout)
ls –l > ls.txt

- 에러 리다이렉션(Redirection stderr)
./backup.sh > backup.log 2>&1

방향 재지정 (Redirection)

fd (파일 기술자, File Descriptor)

각 프로세스를 위한 파일 기술자(File Descriptor)가 쉘에 의해 만들어 진다. 한 개의 파일을 열면 그 연 파일을 나타내는 특별한 숫자를 쉘이 만들어 내는데 이것이 파일 기술자이다. 이런 번호들 중 미리 예약 되어져서 파일을 열 때 주어지지 않는 번호가 있는데 아래 표와 같다.

```
파일 기술자(File Description)란?
        프로세스가 파일을 열때 할당되는 번호
        프로세스의 열린 파일을 구분할 때 사용하는 식별 번호

        [참고] C 언어
        # vi test.c
        ------------------------
        int fd;
        fd=open(.....);
        ------------------------
```

[1] 예약되어진 파일 기술자(File Descriptor)

파일기술자 : 축약의미 :설 명

```
0 : stdin : Standard Input, 표준입력, 특별히 입력이 지정되지 않은 경우 키보드에서 입력을 받는다.
1 : stdout : Standard Ouput, 표준출력, 특별히 출력이 지정되지 않은 경우 정상적인 출력 결과를 모니터로 출력된다.
2 : stderr : Standard Error, 표준에러, 특별히 출력이 지정되지 않은 경우 에러의 출력 결과를 모니터로 출력된다.
```

파일 기술자에 대한 정보는 /proc 디렉토리 하위에 존재하는 각 프로세스의 PID(Process ID) 번호 디렉토리 하위에 fd 디렉토리에서 확인할 수 있고 또한 /dev/fd 디렉토리 하위에서도 파일 기술자에 번호를 확인 할 수 있다. 하지만 번호들은 열어 볼 수는 없는 파일들이다. 다음은 파일기술자에 대한 확인을 하기 위한 테스트이다. 가상터미널1에서 /etc/passwd 파일을 열고 다른 가상터미널2에서 현재 동작중인 프로세스를 확인한 후 프로세스가 열고 있는 파일을 확인해 보면 새로운 파일 기술자(File Descriptor)가 생성된 것을 확인한 것이다.

[FD 확인]

[TERM1]

```
# vi /etc/passwd
```

[TERM2]

```
# ps -ef | grep vi    /* vi가 실행된 PID번호를 볼 수 있다 */
root      2077  2045  0 02:26 pts/10   00:00:00 vi /etc/passwd
root      2081 32326  0 02:26 pts/8    00:00:00 grep vi
# cd /proc/2077/fd/* 단, 프로세스의 번호(2077)은 시스템 마다 번호가 다를 수 있다. */
# ls -l/* 파일 기술자 4번이 할당되어 열린 것을 확인 할 수 있다. */
합계 0
lrwx------ 1 root root 64  3월 17 02:27 0 -> /dev/pts/10
lrwx------ 1 root root 64  3월 17 02:27 1 -> /dev/pts/10
lrwx------ 1 root root 64  3월 17 02:26 2 -> /dev/pts/10
lrwx------ 1 root root 64  3월 17 02:27 4 -> /etc/.passwd.swp
# cd
```

[TERM1]

```
# vi /etc/passwd/* vi 편집기 실행 닫기 */
:q
```

[TERM2]

```
# cd /proc/2077/fd/* 실행중인 vi 편집기의 프로세스 번호 */
-bash: cd: /proc/2077/fd: 그런 파일이나 디렉토리가 없음
```

[EX] proc 디렉토리에 프로세스 번호 생성여부 확인

[TERM2]

```
# cd /proc
# ls > /test/proc_`date +%H%M`.txt
```

[TERM1]

```
# vi /etc/passwd
```

[TERM2]

```
# cd /proc
# ps -ef | grep vi
root     12926  8360  0 16:26 pts/4    00:00:00 vi /etc/passwd
root     12928 12761  0 16:26 pts/1    00:00:00 grep vi
# grep 12926 /test/proc_1415.txt
출력값 없음
# ls > /test/proc_`date +%H%M`.txt
# ls /test
proc_1415.txtproc_1418.txt
# grep 12926 /test/proc_1418.txt
12926
```

[TERM1]

```
# vi /etc/passwd
:q
```

[TERM2]

```
# ls > /test/proc_`date +%H%M`.txt
# ls /test
proc_1415.txtproc_1418.txtproc_1422.txt
# grep 12926 /test/proc_1422.txt
출력값 없음
```

[2]표준 입력, 표준 출력

쉘에서는 명령어 입력을 특별한 지정이 없는 한 키보드(Keyboad)에서 받는다. 그리고 명령어의 출력 결과를 모니터에 출력해준다. 이러한 명령어의 출력 결과나 입/출력을 리다이렉션 심볼(Redirection Symbol)을 사용하여 방향을 재지정 할 수 있다.
방향을 재 지정함으로 인해 기본 출력 방향이 모니터인 것을 file로 변경이 가능하다.

```
표준입력(0, stdin) -----> SHELL(bash) -----> 표준출력(1, stdout)
(EX: keyboard)              |               (EX: Screen)
                            |
                            V
                          표준에러(2, stderr)
                            (EX: Screen)
```

```
# ls /var /nodir
ls: /nodir: 그런 파일이나 디렉토리가 없음
/var:
account  db     games  local  mail   opt       run    tux
cache    empty  gdm    lock   named  preserve  spool  www
crash    ftp    lib    log    nis    racoon    tmp    yp
# cat
hello<Enter>       /* 표준 입력(stdin) */
hello                /* 표준 출력(stdout) */
Linux<Enter>
Linux
<Ctrl + D>
# cat  <  /etc/passwd

# cat  >  testfile
hello
Linux
<Ctrl + D>
--------------------------
# cat testfile
hello
Linux
```

[3] 입력 재지정(입력 리다이렉션, Redirection stdin)

입력 재지정은 표준입력(Stdin)에서 입력을 받지 않고, 다른쪽에서 입력을 받는 것을 입력 재지정(Redirection stdin)이라고 한다. 원래 특별한 지정이 없다면, 키보드에서 입력을 받지만 다른쪽 방향에서 입력을 받는 것을 말한다. 파일 기술자 0번은 생략 가능하다.

```
[형식]
CMD   <  filename (표준입력)
CMD  0<  filename(표준입력 - 파일 기술자 0번)
```

[EX1] 리다이렉션 실습

```
# cat < /etc/passwd
root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/sbin/nologin
daemon:x:2:2:daemon:/sbin:/sbin/nologin
.....
# cat 0< /etc/shadow
root:$1$GUT1Ey/2$ltYR1Bjp6er6klmf1/4DV1:14634:0:99999:7:::
bin:*:14634:0:99999:7:::
daemon:*:14634:0:99999:7:::
....
[TERM1]
# wall < /etc/passwd
[TERM2]
# 터미널 확인
```

[4] 출력 재지정(출력 리다이렉션, Redirection stdout)

출력 재지정은 정상정인 출력 결과를 표준출력(모니터)으로 출력을 하지 않고, 다른 쪽으로 출력하는 것을 출력 재지정(Redirection stdout)이라고 한다. 원래 특별한 지정이 없다면, 모니터로 출력되지만 다른 쪽 방향으로 출력하는 것을 말한다. 파일 기술자 1번은 생략 가능하다.

[명령어 형식]

```
CMD   >  filename /* 표준출력, 덮어쓰기 기능 */
CMD  1>  filename /* 표준출력, 덮어쓰기 기능 */
CMD   >> filename/* 표준출력, 이어쓰기 기능 */
CMD  1>> filename/* 표준출력, 이어쓰기 기능 */
```

[EX1] 리다이렉션(Redirection stdout) 실습

```
# cd
# ls -l
drwxr-xr-x 2 root root  4096  3월 15 15:55 Desktop
-rw------- 1 root root  4210  1월 25 22:54 anaconda-ks.cfg
-rw-r--r-- 1 root root 54631  1월 25 22:54 install.log
-rw-r--r-- 1 root root  9641  1월 25 22:50 install.log.syslog
# ls -l > lsfile     (# ls -l 1> lsfile)
# cat lsfile
drwxr-xr-x 2 root root  4096  3월 15 15:55 Desktop
-rw------- 1 root root  4210  1월 25 22:54 anaconda-ks.cfg
-rw-r--r-- 1 root root 54631  1월 25 22:54 install.log
-rw-r--r-- 1 root root  9641  1월 25 22:50 install.log.syslog
-rw-r--r-- 1 root root     0  3월 17 02:32 lsfile
```

[EX2] > , >> 차이점

```
# echo 1111
1111
# echo 1111 > filename1
# cat filename1
1111
# echo 2222 > filename1 /* 덮어쓰기 기능 */
# cat filename1
2222
# echo 1111 >> filename1 /* 이어쓰기 기능 */
# cat filename1
2222
1111
```

[2] 에러 재지정(에러 리다이렉션, Redirection stderr)

에러 재지정은 표준 에러로 출력하지 않고, 다른 쪽으로 출력하는 것을 에러 재지정(Redirection stderr)이라고 한다. 원래 특별한 지정이 없다면, 모니터로 출력을 하지만 다른 쪽으로 출력하는 것을 말한다. 파일 기술자 2번은 생략 불가능하다. 방향 재지정자를 1번과 동일하게 사용하여 1번 파일 기술자와 구별하기 위해 생략 불가능하다.

[명령어 형식]

```
CMD  2>  filename/* 표준에러출력 */
CMD  2>> filename /* 표준에러출력 */
```

[EX] 표준 에러 실습

```
# cd /test ; rm -rf /test/*
# touch file1 file2 file3

# ls -l /test /nodir
--------------------------------------
정상출력
에러출력
--------------------------------------

# ls -l /test /nodir > dirfilename1
--------------------------------------
정상출력
에러출력
--------------------------------------
# cat dirfilename1
-rw-r--r--  1 root root    0  3월 17 02:34 dirfilename1
-rw-r--r--  1 root root    0  3월 17 02:33 file1
-rw-r--r--  1 root root    0  3월 17 02:33 file2
-rw-r--r--  1 root root    0  3월 17 02:33 file3
# ls -l /test /nodir 2> dirfilename2
--------------------------------------
정상출력
에러출력
--------------------------------------

# cat dirfilename2
ls: /nodir: 그런 파일이나 디렉토리가 없음
# ls /test /nodir > dirfilename1 2> dirfilename2
--------------------------------------
정상출력
에러출력
--------------------------------------

# cat dirfilename1
# cat dirfilename2

# ls /test /nodir > dirfilename 2>&1
# cat dirfilename
```

(실무 예) script.sh(스크립트) 로그 파일 생성

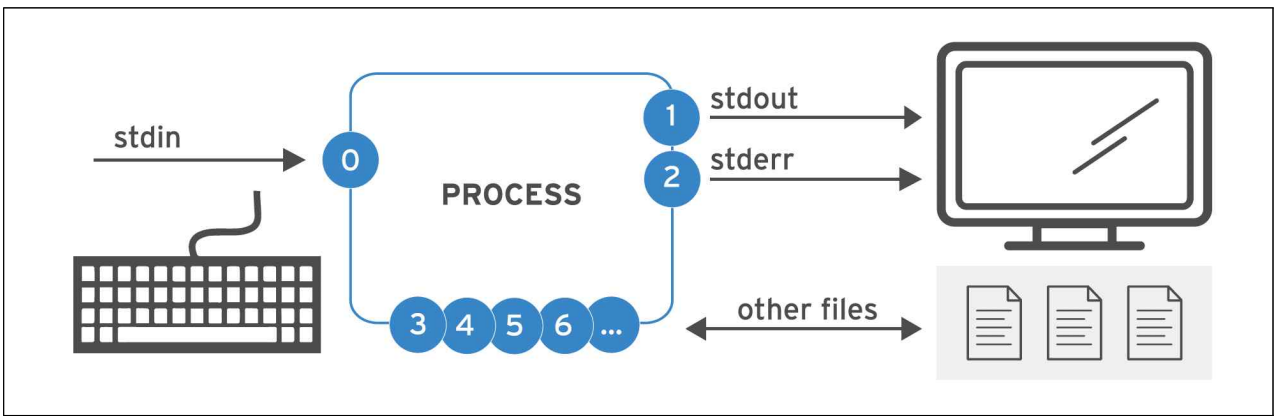
```
(X) # ./script.sh > file.log
(O) # ./script.sh > file.log 2>&1
```

(실무 예) ./configure —prefix=/usr/local/apache2

```
# ./configure --perfix=/usr/local/apache2 2> file.log
# ./configure --perfix=/usr/local/apache2 > file.log 2>&1
```

(실무 예) 일반 사용자(EX: oracle, wasuser)가 검색하는 경우

```
$ find / -name core -type f
$ find / -name core -type f 2> /dev/null
```



출력 리디렉션 연산자

번호	채널 이름	설명	기본 연결	사용법
0	**stdin**	표준 입력	키보드	읽기 전용
1	**stdout**	표준 출력	터미널	쓰기 전용
2	**stderr**	표준 오류	터미널	쓰기 전용
3+	**파일 이름**	기타 파일	none(없음)	읽기 및/또는 쓰기

>> file 2>&1 &>> file	**stdout** 및 **stderr**을 리디렉션 하여 동일한 파일에 첨부	

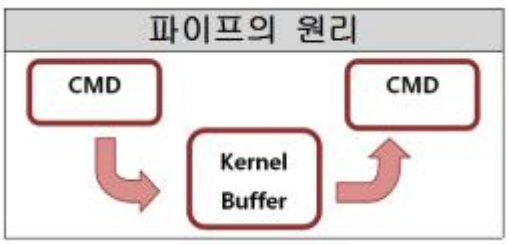
파이프(PIPE)

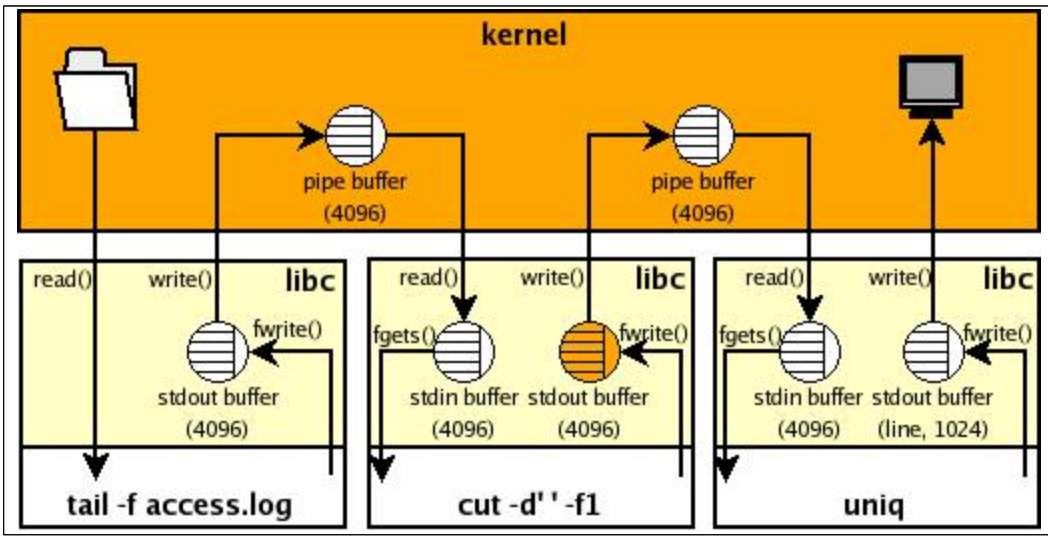
- CMD | CMD
ps – ef | more
ps –ef | grep systemd
CMD > file.log (# CMD >> file.log)
CMD | tee file.log (# CMD | tee –a file.log)

파이프

파이프(Pipe)란? 앞에 실행한 명령어의 출력 결과를 뒤에 실행하는 명령어의 입력 값으로 넣어 준다. 앞의 출력 결과를 뒤에 실행하는 명령어의 입력 결과로 보내줌으로 인해 파이프(|)를 사용하면 여러 명령을 동시에 연결하여 사용 할 수가 있다.

ls -al 명령을 사용하게 되면 모든 파일을 보여주므로 한 화면을 넘는 많은 파일을 보여 주게 될 때 "ls -al | more" 를 쓰게 되면 ls 명령에 대한 결과를 more 명령을 통해 확인해 볼 수 있다.





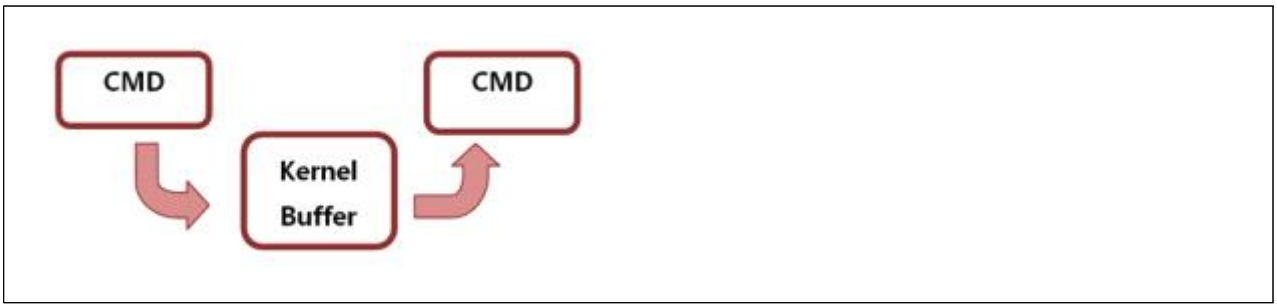
첫 번째 CMD로 출력된 내용을 화면에 그대로 출력하지 않고 파이프 버퍼에 data를 저장 해 두었다가 버퍼에 저장된 결과 값을 가지고 뒤에 있는 CMD의 입력 값으로 넘겨 뒤에 있는 명령어가 실행한 결과를 출력해 주게 된다.

[명령어 형식]

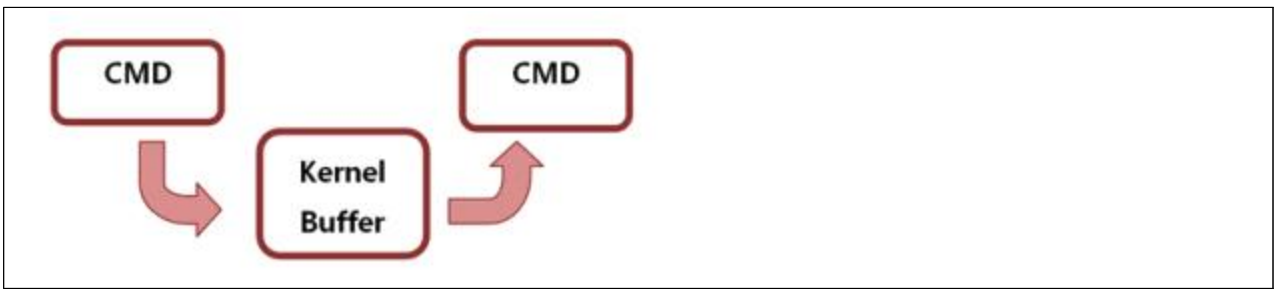
```
# CMD | CMD
```

[EX1] '|'(Pipe) 실습

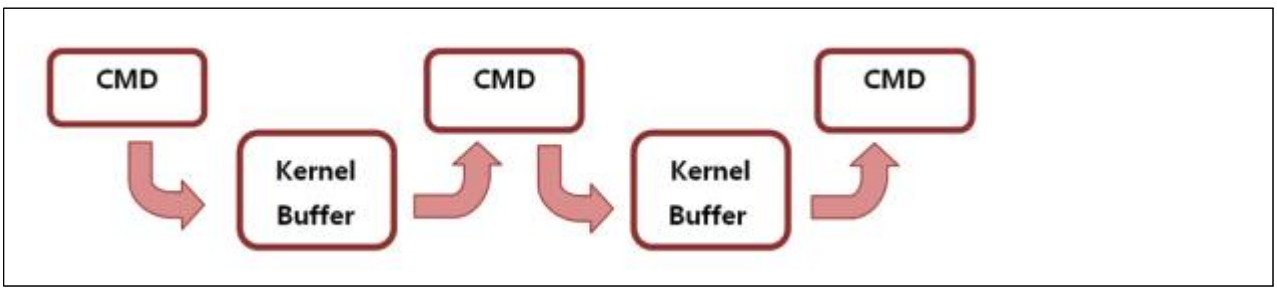
```
# ps -ef          /* 프로세스의 모든 정보 출력 */
# ps -ef | more   /* CMD | more : 한 페이지가 넘는 페이지 출력 방법 지정 */
```




```
# ps -ef | grep sshd    /* 프로세스 결과중에서 sshd 패턴 검색 */
```




```
# ls -al / | sort -r | more /* CMD | sort를 하여 buffer에 넣고 buffer 내용을 한 화면 단위로
출력 */
```



ls 명령을 통해 출력된 결과를 sort 명령으로 정렬하고 more명령을 통해 보기 쉽게 출력하는 세 가지 명령을 사용하고 있다.

```
[참고] 명령어 비교
-------------------------+---------------------------
# ps -ef > file.log      | # ps -ef | grep inetd
# grep inetd file.log    |
-------------------------+---------------------------
```

[참고] Pipe와 함께 사용 할 수 있는 명령어

- tee란? 파이프 중간에 사용하여 입력을 출력으로 보내기 전에 파일로 기록

```
# man tee
개요
       tee  [-ai] [--append] [--ignore-interrupts] [--help] [--version] [화일...]

설명
       이 맨페이지는 GNU 버전의 tee 를 다룬다.  tee 명령은 표준입력으로부터의  입
       력을 표준출력 또는 인수로 주어진 화일에 복사한다.
1
       씌 여질 화일이 존재하지 않으면 생성된다. 만약 이미 존재한다면 -a 옵션을 사
       용하지 않는 한 이전의 자료는 모두 지워진다.

   옵션
       -a, --append
              덮어쓰지 않고 주어진 화일에 표준입력을 추가한다.
```

[EX2] 출력 결과를 파일로 저장

```
        # CMD > file.log
        # CMD | tee file.log

# cal 2002 > file.log
# cal 2002 | tee file2.log
```

[EX3] 실시간 모니터링 하면서 파일로 내용을 저장

```
# while true
> do
> echo
> echo "====================`date`======================"
> ps -ef | head -1
> ps -ef | grep testfile
> sleep 2
> done
```

```
# while true
> do
> echo
> echo "===========`date`=========="
> ps -ef | grep testfile | tee -a file.log    (> w root >> file.log)
> sleep 2
> done
```

[EX4] 다른 터미널 창에서 입력하는 내용이 그대로 지정한 터미널 창으로 출력 되도록 설정

```
[참고] script CMD
# cd /test ; rm -rf /test/*
# script -a file.log
# ls
# date
# cal
# exit
# cat file.log
```

[EX4] 다른 터미널 창에서 입력하는 내용이 그대로 지정한 터미널 창으로 출력 되도록 설정

```
# tty
/dev/pts/1
--> TERM3 에서 입력하는 내용과 출력하는 내용이 그대로 화면에 출력 된다.
```

[TERM2] 두번째 윈도우

```
# tty
/dev/pts/2
```

[TERM3] 세번째 윈도우

```
# script -a /dev/null | tee /dev/pts/1 | tee /dev/pts/2
# ls
# date
# cal
# exit
```

배쉬쉘 기능(Bash Shell Funtion)
- bash shell function
set –o
set –o vi
set +o vi
set –o ignoreeof
set –o noclobber

bash 쉘의 기능(bash Shell Function)

```
# set -o/* 쉘 자체의 기능 전체 목록 확인 */
# set -o vi /* 쉘 자체의 기능 중 vi 기능을 ON */
# set +o vi /* 쉘 자체의 기능 중 vi 기능을 OFF */
```

bash 쉘의 기능

```
allexport            off
braceexpand          on
emacs                on/* 기본적인 편집기 선언 */
errexit              off
errtrace             off
functrace            off
hashall              on
histexpand           on
history              on
ignoreeof            off/* 로그아웃 방지 기능 선언 */
interactive-comments on
keyword              off
monitor              on
noclobber            off/* 덮어쓰기 방지 기능 선언 */
noexec               off
noglob               off
nolog                off
notify               off
nounset              off
onecmd               off
physical             off
pipefail             off
posix                off
privileged           off
verbose              off
vi                   off/* 기본적인 편집기 선언 */
xtrace               off
```

쉘 프롬프트 상태에서 <Ctrl+D>의 역할은 파일의 끝을 알려주거나 현재 쉘프롬프트를 종료하는 기능을 가지고 있다. 잘못하여 <Ctrl+D>를 누르게 되면 관리자가 작업하던 중 쉘프롬프트가 종료되면서 로그아웃이 되어 버린다. 관리자가 작업 중에 <Ctrl+D>를 눌러 로그아웃이 되는 것을 막기 위해 쉘의 기능중 ignoreeof를 이용하여 로그아웃 방지 기능을 설정 하도록 한다. <Ctrl+D>를 이용한 로그아웃 방지 기능이 설정되면 exit를 이용하여 로그아웃을 하도록 한다.

```
set -o ignoreeof
```

EOF(End Of File)의 의미를 갖는 키 <CTRL + D>

① 파일의 끝(EOF)
② 현재 쉘(프로그램) 종료

[EX1] 로그아웃 방지 기능

```
# ssh team01@localhost
Trying 192.168.10.252...
Connected to 192.168.10.252.
Escape character is '^]'.

Kernel 3.10.0-1062.el7.x86_64 on an x86_64
server2 login: root
Password:
Last login: Mon Mar 16 15:43:52 from 192.168.10.200
[root@server2 ~]#
[root@server2 ~]# hostname
[root@server2 ~]# id
[root@server2 ~]# ps
[root@server2 ~]# <CTRL + D>
[root@server2 ~]# bash
[root@server2 ~]# set -o | grep ignoreeof
ignoreeof       off
[root@server2 ~]# set -o ignoreeof /* ignoreeof 기능 on */
[root@server2 ~]# <CTRL + D> Use "exit" to leave the shell. /* 로그아웃 기능 */
[root@server2 ~]# exit
#
```

[참고] 사용자 계정의 자동 로그아웃 기능

```
윈도우에서 사용자가 아무런 동작을 하지 않으면 자동 로그아웃이 되는 기능이 있듯이 Linux에서도
사용자가 지정된 시간동안 아무런 동작을 하지 않으면 자동 로그아웃이 되는 기능이 있다.

▶ /etc/profile  환경 파일 : 모든 사용자에게 적용가능
# vi /etc/profile
...
 HOSTNAME='/bin/hostname'
 ...
 TMOUT=3600          /* 자동 로그아웃 되는 시간 : 초단위로 지정 */
 ...
▶ .bash_profile 환경파일 : 사용자별 홈 디렉토리에 있는 파일을 변경
# vi ~/.bash_profile
...
TMOUT=60             <-- 라인 추가
export PATH TMOUT    /* 지역변수를 환경 변수로 적용 */
...
```

[2] 덮어쓰기 방지기능 (noclobber)

```
방향 재 지정자(Redirection Stdout, '>')를 이용하면 기존에 존재하는 파일이름에 새로운 내용으로
덮어쓰기가 가능 했었다. 중요한 파일의 경우 파일의 이름이 존재하는지 모른 채 덮어쓰기를 할 우
려가 있기 때문에 덮어쓰기 방지 기능을 이용하여 중요한 파일등에 덮어쓰기가 되지 않도록 파일의
내용을 보호 할 수가 있다.
set -o noclobber
```

```
# set -o | grep noclobber
noclobber       off
# set -o noclobber /* 덮어쓰기 방지 기능 on */
# set -o | grep noclobber
noclobber       on
# echo 1111 > test1
# echo 2222 > test1 /* 덮어쓰기가 되지 않는다 */
-bash: test1: cannot overwrite existing file
# cat test1
1111
# set +o noclobber/* 덮어쓰기 방지 기능 off */
# echo 2222 > test1
# cat test1
2222
# set -o noclobber/* 덮어쓰기 방지 기능 on */
# echo 3333 >| test1 /* 덮어쓰기 방지 무시 */
3333
```

<Tab> 파일이름 자동 완성 기능

디렉토리에 있는 파일이나 디렉토리의 첫 번째 문자만 입력 후 Tab키를 누르면 첫 글자로 시작하는 파일이나 디렉토리의 나머지 글자가 자동으로 완성된다. 단, 첫 문자가 같은 디렉토리나 파일이 있는 경우 Tab키 한번만 누르면 아무 변화도 없지만 Tab키 두 번 연속으로 누르면 동일하게 시작하는 모든 디렉토리나 파일을 출력해 준다.

[명령어 형식]

```
# ls t[TAB]
```

[EX1] <TAB> 이용한 디렉토리 이동

```
# cd /etc/sysco<TAB>
/etc/sysconfig/
# cd /etc/sysconfig/netw<TAB>
/etc/sysconfig/network
# cd /etc/sysconfig/network-s<TAB>
/etc/sysconfig/network-scripts/
```

[EX1] <TAB> 이용한 디렉토리 이동

```
# cd /etc/sys<TAB><TAB>
sysconfig/ sysctl.conf  syslog.conf
# cd /etc/sysco<TAB> /* 다른 문자와 구별 할 수 있는 글자까지 적어주고 다시<Tab>을 누르면 완성
*/
/etc/sysconfig
```

[EX3] <TAB> 이용한 명령어의 종류 확인

```
# ch[TAB][TAB]
chacl                check-binary-files  chmod
chage                checkXML            chown
change_console       checkmodule         chpasswd
charmap              checkpolicy         chroot
chat                 chfn                chrt
chattr               chgrp               chsh
chcat                chkconfig           chvt
chcon                chkfontpath
# systemctl [TAB][TAB]
add-requires           hybrid-sleep           reload-or-restart
add-wants              is-active              reload-or-try-restart
cancel                 is-enabled             rescue
cat                    is-failed              reset-failed
condreload             is-system-running      restart
condrestart            isolate                set-default
condstop               kexec                  set-environment
daemon-reexec          kill                   set-property
daemon-reload          link                   show
default                list-dependencies      show-environment
delete                 list-jobs              snapshot
disable                list-sockets           start
edit                   list-timers            status
emergency              list-unit-files        stop
enable                 list-units             suspend
exit                   mask                   switch-root
force-reload           poweroff               try-restart
get-default            preset                 unmask
halt                   reboot                 unset-environment
help                   reenable
hibernate              reload
```

[EX3] <TAB> 이용한 명령어의 종류 확인

```
                        RHEL 3.X(CentOS 3.X) : redhat-config-*
       REEL 4.X(CentOS 4.X) : system-config-*   / service
       REEL 5.X(CentOS 5.X) : system-config-*   / service
       REEL 6.X(CentOS 6.X) : system-config-* ( 기본설치X ) /service systemctl
                                     # yum -y install system-config-*
       REEL 7.X(CentOS 4.X) : systemctl
# set -o vi
(ㄱ) 파일이름 자동 완성 기능(File name completion)
(ㄴ) 명령행 편집(Command Line Edit)

(ㄱ) 파일 이름 자동 완성 기능
# cd /etc/sysco[TAB]
# cd /etc/sysco[ESC][W]

(ㄴ) 명령행 편집
(이전 명령어를 되살려서 쓰는 경우)
# ↑↑↑↓↓↓
# [ESC][k][k][k][j][j][j]

(이전 명령어를 편집해서 사용하는 경우)
# find / -name core -type f
<CTRL + C>
# [ESC][k]
# find / -name file1 -type f

# cp /etc/service file1
# cp file1 file2
# cp file1 file3
# cp file1 file4
# cp file1 file5

(이전에 사용한 명령어를 확장해서 사용하는 경우)
# df -h
# df -h /
# df -h / | tail -1
# df -h / | tail -1 | awk '{print $5}'

# ps -ef | head
# ps -ef | head | sort -k 2
# ps -ef | head | sort -k 2 -n
# ps -ef | head | sort -k 2 -nr

(확인 + 명령어 수행 + 확인)
# chkconfig --list krb5-telnet
# chkconfig krb5-telnet on
# ↑↑
```

변수(Variable)

- 변수의 종류
지역 변수(Local Variable)
환경 변수(Environment Variable)
특수 변수(Special Variable)

- 변수 선언 방법
VAR=5 ; export VAR
echo $VAR
unset VAR

변수 export란?

변수

변수의 종류

지역변수(Local Variable) : # VAR=5

환경변수(Environment Variable) : # export VAR=5

특수변수(Special Variable) : $$, $?, $!, $0, $1, $#, $*, ...

[1] 지역변수 선언

```
변수 선언 방법(EX: bash)
        # VAR=5        (# export VAR=5)
        # export VAR
        # echo $VAR    (# print $VAR)
        # unset VAR
```

지역 변수는 현재 사용하고 있는 쉘에만 적용되는 변수 값으로 선언 하는 것이다. 서브 쉘에서 지역 변수 값을 확인하려고 할 경우 값이 나타나지 않는다.

```
var=5                     echo $var (0)
----------+              +----------
          |              |
          | echo $var (X) |
          +--------------+
```

```
# var=5
# echo $var
5
# var=hello
# echo $var
hello
# bash
# echo $var
#

# exit
# echo $var
```

[2] 환경변수 선언

환경변수는 현재 쉘과 서브쉘에 변수를 모두 적용되는 변수 값으로 선언해 주는 것이다. 환경변수를 사용하는 이유는 이전쉘에서 선언한 변수 값을 서브쉘에서도 동일하게 적용하기 위한 것이다.

```
export var=hello    echo $var (0)
----------+              +----------
          |              |
          | echo $var (0) |
          +--------------+
```

```
# var=hello
# export var
# echo $var
hello
# bash
# echo $var
hello
# exit

# echo $var
hello
export var=hello                  echo $var
----------+                             +----------
          |                             |
          | export var=centos           |
          +--------------+              |
                         |              |
                         | echo $var    |
                         +--------------+
```

```
# var=hello     (# export var=hello)
# export var
# echo $var
hello
# bash
# echo $var
hello
# export var=centos
# echo $var
centos
# bash
# echo $var
centos
# ps
PID TTY          TIME CMD
14072 pts/2    00:00:00 bash
14075 pts/2    00:00:00 bash
14078 pts/2    00:00:00 bash
# exit
# exit
# echo $var
hello
```

```
(결론) 환경 변수는 자신이 선언한 쉘의 서브쉘에서 적용되는 것이다. 상위의 쉘에는 적용되지 않는다.

[참고] env와 set의 차이점 (선언된 변수를 확인하는 명령어)
```

```
■ set : 모든 변수에 대해 출력 (지역변수 + 환경변수)
■ env : 환경변수만 출력

[EX] set/env 명령어 실습
# var1=CentOS
# export var2=Linux

# set | grep var1
var1=CentOS
# set | grep var2
# bash
# env | grep var1
#
-> 출력되는 정보가 없다.

# env | grep var2
var2=Linux
```

[3-1] 시스템 지역변수

[3-1-1] PS1 명령어 프롬프트에 나타내는 시스템 환경 변수

```
# <- 현재 표시되는 이 문자에 대한 시스템 환경 변수를 표시한다.
```

```
[root@server1 ~]# echo $PS1
[₩u@₩h ₩W]₩$

                PS1='[₩u@₩h ₩W]₩$ '
                ------------------------------------------------------------
                ₩u   : username
                ₩h   : hostname
                ₩W   : working directory (예: /home/user01 -> user01)
                ₩w   : working directory (예: /home/user01 -> /home/user01)
                ₩$   : root(#), user01($)
                ------------------------------------------------------------
        현재 경로만 표시
# PS1='₩w> ' /* 윈도우 처럼 절대 경로가 출력 '[절대경로]> ' */
/root>
                # PS1='$PWD> '
                # PS1='₩w> '
# set | grep PS1
$PWD>
# env | grep PS1
#

/root> bash
[root@linux200 ~]#
# PS1='[$PWD]# '
[/root]#
                # PS1='[$PWD]# '
                # PS1='[₩w]# '
[/root]# bash
[root@linux200 ~]#
# export PS1='[₩u@$PWD]# '
[root@/root]#
                # export PS1='[₩u@$PWD]# '
                # export PS1='[₩u@₩w]₩$ '
# bash
[root@/root]#
# export PS1='[₩u@₩h ₩w]₩$ '
[root@linux200 ~]#
# exit
# exit
# exit
#
```

(실무 예) PS1='[₩u@₩h ₩w]₩$ '

```
# vi ~/.bashrc
-------------------------------
.....
#
# Variable Definition
#
export PS1='[₩u@₩h ₩w]₩$ '
-------------------------------

# .  ~/.bashrc   (# source ~/.bashrc)
```

[참고] 디렉토리 이동시 경로 표시하기

```
# vi .bashrc
function cd { builtin cd $* && pwd ; }  디렉토리 이동후 경로 표시

저장후 쉘을 새로 오픈한다.
```

[참고] PS1 시스템 환경 변수 설정하는 방법

```
쉘프롬프트에 보여지는 항목을 설정하는 것으로 아래와 같은 것들을 사용할 수 있다.
ex) [로그인한 계정@호스트네임 현재경로] #        /* 현재 상태 $ : 프롬프트($, root일 경우 #)
*/
---------------------------------------
₩t     현재 시간을 HH:MM:SS 형식으로 표시
₩d     날자를 "요일 월 일" 형식으로 표시 (예, "Tue May 26")
₩n     개행문자
₩s     쉘의 이름, $0 의 베이스 이름 (마지막 슬래쉬 뒷 부분)
₩w     현재 작업 디렉토리
₩W     현재 작업 디렉토리의 베이스 이름
₩u     현재 사용자의 사용자명
₩h     호스트 이름
₩#     이 명령의 명령 번호
₩!     이 명령의 히스토리 번호
₩$     유효 UID가 0 이면 if the effective UID is 0, a #, 그렇지 않으면 $
₩nnn   팔진수 nnn에 해당하는 문자
₩₩     백슬래쉬
₩[     비출력 문자의 시퀀스를 시작한다. 프롬프트에 터미널 제어시퀀스를 넣을때 사용한다.
₩]     비출력 문자의 시퀀스를 마친다.
```

[3-1-2] PS2 명령어가 아직 끝나지 않았음을 나타낼 때 사용

```
PS2='> '
# echo $PS2
>
# ls -a -l
# ls -a ₩
> -l

# echo "hello linux
> "

# while true
> do
> CMD
> sleep 5
> done
```

```
# echo $PS2
>
```

[3-2] 시스템 환경변수

```
# echo $HOME
/root
# cd $HOME
# pwd
/root
# echo $PWD
/root
# echo $LOGNAME
root
# echo $TERM
xterm
# echo $LANG
ko_KR.UTF-8
# echo $USER
root
# echo $UID
0
```

[4] 알아두면 좋은 특수 변수

① $: 현재쉘의 PID를 저장하고 있다. 쉘 스크립트내에서 임시 파일의 이름을 지정할 때 보통 사용이 된다.

```
# echo $$
11991

# ps
PID TTY          TIME CMD
11991 pts/1    00:00:00 bash
```

② ? : 바로 이전 명령어의 정상 실행 여부에 대한 결과값이 들어 있다. 쉘 스크립트내에서 이전 명령어의 정상 수행 여부를 확인할 때 주로 사용된다.

```
# ls /nodir
ls: /nodir: No such file or directory
# echo $?
2
# cd /nodir
-bash: cd: /nodir: No such file or directory
# echo $?
1
# no
-bash: no: command not found
# echo $?
127
# ls
Desktop          hash.tar.gz         mbox              top.seceret.mail.gpg
anaconda-ks.cfg  install.log         serv.pub
hash.md5         install.log.syslog  top.seceret.mail
# echo $?
0
```

(실무 예) 백업 스크립트 작성시

```
# vi backup.sh
-------------------------------------------------------
#!/usr/bin/bash
tar cvzf /backup/backup.tar.gz /home
        if [ $? -eq 0 ] ; then           /* -eq : equal */
        echo "success"
        else
        echo "error"
        fi

-------------------------------------------------------
```

[참고] /root/bin/script.sh 편하게 실행하기

관리자가 생성한 쉘 스크립트(쉘 프로그램)들을 모아 놓은 디렉토리

```
/root/bin or /root/shell
```

```
# echo $PATH
/usr/lib/qt-3.3/bin:/usr/kerberos/sbin:/usr/kerberos/bin:/usr/local/sbin:/usr/local/bin:/sbin
:/bin:/usr/sbin:/usr/bin:/root/bin
                PATH : 명령어들이 위치한 디렉토리를 선언할 때 사용하는 변수
                # ls -al
                (절대경로) # /bin/ls -al
                (상대경로) # cd /bin
                           # ./ls -al

# mkdir -p /backup
# mkdir -p /root/bin
# vi /root/bin/backup.sh
-------------------------------------------------------
#!/bin/bash

tar cvzf /backup/back-`date +%m%d`.tar.gz --absolute-name /test
-------------------------------------------------------
# chmod 755 /root/bin/backup.sh

# export PATH=$PATH:/root/bin
# backup.sh
# cd /backup ; ls
back-0520.tar.gz
```

[참고] 백업 스크립트 실패 유무 확인

```
# cd /test
# vi script.sh
-------------------------------------------------------
#!/bin/bash

/root/bin/backup.sh
                if [ $? -eq 0 ] ; then              /* -eq : equal */
                echo "Backup Success"
                else
                echo "Backup Fail"
                fi
-------------------------------------------------------
# chmod 755 /test/script.sh
# ./script.sh
-------------------------------------------------------
                /test/
                /test/script.sh
                Backup Success
-------------------------------------------------------
# vi /root/bin/backup.sh
-------------------------------------------------------
#!/bin/bash

                tar cpvfz /backup/back-`date +'%m%d'`.tar.gz --absolute-name /nodir
-------------------------------------------------------
# ./script.sh
                tar: /nodir: Cannot stat: 그런 파일이나 디렉토리가 없음
                tar: Error exit delayed from previous errors
                Backup Fail
# ./script.sh > backup.log 2>&1
# cat backup.log
                tar: /nodir: Cannot stat: 그런 파일이나 디렉토리가 없음
                tar: Error exit delayed from previous errors
                Backup Fail
```

③ ! : 바로 이전에 백그라운드로 실행한 프로세스의 PID번호가 저장 된다.

```
# sleep 300 &
[1] 25192
                [1]   : Job ID
                25192 : PID
# echo $!
25192
# ps
PID TTY          TIME CMD
24868 pts/1    00:00:00 bash
25101 pts/1    00:00:00 bash
25192 pts/1    00:00:00 sleep
25219 pts/1    00:00:00 ps
# sleep 400 &
[2] 25233
# ps
PID TTY          TIME CMD
24868 pts/1    00:00:00 bash
25101 pts/1    00:00:00 bash
25192 pts/1    00:00:00 sleep
25233 pts/1    00:00:00 sleep
25234 pts/1    00:00:00 ps
# echo $!
25233
```

④ 인자변수(Argument Variable)

```
$로 표시하여 인자변수의 값이 몇 번째인지 나타낼 수 있다.   인자변수1 | 인자변수2 | 인자변수3
                # ls -a
                # vi ls
                ----------------------------------
                ..... (중략) .....
                echo "$1"
                ..... (중략) .....
                ----------------------------------
```

```
# cd /test
# vi test.sh
-------------------------------------
                #!/usr/bin/bash

                echo $1                     /* 첫번째 인자 */
                echo $2                     /* 두번째 인자 */
                echo $3                     /* 세번째 인자 */
-------------------------------------
# chmod 755 test.sh
# ls -l test.sh
-rwxr-xr-x 1 root root    38 Aug 19 14:22 test.sh
# ./test.sh test1 test2 test3
test1
test2
test3
# date
Thu Aug 19 14:24:33 KST 2010
# set $(date)
# echo $1
Thu
# echo $2
Aug
# echo $3
19
# echo $4
14:24:47
# echo $5
KST
# echo $6
2010
```

[5] PATH 변수

명령어를 검색할 디렉토리를 선언할 때 사용하여 절대 경로를 쓰지 않고 실행파일명만을 이용해서 사용할 수 있게 한다. which라는 명령도 PATH에 설정되어 있는 디렉토리의 파일의 경로만을 알려준다.

```
# gedit &
# which gedit
/usr/bin/gedit
# echo $PATH
/usr/lib/qt-3.3/bin:/usr/kerberos/sbin:/usr/kerberos/bin:/usr/local/sbin:/usr/local/bin:/sbin
:/bin:/usr/sbin:/usr/bin:/root/bin:/root/bin
# cd
# ls -l /test/script.sh
-rwxrwxrwx 1 root root 109  5월 20 15:26 /test/script.sh
```

(필요하면 아래와 같이 /test/script.sh 스크립트를 생성한다.)

```
# vi /test/script.sh
------------------------------
#!/bin/bash

echo "script.sh Excution"
------------------------------
# chmod 755 /test/script.sh
```

```
# script.sh
-bash: script.sh: command not found
                (절대경로)# /test/script.sh
                (상대경로) # cd /test
                # ./script.sh
                # script.sh
```

```
# vi ~/.bash_profile
# .bash_profile

# Get the aliases and functions
if [ -f ~/.bashrc ]; then
        . ~/.bashrc
fi

# User specific environment and startup programs

PATH=$PATH:$HOME/bin
PATH=$PATH:/test                  <----- 라인 추가

export PATH
unset USERNAME

# .  ~/.bash_profile
# echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/root/bin:/root/bin:/custom
# script.sh
script.sh Excution
```

[6] HISTTIMEFORMAT 변수

```
# export LANG=C
# man bash
------------------------------
/HISTTIMEFORMAT
       HISTTIMEFORMAT
              If this variable is set and not null,  its  value
              is  used  as  a  format string for strftime(3) to
              print the time stamp associated with each history
              entry  displayed by the history builtin.  If this
              variable is set, time stamps are written  to  the
              history  file  so  they  may  be preserved across
              shell sessions.
------------------------------
# man 3 strftime
------------------------------
/%F
       %F     Equivalent  to  %Y-%m-%d  (the ISO 8601 date for-
              mat). (C99)
/%T
       %T     The time in 24-hour notation (%H:%M:%S). (SU)
# vi /etc/profile
..... (중략) .....
for i in /etc/profile.d/*.sh ; do
    if [ -r "$i" ]; then
        if [ "${-#*i}" != "$-" ]; then
            . $i
        else
            . $i >/dev/null 2>&1
        fi
    fi
done

#
# (1) Sfecific Configuration
#
export HISTTIMEFORMAT="%F %T     "

unset i
unset pathmunge
------------------------------
# telnet localhost
root 사용자로 로그인

# history
..... (중략) ....
  776  10:07:00     clear
  777  10:07:11     clear
  778  10:10:42     clear
  779  10:12:43     export LANG=C
  780  10:13:12     man bash
  781  10:15:13     clear
  782  10:15:16     man 3 strftime
  783  10:17:49     vi /etc/profile
  784  10:20:15     history
# exit
#
```

[7] 메타캐릭터(Metacharacter)

- ''(작은 따옴표(single quotation))
- ""(큰 따옴표(Double quotation))
- ``(역 따옴표(Back quotation))
- ₩ (역 슬래쉬(Back slash))
- ; (세미콜론(Semicolon))

■ ''(작은 따옴표(single quotation))

쉘이 해석 할 수 없도록 막아 준다.

```
# echo $HOME
/root
# echo '$HOME'
$HOME
```

■ ""(큰 따옴표(Double quotation))

쉘이 해석 할 수 없도록 막아 준다. 단 인식되는 문자($, ``, ₩)들도 있다.

```
# echo $HOME
/root
# echo "$HOME"
/root
# echo "$HOME is my    directory."
/root is my    directory.
```

■ ``(역 따옴표(Back quotation))

쉘이 해석할 때 명령어로 인식한다. 따라서 역 따옴표 안의 내용을 실행한다.

```
# date
# echo `date`     (# echo $(date))
# echo `hostname` (# echo $(hostname))
# echo "`hostname` is my hostname."

# touch server_`date +%m%d`.log
# tar cvzf /backup/home_`date +%m%d`.tar.gz /home
# while true
> do
> echo "------------`date`---------"
......
> sleep 2
> done
```

■ ₩ (역 슬래쉬(Back slash))

바로 이후에 있는 문자을 쉘이 해석 할 수 없도록 막아 준다.

```
# echo $HOME
# echo ₩$HOME    (# echo '$HOME')

# find / -name core -type f -exec rm -f {} ₩;
# find / ₩( -perm -4000 -o -perm -2000 ₩) -type f

                [참고] ₩(역 슬래쉬)
                # ₩CMD
                # CMD₩

                #  ./configure  --prefix=/usr/local/apache2  --options2=....  --options3=....
.....
                # ./configure -prefix=/usr/local/apache2 ₩
                > --options2=.... ₩
                > --options3=.... ₩
                > --options4=....
```

■ ; (세미콜론(Semicolon))

한개의 라인에 여러개 명령어 수행할 때 사용한다.

```
# ls
# date
# cal

# ls ; date ; cal

# alias pps='ps -ef | head -1 ; ps -ef | grep $1'
# EDITOR=/usr/bin/vim ; export EDITOR    (# export EDITOR=/usr/bin/vim)
```

히스토리(HISTORY)
- History 관련 명령어
history
history 5
cat ~/.bash_history
![history_number]

- History 관련 변수

HISTSIZE
HISEFILE
HISTFILESIZE
history 명령어

사용자가 로그인 하게 되면 사용자의 명령어를 저장하기 위해서 Stack 공간이 할당된다. Stack 공간은 ksh 쉘을 사용하고 있다면, 기본값은 1000개의 명령어를 저장할 수 있는 공간이 할당된다. bash 쉘은 1000개의 명령어를 저장 할 수 있다. 또한 sh 쉘은 히스토리 기능이 없다.

```
|        |        -----
|        |          A
|        |          |  1000
|        |          V
+--------+       ------
        ~/.bash_history
```

```
HISTSIZE              /* 히스토리를 기록하는 스택의 크기를 지정 */
HISTFILE              /* 히스토리 내용을 지속적으로 저장하는 파일 이름 지정 */
                      /* 기본값: ~/.bash_history */
HISTFILESIZE          /* 히스토리파일의 크기 지정 */
```

```
# grep -i histsize /etc/profile
HISTSIZE=1000
# history | less /* 현재까지 사용한 스택에 들어간 명령이 출력 */
1  ifconfig
    2  ping 163.126.63.1
    3  ping 168.126.63.1
    4  vi /etc/sysconfig/network-scripts/ifcfg-eth0
    5  vi /etc/sysconfig/network-scripts/ifcfg-eth1
    6  vi /etc/hosts
    7  vi /etc/resolv.conf
....
# history -c /* (clear) 현재까지 사용한 명령을 삭제 */
# history
```

alias

별칭(Alias)

```
# alias cp='cp -i'
# alias   (# alias cp)
# unalias cp     < cp로 등록되어 있는 alias 해제


# alias cp='cp -i'
# alias mv='mv -i'
# alias rm='rm -i'

# alias vi='/usr/bin/vim'
# alias pps='ps -ef | head -1 ; ps -ef | grep $1'



[EX1] 별칭 테스트
# alias a='cd /test && rm -rf /test/*'
# alias b='cp /etc/passwd file1 ; cp file1 file2 ; cp file1 file3'
# a
# b
# ls
```

환경 파일(Environment File)

- 환경파일의 종류

/etc/profile
~/.bash_profile
~/.bashrc

- 사용자 환경 파일 개요

- 사용자 환경파일 읽혀 지는 순서

  로그인시 읽혀 지는 환경파일

  쉘이 실행 될때 마다 읽혀 지는 환경파일

  로그아웃 할 때 마다 읽혀 지는 환경파일

- 사용자 환경파일 분석
- 사용자 환경파일 활용

사용자 환경 파일(Bash Initialization)

[1] 환경 파일이 읽혀 지는 순서

[1-1] 로그인시에 읽혀 지는 환경 파일

```
■ /etc/profile
        ■ /etc/profile.d/*.sh

■ ~/.bash_profile (~/.bash_profile -> 파일이 없으면 -> ~/.bash_login -> 파일이 없으면 -> ~/.profile)
        ■ ~/.bashrc
                ■ /etc/bashrc
```

[1-2] 쉘이 실행 될때 마다 읽혀 지는 환경 파일

```
■ ~/.bashrc
        ■ /etc/bashrc
                ■ /etc/profile.d/*.sh
```

[1-3] 로그아웃 할 때 마다 읽혀 지는 환경 파일

```
■ ~/.bash_logout
```

[2] 사용자 환경 초기값 설정

[2-1] 관리자가 일반사용자의 환경을 설정 시켜 주는 경우

```
■ /etc/profile : 로그인 할 때만 읽혀짐
■ /etc/bashrc  : 쉘이 실행 될때 마다 읽혀짐
■ /etc/profile.d/*.sh : 쉘이 실행 될때 마다 읽혀짐
```

[2-2] 일반 사용자가 자신의 환경을 설정 하는 경우

```
■ ~/.bash_profile : 로그인 할 때만 읽혀짐
■ ~/.bashrc       : 쉘이 실행 될때 마다 읽혀짐
```

[3] 환경파일 분석

[3-1] /etc/profie 파일

```
# cat /etc/profile
# /etc/profile

# System wide environment and startup programs, for login setup
# Functions and aliases go in /etc/bashrc

pathmunge () {
        if ! echo $PATH | /bin/egrep -q "(^|:)$1($|:)" ; then /* PATH 변수 내용 순서를 설정
*/
           if [ "$2" = "after" ] ; then
              PATH=$PATH:$1
           else
              PATH=$1:$PATH
           fi
        fi
}

# ksh workaround
if [ -z "$EUID" -a -x /usr/bin/id ]; then    /* EUID 없으면 선언 */
        EUID=`id -u`
        UID=`id -ru`
fi

# Path manipulation
if [ "$EUID" = "0" ]; then       /* root 사용자이면 PATH 변수 선언, 위에 존재하는 pathmunge
() 사용 */
        pathmunge /sbin
        pathmunge /usr/sbin
        pathmunge /usr/local/sbin
fi

# No core files by default
ulimit -S -c 0 > /dev/null 2>&1  /* -S : Soft 설정, -c : core 파일의 최대 크기 설정 */

if [ -x /usr/bin/id ]; then      /* USER, LOGNAME, MAIL 변수 선언 */
        USER="`id -un`"
        LOGNAME=$USER
        MAIL="/var/spool/mail/$USER"
fi

HOSTNAME=`/bin/hostname`         /* HOSTNAME, HISTSIZE 변수 선언 */
HISTSIZE=1000

if [ -z "$INPUTRC" -a ! -f "$HOME/.inputrc" ]; then
    INPUTRC=/etc/inputrc
fi

export PATH USER LOGNAME MAIL HOSTNAME HISTSIZE INPUTRC

# By default, we want umask to get set. This sets it for login shell
# Current threshold for system reserved uid/gids is 200
# You could check uidgid reservation validity in
# /usr/share/doc/setup-*/uidgid file
if [ $UID -gt 99 ] && [ "`id -gn`" = "`id -un`" ]; then
    umask 002
else
    umask 022
fi

for i in /etc/profile.d/*.sh ; do    /* /etc/profile.d/*.sh 스크립트 실행 */
    if [ -r "$i" ]; then
        if [ "${-#*i}" != "$-" ]; then
            . $i
        else
            . $i >/dev/null 2>&1
        fi
    fi
done

unset i                 /* i 변수 unset */
unset pathmunge         /* pathmunge 함수 unset */
```

[3-2] /etc/profile.d 디렉토리

```
# cd /etc/profile.d
# ls
colorls.csh  gnome-ssh-askpass.csh  lang.csh  vim.csh
colorls.sh   gnome-ssh-askpass.sh   lang.sh   vim.sh
glib2.csh    krb5-workstation.csh   less.csh  which-2.sh
glib2.sh     krb5-workstation.sh    less.sh
               쉘(Shell)
               - sh  style : sh  -> ksh -> zsh -> bash
               - csh style : csh -> tcsh
```

[3-3] $HOME/.bash_profile 파일

```
# cat ~/.bash_profile
# .bash_profile

# Get the aliases and functions    /* ~/.bashrc 파일 실행 */
if [ -f ~/.bashrc ]; then
        . ~/.bashrc
fi

# User specific environment and startup programs

PATH=$PATH:$HOME/bin               /* PATH 변수에 $HOME/bin 추가 */

export PATH                        /* PATH 변수 export */
unset USERNAME                     /* USERNAME 변수 unset */
```

[3-4] $HOME/.bashrc 파일

```
# cat ~/.bashrc
# .bashrc

# User specific aliases and functions

alias rm='rm -i'                   /* 기본적인 명령어에 대한 alias 선언 */
alias cp='cp -i'
alias mv='mv -i'

# Source global definitions        /* /etc/bashrc 실행 */
if [ -f /etc/bashrc ]; then
        . /etc/bashrc
fi
```

[3-5] /etc/bashrc 파일

```
# cat /etc/bashrc
# /etc/bashrc

# System wide functions and aliases
# Environment stuff goes in /etc/profile

# are we an interactive shell?
if [ "$PS1" ]; then
  if [ -z "$PROMPT_COMMAND" ]; then
    case $TERM in
        xterm*)
                if [ -e /etc/sysconfig/bash-prompt-xterm ]; then
                        PROMPT_COMMAND=/etc/sysconfig/bash-prompt-xterm
                else
                        PROMPT_COMMAND='printf         "₩033]0;%s@%s:%s₩007"         "${USER}"
"${HOSTNAME%%.*}" ₩
                        "${PWD/#$HOME/~}"'
                fi
                ;;
        screen)
                if [ -e /etc/sysconfig/bash-prompt-screen ]; then
                        PROMPT_COMMAND=/etc/sysconfig/bash-prompt-screen
                else
                        PROMPT_COMMAND='printf        "₩033]0;%s@%s:%s₩033₩₩"        "${USER}"
"${HOSTNAME%%.*}" ₩
       "${PWD/#$HOME/~}"'
                fi
                ;;
```

```
        *)
                [ -e /etc/sysconfig/bash-prompt-default ] && ₩
       PROMPT_COMMAND=/etc/sysconfig/bash-prompt-default
            ;;
    esac
  fi
  # Turn on checkwinsize
  shopt -s checkwinsize
  [ "$PS1" = "₩₩s-₩₩v₩₩₩$ " ] && PS1="[₩u@₩h ₩₩]₩₩$ "
fi

if ! shopt -q login_shell ; then
    # We're not a login shell
    # Need to redefine pathmunge, it get's undefined at the end of /etc/profile
    pathmunge () {
                if ! echo $PATH | /bin/egrep -q "(^|:)$1($|:)" ; then
                        if [ "$2" = "after" ] ; then
                                PATH=$PATH:$1
                        else
                                PATH=$1:$PATH
                        fi
                fi
        }

    # By default, we want umask to get set. This sets it for non-login shell.
    # You could check uidgid reservation validity in
    # /usr/share/doc/setup-*/uidgid file
    if [ $UID -gt 99 ] && [ "`id -gn`" = "`id -un`" ]; then   /* UMASK 값 설정 */
       umask 002
    else
       umask 022
    fi

    # Only display echos from profile.d scripts if we are no login shell
    # and interactive - otherwise just process them to set envvars
    for i in /etc/profile.d/*.sh; do
        if [ -r "$i" ]; then
            if [ "$PS1" ]; then
                . $i
            else
                . $i >/dev/null 2>&1
            fi
        fi
    done

        unset i
        unset pathmunge
fi
# vim:ts=4:sw=4
```

[3-6] $HOME/.bash_logout

```
# cat ~/.bash_logout
# ~/.bash_logout

clear
```

[EX1] 사용자 환경 파일이 읽혀 지는 순서에 대한 실습

① /etc/profile 파일에 테스트 내용 추가

■ /etc/profile -> /etc/profile.d/*.sh

```
# cat /etc/profile | head -10
echo "|---> /etc/profile read"            <----- 새로운 라인 추가

# /etc/profile

# System wide environment and startup programs, for login setup
# Functions and aliases go in /etc/bashrc

pathmunge () {
        if ! echo $PATH | /bin/egrep -q "(^|:)$1($|:)" ; then
           if [ "$2" = "after" ] ; then
              PATH=$PATH:$1
           else
```

② /etc/profile.d 디렉토리에 테스트용 스크립트 생성

```
# cd /etc/profile.d
# cat test.sh
-----------------------------------------------
#!/bin/bash
echo "|---> /etc/profile.d/*.sh read"
-----------------------------------------------
```

③ ~/.bash_profile 파일에 테스트 내용 추가

■ ~/.bash_profile -> ~/.bashrc -> /etc/bashrc

```
# cat ~/.bash_profile | head -10
echo "|---> ~/.bash_profile read"        <----- 새로운 라인 추가

# .bash_profile

# Get the aliases and functions
if [ -f ~/.bashrc ]; then
        . ~/.bashrc
fi

# User specific environment and startup programs

PATH=$PATH:$HOME/bin
```

④ ~/.bashrc 파일에 테스트 내용 추가

```
# cat ~/.bashrc | head -10
echo "|---> ~/.bashrc read"             <----- 새로운 라인 추가

# .bashrc

# User specific aliases and functions

alias rm='rm -i'
alias cp='cp -i'
alias mv='mv -i'

# Source global definitions
if [ -f /etc/bashrc ]; then
```

```
⑤ /etc/bashrc 파일에 테스트 내용 추가
# cat /etc/bashrc | head -10
echo "|---> /etc/bashrc read"              <---- 새로운 라인 추가

# /etc/bashrc

# System wide functions and aliases
# Environment stuff goes in /etc/profile

# are we an interactive shell?
if [ "$PS1" ]; then
  if [ -z "$PROMPT_COMMAND" ]; then
    case $TERM in
        xterm*)
```

```
⑥ ~/.bash_logout 파일에 테스트 내용 추가
# cat ~/.bash_logout
echo "|---> ~/.bash_logout read"          <----- 새로운 라인 추가

# ~/.bash_logout
```

```
⑦ 사용자로 로그인
[root@server1 ~]# ssh root@localhost
root@localhost's password:
Activate the web console with: systemctl enable --now cockpit.socket

Last login: Thu Dec 10 18:37:17 2020 from 192.168.10.1
|---> /etc/profile read
|---> /etc/profile.d/*.sh test.sh read
|---> /etc/bashrc read
|---> /etc/bashrc out
|---> ~/.bash_profile read
|---> ~/.bashrc read
|---> /etc/bashrc read
|---> /etc/bashrc out
|---> ~/.bashrc out
|---> ~/.bash_profile out
```

```
⑧ 하위 쉘(Sub Shell) 실행
# bash
|---> ~/.bashrc read
|---> /etc/bashrc read
|---> /etc/profile.d/*.sh read
```

```
⑨ 새로운 윈도우(Window) 실행
-> 바탕화면(Work Space)에서 오른쪽 마우스 클릭
-> 터미널 열기
-> 출력 내용 확인
|---> ~/.bashrc read
|---> /etc/bashrc read
|---> /etc/profile.d/*.sh read
# exit
```

```
⑩ 이전 명령어 윈도우에서 로그 아웃(Logout)
# exit
# exit
echo "|---> ~/.bash_logout read"
```

파일이 읽혀지는 순서

```
로그인 이후 쉘 할당 까지
|---> /etc/profile read
|---> /etc/profile.d/*.sh read
|---> /etc/bashrc read
|---> ~/.bash_profile read
|---> ~/.bashrc read
|---> /etc/bashrc read

하위 쉘 생성
|---> ~/.bashrc read
|---> /etc/bashrc read
|---> /etc/profile.d/*.sh read

로그아웃
|---> ~/.bash_logout read
```

[4] 환경 파일의 활용 예

[4-1] 사용자 환경 파일에 등록 될수 있는 내용들

```
■ 변수 설정
              PATH, PS1, 사용자 정의 변수 설정
■ 엘리어스(Alias) 설정
              alias ls='ls -hF'
■ 쉘 자체의 기능
```

[4-2] 선언 예

```
# cat ~/.bashrc
# .bashrc

# User specific aliases and functions

alias rm='rm -i'            /* 기본적인 명령어에 대한 alias 선언 */
alias cp='cp -i'
alias mv='mv -i'

# Source global definitions  /* /etc/bashrc 실행 */
if [ -f /etc/bashrc ]; then
. /etc/bashrc
fi

---------------------------- ~/.bashrc 파일 하단에 추가 ----------------------------
# (a). 기본적인 alias
alias c='clear'      /* 명령어를 짧게 사용하기 위한 설정 */
alias h='history'
alias t='/usr/kerberos/bin/telnet 172.16.9.252'
alias d='cd /test && rm -rf /test/*'

alias grep='/bin/grep -i --color' /* 명령어 옵션 */
alias cat='/bin/cat -n'
alias df='/bin/df -h -T'
alias ls='ls -h --color=tty'

# (b). 필요한 경우 선언하는 alias
alias lsf='/bin/ls -al | grep ^-'      /* 새로운 명령어를 위한 alias */
alias lsd='/bin/ls -al | grep ^d'
alias pps='/bin/ps -ef | head -1 ; ps -ef | grep $1'
alias ddf='/bin/df -h -F ext3 ; echo ; /bin/df -i'

alias dir='/bin/ls -ailhF | more'   -----> 윈도우즈 명령어 실행 alias
alias ipconfig='ifconfig'           -----> 윈도우즈 명령어 실행 alias
alias topas='top'                   -----> AIX
alias bdf='df -h'                   -----> HP-UX
alias prstat='top'                  -----> Solars

alias nslookup='/usr/local/bin/nslookup' /* 다른 위치의 명령어 실행 */
alias vi='/usr/bin/vim'

# (c). 편리한 기능의 alias
/*
Webserver(Apache)
/etc/httpd/conf/httpd.conf -----> Configuration File
/var/www/html              -----> Source Directory
/etc/httpd/logs            -----> Log Directory
*/

# 설정 파일
alias fconf='vi /etc/vsftpd/vsftpd.conf'
alias wconf='vi /etc/httpd/conf/httpd.conf'
alias vsftpd.conf='vi /etc/vsftpd/vsftpd.conf'
alias httpd.conf='vi /etc/httpd/conf/httpd.conf'
# 소스디렉토리 이동
alias wdir='cd /var/www/html'
# 로그파일 모니터링
alias mlog='tail -f /var/log/messages'
alias wlog='tail -f /etc/httpd/logs/access_log'
alias welog='tail -f /etc/httpd/logs/error_log'
alias slog='tail -f /var/log/secure'
/* 실무에서 많이 사용되는 로그 파일 이름 형식 */ (예) file_1210.log
# tail -f /logs/file_1210.log
# tail -f /logs/file_`date +%m%d`.log
# alias slog='tail -f /logs/file_`date +%m%d`.log'
# alias slog="view /logs/file_`date --date '1 days ago' +%m%d`.log"
# alias slog="view /logs/file_`date --date '2 days ago' +%m%d`.log"
---------------------------- ~/.bashrc 파일 하단에 추가 ----------------------------
```

검색 관련 명령어
단원목표

- grep 명령어
- find 명령어

grep 명령어
- grep 명령어

grep OPTIONS PATTERN ARG1

OPTIONS : -l, -n, -l, -v, -w

PATTERN : * . ^root root$ [abv] [^a]

CMD | grep sshd
ps –ef | grep systemd
rpm –qa | grep open-ssh
cat /etc/passwd | grep root
cat /etc/group | grep root
cat /var/log/messages | egrep '(warn|err|crti|alert|emerg)'

grep 명령어

파일 내에서 특정한 패턴을 검색하여 그 패턴을 포함하는 모든 줄을 화면에 출력하는 명령어. 파일 내에 특정한 패턴을 찾기 위해 많이 사용한다. 찾으려고 하는 패턴을 정규식(g/re/p, Globally/Regular Expression/Print)이라고 표현한다. grep 명령어의 약자에서 볼 수 있듯이 많은 패턴을 제공하고 있다.

[명령어 형식]

```
# grep OPTIONS PATTERN file1
```

(기본 사용법)

```
# grep root /etc/passwd    (# cat /etc/passwd | grep root)
# CMD | grep root
# cat /etc/passwd | grep root
# rpm -qa | grep ssh
# ps -ef | grep socket
```

(옵션 사용법) "# grep OPTIONS PATTERN file1"

```
# grep -l root /etc/hosts /etc/passwd /etc/group /* 여러 파일 중 검색 문자열 존재 파일 출력 */
# grep -n root /etc/group  /* -n: number line, 파일에서 root 문자열을 검색하고 라인 번호도 같이 출력 # cat -n /etc/passwd | grep root */
# grep -v root /etc/passwd  /* -v: inverse, 파일에서 검색 문자열을 제외하고 나머지 출력 */
# grep -i root /etc/passwd  /* -i: ignore case, 파일에서 검색 문자열의 대소문자를 구분하지 않음 */
# grep -w root file1        /* -w: word , 단어의 일부분이 아닌 단어의 전체가 일치하는 경우 출력 */
# grep --color root /etc/passwd
```

[명령어 옵션]

옵션 : 설명

-l : (-l : list files) 패턴이 있는 파일이름만을 출력한다.
-r : -l 옵션을 사용할 때 디렉토리를 대상을 검색시에 사용한다. grep -lr 형식으로 사용한다.
-n(: -n : number line) 패턴을 포함하는 줄을 출력할 때 줄번호와 함께 출력한다.
-v : (-v : inVerse, except) 패턴을 포함하는 줄을 제외하고 출력한다.
-c : (-c : count) 패턴을 찾은 줄의 수를 출력한다.
-i : (-i : ignore case, 대문자/소문자) 패턴을 찾을 때 대소문자를 구분하지 않는다.
-w : 정확한 단어를 찾는다.

(패턴 사용법) "# grep OPTIONS PATTERN file1"

```
*        # grep 'ro*t' /etc/passwd          모든문자를 포함(영문자,숫자)
.        # grep 'no...y' /etc/passwd        (.에 하나의 영문자나 숫자를 치환)
^root    # grep '^root' /etc/passwd         라인의 시작이 root인 라인을 검색
root$    # grep 'root$' /etc/group          라인의 끝이 root로 끝나는 라인을 검색
[abc]    # grep 'user0[123]' /etc/passwd  user0으로 시작하여 1이나 2나,3으로 끝나는 문자열
```

[EX1] grep 명령어 옵션 실습

```
# useradd FEDORA
# passwd FEDORA
-> 사용자 암호 입력

# grep fedora /etc/passwd    (# cat /etc/passwd | grep fedora)
fedora:x:1001:1001::/home/fedora:/bin/bash
# grep FEDORA /etc/passwd
FEDORA:x:1004:1004::/home/FEDORA:/bin/bash

리눅스는 대소문자를 구별하기 때문에 검색을 하는 경우에 대소문자를 정확히 구별하여 패턴 작성을 해야 한다.
하지만 이것을 무시하고 검색을 해야 하는 경우에는 -i 옵션을 사용해준다.
# grep -i fedora /etc/passwd
fedora:x:1001:1001::/home/fedora:/bin/bash
FEDORA:x:1004:1004::/home/FEDORA:/bin/bash
```

(실무 예) 환경 파일에 등록

```
# alias grep='grep -i'          => $HOME/.bashrc
# alias grep='grep -i --color'  => $HOME/.bashrc < 검색된 단어에 색상을 입혀 가독성을 높여준다 >

# vi ~/.bashrc
alias grep='grep -i --color'
# . ~/.bashrc
# alias grep
```

```
# grep -n fedora /etc/passwd  (# cat -n /etc/passwd | grep fedora)
48:fedora:x:1001:1001::/home/fedora:/bin/bash
# grep -n root /etc/group
1:root:x:0:
# grep -l root /etc/group /etc/passwd /etc/hosts
/etc/group
/etc/passwd
# vi file.txt



# grep -w root file.txt
root hello
```

[EX2] grep 명령어 패턴 실습

```
# nmcli con show
NAME   UUID                                   TYPE      DEVICE
ens33  7da16185-c5c7-47de-8e44-07c3276d5006  ethernet  ens33

# grep -lr ens33 /etc
/etc/sysconfig/network-scripts/ifcfg-ens33

# cat /var/log/messages | grep -i aug | head -5
Aug     21     14:33:19     server1     kernel:     Linux     version     4.18.0-193.el8.x86_64
(mockbuild@kbuilder.bsys.centos.org) (gcc version 8.3.1 20191121 (Red Hat 8.3.1-5) (GCC)) #1
SMP Fri May 8 10:59:10 UTC 2020
Aug         21         14:33:19         server1         kernel:         Command         line:
BOOT_IMAGE=(hd0,msdos1)/vmlinuz-4.18.0-193.el8.x86_64       root=/dev/mapper/cl-root       ro
resume=/dev/mapper/cl-swap rd.lvm.lv=cl/root rd.lvm.lv=cl/swap rhgb quiet
Aug 21 14:33:19 server1 kernel: Disabled fast string operations
Aug 21 14:33:19 server1 kernel: x86/fpu: Supporting XSAVE feature 0x001: 'x87 floating point
registers'
Aug 21 14:33:19 server1 kernel: x86/fpu: Supporting XSAVE feature 0x002: 'SSE registers'
# cat /var/log/messages | grep -i 'aug 26'

# ps
PID TTY          TIME CMD
13929 pts/2    00:00:00 bash
# ps -ef | grep 13929
root     13929 13927  0 14:00 pts/2    00:00:00 -bash
# ps -ef | grep bash
root      7117  7115  0 00:53 pts/3    00:00:00 -bash
root      7364  7117  0 01:47 pts/3    00:00:00 -bash
root     27308 27302  0 Jan26 pts/2    00:00:00 bash
# rpm -qa | grep postfix
postfix-8.13.8-2.el5
postfix-cf-8.13.8-2.el5

                # rpm -q postfix
                # rpm -qa postfix
                # rpm -qa | grep postfix
```

[EX3] 파일내의 특정 패턴 여러개 검색하기

```
grep의 태생적 한계 -> 하나이상의 패턴 검색이 불가능
■ egrep(Extended grep) CMD /* 동시에 여러개의 패턴을 검색하는 경우에 사용 &/
# cat /var/log/messages | egrep -i '(warn|err|crit|alert|emerg)'
# cat /etc/passwd | egrep -i '(root|fedora)'
root:x:0:0:root:/root:/bin/bash
operator:x:11:0:operator:/root:/sbin/nologin
fedora:x:1001:1001::/home/fedora:/bin/bash
FEDORA:x:1002:1002::/home/FEDORA:/bin/bash

# cat file1 | grep -E  '(one|two)'

■ fgrep(Fixed grep) CMD
# fgrep '^root' file1

# cat file1 | grep - f   ^one
```

(실무 예) 로그 파일에서 에러 메세지 검색

```
# cat /var/log/messages | egrep -i '(warn|err|crit|alert|emerg)'
```

```
# egrep -v "fedora|user01" /etc/passwd    (# egrep -v "(fedora|user01)" /etc/passwd)
root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/sbin/nologin
daemon:x:2:2:daemon:/sbin:/sbin/nologin
..... (중략) .....
```

(실무) 특정한 단어를 제외하고 검색

```
# ps -ef | grep sshd | grep -v grep

grep : 강력한 패턴 매칭 템플릿을 정의하기 위해 "정규 표현식"을 사용할 수 있다.
egrep[옵션] "패턴|패턴|..." [대상파일들] : 확장된 정규 표현식을 사용하며, 찾아낼 패턴을 여러
개 지정할 수 있다.
'|'기호는 불린 연산자 "OR"에 해당하므로, 정해진 패턴들에 포함되는 모든 라인을 보여준다.
fgrep[옵션] 패턴 [대상파일들] : 패턴과 정확히 일치하는 것만을 찾아 준다.
```

grep 출력 내용 추가하기

```
[root@localhost test]# cat -n /etc/ssh/sshd_config | grep -A 3 -i 'port 22'
    21  #Port 22
    22  #AddressFamily any
    23  #ListenAddress 0.0.0.0
    24  #ListenAddress ::
[root@localhost test]# cat -n /etc/ssh/sshd_config | grep -B 3 -i 'port 22'
    18  # SELinux about this change.
    19  # semanage port -a -t ssh_port_t -p tcp #PORTNUMBER
    20  #
    21  #Port 22
[root@localhost test]# cat -n /etc/ssh/sshd_config | grep - 3 -i 'port 22'
grep: 3: No such file or directory
grep: port 22: No such file or directory
[root@localhost test]# cat -n /etc/ssh/sshd_config | grep -C 3 -i 'port 22'
    18  # SELinux about this change.
    19  # semanage port -a -t ssh_port_t -p tcp #PORTNUMBER
    20  #
    21  #Port 22
    22  #AddressFamily any
    23  #ListenAddress 0.0.0.0
    24  #ListenAddress ::
```

find

find / locate

inode 정보로 검색

locate

```
# locate filename
단점:db를 이용하기 때문에 갱신되지 않은 파일의 경우에는 찾을수 없다.

# updatedb ; locate file1  << updatedb를 같이 사용해야함.
단점: 파일명만 가지고 검색


# alias locate='updatedb ; locate $*'
```

```
# find 명령어
# find / -name core –type f
# find / -user user-1 –group class1
# find / -mtime [-7|7|+7]
# find / -perm [755|-755]
# find / -size [-300M|300M|+300M]
# find / -name core –type f –exec rm –f {} ₩;
# find / -name core –type -ls
# find / -name core –type f –ok rm {} ₩;
# find /Log_Dir –name "*.log" –type f ₩
> -mtime +30 –exec rm 0rf {} ₩;
# find /Log_Dir –mtime -2 –size +1G –type
```

find 명령어

디렉토리안에서 원하는 파일을 찾고자 할 때 사용하는 명령어이다. find 명령 다음에 시작 디렉토리를 정해 주고 찾고자하는 파일 이름 앞에 옵션을 주면 된다.

[명령어 형식]

```
# find [검색시작위치] [옵션1] [인자값1] [옵션2] [인자값2] ...
```

[명령어 옵션]

옵션 : 설명
-name : 파일 이름을 기준으로 검색
-iname: 대소문자를 구별하지 않고 파일 이름을 기준으로 검색
-perm : 파일 권한을 기준으로 검색한다.
-type : 파일의 종류를 기준으로 검색
b : 블록 파일
c : 문자
d : 디렉토리
f : 파일
l : 링크
s : 소켓
-size : 파일의 크기를 기준으로 검색
+n : n보다 크다
-n : n보다 작다
n : n이다
b : 512-byte
c : Bytes (Character = Byte)
k : Kilo Byte
M : Mega Bytes
G : Giga Bytes
w : 2-byte
-links : 링크의 개수를 기준으로 검색
-user : 사용자 ID를 기준으로 검색
-group: 그룹 ID를 기준으로 검색
-atime : 특정 기간 이상 접근하지 않은 파일을 기준으로 검색
-mtime : 특정 기간 이상 수정되지 않은 파일을 기준으로 검색
-inum : number 지정된 inode 번호와 파일을 찾는다.
-print : 표준출력으로 검색된 파일 출력: GNU는 디펄트, Unix는 필수 입력
-exec command {} ₩; :찾은 각 파일에 대해 지정된 명령을 실행
-ok command {} ₩; : 실행여부(실행 되어 있는지 아닌지)를 사용자에게 확인 후 명령을 실행

```
-----------------------------------------------------------------------------
(형식1) # find / -name core -type [f|d]    (# find / -name "*oracle*" -type f)
(형식2) # find / -user user01 -group class1
(형식3) # find / -mtime [-7|7|+7]
(형식4) # find / -perm [-755|755]
(형식5) # find / -size [-300M|300M|+300M]
(형식6) # find / -name core -type f -exec rm {} W;
-----------------------------------------------------------------------------
```

[EX1] 파일 이름 검색(예: # find / -name core –type f)

```
/test -+- dir1 - file1
        |
        +- dir2 - file3
        |
        +- file1
        |
        +- file2
```

```
# cd /test
# find . -name file1
./dir1/file1
./file1
-> "."은 현디렉토리, 이름이 file1인 찾아라 디렉토리이던 파일이던 type과 상관없이 모두 찾는 것
# find . -name file1 -type f
./dir1/file1
./file1
# find /var -name "*.log" -type f
/var/log/audit/audit.log
/var/log/sssd/sssd.log
/var/log/sssd/sssd_implicit_files.log
/var/log/sssd/sssd_nss.log
/var/log/sssd/sssd_kcm.log
/var/log/tuned/tuned.log
..... (중략) .....
```

[EX2] 사용자/그룹 검색(예: # find / -user user01 -group class1)

```
# find /home -user fedora -group fedora
/home/fedora
/home/fedora/top.seceret.mail
/home/fedora/.bashrc
..... (중략) .....
# find /home -user fedora -group fedora -ls
-> 출력 결과 확인
```

[EX3] 날짜 검색(예: # find / -mtime 7)

```
# cd dir2
# touch file1 file2 file3 file4 file5 file6

# date 08191111
Thu Aug 19 11:11:00 KST 2010

참고 - vmware에서 실습을 진행하는 경우 NTP로 인하여 , VMware tools로 인하여 시간이 계속 변경되기 때문에
터미널을 하나 더 실행하여 다음 명령어로 현재 시간을 확인하며 실습을 진행한다.
# watch -n date
# while : ; do date 08191111; sleep 1; echo "date resetting"; done
```

[참고] 파일 속성 정보

```
     file6      file5       file4        file3      file2      file1
  -----+----------+------------+----------+----------+----------+---------+
     0813        0814         0815       0816       0817        0818      0819
```

```
# touch -t 08181111 file1
# touch -t 08171111 file2
# touch -t 08161111 file3
# touch -t 08151111 file4
# touch -t 08141111 file5
# touch -t 08131111 file6
# ls -l file*
-rw-r--r-- 1 root root 0 Aug 18 11:11 file1
-rw-r--r-- 1 root root 0 Aug 17 11:11 file2
-rw-r--r-- 1 root root 0 Aug 16 11:11 file3
-rw-r--r-- 1 root root 0 Aug 15 11:11 file4
-rw-r--r-- 1 root root 0 Aug 14 11:11 file5
-rw-r--r-- 1 root root 0 Aug 13 11:11 file6
# find . -mtime 3 -type f /* 수정 날짜가 3일전인 파일 */
./file3
# find . -mtime -3 -type f /* 수정한 날짜가 3일이 안된 파일 */
./file3
./file2
./file1
# find . -mtime +3 -type f /* 수정한 날짜가 3일이 지난 파일 */
./file6
./file5
./file4
```

[EX4] 퍼미션 검색(예: # find / -perm 755 -type f)

```
# cd /test
# mkdir dir3
# cd dir3
# touch file1 file2 file3 file4 file5 file6 file7 file8

# chmod 000 file1  (---)
# chmod 100 file2  (--x)
# chmod 200 file3  (-w-)
# chmod 300 file4  (-wx)
# chmod 400 file5  (r--)
# chmod 500 file6  (r-x)
# chmod 600 file7  (rw-)
# chmod 700 file8  (rwx)

# ls -l file*
---------- 1 root root3043 3월 18 14:54 file1
---x------ 1 root root362131 3월 18 14:52 file2
--w------- 1 root root  0 3월 18 14:52 file3
--wx------ 1 root root  0 3월 18 14:52 file4
-r-------- 1 root root  0 3월 18 14:52 file5
-r-x------ 1 root root 0 3월 18 14:52 file6
-rw------- 1 root root  0 3월 18 14:52 file7
-rwx------ 1 root root 0 3월 18 14:52 file8
# find . -perm 600 -type f -ls
-> file7

# find . -perm -400 -type f -ls
-> file5, file6, file7, file8

SETUID SETGID 동시에 검색
# find /  -perm -2000 -o -perm -4000
# find / -perm -4000 -not -path "/proc/*" 2> /dev/null
# find / -perm -4000  2> /dev/null
```

[EX5] 파일 크기 검색(예: # find / -size 50k -type f)

```
# cp /etc/passwd file10
# cp /etc/services file11
# ls -l

* file1-8   0    Bytes
* file10  3228   Bytes
* file11  692252 Bytes

# find . -size 2490c -type f    /* 정확이 일치되는 용량만 검색 */
-> file10

# find . -size -1980c -type f   /* 일치되는 것 제외한 파일 크기가 2027bytes 미만 */
-> file1-8

# find . -size +1980c -type f   /* 파일 크기가 2027bytes 보다 큰것 */
-> file11
```

[EX6] 디렉토리안에 특정한 패턴을 가진 파일들을 삭제(# find / -name file -type f -exec rm {} ₩;)

```
# cd /test
# find . -name file1 -type f /* 파일의 이름이 file1인 것 */
./file1
./dir2/file1
```

```
# find . -name file1 -type f -ls
# find . -name file1 -type f -exec chown -v user01 {} ₩;
# find . -name file1 -type f -ls

# find . -name file1 -type f -exec chmod -v 640 {} ₩;
# find . -name file1 -type f -ls

# find . -name file1 -type f -exec rm -fv {} ₩;
# find . -name file1 -type f

/proc 디렉토리 제외하고 검색하기
# find / -name file1 -not -path '/proc/*'
```

```
find 명령어에서 -exec 옵션에서 중괄호 {}의 역할
find 명령어는 검색으로 출력되는 하나 이상의 파일 및 디렉토리 결과를 가지고 있다. 이 결과를 바탕으로 -exec 옵션에서는 2차적으로 명령을 진행가능하다.
이때 {}는 버퍼의 역할을 하며 순차적으로 처음 검색된 내용부터 차례대로 치환된다. 이 치환은 마지막 검색내용이 입력될때까지 반복된다.

find 명령어에서 -exec 옵션에서 세미콜론(;)의 역할
find 명령어를 사용하면서 출력된 결과를 가지고 바로 후처리 실행으로 이어질수 있는 옵션인 exec ok가 존재한다. 해당 문법의마지막에는 세미콜론(;)이 입력된다. 하지만 쉘에서는 세미콜론(;)을 기준으로 앞 명령어와 뒤 명령어를 구분하는 구분자로 사용하기 때문에 그냥 사용할수는 없으며 바로 앞에 역슬래시를 붙여 ₩; 구조로 만들어줘야 find 명령어에서 정상적으로 해석가능하다.
```

(실무 예) 오래된 로그 기록 삭제

```
한달(시스템 생성일 ~ 30일)이 지난 로그파일은 그 의미를 상실하게 된다. 따라서 일정 시간이 지난 로그파일의 경우 find라는 명령어를 이용하여 파일을 주기적으로 삭제해 주도록 한다.

# find /Log_Dir1 -name "*.log" -type f -mtime +30 -exec rm -f {} ₩;
# find /Log_dir2 -name "*.log" -type f -mtime +60 -exec rm -f {} ₩;
```

(실무 예) 파일시스템이 갑자기 풀(Full) 나는 경우

```
# find /var -mtime -2 -size +1G -type f
# find /var -mtime -2 -size +512M -type f
/var/server/log/file.log

[참고] lsof(list open file)
# lsof | grep /var/server/log/file.log
```

(실무 예) 에러메세지가 들어 있는 startup script 검색

```
# /was/bin/startup.sh
..... Server Error .....
# find /was -type f -exec grep -l 'Server Error' {} ₩;
/was/conf/server.xml
# vi /was/conf/server.xml
/Server Error
......
if 조건 ; then

else
echo "Server Error"
fi
```

(실무 예) 에러 메세지를 검색하는 방법

```
http://www.google.co.kr
-> site:.redhat.com "Server Error"
-> 가상화 .pdf
-> 가상화 .ppt
-> "Server Error1" AND "Server Error2" (AND/OR)
-> "Server Error1"
```

기본 스크립트

bash shell script
1. bash shell에서만 사용되는 것 X
2. top down 방식으로 작성
3. 작성이 끝이 아님. 꼭 검증

```
#!/usr/bin/bash
action1
action2
action3
action4
```

이 것만 아시면 사실 스크립트 절반은 이해한겁니다.

```
#!/usr/bin/bash
```

명령 인터프리터 지정 : bash shell을 사용하겠다.
bash file

내부에서 사용되는 명령들은 새로운 것이 아님
기존에 shell에서 사용하던 명령들이 대부분

```
#!/usr/bin/bash
echo 111
```

위와 같이 작성해도 하나의 스크립트로 완성
하나 추가할 점: 실행 권한을 줘야 한다.

for

```
for VARIABLE in LIST; do
COMMAND VARIABLE
done
```

```
for 변수 in 변수에 치환될 값; do
COMMAND VARIABLE
done
```

```
for i in one two three ; do
echo ${i}
done
```

```
#!/usr/bin/bash

for i in $(cat /test/list) ; do
echo file${i}
done
```

while

```
while true
do
COMMAND VARIABLE
done
```

무한 반복: 끊어내기 전까지는 계속 진행

if

```
if <CONDITION>; then
 <STATEMENT>
 ...
 <STATEMENT>
fi

if 조건식 ; then
  실행될 내용
fi
```

```
if <CONDITION>; then
 <STATEMENT>
 ...
 <STATEMENT>
else
 <STATEMENT>
...
 <STATEMENT>
fi
```

<사용에 맞추어 리눅스 설정하기>
소프트웨어 설치 및 업데이트

[1] RPM(Redhat Package Manager)

```
(EX) kernel-3.10.0-1062.18.1.el7.rpm
                -> Redhat, RHEL, Fedora, CentOS
```

RPM(Redhat Package Manager)이란 Redhat사에서 패키지를 좀 더 쉽게 설치하고 관리하기 위해 만든 패키지 관리 프로그램이다. 복잡한 컴파일 과정 없이 간단하게 패키지를 설치할 수 있는 장점이 있지만 반대로 사용자에게 맞는다양한 옵션들은 설정할 수 없다는 것이 단점이다. 그래서 간단한 프로그램의 경우에는 RPM으로 설치하고 서버와 같은 중요한 프로그램은 소스를 컴파일 하여 설치하는 것이 좋다. rpm명령어는 rpm패키지를 설치하고 삭제 또는 관리하는 명령어이다.

■ rpm 패키지 다운로드 사이트

http://rpmfind.net

http://rpm.pbone.net < 실습에서 사용하는 사이트

ftp://ftp.daum.net

[명령어 형식]

```
# rpm [옵션]

질의 옵션 (-q 또는 --query 옵션과 함께 사용):
-c, --configfiles 모든 설정 파일을 나열합니다
-d, --docfiles 모든 문서 파일을 나열합니다
--dump 기본 파일 정보를 보여줍니다
-l, --list 패키지 안의 파일을 나열합니다
-s, --state 나열된 파일의 상태(state)를 보여줍니다
-a, --all 모든 패키지에 대해 질의/검증합니다
-f, --file 파일이 들어있는 패키지에 대해 질의/검증 합니다
-g, --group 그룹 안의 패키지를 질의/검증합니다
-p, --package query/verify a package file
-i, --info  패키지의 정보를 볼수 있다.

검증 옵션 (-V 또는 --verify 옵션과 함께 사용):
--nodeps(Dependency) 패키지의 의존성을 검사하지 않습니다.
                -> 의존성 : 특정 프로그램을 설치하기 위하여 사전 설치되어야 하는 기능이나 프
로그램을 뜻합다.
-a, --all 모든 패키지에 대해 질의/검증합니다.
-f, --file 파일이 들어있는 패키지에 대해 질의/검증 합니다.
-g, --group 그룹 안의 패키지를 질의/검증 합니다
-p, --package query/verify a package file

서명 옵션:
--addsign sign package(s) (identical to --resign)
-K, --checksig verify package signature(s)
--delsign delete package signatures
--import import an armored public key
--resign sign package(s) (identical to --addsign)
--nodigest do not verify package digest(s)
--nosignature don't verify package signature(s)

데이터베이스 옵션:
--initdb 데이터베이스를 초기화 합니다.
--rebuilddb 설치된 패키지 헤더에서 상반된 목록(inverted lists)의 데이터베이스를 재구축
합니다

설치/업그레이드/삭제 옵션:
-e, --erase=<패키지>+ 패키지를 (제거) 삭제합니다
--excludedocs 패키지에 포함된 문서 파일을 설치하지 않습니다
--excludepath=<경로> <경로>로 시작되는 파일은 설치하지 않습니다
--force --replacepkgs 와 --replacefiles 옵션을 동시에 사용합니다
-F, --freshen=<패키지파일>+ 기존에 설치된 패키지를 업그레이드 합니다
-h, --hash 패키지 설치를 해시마크(#)로 표시합니다
(-v 옵션과 함께 사용하는 것이 좋습니다)
-i, --install install package(s)
--nodeps 패키지의 의존성을 검사하지 않습니다
--oldpackage 이전 버전의 패키지로 다운그레이드 합니다
(--force 옵션을 사용시에는 이 옵션이 자동으로 적용됩니다)
--percent 패키지 설치를 퍼센트(%)로 표시합니다
--prefix=<디렉토리> 재배치 기능이 있는 패키지의 경우, 지정한 <디렉토리>로 재배치하여 설치합니
다.
--replacepkgs 패키지가 이미 설치되어 있는 경우에도 설치합니다
--test 패키지를 설치하지 않고, 제대로 설치되는지만 확인합니다
-U, --upgrade=<패키지파일>+ 패키지를 업그레이드 합니다

Common options for all rpm modes and executables:
-v, --verbose 자세한 출력을 제공합니다
```

[명령어 사용 예제]

```
■ RPM 패키지 설치 및 업데이트
# rpm -ivh package-filename.rpm /* 패키지 설치, 이전 버전이 존재하는 경우 설치하지 않음 */
(-i: install, -v: verbose, -h: hash)
# rpm -Fvh package-filename.rpm /* 패키지 없데이트, 이전 버전의 패키지가 있을 경우만 패키지를
설치 */
(-F: Freshen)
# rpm -Uvh package-filename.rpm /* 패키지 업데이트, 이전 버전이면 업그레이드 하고 없으면 설치
*/
(-U: Upgrade)
# rpm -ivh --nodeps package-filename.rpm /* 의존성 체크 안함, 의존성 문제를 무시하고 설치 */
(-nodeps: no dependency)
# rpm -Uvh --force package-filename.rpm  /* 패키지 충돌이 방생해도 강제적으로 설치 */

■ RPM 패키지 삭제
# rpm -e package-filename /* 패키지 삭제 */
(-e: erase)
# rpm -e --nodeps package-filename /* 패키지 삭제, 의존성 문제를 무시하고 패키지 삭제 */
(--nodeps: no dependency)
■ RPM 패키지 정보 확인
# rpm -qa package-filename /* 지정된 패키지 설치 유무 확인 */
(-q: query, -a: all)
# rpm -qa | grep package-filename /* 전체 패키지 목록에서 지정된 패키지 설치 유무 확인 */
# rpm -qi package-filename /* 설치된 패키지의 자세한 정보 확인 */
(-i: information)
# rpm -ql package-filename /* 설치된 패키지의 파일과 디렉토리 목록 확인 */
(-l: list)
# rpm -qf /usr/bin/ls /* 지정된 파일이 포함된 패키지 이름 확인 */
(-f: file)
# rpm -qip package-filename.rpm /* 설치하고자 하는 패키지의 자세한 정보 확인 */
(-p: packages file)
```

패키지 파일 이름 형식

```
name-version-release.architecture.rpm
     kernel-2.6.9-42.0.2.EL.i686.rpm
     kernel-2.6.18-164.el5.x86_64.rpm
     kernel-2.6.18-164.el5.src.rpm

종류       필드-이름 설명
kernel    패키지 이름   어떤 패키지인가를 설명해주는 패키지의 이름을 나타낸다.
2.6.9     패키지 버전    패키지의 버전을 의미한다.
49.0.2.EL5  패키지 릴리즈    한 버전의 패키지를 몇 번 빌드했는지알 수 있다. EL(Enterprise
Linux)
i686      패키지 아키텍쳐  패키지 아키텍쳐를 나타낸다.
                                          source : .src.rpm
                                          noarch : .noarch.rpm
                                          binary : .i386.rpm
                                          (i386, i586, i686, x86, x86_64 등)
rpm       패키지 확장자     레드햇(RedHat) 운영체제에서 사용하는 패키지의 이름은 .rpm이다.
```

[참고] 패키지 아키텍쳐(Package Architecture) 종류의 차이점

```
패키지 아키텍쳐(Package Architecture) 종류의 차이점

해당 패키지 다운 받을때에 i386 이니 뭐니 이런 것들은 소스코드를 각각의 CPU 에 적합하게 컴파일
된 것을
의미하게 됩니다.

i386 : 인텔(Intel) 80386 CPU 계열의 CPU 의미(32bit intel-compatible CPU)
i586 : 인텔(Intel) 80586 CPU 계열의 Pentium CPU 의미
i686 : 인텔(Intel) 80686 CPU 계열의 Pentium Pro 이상의 CPU 의미        (686 class 32bit
intel-compatibel CPU)
x86 : 위에서 언급한 CPU(i386/i586/i686) 의미
(x386, x586, x686 모든 플랫폼에서 동작하는 것을 의미)
x86_64 : 64비트 CPU 환경에서 동작하는 것을 의미
athlon : AMD Athlon CPU
sparc  : 32bit or 64bit SPARC CPU
sparc64: 64bibt UltraSPARC CPU
```

[EX1] 패키지 설치 확인

```
① 패키지 전체 목록 확인
# rpm -qa | more
centos-release-notes-5.9-0
rootfiles-8.1-1.1.1
atk-1.12.2-1.fc6
nspr-4.9.1-6.el5
bzip2-libs-1.0.3-6.el5_5
nss-3.13.5-8.el5
audiofile-0.2.6-5
libgcrypt-1.4.4-5.el5_8.2
procps-3.2.7-22.el5
..... (중략) .....
-> 서버에 설치되어 있는 모든 패키지가 출력된다.

# rpm -q openssh    (# rpm -qa | grep openssh)
openssh-4.3p2-82.el5
-> 특정 패키지를 검색한다.

                [참고] 패키지 이름을 가지고 패키지 목록 점검
                # rpm -q openssh
                or
                # rpm -qa openssh
                or
                # rpm -qa | grep openssh

# rpm -qa | grep openssh
openssl-0.9.8e-26.el5_9.1
openssh-askpass-4.3p2-82.el5
openssh-4.3p2-82.el5
openssh-server-4.3p2-82.el5
openssh-clients-4.3p2-82.el5
# rpm -qa openssh openssh-server openssh-clients
openssh-4.3p2-82.el5
openssh-server-4.3p2-82.el5
openssh-clients-4.3p2-82.el5
```

[EX2] 시스템에 설치된 패키지 삭제

설치된 패키지를 제거하는 것은 -e 옵션을 사용한다. 제거할 패키지가 다른 패키지에 영향을 미친다면 패키지를 제거할 수 없다는 메시지가 나타나는데 이것을 의존성이라 한다. 의존성을 무시하고 함부로 제거하면 다른 패키지를 사용하는데 문제가 생길 수 있다. 꼭 패키지를 삭제해야하는 경우 먼저 의존적인 패키지를 먼저 삭제한 후 원하는패키지 삭제 해야지만 된다.

```
[선수작업] php 패키지를 설치한다.
# yum -y install php
① php 패키지 삭제
# rpm -qa | grep php
php-5.1.6-39.el5_8
php-common-5.1.6-39.el5_81
php-cli-5.1.6-39.el5_8
php-ldap-5.1.6-39.el5_8
# rpm -e php        (# rpm -e php-5.1.6-39.el5_10)
# rpm -qa | grep php
php-common-5.1.6-39.el5_8
php-cli-5.1.6-39.el5_8
php-ldap-5.1.6-39.el5_8
-> 정상적으로 잘 삭제 되었다.
-> php-5.1.6-39.el5_4.5 패키지와 의존성 관계에 있는 패키지는 없는 것 같다.
```

[EX3] 시스템에 패키지 설치

```
■ 시스템 패키지 설치 방법
   - CD 안에서 패키지를 검색하여 설치하는 방법
   - 인터넷상에서 패키지를 다운로드 하여 설치하는 방법
- http://rpmfind.net
- http://rpm.pbone.net
- ftp://ftp.daum.net

■ CD 안에 패키지 검색하여 설치하는 방법

(전제 조건) CD가 마운트 되어 있어야 한다.
   - VMware > VM > Settings > CD/DVD
```

```
① CD 마운트
# df -h
Filesystem            Size  Used Avail Use% Mounted on
/dev/sda1              26G  3.2G   22G  13% /
/dev/sda8             487M   11M  451M   3% /data1
/dev/sda7             487M   11M  451M   3% /data2
/dev/sda6             487M   11M  451M   3% /data3
/dev/sda5             487M   11M  451M   3% /data4
/dev/sda3             487M   11M  451M   3% /home
tmpfs                 1.5G     0  1.5G   0% /dev/shm
/dev/sr0              3.9G  3.9G     0 100% /run/media/root/CentOS 7 x86_64
② php 패키지 설치
# cd /run/media/root/CentOS 7 x86_64/Packages
# ls php*
php-5.1.6-39.el5_8.i386.rpm           php53-bcmath-5.3.3-13.el5_8.i386.rpm
php-bcmath-5.1.6-39.el5_8.i386.rpm    php53-cli-5.3.3-13.el5_8.i386.rpm
php-cli-5.1.6-39.el5_8.i386.rpm       php53-common-5.3.3-13.el5_8.i386.rpm
php-common-5.1.6-39.el5_8.i386.rpm    php53-dba-5.3.3-13.el5_8.i386.rpm
php-dba-5.1.6-39.el5_8.i386.rpm       php53-devel-5.3.3-13.el5_8.i386.rpm
php-devel-5.1.6-39.el5_8.i386.rpm     php53-gd-5.3.3-13.el5_8.i386.rpm
php-gd-5.1.6-39.el5_8.i386.rpm        php53-imap-5.3.3-13.el5_8.i386.rpm
php-imap-5.1.6-39.el5_8.i386.rpm      php53-intl-5.3.3-13.el5_8.i386.rpm
php-ldap-5.1.6-39.el5_8.i386.rpm      php53-ldap-5.3.3-13.el5_8.i386.rpm
php-mbstring-5.1.6-39.el5_8.i386.rpm  php53-mbstring-5.3.3-13.el5_8.i386.rpm
php-mysql-5.1.6-39.el5_8.i386.rpm     php53-mysql-5.3.3-13.el5_8.i386.rpm
php-ncurses-5.1.6-39.el5_8.i386.rpm   php53-odbc-5.3.3-13.el5_8.i386.rpm
php-odbc-5.1.6-39.el5_8.i386.rpm      php53-odbc64-5.3.3-2.el5.i386.rpm
php-pdo-5.1.6-39.el5_8.i386.rpm       php53-pdo-5.3.3-13.el5_8.i386.rpm
php-pear-1.4.9-8.el5.noarch.rpm       php53-pgsql-5.3.3-13.el5_8.i386.rpm
php-pgsql-5.1.6-39.el5_8.i386.rpm     php53-process-5.3.3-13.el5_8.i386.rpm
php-snmp-5.1.6-39.el5_8.i386.rpm      php53-pspell-5.3.3-13.el5_8.i386.rpm
php-soap-5.1.6-39.el5_8.i386.rpm      php53-snmp-5.3.3-13.el5_8.i386.rpm
php-xml-5.1.6-39.el5_8.i386.rpm       php53-soap-5.3.3-13.el5_8.i386.rpm
php-xmlrpc-5.1.6-39.el5_8.i386.rpm    php53-xml-5.3.3-13.el5_8.i386.rpm
php53-5.3.3-13.el5_8.i386.rpm         php53-xmlrpc-5.3.3-13.el5_8.i386.rpm
# rpm -Uvh php-5.1.6-39.el5_8.i386.rpm
Preparing...                ########################################### [100%]
   1:php                    ########################################### [100%]
-> 설치가 완료되었다.

                [참고]
                Failed dependencies 에러가 나는 경우 >
                # rpm -qa | grep php
                php-common-5.1.6-45.el5_11
                php-cli-5.1.6-45.el5_11
                php-ldap-5.1.6-45.el5_11

                # rpm -Uvh php-5.1.6-44.el5_10.x86_64.rpm
                오류: Failed dependencies:
                php-cli = 5.1.6-44.el5_10 is needed by php-5.1.6-44.el5_10.x86_64
                php-common = 5.1.6-44.el5_10 is needed by php-5.1.6-44.el5_10.x86_64
                현재 설치하는 패키지를 포함한 의존성이 있는 패키지의 버전도 포함하여 파악해야
한다.
                에러가 발생한다면 의존성 패키지를 삭제한뒤 해당버전으로 재설치후 설치를 시도
한다.
                업데이트가 되어있는 버전의 경우에는 정상적으로 설치미디어에 있는 파일을 사용
할 수 없는 경우도 발생한다.
# rpm -qa | grep php
php-5.1.6-39.el5_8
php-common-5.1.6-39.el5_8
php-cli-5.1.6-39.el5_8
php-ldap-5.1.6-39.el5_8
③ php 패키지 삭제
# rpm -e php
# rpm -qa | grep php
php-common-5.1.6-39.el5_8
php-cli-5.1.6-39.el5_8
php-ldap-5.1.6-39.el5_8
```

■ 인터넷상에서 패키지를 다운로드 하여 설치하는 방법

```
- http://rpmfind.net
- http://rpm.pbone.net
- ftp://ftp.daum.net
```

① http://rpm.pbone.net 사이트에서 패키지 다운로드

혹시나 접속이 되지 않는다면 http://mirror.kakao.com/

```
# cd /test
#                                                                                                                wget
ftp://ftp.pbone.net/mirror/ftp.centos.org/5.10/os/i386/CentOS/php-5.1.6-39.el5_9.i386.rpm
--2014-01-13                                                                                                  18:29:32--
ftp://ftp.pbone.net/mirror/ftp.centos.org/5.10/os/i386/CentOS/php-5.1.6-39.el5_9.i386.rpm
           => `php-5.1.6-39.el5_9.i386.rpm'
Resolving ftp.pbone.net... 85.14.85.4
Connecting to ftp.pbone.net|85.14.85.4|:21... connected.
Logging in as anonymous ... Logged in!
==> SYST ... done.    ==> PWD ... done.
==> TYPE I ... done.  ==> CWD /mirror/ftp.centos.org/5.10/os/i386/CentOS ... done.
==> SIZE php-5.1.6-40.el5_9.i386.rpm ... 2426228
==> PASV ... done.    ==> RETR php-5.1.6-40.el5_9.i386.rpm ... done.
Length: 2426228 (2.3M)

100%[=========================================>] 2,426,228    600K/s   in 3.9s

2014-01-13 18:29:41 (600 KB/s) - `php-5.1.6-39.el5_9.i386.rpm' saved [2426228]

-> 만약 링크에서 에러가 난다면 링크를 클릭하여 다운로드 하거나 내부에있는 여러 링크중에 하나를 새로 선택하여 링크주소복사를 새로한다.
```

```
② 패키지 설치

                만약 rpm.pbone.net 사이트에서 원하는 패키지 버전이 없다면 172.16.9.252 사이트에서 다운로드 한다.
                실습에서는 서버에서 php 패키지를 복사해서 사용한다.
                # cd /test
                # ssh 172.16.9.252 ls /root/pkg/php
                /root/pkg/php-5.1.6-23.2.el5_3.i386.rpm
                /root/pkg/php-5.1.6-39.el5_8.i386.rpm
                /root/pkg/php-5.1.6-40.el5_9.i386.rpm
                # scp 172.16.9.252:/root/pkg/php/*.rpm /test

                운영체제의 버전에 따라서 다음과 같은 버전을 가지고 실습한다.
                (CentOS 5.4 ) php-5.1.6-23.2.el5_3.i386.rpm
                (CentOS 5.9 ) php-5.1.6-39.el5_8.i386.rpm
                (CentOS 5.10) php-5.1.6-40.el5_9.i386.rpm

# ls
php-5.1.6-39.el5_9.i386.rpm
                [참고] 패키지 의존성 관계에 걸려서 설치가 안되는 경우
                # rpm -Uvh php-5.1.6-40.el5_9.i386.rpm
                error: Failed dependencies:
                        php-cli = 5.1.6-40.el5_9 is needed by php-5.1.6-40.el5_9.i386
                        php-common = 5.1.6-40.el5_9 is needed by php-5.1.6-40.el5_9.i386
                -> php-cli, php-common 프로그램도 같은 버전으로 다운로드하고 같이 설치 해야 한다.

# rpm -qip php-5.1.6-39.el5_8.i386.rpm
Name        : php                           Relocations: (not relocatable)
Version     : 5.1.6                             Vendor: CentOS
Release     : 39.el5_8                       Build Date: Thu Jun 28 01:29:57 2012
Install Date: (not installed)                Build Host: builder10.centos.org
Group       : Development/Languages          Source RPM: php-5.1.6-39.el5_8.src.rpm
Size        : 6191674                           License: The PHP License v3.01
Signature   : DSA/SHA1, Thu Jun 28 05:21:35 2012, Key ID a8a447dce8562897
URL         : http://www.php.net/
Summary     : The PHP HTML-embedded scripting language. (PHP: Hypertext Preprocessor)
Description :
PHP is an HTML-embedded scripting language that allows developers to write
```

```
dynamically generated web pages. PHP is ideal for writing database-enabled
websites, with built-in integration for several commercial and non-commercial
database management systems. PHP is often used as a replacement for CGI
scripts.

The php package contains a module that adds support for the PHP language to the
Apache HTTP Server.
# rpm -Uvh php-5.1.6-39.el5_8.i386.rpm
Preparing...                ########################################### [100%]
   1:php                    ########################################### [100%]
# rpm -qa | grep php
php-common-5.1.6-39.el5_8
php-cli-5.1.6-39.el5_8
php-5.1.6-39.el5_8
php-ldap-5.1.6-39.el5_8

③ 설치된 패키지 정보 확인
# which php
/usr/bin/php
# rpm -qf /usr/bin/php
php-cli-5.1.6-39.el5_8
# rpm -ql php
/etc/httpd/conf.d/php.conf
/usr/lib/httpd/modules/libphp5-zts.so
/usr/lib/httpd/modules/libphp5.so
/var/lib/php/session
/var/www/icons/php.gif
                # rpm -qi package          /* 설치된 패키지의 자세한 정보 확인 */
                # rpm -qi -p package.rpm   /* 현재 디렉토리에 존재하는 패키지의 자세한 정보
확인 */

# rpm -qi php    /* -i : information */
Name        : php                           Relocations: (not relocatable)
Version     : 5.1.6                             Vendor: CentOS
Release     : 39.el5_8                       Build Date: Thu Jun 28 01:29:57 2012
Install Date: Mon Jan 13 18:41:45 2014          Build Host: builder10.centos.org
Group       : Development/Languages          Source RPM: php-5.1.6-39.el5_8.src.rpm
Size        : 6191674                           License: The PHP License v3.01
Signature   : DSA/SHA1, Thu Jun 28 05:21:35 2012, Key ID a8a447dce8562897
URL         : http://www.php.net/
Summary     : The PHP HTML-embedded scripting language. (PHP: Hypertext Preprocessor)
Description :
PHP is an HTML-embedded scripting language that allows developers to write
dynamically generated web pages. PHP is ideal for writing database-enabled
websites, with built-in integration for several commercial and non-commercial
database management systems. PHP is often used as a replacement for CGI
scripts.

The php package contains a module that adds support for the PHP language to the
Apache HTTP Server.

                (정리) 패키지 관리

                rpm CMD(RedHat Packages Manager)
                # rpm -ivh [--nodeps] pkg.rpm
                # rpm -Uvh [--nodeps] pkg.rpm
                # rpm -Fvh [--nodeps] pkg.rpm

                # rpm -qa | grep pkg
                # rpm -q pkg

                # rpm -qf /etc/passwd
                # rpm -ql pkg
                # rpm -qi pkg
                # rpm -qi -p pkg.rpm

                # rpm -e [--nodeps] pkg
```

DNF (Dandified Yum)

기존의 rpm의 경우 wget을 이용하여 rpm패키지가 저장된 서버의 주소를 모두 입력하고 rpm -i옵션을 이용해 패키지를 설치해야만 했다. 그러나 dnf는 rpm패키지가 저장된 서버에 자동으로 접속하고 설치하고자 하는 패키지를 다운로드하여 설치한다. 또한 의존성을 가지는 다른 rpm패키지까지 알아서 다운로드하여 설치하며 rpm 패키지들을 안전하게 설치, 삭제 및 업데이트를 한다. rpm에 비해 이미 설치되었거나 혹은 저장소에 설치 가능한 패키지에 관한 정보를 효율적이고 쉽게 검색 할 수 있고, 일일이 수동 업그레이드를 할 필요가 없으므로 많은 시스템을 관리할 때 더더욱 유용하게 쓰인다.

DNF(Dandified Yum)는 RPM 기반 리눅스 배포판을 위한 패키지 관리도구이다.
페도라 18에 처음 도입되었고,페도라 22와 레드햇 엔터프라이즈 리눅스 8부터 기존의 Yum을 대신하여 기본 패키지 관리도구로 채택되었다.
DNF는 Yum의 낮은 성능, 많은 메모리 사용량, 비효율적인 중복 의존성 해결 메커니즘 등의 문제를 해결하기 위해 개발되었고, Yum과 달리, 외부 라이브러리인 libsolv를 통해 의존성 문제를 처리한다.
DNF는 RPM 패키지 매니저와 여러 지원 라이브러리를 이용하여 패키지 관리 작업을 수행한다.
처음에는 Yum 3.4를 기반으로 파이썬으로 개발되었지만, 지금은 대부분의 기능을 C로 포팅하여 libdnf 라이브러리로 옮기는 작업이 진행중이다. - 위키백과

[명령어 형식]

```
# dnf [options] COMMAND

반환값
0 : 작업이 성공했습니다.
1 : dnf에서 처리 한 오류가 발생했습니다.
3 : 조작 중에 알 수없는 처리되지 않은 오류가 발생했습니다.
100: 업데이트 확인 참조
200: 잠금 획득 또는 해제에 문제가 있었습니다.


(명령어 사용예)
# dnf list      (# dnf list all)
# dnf list installed
# dnf list available

# dnf -y update
# dnf -y update php
# dnf -y install php

# dnf -y localinstall php

# dnf (erase|remove) php
```

```
# dnf --help
usage: dnf [options] COMMAND

주요 명령 목록 :

alias                     List or create command aliases
autoremove                원래 종속물로 설치된 모든 불필요한 패키지 제거
check                     check for problems in the packagedb
check-update              사용 가능한 패키지 업그레이드 확인
clean                     remove cached data
deplist                   List package is dependencies and what packages provide them
distro-sync               설치된 패키지를 최신 버전과 동기화한다.
downgrade                 Downgrade a package
group                     표시하거나 사용하십시오.
help                      유용한 메시지 표시
history                   표시 또는 사용, 거래 내역
info                      패키지 또는 패키지 그룹에 대한 세부 정보 표시
install                   install a package or packages on your system
list                      패키지 또는 패키지 그룹 나열
makecache                 메타 데이터 캐시를 생성한다.
mark                      mark or unmark installed packages as installed by user.
module                    Interact with Modules.
provides                  주어진 값을 제공하는 패키지 찾기
reinstall                 패키지 다시 설치
remove                    remove a package or packages from your system
repolist                  display the configured software repositories
repoquery                 키워드와 일치하는 패키지 검색
repository-packages       지정된 저장소의 모든 패키지 위에 명령을 실행합니다.
search                    search package details for the given string
```

```
shell                     run an interactive DNF shell
swap                      하나의 스펙을 제거하고 설치하기위한 대화 형 dnf mod 실행
updateinfo                display advisories about packages
upgrade                   시스템에서 패키지를 업그레이드하십시오.
upgrade-minimal           upgrade, but only 'newest' package match which fixes a problem that
affects your system

플러그인 명령리스트 :

builddep                  Install build dependencies for package or spec file
changelog                 Show changelog data of packages
config-manager            dnf 구성 옵션 및 저장소 관리
copr                      Copr 저장소와 상호 작용하십시오.
debug-dump                설치된 rpm 패키지에 대한 정보를 파일에 덤프하십시오.
debug-restore             디버그 덤프 파일에 기록 된 패키지 복원
debuginfo-install         debuginfo 패키지 설치
download                  현재 디렉토리에 패키지 다운로드
needs-restarting          다시 시작해야하는 업데이트 된 바이너리 결정
playground                놀이터 저장소와 상호 작용하십시오.
repoclosure               저장소에 대한 확인되지 않은 종속성 목록 표시
repodiff                  List differences between two sets of repositories
repograph                 도트 형식의 전체 패키지 종속성 그래프 출력
repomanage                rpm 패키지 디렉토리 관리
reposync                  원격 저장소에서 모든 패키지를 다운로드하십시오.

Optional arguments:
  -c [config file], --config [config file]
                        config file location
  -q, --quiet           quiet operation
  -v, --verbose         verbose operation
  --version             show DNF version and exit
  --installroot [path]  set install root
  --nodocs              do not install documentations
  --noplugins           disable all plugins
  --enableplugin [plugin]
                        enable plugins by name
  --disableplugin [plugin]
                        disable plugins by name
  --releasever RELEASEVER
                        override the value of $releasever in config and repo
                        files
  --setopt SETOPTS      set arbitrary config and repo options
  --skip-broken         resolve depsolve problems by skipping packages
  -h, --help, --help-cmd
                        show 명령 도움말
  --allowerasing        종속성을 해결하기 위해 설치된 패키지 지우기 허용
  -b, --best            트랜잭션에서 사용 가능한 최상의 패키지 버전을 사용해보십시오.
  --nobest              do not limit the transaction to the best candidate
  -C, --cacheonly       시스템 캐시에서 완전히 실행하고, 캐시를 업데이트하지 않습니다.
  -R [minutes], --randomwait [minutes]
                        최대 명령 대기 시간
  -d [debug level], --debuglevel [debug level]
                        디버깅 출력 레벨
  --debugsolver         자세한 해결 결과를 파일로 덤프합니다.
  --showduplicates      repos에있는 중복 목록을 목록 / 검색 명령에 표시합니다.
  -e ERRORLEVEL, --errorlevel ERRORLEVEL
                        오류 출력 레벨
  --obsoletes           패키지가 정보, 목록 및 리포 눅스에 대해 사용하지 않는 업그레이드 또는
표시 기능을 위해
                        dnf의 처리 논리를 사용하지 못하게합니다.
  --rpmverbosity [debug level name]
                        rpm 용 디버깅 출력 레벨
  -y, --assumeyes       모든 질문에 대해 자동으로 대답하십시오.
  --assumeno            모든 질문에 대해 자동으로 대답합니다.
  --enablerepo [repo]
  --disablerepo [repo]
  --repo [repo], --repoid [repo]
                        id 나 glob로 특정 리포지토리를 활성화 할 수 있습니다. 여러 번 지정할
수 있습니다.
  --enable, --set-enabled
```

```
                        enable repos with config-manager command
                        (automatically saves)
  --disable, --set-disabled
                        disable repos with config-manager command
                        (automatically saves)
  -x [package], --exclude [package], --excludepkgs [package]
                        패키지를 이름이나 glob로 제외합니다.
  --disableexcludes [repo], --disableexcludepkgs [repo]
                        excludepkgs 사용 중지
  --repofrompath [repo,path]
                        레이블 및 추가 저장소의 경로는 여러 번 지정할 수 있습니다.
  --noautoremove        더 이상 사용되지 않는 종속성 제거 사용 안 함
  --nogpgcheck          disable gpg signature checking (if RPM policy allows)
  --color COLOR         색상 사용 여부 제어
  --refresh             명령을 실행하기 전에 만료 된 메타 데이터 설정
  -4                    IPv4 주소로만 해결
  -6                    IPv6 주소로만 해결
  --destdir DESTDIR, --downloaddir DESTDIR
                        패키지를 복사 할 디렉토리를 설정하십시오.
  --downloadonly        다운로드 패키지 만
  --comment COMMENT     거래에 의견을 추가하십시오.
  --bugfix              버그 수정 관련 패키지를 업데이트에 포함 시키십시오.
  --enhancement         업데이트 관련 향상 패키지 포함
  --newpackage          새 패키지 관련 패키지를 업데이트에 포함하십시오.
  --security            업데이트에 보안 관련 패키지 포함
  --advisory ADVISORY, --advisories ADVISORY
                        업데이트에서 주어진 권고를 수정하는 데 필요한 패키지를 포함하십시오.
  --bz BUGZILLA, --bzs BUGZILLA
                        업데이트에서 주어진 BZ를 수정하는 데 필요한 패키지를 포함하십시오.
  --cve CVES, --cves CVES
                        업데이트에서 주어진 CVE를 수정하는 데 필요한 패키지를 포함하십시오
  --sec-severity              {Critical,Important,Moderate,Low},              --secseverity
{Critical,Important,Moderate,Low}
                        업데이트에서 심각도와 일치하는 보안 관련 패키지를 포함합니다.
  --forcearch ARCH      아키텍처의 사용을 강요하십시오
```

로그를 보기 위한 기본 지식
네트워크와 스토리지 설정

<자원 관리를 위한 중요 포인트 이해>
서비스와 프로세스 이해
프로세스 관리

단원목표

- 프로세스 정보
- PID / PPID
- 하드웨어 리소스
- 프로세스 관리
- 프로세스 모니터링

프로세스 정보

- 프로세스 정보

    /proc/PID

프로세스(process)란?

프로세스(process)란? 실행중인 프로그램이다.

DISK(한글.exe) ----> MEM(한글 process) ----> CPU(중앙 처리 장치)

프로세스는 프로그램이 메모리에 적재되어 실제로 실행되고 있는 상태를 말한다. 프로세스는 컴퓨터 내에서 실행중인 프로그램의 인스턴스이다. 여러 명의 사용자들에 의해 공유되고 있는 응용프로그램은 일반적으로 각 사용자들의 실행단계에서 하나의 프로세스를 갖는다. 프로세스는 자식프로세스라고도 불리는 서브프로세스를 시작시킬 수 있다. 자식프로세스는 부모프로세스의 복제로서 부모프로세스의 자원을 일부 공유하는데, 부모프로세스가 종료되면 더 이상 존재할 수 없다. 프로세스들은 몇 가지 IPC 방식을 통하여 정보를 교환하거나 그들의 연산을 동기화할 수 있다.

모든 프로그램은 실행될 때 하나 이상의 프로세스를 갖는다. 하나의 프로세스에는 여러 명령어와 카운트, CPU 레지스터, 그리고 루틴 인자, 복귀 주소, 저장된 변수 등의 테이터 스택이 포함되어 있다. 각 프로세스는 고유의 권한과 책임을 가지고 서로 통신하며, 시스템에서 동작중인 하나의 프로세스가 잘못된 연산을 수행하여 에러를 일으키더라도 다른 프로세스는 정상적으로 작동한다. 개별 프로세스는 자신의 가상 주소 공간에서 실행되며, 커널이 제공하는 인터페이스를 통해서만 다른 프로세스와 연동할 수 있다.

프로세스는 여러 가지 자원을 사용한다. 프로세스는 해당 명령을 수행하기 위해 운영체제에 따라 CPU를 점유할 수 있다. 명령어와 데이터를 저장하기 위해 물리적인 메모리를 사용한다. 프로세스는 운영체제의 제어를 받으면서 실행(running), 대기(waiting), 중단(stopped), 좀비(zombie) 중 한 상태에 있게 된다.

[1] 자식프로세스(Child Process) & 부모프로세스(Parent Process)

- PID(Process Identification) : 프로세스가 시작할 때 할당받는 프로세스 식별번호
- PPID(Parent Process Identification) : 부모 프로세스 식별번호 (서브 프로세스를 실행시킨 프로세스)

```
# ps
 PID TTY          TIME CMD
 7321 pts/3    00:00:00 bash
# bash
# ps -f
UID        PID  PPID  C STIME TTY          TIME CMD
root      7321  7319  0 01:19 pts/3    00:00:00 -bash
root      7354  7321  0 01:19 pts/3    00:00:00 bash

PID=7321
----------+
          |
          | PID=7354, PPID=7321
          +----------------------
```

```
# ps -ef | more
UID        PID   PPID  C STIME TTY          TIME CMD
root         1      0  0 19:33 ?        00:00:02 /usr/lib/systemd/systemd --switched-root
--system --deserialize 22
root         2      0  0 19:33 ?        00:00:00 [kthreadd]
root         4      2  0 19:33 ?        00:00:00 [kworker/0:0H]
root         6      2  0 19:33 ?        00:00:00 [ksoftirqd/0]
root         7      2  0 19:33 ?        00:00:00 [migration/0]
root         8      2  0 19:33 ?        00:00:00 [rcu_bh]
root         9      2  0 19:33 ?        00:00:00 [rcu_sched]
root        10      2  0 19:33 ?        00:00:00 [lru-add-drain]
root        11      2  0 19:33 ?        00:00:00 [watchdog/0]
root        12      2  0 19:33 ?        00:00:00 [watchdog/1]

(ps -ef 명령어 출력 결과 해석)
--------------------------------------------------------------------------------
UID,   프로세스를 실행시킨 사용자, 프로세스의 주인
PID,   프로세스 아이디, 프로세스 식별 번호
PPID,  부모 프로세스 아이디
C,     현재 사용되지 않는 필드
STIME(Start Time), 프로세스 실행 시작 시간
TTY,   제어 터미널, 프로세스가 실행된 터미널
TIME,  CPU 사용 누적 시간
CMD,   명령어(옵션 + 인자 포함)
--------------------------------------------------------------------------------
```

[참고] 데몬(Daemon)의 정의

```
Deamon 이란?

시스템(System)을 위해 또는 서비스(Service)를 위해 백그라운드에서 동작하는 프로세스이다.
커널상에서 백그라운드 모드로 작동하여 비활성화 상태에서 요청이 있을 때만 동작하는 프로세스를
말한다. 커널상에 백그라운드 모드로 실행되어 작동하지 않고 있을 때는 CPU에 부하를 주지 않지만
메모리의 공간은 차지하고 있으므로 데몬이 커널상에 많이 존재한다면 시스템의 자원을 많이 사용하
게 된다.

EX) 웹 데몬     ----> httpd      service, systemd
    메일 데몬   ----> sendmail
    ftp 데몬    ----> vsftpd
    ------------------------
    telnet 데몬 ----> xinetd , socket

데몬, 서비스, 백그라운드 프로세스
```

프로세스의 정보 (Process Information)

[1] 프로세스 정보가 존재하는 디렉토리

/proc 라는 디렉토리에 각 프로세스에 해당되는 PID번호로 디렉토리들이 있다.

/proc 라는 디렉토리는 커널 메모리를 마운트 한 것이다.

```
# ls /proc
1     2525  3065  3780  4103  4330  4431  6165        filesystems  net
10    2526  3224  3793  4108  4334  4443  635         fs           partitions
11    2527  3250  3794  4109  4335  4476  6361        ide          schedstat
12    2528  3278  3802  4110  4337  4478  658         interrupts   scsi
13    2562  3541  3808  4111  4355  4480  689         iomem        self
14    2564  3543  3817  4112  4359  4530  7           ioports      slabinfo
15    2566  3565  3842  4115  4361  5     8           irq          stat
16    2568  3569  3870  4116  4365  507   9           kallsyms     swaps
17    2570  3579  3887  4209  4368  572   acpi        kcore        sys
18    26    3590  3892  4211  4370  573   asound      key-users    sysrq-trigger
19    27    3619  3919  4212  4371  579   buddyinfo   keys         sysvipc
1932  28    3620  3937  4236  4378  580   bus         kmsg         tty
196   283   3621  3945  4238  4391  581   cmdline     loadavg      uptime
197   284   3622  3955  4254  4396  582   cpuinfo     locks        version
198   285   3631  3964  4286  4401  583   crypto      mdstat       vmcore
199   286   3655  3997  4288  4406  5877  devices     meminfo      vmmemctl
2     287   3677  4     4314  4408  599   diskstats   misc         vmstat
202   288   3696  4006  4316  4423  6     dma         modules      zoneinfo
```

```
204   289   3702  4014  4322  4427  6159  driver       mounts
25    29    3732  4041  4323  4428  6163  execdomains  mpt
2524  3     3770  4042  4328  4430  6164  fb           mtrr
# sleep 600 &
[1] 6180
# ps
 PID TTY          TIME CMD
 6165 pts/1    00:00:00 bash
 6180 pts/1    00:00:00 gedit
# ls -ld /proc/6180
dr-xr-xr-x 5 root root 0 Jun 18 14:21 /proc/6180
                # ls -l /proc/6180
                # file /proc/6180/*
-> gedit 툴을 종료

# ls -ld /proc/6180
ls: /proc/6180: No such file or directory
```

프로세스 관리

- 프로세스 실행
fg) # sleep 300
bg) # sleep 300 &

- 프로세스 확인
ps –ef | grep xinetd
ps aux | grep xinetd

- 프로세스 종료
kill –[1|2|9|15] PID
killall httpd

[1] 프로세스 관리 (백그라운드와 포그라운드 관리 - &, bg, fg, jobs)

쉘 프롬프트에 명령어를 입력해서 포그라운드 상태($ cat > testfile)로 프로그램을 실행시킴. 그 프로세스가 실행되는 동안 셸 프롬프트를 볼 수 없다. <Ctrl + D>로 프로그램이 종료 되어야 쉘프롬프트 상태로 돌아오게 된다. 백그라운드는 명령행 끝에 &(앰퍼샌드)를 붙여서 사용하면 쉘 프롬프트가 떨어지고 대신 프로세스는 사용자가 보지 않는 상태로 실행된다.

백그라운드 프로세스는 $ bg 명령어로 확인가능 하고 $ fg 명령어로 포그라운드 전환이 가능하다. 반대로 하는 방법은 포그라운드에서 돌아가는 중에 <Ctrl+Z> 으로 멈추고 $ bg 명령을 내리면 된다.

(프로그램이 실행되는 형태)

(fg) # ls (# gedit)

(bg) # ls & (# gedit &)

```
(fg 형태로 프로그램이 실행되면)
# ps
 PID TTY          TIME CMD
 8711 pts/1    00:00:00 bash
# sleep 600
-> 명령어 창으로 돌아와서 "# ls" 명령어를 수행해도 명령어가 출력결과가 보이지 않는다.
-> 부모 프로세스(bash)가 종료되면 자식 프로세스가 종료된다.


(bg 형태로 프로그램이 실행되면)
# ps
 PID TTY          TIME CMD
 8711 pts/1    00:00:00 bash
# gedit &
-> 명령어 창으로 돌아와서 "# ls" 명령어를 수행하면 정상적으로 수행이 된다.
-> 부모 프로세스(bash)가 종로되었다고 해서 자식 프로세스가 종료되지 않는다.(# exit 명령어 사용)
```

[EX1] 프로그램 실행 실습

```
# cat & <ENTER> /* 백그라운드 상태로 프로세스를 실행 */
[1] 7827
        [1]  : 잡 아이디(Job ID)
```

```
        7827 : 프로세스 아이디(PID)
# ps -ef | grep cat
root      7827  7601  0 02:46 pts/3    00:00:00 cat
root      27281      1   0 Jan26 ?          00:00:00 /usr/libexec/notification-area-applet
--oaf-activate-ii d=OAFIID:GNOME_NotificationAreaApplet_Factory --oaf-ior-fd=21

[1]+  Stopped                 cat
# ls -ld /proc/7827
dr-xr-xr-x 5 root root 0  1월 27 02:46 /proc/7827/
# fg        (# fg 1)
cat
<CTRL + C>                                /* 포그라운드 상태의 프로세스를 종료시킴 */
# ls -ld /proc/7827 /* 지워진 것을 확인 할 수 있다. */
ls: /proc/7827/: 그런 파일이나 디렉토리가 없음
```

[EX2] 포그라운드 & 백그라운드 실행 실습

```
# sleep 500 /* 포그라운드 상태로 프로세스를 실행 */
[1]+  Stopped                 sleep 500
<CTRL + Z>
# sleep 600
[2]+  Stopped                 sleep 600
<CTRL + Z>
# sleep 700
[3]+  Stopped                 sleep 700
<CTRL + Z>
# jobs
[1]   Stopped                 sleep 500
[2]-  Stopped                 sleep 600
[3]+  Stopped                 sleep 700
# bg 1/* 일시중지된 포그라운드 작업을 백그라운드에서 실행 */
[1] sleep 500 &
# bg 2
[2]- sleep 600 &
# bg 3
[3]+ sleep 700 &
# jobs
[1]   Running                 sleep 500 &
[2]-  Running                 sleep 600 &
[3]+  Running                 sleep 700 &
# fg 1
sleep 500
<CTRL + C>
# fg 2
sleep 600
<CTRL + C>
# fg 3
sleep 700
<CTRL + C>
# jobs
#
```

[2] 프로세스 정보 확인

[명령어 형식]

```
# ps /* 현재 터미널에서 실행된 프로세스의 간략한 정보 확인 */
# ps -l /* 현재 터미널에서 실행된 프로세스의 자세한 정보 확인 */
# ps -a /* 사용자 프로세스에 대한 간략한 정보 확인 */
# ps -U <사용자명> (# ps -U user01, # ps -U user01,user02,user03)
/* 특정한 사용자가 실행시킨 프로세스의 간략한 정보 확인 */
# ps -t pts/5 /* 특정한 터미널에서 실행시킨 프로세스의 간략한 정보 확인 */

# ps aux | grep xinetd    (# ps -ef | grep xinetd)
# ps -aux | grep xinetd
```

[명령어 옵션]

옵션 : 내용

-e : 모든 프로세스 리스트를 출력한다.
-f : 프로세스 시작시간, 프로세스의 부모 ID, 그 프로세스에 관련된 사용자 ID, 명령 이름과 가능한 매개변수등 모든 정보를 출력한다. (full format)
-l : 자세한 정보 보기

-p (pid) : 지정된 프로세스에 대한 정보를 출력한다.
-t (tty) : 지정된 터미널에 연관된 프로세스에 대한 정보를 출력한다.
-u (uid) : 지정된 사용자에 연관된 프로세스에 대한 정보를 출력한다.
a : 다른 사용자의 프로세스 상태도 표시된다.
x : 화면에 보이지 않는 프로세스까지 모두 표시

[EX1] ps 명령어 실습

```
# ps /* 자신을 포함하여 자신이 실행한 프로세스를 확인 */
 PID TTY          TIME CMD
 6165 pts/1    00:00:00 bash
# ps -U root /* root 사용자가 사용하는 프로세스 정보를 보여줌 */
PID TTY          TIME CMD
    1 ?        00:00:00 init
    2 ?        00:00:00 migration/0
    3 ?        00:00:00 ksoftirqd/0
    4 ?        00:00:00 watchdog/0
    5 ?        00:00:00 events/0
..... (중략) .....
# ps /* 현재 터미널에서 실행된 프로세스의 간략한 정보 확인 */
 PID TTY          TIME CMD
 7601 pts/3    00:00:00 bash
 7714 pts/3    00:00:00 ps
# ps -t pts/3 /* 터미널창 번호를 확인 후에 해당 터미널의 프로세스 정보를 확인 */
 PID TTY          TIME CMD
 7601 pts/3    00:00:00 bash
 7717 pts/3    00:00:00 ps
# ps a /* 다른 사용자의 프로세스 상태도 표시 */
 PID TTY      STAT   TIME COMMAND
 2450 tty1     Ss+    0:00 /sbin/mingetty tty1
 2453 tty2     Ss+    0:00 /sbin/mingetty tty2
 2454 tty3     Ss+    0:00 /sbin/mingetty tty3
 2455 tty4     Ss+    0:00 /sbin/mingetty tty4
 2456 tty5     Ss+    0:00 /sbin/mingetty tty5
 2457 tty6     Ss+    0:00 /sbin/mingetty tty6
 2562 tty7     Ss+    0:11 /usr/bin/Xorg :0 -br -audit 0 -auth /var/gdm/:0.Xauth
 7563 pts/1    Ss+    0:00 -bash
 7601 pts/3    Ss     0:00 -bash
 7731 pts/3    R+     0:00 ps a
27308 pts/2    Ss+    0:00 bash
# ps -ef | less /* 모든 프로세스 리스트를 출력하면서 모든 정보를 함께 출력 함 */
UID        PID  PPID  C STIME TTY          TIME CMD
root         1     0  0 Jan26 ?        00:00:00 init [5]
root         2     1  0 Jan26 ?        00:00:00 [migration/0]
root         3     1  0 Jan26 ?        00:00:00 [ksoftirqd/0]
root         4     1  0 Jan26 ?        00:00:00 [watchdog/0]
root         5     1  0 Jan26 ?        00:00:00 [events/0]
# ps aux | less    /* a : 다른 사용자의 프로세스 상태도 표시
                                                          x : 화면에 보이지 않는 프로세
스까지 모두 표시
                                                          u : 프로세스를 사용한 사용자
와 실행 시간까지 표시 */
USER       PID %CPU  %MEM   VSZ   RSS TTY    STAT   START   TIME COMMAND
root         1  0.0  0.1   2068   576 ?        Ss   Jan26   0:00 init [5]
root         2  0.0  0.0      0     0 ?        S<   Jan26   0:00 [migration/0]
root         3  0.0  0.0      0     0 ?        SN   Jan26   0:00 [ksoftirqd/0]
root         4  0.0  0.0      0     0 ?        S<   Jan26   0:00 [watchdog/0]
root         5  0.0  0.0      0     0 ?        S<   Jan26   0:00 [events/0]
root         6  0.0  0.0      0     0 ?        S<   Jan26   0:00 [khelper]
root         7  0.0  0.0      0     0 ?        S<   Jan26   0:00 [kthread]
root        10  0.0  0.0      0     0 ?        S<   Jan26   0:01 [kblockd/0]
[참고] ps aux를 통해 명령어 출력시 나오는 헤더부분 설명
user는 프로세스 소유자의 이름, ( u 옵션 )
PID는 프로세스 식별자 번호
%CPU : 마지막 1분 동안 프로세스가 사용한 CPU 점유율 ( u 옵션 )
%MEN : 마지막 1분 동안 프로세스가 사용한 메모리의 점유율 ( u,v 옵션 )
       -u 옵션으로 사용자의 ID나 이름을 지정하여 지정한 사용자의 프로세스만을 살펴 볼수 있다.
VSZ : 가상 메모리에 적재된 프로세스의 kb단위 크기
START는 프로세스가 시작된 시간

STAT 필드의 상태
```

```
-D : 디스크 입출력 대기 상태로 interrupts를 걸 수 없는 상태
-R : 실행중
-S : 짧은 sleep 상태
-T : 정지 상태
-Z : 좀비 상태
-W : 상주한 페이지가 없는 프로세스
-< : 높은 우선권 프로세스
-N : 낮은 우선권 프로세스
-L : 페이지가 락이 걸린 상태
```

프로세스 종료 명령어(# kill)

실행 중인 프로세스를 강제로 종료(인터럽트)시킬 수 있지만 백그라운드에서 실행되는 프로세스는 종료시키지 못하게 된다. 이때 kill 명령을 사용해서 프로세스를 종료시킨다다. kill 명령은 사용자가 프로세스에 일정한 신호를 보내서 프로세스를 종료시킨다, kill -l 명령을 사용하면 신호의 종류를 확인 할 수 있다.

[명령어 형식]

```
# kill PID          (# kill -15 PID, # kill -TERM PID)
# kill PID PID PID
# kill -9 PID       (# kill -KILL PID)
# kill -2 PID       (# kill -INT PID)
```

■ 시그널(Signal)? 프로세스가 다른 프로세스에게 보내는 비동기적 알림 메세지

[참고] 일반적으로 많이 쓰이는 시그널(Signal)

```
시그널 번호 시그널 이름 설  명
1       SIGHUP   프로세스 재시작(HangUp) (EX: # kill  -1 450)
2       SIGINT   인터럽트(Interrupt, <Ctrl + C>) (EX: # kill  -2 450)
9       SIGKILL  강제 종료(force exit signal) (EX: # kill  -9 450)
15      SIGTERM  정상 종료(exit), 기본 시그널 (EX: # kill -15 450)
kill 명령에 아무 신호도 입력하지 않으면 15번 신호인 SIGTERM을 보낸다. (기본값)
```

```
# man 7 signal
..... (중략) .....
       Signal     Value     Action   Comment
       ----------------------------------------------------------------------
       SIGHUP       1       Term    Hangup detected on controlling terminal
                                    or death of controlling process
       SIGINT       2       Term    Interrupt from keyboard
       SIGQUIT      3       Core    Quit from keyboard
       SIGILL       4       Core    Illegal Instruction
       SIGABRT      6       Core    Abort signal from abort(3)
       SIGFPE       8       Core    Floating point exception
       SIGKILL      9       Term    Kill signal
       SIGSEGV     11       Core    Invalid memory reference
       SIGPIPE     13       Term    Broken pipe: write to pipe with no readers
       SIGALRM     14       Term    Timer signal from alarm(2)
       SIGTERM     15       Term    Termination signal
       SIGUSR1   30,10,16   Term    User-defined signal 1
       SIGUSR2   31,12,17   Term    User-defined signal 2
       SIGCHLD   20,17,18   Ign     Child stopped or terminated
       SIGCONT   19,18,25   Cont    Continue if stopped
       SIGSTOP   17,19,23   Stop    Stop process
       SIGTSTP   18,20,24   Stop    Stop typed at tty
       SIGTTIN   21,21,26   Stop    tty input for background process
..... (중략) .....
```

```
# kill -l
1) SIGHUP       2) SIGINT       3) SIGQUIT      4) SIGILL
 5) SIGTRAP      6) SIGABRT      7) SIGBUS       8) SIGFPE
 9) SIGKILL     10) SIGUSR1     11) SIGSEGV     12) SIGUSR2
13) SIGPIPE     14) SIGALRM     15) SIGTERM     16) SIGSTKFLT
17) SIGCHLD     18) SIGCONT     19) SIGSTOP     20) SIGTSTP
21) SIGTTIN     22) SIGTTOU     23) SIGURG      24) SIGXCPU
25) SIGXFSZ     26) SIGVTALRM   27) SIGPROF     28) SIGWINCH
29) SIGIO       30) SIGPWR      31) SIGSYS      34) SIGRTMIN
35) SIGRTMIN+1  36) SIGRTMIN+2  37) SIGRTMIN+3  38) SIGRTMIN+4
39) SIGRTMIN+5  40) SIGRTMIN+6  41) SIGRTMIN+7  42) SIGRTMIN+8
43) SIGRTMIN+9  44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12
```

```
47) SIGRTMIN+13 48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14
51) SIGRTMAX-13 52) SIGRTMAX-12 53) SIGRTMAX-11 54) SIGRTMAX-10
55) SIGRTMAX-9  56) SIGRTMAX-8  57) SIGRTMAX-7  58) SIGRTMAX-6
59) SIGRTMAX-5  60) SIGRTMAX-4  61) SIGRTMAX-3  62) SIGRTMAX-2
63) SIGRTMAX-1  64) SIGRTMAX
```

[EX1] kill 명령어 실습

```
# ps -ef | grep postfix
root      23476      1  0 06:58 ?        00:00:00 /usr/libexec/postfix/master -w
postfix   23478  23476  0 06:58 ?        00:00:00 qmgr -l -t unix -u
postfix   71618  23476  0 19:30 ?        00:00:00 pickup -l -t unix -u
root      73004  72956  0 19:52 pts/4    00:00:00 grep --color=auto postfix
                                                  /var/spool/clientmqueue
# kill 23476 (# kill -15 23476, # kill -TERM 23476)
# ps -ef | grep postfix
root      73074  72956  0 19:53 pts/4    00:00:00 grep --color=auto postfix
# systemctl restart postfix
# systemctl status postfix | grep -i active
   Active: active (running) since 수 2020-08-19 19:54:18 KST; 22s ago
# ps -ef | grep postfix
root      73191      1  0 19:54 ?        00:00:00 /usr/libexec/postfix/master -w
postfix   73192  73191  0 19:54 ?        00:00:00 pickup -l -t unix -u
postfix   73193  73191  0 19:54 ?        00:00:00 qmgr -l -t unix -u
root      73245  72956  0 19:55 pts/4    00:00:00 grep --color=auto postfix
# kill -9 73191  (# kill -KILL 7908)
        일반적인 상황에서는 -15 시그널과 -9 시그널에 차이가 없다.
        강제종료 상황이 다가왔을대는 -15로 장애가 난 프로세스를 종료하는 경우 정상적으로 종료
가 되지 않는다. 이러한 경우에는 어쩔수 없이 -9 시그널로 종료해야한다.
# ps -ef | grep postfix
root      73294  72956  0 19:55 pts/4    00:00:00 grep --color=auto postfix
# systemctl restart postfix
# ps -ef | grep postfix
root      73446      1  0 19:56 ?        00:00:00 /usr/libexec/postfix/master -w
postfix   73447  73446  0 19:56 ?        00:00:00 pickup -l -t unix -u
postfix   73448  73446  0 19:56 ?        00:00:00 qmgr -l -t unix -u
root      73468  72956  0 19:57 pts/4    00:00:00 grep --color=auto postfix
# kill -1 73446
# ps -ef | grep postfix
root      73446      1  0 19:56 ?        00:00:00 /usr/libexec/postfix/master -w
postfix   73475  73446  0 19:57 ?        00:00:00 pickup -l -t unix -u
postfix   73476  73446  0 19:57 ?        00:00:00 qmgr -l -t unix -u
root      73482  72956  0 19:57 pts/4    00:00:00 grep --color=auto postfix
```

[참고] kill & killall 명령어

```
# kill PID
# killall httpd
```

프로세스 모니터링
- top
top
top -u oracle
top -p [PID]

프로세스 모니터링

[1] top 명령 (ps와 같은 명령어 - 차이점? top은 실시간 모니터링)

[참고] 서버에 접속하여 시스템 모니터링

[TERM1] (TUI) # top

[TERM2] (TUI) # tail -f /var/log/messages

실시간으로 프로세스의 상태나 CPU, 메모리, 접속한 사용자의 수 등을 알 수 있게 된다. 시스템의 성능을 확인하려 할 때 많이 사용하게 된다.

기본설정

- 명령어 수행 시간 간격(time interval)은 3 ~ 4초
- CPU 사용량별로 정렬(Sorting)

[명령어 형식]

```
# top
# top -u oracle /* 해당 사용자의 프로세스 정보를 실시간 모니터링 */
# top -p PID1,PID2,PID3/* 해당 번호의 프로세스 정보를 실시간 모니터링 */
# top -d [TIME]/* 새로고침의 시간을 변경한다. */
```

[EX1] top 명령어 기본 사용방법 실습

```
# top -h      (# top --help)
top: procps version 3.2.7
usage:  top -hv | -bcisSHM -d delay -n iterations [-u user | -U user] -p pid [,pid ...]
```

```
# top
top - 17:46:37 up  2:25,  4 users,  load average: 0.03, 0.03, 0.00
Tasks: 136 total,   1 running, 134 sleeping,   0 stopped,   1 zombie
Cpu(s):  2.0%us,  1.7%sy,  0.0%ni, 96.3%id,  0.0%wa,  0.0%hi,  0.0%si,  0.0%st
Mem:   1035096k total,   718752k used,   316344k free,    68376k buffers
Swap:   522104k total,        0k used,   522104k free,   492140k cached

  PID USER      PR  NI  VIRT  RES  SHR S %CPU %MEM    TIME+  COMMAND
 4203 root      16   0 84828  16m  11m S  2.7  1.6   0:09.65 gnome-terminal
 3624 root      15   0 35972  10m 6360 S  0.7  1.1   0:16.60 Xorg
 3813 root      15   0 42812  10m 8520 S  0.2  1.1   0:01.10 vmware-user
    1 root      18   0  2072  616  532 S  0.0  0.1   0:01.26 init
    2 root      RT  -5     0    0    0 S  0.0  0.0   0:00.35 migration/0
    3 root      34  19     0    0    0 S  0.0  0.0   0:00.00 ksoftirqd/0
    4 root      RT  -5     0    0    0 S  0.0  0.0   0:00.00 watchdog/0
    5 root      RT  -5     0    0    0 S  0.0  0.0   0:00.15 migration/1
    6 root      36  19     0    0    0 S  0.0  0.0   0:00.00 ksoftirqd/1
    7 root      RT  -5     0    0    0 S  0.0  0.0   0:00.00 watchdog/1
    8 root      10  -5     0    0    0 S  0.0  0.0   0:00.03 events/0
    9 root      10  -5     0    0    0 S  0.0  0.0   0:00.00 events/1
   10 root      10  -5     0    0    0 S  0.0  0.0   0:00.01 khelper
   11 root      10  -5     0    0    0 S  0.0  0.0   0:00.00 kthread
   15 root      18  -5     0    0    0 S  0.0  0.0   0:00.03 kblockd/0
   16 root      10  -5     0    0    0 S  0.0  0.0   0:00.02 kblockd/1
   17 root      15  -5     0    0    0 S  0.0  0.0   0:00.00 kacpid
  183 root      11  -5     0    0    0 S  0.0  0.0   0:00.00 cqueue/0
  184 root      11  -5     0    0    0 S  0.0  0.0   0:00.00 cqueue/1
  187 root      10  -5     0    0    0 S  0.0  0.0   0:00.00 khubd
  189 root      10  -5     0    0    0 S  0.0  0.0   0:00.00 kseriod
  256 root      16   0     0    0    0 S  0.0  0.0   0:00.00 pdflush
  257 root      15   0     0    0    0 S  0.0  0.0   0:00.15 pdflush
..... (중략) .....
h  <----- 'h' 입력

Help for Interactive Commands - procps version 3.2.7
Window 1:Def: Cumulative mode Off.  System: Delay 3.0 secs; Secure mode Off.

  Z,B       Global: 'Z' change color mappings; 'B' disable/enable bold
  l,t,m     Toggle Summaries: 'l' load avg; 't' task/cpu stats; 'm' mem info
  1,I       Toggle SMP view: '1' single/separate states; 'I' Irix/Solaris mode

  f,o     . Fields/Columns: 'f' add or remove; 'o' change display order
  F or O  . Select sort field
  <,>     . Move sort field: '<' next col left; '>' next col right
  R,H     . Toggle: 'R' normal/reverse sort; 'H' show threads
  c,i,S   . Toggle: 'c' cmd name/line; 'i' idle tasks; 'S' cumulative time
  x,y     . Toggle highlights: 'x' sort field; 'y' running tasks
  z,b     . Toggle: 'z' color/mono; 'b' bold/reverse (only if 'x' or 'y')
  u       . Show specific user only
  n or #  . Set maximum tasks displayed

  k,r       Manipulate tasks: 'k' kill; 'r' renice
  d or s    Set update interval
  W         Write configuration file
  q         Quit
          ( commands shown with '.' require a visible task display window )
Press 'h' or '?' for help with Windows,
any other key to continue
```

```
PID to kill: <firedox의 PID번호 입력>
Kill PID 23846 with signal [15]: <9>

정렬 기준 변경 : F입력

Current Sort Field:  K  for window 1:Def
Select sort field via field letter, type any other key to return

  a: PID        = Process Id              s: DATA       = Data+Stack size (kb)   not those in
column display.  Thus,       t: SHR         = Shared Mem size (kb)   the TTY & WCHAN fields
will violate       u: nFLT        = Page Fault count        strict ASCII collating sequence.
    v: nDRT       = Dirty Pages count      (shame on you if WCHAN is chosen)
  f: GROUP      = Group Name              x: COMMAND    = Command name/line
  g: TTY        = Controlling Tty         y: WCHAN      = Sleeping in Function
  h: PR         = Priority                z: Flags      = Task Flags <sched.h>
  i: NI         = Nice value
  j: P          = Last used cpu (SMP)   Note1:
* K: %CPU       = CPU usage               If a selected sort field can't be
  l: TIME       = CPU Time                shown due to screen width or your
  m: TIME+      = CPU Time, hundredths    field order, the '<' and '>' keys
  n: %MEM       = Memory usage (RES)      will be unavailable until a field
  o: VIRT       = Virtual Image (kb)      within viewable range is chosen.
  p: SWAP       = Swapped size (kb)
  q: RES        = Resident size (kb)    Note2:
  r: CODE       = Code size (kb)          Field sorting uses internal values,
```

- (첫 번째 줄) 시스템의 현재 시간과 시스템이 부팅된 후 작동한 시간, 현재 사용자 수를 보여주며, load average는 CPU 로드의 평균 값을 보여준다. (1분 , 5분, 15분)
- (두 번째 줄) 시스템에 동작중인 프로세스의 상태를 보여준다.
- (세 번째 줄) CPU의 상태를 보여준다
- (네 번째 / 다섯 번째 줄) 총 사용 가능 메모리, 사용된 메모리, 사용할 수 있는 메모리, 공유 메모리, 버퍼로사용된 메모리, 스왑 메모리등 메모리에 관한 정보를 보여준다. 각각의 시스템 자원에 따른 프로세스의 점유율을 확인 할 수 있다.

(top 명령어 필드 해석)

```
a: PID  --  Process Id
          The  task's  unique process ID, which periodically wraps, though never
          restarting at zero.

       b: PPID  --  Parent Process Pid
          The process ID of a task's parent.

       c: RUSER  --  Real User Name
          The real user name of the task's owner.

       d: UID  --  User Id
          The effective user ID of the task's owner.

       e: USER  --  User Name
          The effective user name of the task's owner.

       f: GROUP  --  Group Name
          The effective group name of the task's owner.

       g: TTY  --  Controlling Tty
          The name of the controlling terminal.  This  is  usually  the  device
          (serial port, pty, etc.) from which the process was started, and which
          it uses for input or output.  However, a task need not  be  associated
          with a terminal, in which case you'll see '?' displayed.

       h: PR  --  Priority
          The priority of the task.

       i: NI  --  Nice value
          The nice value of the task.  A negative nice value means higher prior-
          ity, whereas a positive nice value means lower priority.  Zero in this
          field  simply  means  priority  will  not be adjusted in determining a
          task's dispatchability.

       j: P  --  Last used CPU (SMP)
```

A number representing the last used processor. In a true SMP environment this will likely change frequently since the kernel intentionally uses weak affinity. Also, the very act of running top may break this weak affinity and cause more processes to change CPUs more often (because of the extra demand for cpu time).

k: %CPU -- CPU usage
The task's share of the elapsed CPU time since the last screen update, expressed as a percentage of total CPU time. In a true SMP environment, if 'Irix mode' is Off, top will operate in 'Solaris mode' where a task's cpu usage will be divided by the total number of CPUs. You toggle 'Irix/Solaris' modes with the 'I' interactive command.

l: TIME -- CPU Time
Total CPU time the task has used since it started. When 'Cumulative mode' is On, each process is listed with the cpu time that it and its dead children has used. You toggle 'Cumulative mode' with 'S', which is a command-line option and an interactive command. See the 'S' interactive command for additional information regarding this mode.

m: TIME+ -- CPU Time, hundredths
The same as 'TIME', but reflecting more granularity through hundredths of a second.

n: %MEM -- Memory usage (RES)
A task's currently used share of available physical memory.

o: VIRT -- Virtual Image (kb)
The total amount of virtual memory used by the task. It includes all code, data and shared libraries plus pages that have been swapped out. (Note: you can define the STATSIZE=1 environment variable and the VIRT will be calculated from the /proc/#/state VmSize field.)

VIRT = SWAP + RES.

p: SWAP -- Swapped size (kb)
The swapped out portion of a task's total virtual memory image.

q: RES -- Resident size (kb)
The non-swapped physical memory a task has used.

RES = CODE + DATA.

r: CODE -- Code size (kb)
The amount of physical memory devoted to executable code, also known as the 'text resident set' size or TRS.

s: DATA -- Data+Stack size (kb)
The amount of physical memory devoted to other than executable code, also known as the 'data resident set' size or DRS.

t: SHR -- Shared Mem size (kb)
The amount of shared memory used by a task. It simply reflects memory that could be potentially shared with other processes.

u: nFLT -- Page Fault count
The number of major page faults that have occurred for a task. A page fault occurs when a process attempts to read from or write to a virtual page that is not currently present in its address space. A major page fault is when disk access is involved in making that page available.

v: nDRT -- Dirty Pages count
The number of pages that have been modified since they were last written to disk. Dirty pages must be written to disk before the corresponding physical memory location can be used for some other virtual page.

w: S -- Process Status
The status of the task which can be one of:

```
            'D' = uninterruptible sleep
            'R' = running
            'S' = sleeping
            'T' = traced or stopped
            'Z' = zombie

         Tasks shown as running should be more properly thought of as 'ready to
         run'   --   their  task_struct is simply represented on the Linux run-
         queue.  Even without a true SMP machine, you may see numerous tasks in
         this state depending on top's delay interval and nice value.

      x: Command  --  Command line or Program name
         Display the command line used to start a task or the name of the asso-
         ciated program.  You toggle between command line and  name  with  'c',
         which is both a command-line option and an interactive command.

         When  you've chosen to display command lines, processes without a com-
         mand line (like kernel threads) will be shown with  only  the  program
         name in parentheses, as in this example:
               ( mdrecoveryd )

         Either  form of display is subject to potential truncation if it's too
         long to fit in this field's current width.  That  width  depends  upon
         other fields selected, their order and the current screen width.

         Note:  The  'Command' field/column is unique, in that it is not fixed-
         width.  When displayed, this column will be  allocated  all  remaining
         screen  width  (up  to  the maximum 512 characters) to provide for the
         potential growth of program names into command lines.

      y: WCHAN  --  Sleeping in Function
         Depending on the availability of the kernel link  map  ('System.map'),
         this field will show the name or the address of the kernel function in
         which the task is currently sleeping.  Running tasks  will  display  a
         dash ('-') in this column.

         Note:  By  displaying  this  field,  top's  own  working  set  will be
         increased by over 700Kb.  Your only means of  reducing  that  overhead
         will be to stop and restart top.

      z: Flags  --  Task Flags
         This  column  represents the task's current scheduling flags which are
         expressed in hexadecimal notation and with  zeros  suppressed.   These
         flags are officially documented in <linux/sched.h>.  Less formal docu-
         mentation can also be found on the 'Fields select' and 'Order  fields'
         screens.
------------------------------------------------------------------------------
```

[EX2] 사용자 별로 모니터링

[명령어형식]

```
# top -u root
# top -u oracle
# top -u wasuser
```

[EX2] 사용자 별로 모니터링

[명령어형식]

```
# top -u fedora
```

[TERM1] root 사용자 윈도우

```
# telnet localhost
fedora 사용자로 로그인

$ id
$ bash   -----> 관리자 윈도우 확인
$ ksh    -----> 관리자 윈도우 확인
$ vi /etc/passwd
$ exit
$ exit
$ exit
```

[EX3] 몇개의 프로세스만 모니터링

```
[명령어 형식]
# top -p 1
# top -p 1,3624

# ps    (# pstree 2371)
 PID TTY          TIME CMD
 2371 pts/3    00:00:00 bash
 6554 pts/3    00:00:00 ps
# top -p 1,2371
top - 17:51:27 up 1 day, 8 min,  4 users,  load average: 0.12, 0.16, 0.10
Tasks:   2 total,   0 running,   2 sleeping,   0 stopped,   0 zombie
Cpu(s):  7.4%us,  1.5%sy,  0.0%ni, 90.9%id,  0.0%wa,  0.0%hi,  0.3%si,  0.0%st
Mem:    515340k total,   500336k used,    15004k free,    63108k buffers
Swap:  1052248k total,        0k used,  1052248k free,   255872k cached

  PID USER      PR  NI  VIRT  RES  SHR S %CPU %MEM    TIME+  COMMAND
    1 root      18   0  2068  620  532 S  0.0  0.1   0:00.49 init
 2371 root      15   0  5872 1496 1188 S  0.0  0.3   0:00.62 bash
-> 프로세스 PID 번호 선택은 출력 화면 참조
```